Snapper Documentation

Comprehensive project guide

Generated on 2026-06-23

Table of Contents

Snapper

Trading platform with market data collection, trade runtime, and backtester. Supports Kraken (WebSocket; spot + futures + equities), Walutomat (REST polling), and Polygon.io (REST) market data.

Quick Steps

```
# One-time: install frontend deps and build the static UI so the
# FastAPI server can serve the dashboard at `/`.
make ui-setup
make ui-build

# Initialize the database + seed dev data, then refresh the verified
# symbol mappings and market snapshots so the dashboard has data to show.
make migrate-dev run-static

# Start the server
make run-server
```

Open http://localhost:8000/ and log in with the dev seed credentials — the default values are defined in the bundled seed file src/snapper/data/seed/dev.toml (admin / change-me-after-first-login); maintainers with the proprietary submodule get proprietary/data/seed/dev.toml via the three-tier seed lookup. Override per-environment by editing the seed file before running make migrate-dev, or rotate after first login via POST /api/auth/users/{user_id}/change-password (self-service) or POST /api/auth/users/{user_id}/admin-reset-password (admin reset).

Features

- **Market data collection** — Kraken (WebSocket; spot + futures + equities), Walutomat (REST polling), Polygon.io (REST)
- **Candle synthesis** — Multi-timeframe candle synthesis from the 1m base stream (in-process per-publisher rollups) with provenance tagging (native / calculated / synthesized), controlled by the timeframes, candle_forward_fill, and persist_intermediate_candles settings
- **Egress routing** — Per-venue VPN egress routing gated by feed_egress_enabled: public feed traffic routes through the WireGuard/SOCKS5 snapper-egress sidecar while private/executor traffic stays direct-first, with private_fallback_route_id used only when direct is unavailable; private mutations remain direct-only

- **Symbol correlation** — Underlying asset model linking instruments across exchanges (e.g., SPY, ESM6-CME, SPYX-USD-PERP all map to S&P 500). YAML-driven pattern matching, front-month rollover, contract ladder API
- **Trade runtime** — Facts-canonical live and paper trading with canonical Order and Execution facts, rebuildable Position and Balance projections, dispatched via a durable outbox
- **Strategies** — Framework for creating strategies based on RSI, MACD, cointegration, and TA-Lib indicators
- **ZeroMQ messaging** — Pub/sub architecture for market data and signals
- **Provenance and audit** — Every payload item carries session_id and sequence_id for gap detection. Control table (always-on) and telemetry table (toggleable) record all commands and operational events
- **Web dashboard** — FastAPI + React with real-time WebSocket
- **CLI** — Full system management via Typer CLI
- **Backtesting** — Strategy testing on historical data

Quick Start

Requirements

- Python 3.14+
- Poetry
- Node.js 26+ and pnpm 11+ (for frontend)
- Build tools for Python packages. The TA-Lib adapter uses TA-Lib when importable and falls back to pure Python indicators otherwise; install the native TA-Lib C library only if your local wheel build requires it.

Installation

```
# Clone repository
git clone https://github.com/mateusz-klatt/snapper.git
cd snapper

# Install system dependencies
make system-deps

# Install Python dependencies
make setup

# Install frontend dependencies
make ui-setup

# Copy and configure environment variables
cp .env.example .env
```

Configuration

Edit .env file:

```
# Database (SQLite dev, PostgreSQL prod)
DB_URL=sqlite+aiosqlite:///./data/snapper.db

# Deployment environment — production/prod/staging refuse placeholder
# secrets (MASTER_PASSWORD, auth_secret_key, csrf_secret_key)
SNAPPER_ENV=development

# Settings encryption in database
MASTER_PASSWORD=your_master_password

# HTTP Server
SERVER_HOST=0.0.0.0
SERVER_PORT=8000
SERVER_RELOAD=false
SERVER_API_ONLY=false
SERVER_PROXY_HEADERS=true
SERVER_FORWARDED_ALLOW_IPS=127.0.0.1,172.17.0.1

# Telemetry recording (pings, heartbeats, GET reads)
TELEMETRY_RECORDING_ENABLED=false

# ZeroMQ broker
ZMQ_BROKER_XSUB=tcp://127.0.0.1:7500
ZMQ_BROKER_XPUB=tcp://127.0.0.1:7501

# Max command age before the outbox expires stale create/submit
# commands instead of publishing them (<= 0 disables)
TRADE_COMMAND_DISPATCH_TTL_S=30.0
```

Running

```
# Initialize database and seed data
make migrate-dev

# Start server
snapper server
```

Two ways to bring up the dashboard:

- **Backend-served (production / single port).** Run make ui-setup + make ui-build to produce frontend/dist/, then start the server. The FastAPI app mounts the static UI at / only when frontend/dist/ exists; the dashboard is then at http://localhost:8000/.
- **Vite dev server (hot reload).** Run make ui-dev in a separate terminal — Vite serves the dashboard at http://localhost:3000/ with a backend proxy for /api and /api/ws. This does NOT produce frontend/dist/, so http://localhost:8000/ will still return 404 for the UI until the build step is run.

Trade Runtime Dispatch

The trade runtime always uses durable, outbox-driven dispatch: commands are persisted as TradeCommand rows and published by the outbox dispatcher. On the accepted/fill paths, executor VenueEvent persistence is fail-closed — a failed write raises before the corresponding orders.events.* publish. An ambiguous submit failure never fabricates a rejection: the executor verifies venue truth via find_order_by_client_id and, when the venue cannot answer, parks the order in the non-terminal UNKNOWN state — the engine holds its in-flight guard, the recon loop re-verifies the order each cycle until resolved, and the order_unknown notification rule alerts the operator.

Replayed submits of an already-evidenced client_order_id (outbox re-publishes after a crash between publish and the dispatched commit) are dropped by a duplicate-submit guard; its durable venue-evidence probe is fail-closed — a failed check drops the command rather than risk double-placing a market order. Dispatch is also bounded by a max-age TTL (TRADE_COMMAND_DISPATCH_TTL_S, default 30 s): the outbox CAS-expires stale CREATED create/submit commands to EXPIRED instead of publishing them, so an outage backlog cannot fire orders priced off old signals. Cancels are exempt — expiring a stale cancel would strand a live order. A submit refused by an open venue circuit breaker is neither left ambiguous nor blind-retried: the executor records durable breaker-open evidence (counted as duplicate-submit evidence), marks the command FAILED, and publishes a rejection with reason circuit_breaker_open so the engine releases its in-flight intent; if any step fails, the order is parked and the reconciliation loop reruns the disposition.

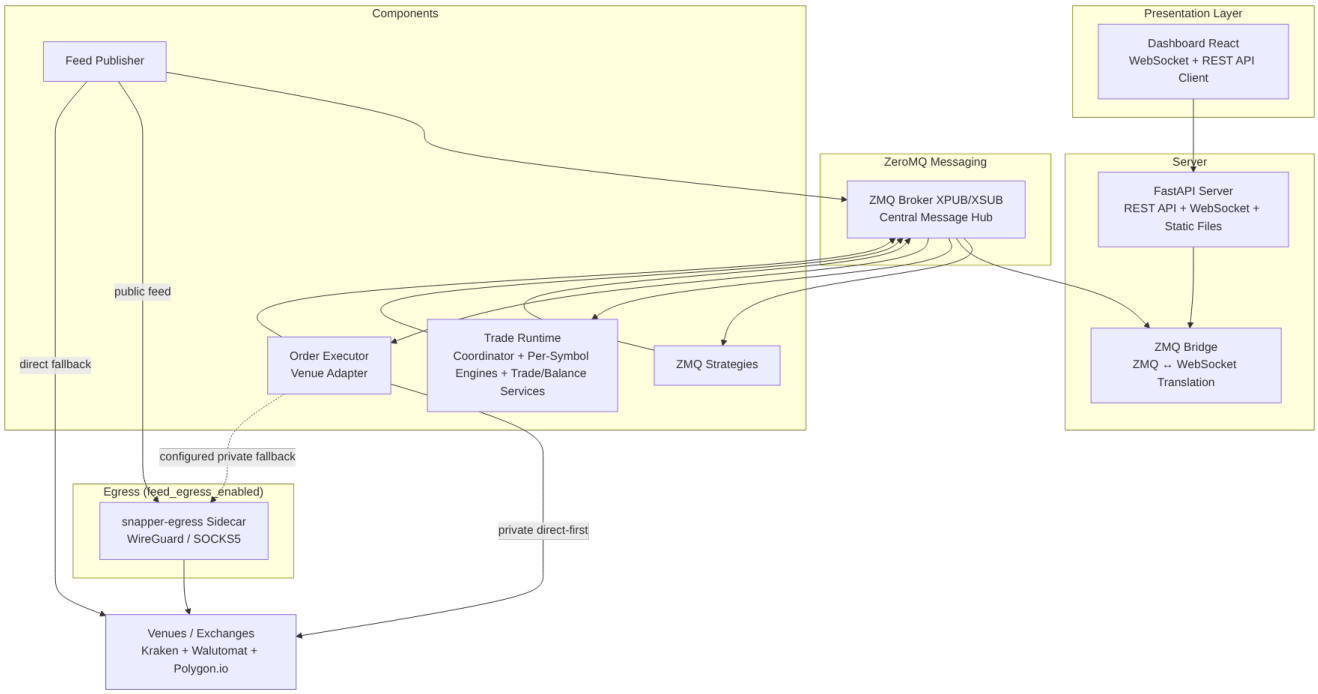
Private fill streams are supervised: a dead venue execution stream is respawned with capped backoff, each respawn reconciles fills missed during the dark window before resubscribing, and executor startup recovery emits corrective fills for anything filled while the executor was down instead of re-baselining to the venue's current cumulative. Corrective fills carry venue-reported fees rather than fabricating fee-less executions; when the venue's fee source is transiently unavailable, the corrective is deferred for a bounded number of reconciliation cycles before degrading to a fee-less emission with a critical alert. The same supervision wraps every executor core loop (order handling, reconciliation, heartbeat): a died loop respawns with capped backoff, and a persistent death streak escalates by crashing the executor so the process launcher rebuilds a fresh instance. Executor heartbeats report status derived from actual loop health — death streaks, reconciliation progress, stuck in-flight commands — instead of a hardcoded healthy, and an executor parked after exhausting its restart budget is alerted through launcher-published synthetic error heartbeats rather than vanishing silently.

Execution-plan decision events (plans.decisions.*) get the same durability: each decision row the plan executor logs commits together with an outbox row in one transaction, the plan executor publishes immediately, and a background drain loop replays unsent rows with capped exponential backoff after broker outages.

Process-Managed Executors

Process-managed deployments run one executor per (exchange, wallet) credential row. Bare executor_<exchange> process configs are templates only; the launcher expands active wallet_credentials rows into executor_<exchange>_w<wallet_short> instances, where wallet_short is the last 12 lowercase hex characters of the wallet UUID7. Each instance loads only its wallet credentials and filters incoming command frames by wallet_public_id.

System Overview



CLI Commands

Server and Infrastructure

```
snapper server              # Start FastAPI server
snapper broker              # Start ZMQ broker
snapper trade-zmq           # Trade runtime / coordinator (pass --instance-id +
--instance-count for N>=2, see docs/operations.md)
snapper executor            # Standalone order executor helper
snapper feed                # Direct Kraken market data publisher helper
snapper feed-engine         # Dedicated feed container entrypoint
snapper egress              # WireGuard + SOCKS5 egress sidecar entrypoint
```

Database

```
snapper db-init             # Initialize schema
snapper db-upgrade          # Alembic migrations
snapper db-downgrade        # Rollback migrations
```

Users

```
snapper init-admin          # Create admin user
snapper list-users          # List users
snapper reset-password      # Reset password
```

Local AI delegate PAT (for @mateusz-klatt/snapper-mcp bridge)

After make dev-backend is running and the seed has provisioned the admin user (per dev.toml), mint a long-lived AI delegate PAT into a JSON file the bridge consumes via --config=PATH:

```
snapper dev-mint-pat        # writes data/dev-pat.json (mode 0600)
```

Wire your ~/.claude.json mcpServers entry once with the file path:

```
"snapper-local": {
  "command": "node",
  "args": [
    "/path/to/snapper-mcp/dist/index.js",
    "--config=/path/to/snapper/data/dev-pat.json"
  ]
}
```

After running, in this order, (a) rm data/snapper.db, (b) make migrate-dev, (c) make dev-backend (in a separate terminal — it blocks the foreground with the live backend), and (d) snapper dev-mint-pat, the bridge picks up the refreshed token automatically — no manual edits to ~/.claude.json per DB wipe. CLI flags + environment overrides:

```
--base-url URL        SNAPPER_DEV_BASE_URL          (default http://
localhost:8000)
--admin-username USER SNAPPER_DEV_ADMIN_USERNAME    (default None → resolved
from seed profiles, mcp then dev)
--admin-password PASS SNAPPER_DEV_ADMIN_PASSWORD    (default None → resolved
from seed profiles, mcp then dev)
--output PATH         SNAPPER_DEV_PAT_OUTPUT        (default data/dev-pat.json)
--label LABEL                                         (default "Local Dev MCP")
```

The script drives production REST endpoints (POST /api/auth/login + POST /api/ai-delegates) so the seeded delegate exercises the same code path the AI Integration UI uses. Tokens never appear in stderr — HTTP error bodies are passed through a JWT-shape redaction helper before printing.

Market Data

```
snapper update-kraken-symbols        # Sync Kraken symbols
snapper update-kraken-futures-symbols
snapper update-kraken-equities-symbols
snapper update-walutomat-symbols
snapper update-polygon-symbols       # Sync Polygon symbols
snapper update-underlyings           # Sync underlying asset mappings from YAML
snapper update-kraken-market-snapshot
snapper update-kraken-futures-market-snapshot
snapper update-kraken-equities-market-snapshot
snapper update-walutomat-market-snapshot
snapper polygon-backfill-aggregates
# Step 1: download history to CSV cache (no DB write)
snapper polygon-load-csv --all       # Step 2: load the CSV cache into the
database
snapper kraken-futures-backfill-candles
snapper kraken-equities-backfill-candles
snapper update-kraken-futures-funding-rates
snapper archive --day 2024-01-15     # Export candle cache to CSV
snapper archive --from 2024-01-01 --to 2024-01-31 --exchange polygon
snapper archive --table ticks --day 2024-01-15 --exchange polygon
snapper archive --table candles-audit --day 2024-01-15 --closed-only --purge
```

See docs/cli.md for full options and examples.

Strategies

Creating your own strategy:

```python
from snapper.strategies.base import BaseStrategy, StrategySignal, StrategyConfig
from snapper.strategies.decorators import register_strategy,
create_strategy_process
from snapper.messaging.schemas.data import CandleData

@register_strategy("MyStrategy")
@create_strategy_process(
    process_name="my_strategy",
    default_config={
        "name": "my_strategy",
        "inputs": ["market.kraken.BTC-USD.candles.1h"],
        "outputs": ["BTC-USD"],
        "exchange": "paper",
        "params": {"threshold": 0.5},
    },
)
class MyStrategy(BaseStrategy):
    def __init__(self, config: StrategyConfig) -> None:
```

```
        super().__init__(config)
        self.threshold = self.params.get("threshold", 0.5)

    async def reset(self) -> None:
        """Clear any per-instrument indicator/state buffers."""

    async def on_candle(self, instrument: str, candle: CandleData) ->
StrategySignal | None:
        if some_condition:
            return StrategySignal(
                instrument=instrument,
                side="buy",
                strength=1.0,
                price=candle.close,
                reason="Condition met",
            )
        return None
```

reset() is abstract on BaseStrategy — every subclass must implement it so the runtime can recycle per-instrument state on restart / replay.

Development

Quality Gates

```
make check-all   # Full quality gate (backend + frontend + tests + 100%
coverage)
make fix-all     # Automatic formatting and linting fixes
```

Individual Steps

```
make fmt         # Check formatting
make lint        # Linting (ruff)
make typecheck   # Type checking (mypy)
make test        # Unit tests
make cov         # Tests with coverage (100% required)
```

Generated contracts are split by target: make ui-gen-types refreshes frontend OpenAPI/WebSocket/Zod/entity/permission types, make ios-gen-types refreshes Swift models, and make ts-bridge (or make bridge-regen) refreshes the opt-in snapper-mcp bridge wire contract.

Frontend

```
make ui-dev      # Vite development server
make ui-build    # Production build
make ui-lint     # ESLint
make ui-format   # Prettier
```

Docker

```
make docker-build-dev    # Build dev image (backend + caddy binary baked in)
make docker-build-prod   # Build production image
make docker-migrate-dev  # Initialize and seed the Docker SQLite database
make docker-run          # Run container
make docker-stop         # Stop containers
```

Or with docker compose:

```
make docker-migrate-dev
docker compose up -d
```

Compose topology

The compose stack runs four application services on the internal snapper-internal bridge network, all from a **single image** (klattm/snapper:latest) that bundles the python runtime + the Caddy binary. Each service overrides entrypoint / command to launch the right process. The optional postgres service is gated behind the dev compose profile.

- snapper — FastAPI backend, ZMQ broker, strategy/process control, and non-publisher runtime processes. PROCESS_AUTOSTART_PROFILE=api, command: ["server"], expose: 8000/7500/7501 internally.
- snapper-feed — dedicated market-data publisher container. PROCESS_AUTOSTART_PROFILE=feed, command: ["feed-engine"]; it connects to the backend broker via tcp://snapper:7500/7501 and uses reduced DB pool settings so publisher subprocesses do not overrun PostgreSQL connection limits.
- snapper-egress — WireGuard + SOCKS5 sidecar for outbound publisher traffic. command: ["egress"], cap_add: NET_ADMIN, kernel module bind.
- snapper-web — Caddy serving the React SPA from /srv/dist and reverse-proxying /api/*, /api/ws, /api/mcp, /docs, /redoc, /openapi.json to snapper:8000. entrypoint: ["/usr/bin/caddy"], command: ["run", "--config", "/etc/caddy/Caddyfile"]. Owns the host 127.0.0.1:8000:8000 bind.

All four application services are restart: unless-stopped so they come back automatically after a host reboot (assuming systemctl is-enabled docker returns enabled). Services explicitly stopped via docker compose stop stay stopped.

Targeted restarts (no tick-stream drop)

Restart targets are now strictly recreate-only — they use whatever image tag exists locally. Rebuild explicitly first if you need new code baked in:

```
# Build (explicit, when you want new code in the image):
make docker-build-dev    # caller UID (host user)
make docker-build-prod   # UID 888 (matches log file ownership)

# Recreate containers (uses current image, no rebuild):
make restart-frontend    # Recreate Caddy sidecar [does NOT touch backend]
make restart-backend     # Recreate backend [drops ticks 30-90s]
make restart-all         # Recreate full stack [drops ticks]

# Common idiom — build + restart in one shot:
make docker-build-prod restart-backend
```

make restart-frontend is the fast path for shipping UI changes without restarting publishers. Only the Caddy container is recreated; WS clients reconnect within ~3 s but the backend snapper container's PID is unchanged and the Kraken / Walutomat tick streams continue uninterrupted.

Footgun: a plain docker compose up -d (without --no-deps) follows depends_on edges and may recreate the backend too. Use the targeted Makefile commands for selective restarts, and reserve docker compose up -d for full-stack restarts when both should restart anyway.

Documentation

Detailed documentation in docs/ directory:

- Architecture — System structure and layers
- Configuration — Environment variables and settings
- CLI — Full command documentation
- Strategies — Creating trading strategies
- Backtesting — Backtest engines and artifacts
- API — REST API and WebSocket
- Messaging — ZeroMQ architecture
- Operations — Multi-instance coordinator runbook
- Observability — Process, notification, retention, and DB table metrics surface
- AI integration — MCP endpoint and AI delegate tokens
- Development — Developer guidelines (incl. Internationalization)
- Paired execution — Multi-leg guard operator runbook
- Egress — WireGuard + SOCKS5 egress sidecar runbook

Regenerate the bundled frontend documentation PDF with make docs-pdf.

License

MIT License — see LICENSE file.

AI Integration

Snapper exposes a **Model Context Protocol (MCP)** endpoint at /api/mcp so Claude Desktop, Cursor, Windsurf, or a plain curl client can authenticate with a long-lived AI delegate access JWT and call a narrow set of trading + market-data tools. The surface is vendor-neutral — no Anthropic or OpenAI SDK is bundled in Snapper core.

This doc covers:

Feature flag

The /api/mcp sub-app is always mounted and gated by the ai_integration_enabled database setting.

1. The flag **defaults to true** — a fresh install exposes the MCP endpoint and the AI Integration navigation entry with no manual setup. Admins who need to disable the feature flip the setting to false via the Settings UI (admin) or directly:

   ```
   curl -X POST http://localhost:8000/api/settings/ai_integration_enabled/set \
     -H "Authorization: Bearer <admin-jwt>" \
     -H "Content-Type: application/json" \
     -d '{"session_id":"cli","sequence_id":1,"public_id":"$
   (uuidgen)","timestamp":"2026-04-20T00:00:00Z","payload":
   {"value":"false","category":"system"}}'
   ```

2. The feature endpoint itself is public, but the frontend route and navigation entry are role/permission-gated. After authentication, the AI Integration page reads GET /api/settings/features and renders the enabled or disabled state. The MCP endpoint and /api/ai-delegates/* return 503 feature_disabled only when the flag is explicitly set to false.

3. Whether the flag is on or off, the MCP endpoint requires every request to carry a valid Authorization: Bearer <jwt> header. Anonymous requests receive 401 missing_bearer_token.

Creating an AI delegate

An **AI delegate** is a dedicated AI_DELEGATE-role user an operator mints per MCP client. Delegates:

- Cannot log in via the web UI password form (the placeholder password hash is opaque to humans).
- Cannot see other delegates, operators, or wallets — their wallet scope comes from the operator they are bound to.
- Carry per-delegate safety caps independent of the operating operator's caps.

Create one via POST /api/ai-delegates:

```
curl -X POST http://localhost:8000/api/ai-delegates \
  -H "Authorization: Bearer <operator-jwt>" \
  -H "Content-Type: application/json" \
  -H "X-CSRF-Token: <csrf>" \
  -d '{
    "session_id": "cli",
    "sequence_id": 1,
    "public_id": "'$(uuidgen)'",
    "timestamp": "2026-04-20T00:00:00Z",
    "payload": {
      "label": "Claude Desktop",
      "operator_public_id": "<operator-public-id>",
      "caps": {
        "max_open_orders": 3,
        "max_daily_notional_usd": 1000.0,
        "max_cancels_per_minute": 10
      }
    }
  }'
```

payload.operator_public_id is optional. When omitted, Snapper binds the delegate to the caller's primary_operator_public_id; if the caller has no primary operator, create returns 422 until an explicit operator is supplied. When supplied by a non-admin caller, the operator must be in the caller's authenticated operator claims. Admin callers have the admin operator bypass and may bind explicitly to any operator. The minted delegate token inherits scope from that bound operator, so later scope grant changes for that operator control which wallets/instruments the delegate can act on.

The response is **one-shot**. Copy the token out of the HTTP session immediately — Snapper never re-serves it:

```
{
  "type": "delegate_created_response",
  "sequence_id": 1,
  "public_id": "<envelope-uuid7>",
  "timestamp": "2026-04-20T00:00:00Z",
  "session_id": "<server-session>",
  "topic": null,
  "payload": {
    "delegate": {
      "public_id": "019da9e...",
      "username": "ai-claudedesktop-a1b2c3",
      "label": "claudedesktop",
      "created_by_user_public_id": "<operator-id>",
      "created_at": "2026-04-20T00:00:00Z",
      "is_active": true,
      "caps": { "max_open_orders": 3, "max_daily_notional_usd": 1000.0,
"max_cancels_per_minute": 10 }
    },
    "access_token": "<jwt-with-90d-exp>",
    "expires_in": 7776000
  }
}
```


Other endpoints on /api/ai-delegates:

- GET /api/ai-delegates — list the caller's delegates (no tokens re-served).
- GET /api/ai-delegates/{id} — single delegate detail.
- PATCH /api/ai-delegates/{id} — update caps (SCD2 close+insert). label/username are immutable post-mint.
- POST /api/ai-delegates/{id}/deactivate — kill switch. Publishes admin.user_deactivated on the bus for immediate fanout; every Snapper instance also polls the DB-backed deactivation registry so matching WebSocket sessions close and token LRU entries are evicted even if the broker is unavailable.

Token model

Each AI delegate mints a single long-lived (~3-month) access JWT. The same token authenticates both the proxy MCP server and the optional push-wakeup watch monitor. Revocation is server-side: POST /api/ai-delegates/{id}/deactivate flips users.is_active=False, revokes the delegate's user_active_tokens row, publishes admin.user_deactivated on the bus, and each Snapper instance evicts matching verify-cache entries on receipt or through the DB-backed fallback scanner. The local JTI blacklist uses a 10-second grace window for requests that raced the kill switch. The 90-day exp is a ceiling, not a commitment; operators are expected to rotate delegate tokens on the cadence that fits their key-management hygiene.

Client configuration examples

Claude Code plugin (recommended)

In any Claude Code session:

```
/plugin marketplace add mateusz-klatt/snapper-mcp
/plugin install snapper-mcp@mateusz-klatt-snapper-mcp
/reload-plugins
```

Claude Code prompts for two required values at install time (per the plugin's userConfig schema in .claude-plugin/plugin.json):

- **Snapper API URL** -- your backend's /api/mcp endpoint. The @mateusz-klatt/snapper-mcp bridge accepts the value with or without a trailing slash and normalizes it before connecting.
- **Access token** -- the access_token from the delegate_created response (or paste from the Settings -> AI Delegates config-snippet generator). Delegates are PAT-style: the JWT lifetime is ~3 months (LONG_LIVED_TOKEN_EXPIRE_DAYS = 90); operators are expected to rotate by minting a fresh delegate via the same flow before the existing one expires. Revocation is immediate: deactivating the delegate in Snapper invalidates the associated user_active_tokens row server-side within one bus round-trip.

Run /mcp list to confirm the snapper server is connected. The plugin pins the runtime to a specific @mateusz-klatt/snapper-mcp version — the manifest hardcodes the exact version string in mcpServers.snapper.args (currently @0.11.0), kept in lockstep with the plugin's own version field by the integrations/snapper-mcp/test/plugin_manifest.test.ts parity test so a bump to either side fails CI until both match. Future runtime publishes do not silently upgrade existing installs; /plugin update snapper-mcp opts in. Sensitive values land in the OS keychain (with ~/.claude/.credentials.json fallback) -- never in settings.json or the plugin manifest.

Claude Desktop

Add to ~/Library/Application Support/Claude/claude_desktop_config.json:

```
{
  "mcpServers": {
    "snapper": {
      "command": "npx",
      "args": ["-y", "@mateusz-klatt/snapper-mcp"],
      "env": {
        "SNAPPER_BASE_URL": "https://snapper.example.com/api/mcp",
        "SNAPPER_ACCESS_TOKEN": "<copied-from-create-response>"
      }
```

```
    }
  }
}
```

The @mateusz-klatt/snapper-mcp npm wrapper presents the access token as a Bearer header on every request. On 401 the bridge surfaces the auth failure to the MCP host once per session; recovery is via recreating the AI delegate in Snapper.

Cursor

In ~/.cursor/config.json:

```
{
  "mcpServers": {
    "snapper": {
      "url": "https://snapper.example.com/api/mcp/",
      "headers": {
        "Authorization": "Bearer <access-token>"
      }
    }
  }
}
```

The delegate access token Cursor sends as a Bearer header is a long-lived PAT JWT (~3 months), so day-to-day operation does not require token rotation more often than the 90-day expiry. Revocation is by deactivating the delegate in Snapper (which invalidates the underlying user_active_tokens row); recovery is to recreate the delegate and update Cursor's Authorization header with the new token.

Windsurf

~/.codeium/windsurf/mcp_config.json:

```
{
  "mcpServers": {
    "snapper": {
      "serverUrl": "https://snapper.example.com/api/mcp/",
      "auth": { "type": "bearer", "token": "<access-token>" }
    }
  }
}
```

curl (diagnostics)

Quick reachability check:

```
# Feature flag
curl http://localhost:8000/api/settings/features
```

```
# Hello-world MCP initialise
curl -X POST http://localhost:8000/api/mcp/ \
  -H "Authorization: Bearer <access-token>" \
  -H "Content-Type: application/json" \
  -H "Accept: application/json, text/event-stream" \
  -d '{"jsonrpc":"2.0","id":1,"method":"initialize","params":
{"protocolVersion":"2024-11-05","capabilities":{},"clientInfo":
{"name":"curl","version":"8"}}}'
```

Available tools

Read-only tools surface the delegate's order book, position state, signals, and venue market data. Write and decision tools (submit_manual_order, cancel_order, submit_ai_review_decision) are gated by per-tool permissions. Only submit_manual_order is unconditionally guarded by TradingCapsEnforcer.guard; cancel_order requires the caps enforcer to be initialized, then the cancel service enters the guard only when emitting a venue cancel command for a plan with a child venue order. Plans without a child venue order use the bare cancellation transition and do not enter the guard. submit_ai_review_decision is not wired to caps_enforcer_getter at registration.

- **list_instruments(exchange: str)** — returns sorted native symbols for the exchange inventory. It is not wallet/operator scoped; read-only access is gated by READ_MARKET_DATA permission (AI_DELEGATE role satisfies).
- **list_orders(wallet_public_id?, status?, exchange?, instrument?, limit=50, offset=0)** — paged read of the delegate's order history within accessible wallets. Requires READ_ORDERS. Surfaces order_not_found for inaccessible wallets (anti-enumeration).
- **get_order_status(command_public_id: str)** — single-order lookup keyed by the trade-command public id returned from submit_manual_order. Requires READ_ORDERS; unknown or out-of-scope command ids return order_not_found (anti-enumeration).
- **list_positions(wallet_public_id?, exchange?, instrument?)** — active positions across the caller's accessible wallets. Requires READ_POSITIONS; wallet scope violations return position_not_found (anti-enumeration).
- **get_position_cycle(cycle_public_id: str)** — full open→close audit trail for a position cycle. Requires READ_POSITIONS; unknown or out-of-scope cycles return position_cycle_not_found (anti-enumeration).
- **get_ohlcv(exchange, instrument, timeframe, since?, until?, limit=200)** — OHLCV candles for a venue + instrument. Range mode (both since + until ISO 8601 UTC) returns chronological candles in the closed window; latest-as-of mode (both

omitted) returns the most recent limit candles in DESC order. Allowed timeframes: 1m, 5m, 15m, 1h, 4h, 1d. Requires READ_MARKET_DATA; market data is public so no wallet scope applies.

- **list_recent_signals(since, instrument?, strategy?, exchange?, wallet_public_id?, limit=50)** — strategy signals fired after a watermark, ordered by fired_at DESC. since is REQUIRED ISO 8601 UTC. Requires READ_SIGNALS; surfaces signal_not_found on wallet scope violation (anti-enumeration).
- **submit_manual_order(exchange, instrument, instrument_public_id, side, order_type, quantity, wallet_public_id, idempotency_key, price?, stop_price?, operator_public_id?, ai_review_public_id?)** — enqueues a trade command under the delegate's user_public_id with source_surface='mcp' + the caps check from TradingCapsEnforcer.guard. Requires CREATE_ORDERS, rejects non-admin operator_public_id values outside the caller's authenticated operator set, rechecks wallet scope against active grants on every call, and fails closed on any cap violation. Validates order params with the same evaluator rule as REST: price is required for limit/stop_limit and stop_price is required for stop/stop_limit order types. Wraps the REST create_order route.
- **cancel_order(plan_public_id, idempotency_key)** — cancels an active execution plan via the same PlansCancelService REST goes through. Idempotent: same idempotency_key on retry returns the current plan state without re-executing. Requires CANCEL_ORDERS; unknown and out-of-scope plans collapse to order_not_found (anti-enumeration).
- **submit_ai_review_decision(review_id, decision, rationale?)** — REST mirror of the POST /api/ai-reviews/{review_public_id}/decision route for the in-process MCP surface; lets the delegate approve or reject a pending CONSULT review. review_id is the UUID7 of the ai_reviews row. Requires CREATE_ORDERS; the review service additionally verifies the caller is a registered AI delegate that still holds an active scope grant for the review wallet and instrument. Any such delegate may decide — the selected delegate is who was consulted, not an exclusive decision authority — and a delegate racing an already-resolved review receives review_already_resolved_by_peer. Not gated by the caps enforcer (the underlying review-decision path does its own SCD2 close-and-insert + bus fanout).

The tool catalog is discoverable via the MCP tools/list JSON-RPC method:

```
curl -X POST http://localhost:8000/api/mcp/ \
  -H "Authorization: Bearer <access-token>" \
  -H "Content-Type: application/json" \
  -H "Accept: application/json, text/event-stream" \
  -d '{"jsonrpc":"2.0","id":2,"method":"tools/list","params":{}}'
```

Safety caps

Every delegate has its own user_trading_caps row. All fields are optional; null means "unbounded on this axis".

- max_order_quantity_per_instrument — JSON dict {instrument: max_qty} OR a scalar applied to every instrument.
- max_open_orders — all-time count of the delegate's in-flight commands (every non-terminal status: created/dispatched/ direct_dispatched/accepted/partially_filled).
- max_daily_notional_usd — rolling 24h sum of submit_quantity * submit_price_usd across non-rejected commands (submit-time commitment basis; partial fills don't change accounting).
- max_cancels_per_minute — sliding-window cancel rate.

Caps are enforced at trade-command insert sites via the TradingCapsEnforcer.guard() surface — submit_manual_order routes through it unconditionally; cancel_order requires the caps enforcer to be initialized and only enters the guard when the cancel service emits a venue cancel command for a plan with a child venue order (the bare cancellation transition skips the guard). The other MCP tools (submit_ai_review_decision, etc.) are not currently wired to caps_enforcer_getter at registration.

Update caps via PATCH /api/ai-delegates/{id} — the write is SCD2 close+insert, so cap history is auditable.

Kill switch + deactivation

POST /api/ai-delegates/{id}/deactivate runs the same code path as operator deactivation:

1. SCD2 close+insert on the users row flipping is_active=False.
2. TokenManager.revoke_user_sessions loads every user_active_tokens.jti for the delegate, flips revoked_at=NOW() in one SQL UPDATE, and seeds the in-memory JTI blacklist with a 10-second grace period.
3. UserService.deactivate_user publishes admin.user_deactivated on the bus as the immediate fanout path (sole publisher).
4. Every Snapper instance's WebSocketAuthManager subscribes to the topic and closes matching WebSocket connections with code 4003.
5. Every Snapper instance's TokenManager subscribes to the topic and evicts matching verify-cache entries via invalidate_user_cache.
6. Every lifespan-wired WebSocketAuthManager and TokenManager also runs a DB-backed fallback scan against the SCD2-active users.is_active row, so broker outages delay fanout by the 5-second scan interval (DEACTIVATION_FALLBACK_SCAN_INTERVAL_S) rather than by token expiry or reconnect.

Latency: the token inventory is revoked before the deactivated-user bus event is published. Existing positive verify-cache entries are evicted when each instance receives admin.user_deactivated; if the bus listener is unavailable, the fallback scanner evicts cache entries and closes matching WebSockets by reading the committed users.is_active=False row. The local JTI blacklist is consulted before the LRU but has a 10-second grace window, so requests that race deactivation can still complete.

In-flight MCP tool handlers are **not** force-cancelled. The guarantee is narrowly "later MCP requests are rejected once the inventory/cache revocation path has observed the kill switch."

Rate limits

REST rate limits apply globally via RestCallTracker — observed via GET /api/metrics/rest-rate. MCP-specific rate limits are enforced by a separate per-principal middleware on /api/mcp (default 60/minute); the per-delegate max_cancels_per_minute cap then applies on top inside cancel_order. Bursts are served best-effort; sustained abuse over published exchange limits (Walutomat 20 req/s, Kraken 15 req/s, Polygon 5/min) surfaces as rate_limited warnings at 80/95% utilisation.

Error catalog

Transport and middleware failures on /api/mcp use standard HTTP status codes plus a Snapper-specific error_code in the JSON body. Clients should branch on error_code, not on status text.

Envelope-based MCP tools return a normal MCP CallToolResult instead: the single TextContent.text entry contains JSON with success, error_code, message, and details, and CallToolResult.isError mirrors not success. A few older tools still return raw dictionaries on success or surface FastMCP ToolError/permission exceptions on failure; client code should handle both shapes until those tools are migrated.

Status	error_code	When it fires	Client action
503	feature_disabled	Flag explicitly set to false	Show "AI Integration disabled" banner
429	rate_limit_exceeded	Per-principal MCP middleware quota exhausted	Back off and retry after Retry-After seconds

Status	error_code	When it fires	Client action
503	mcp_unavailable	Repository dep unavailable (lifespan not ready)	Retry with backoff
401	missing_bearer_token	No Authorization: Bearer … header	Prompt user to authenticate
401	invalid_bearer_token	JWT signature/expiry/blacklist/inventory failure	AI delegates have no refresh token (90-day PAT); deactivate + recreate the delegate in Snapper, then update the client's bearer token. Operator (cookie) sessions can fall back to POST /api/auth/refresh.
401	user_deactivated	Owner account deactivated	Prompt re-login; don't auto-refresh
401	Refresh token redeemed	Replay of a spent refresh JWT	Re-login
401	Account deactivated	Session cookie flow	Re-login
403	MCP write helpers: wallet_out_of_scope: *(prefixed message)* / REST: "Wallet not in accessible set" *(detail string)*	Mutating tool targets a wallet outside the caller's scope. The helper path raises PermissionError(f"wallet_out_of_scope: ...") lifted by FastMCP into a ToolError; REST raises HTTPException(403, detail="Wallet not in accessible set") from server/scoping.py. Tool-level read/cancel paths may instead return structured anti-enumeration envelopes such as order_not_found, position_not_found, or signal_not_found.	Pick a wallet the caller still has a live grant on
403			

Status	error_code	When it fires	Client action
	MCP write helpers: operator_out_of_scope: *(prefixed message)* / REST: "Operator not in accessible set" *(detail string)*	Same write-helper shape as the wallet variant. Non-admin callers must pick an operator from their authenticated set; ADMIN bypasses the operator-set check. Read/cancel tools can intentionally collapse out-of-scope and not-found cases into structured not-found envelopes to avoid leaking resource existence.	Pick an operator from the caller's authenticated set unless the caller is ADMIN

Delegate CRUD:

Status	Detail	When it fires
401	Requires populated user_public_id	Principal has blank user_public_id (older token rollout)
403	require_role(OPERATOR)	AI_DELEGATE or VIEWER trying to manage delegates
404	Delegate not found	Unknown ID OR cross-tenant (no existence leak)
409	Could not derive a unique username …	Label slug collides 8+ times (pathological)
422	Operator '<id>' is not in …	Non-admin caller picked operator_public_id outside their claim set
422	Caller has no primary operator …	No explicit operator and no primary → binding is ambiguous

---

Security model

Token lifetime

- **Operator** access tokens live **15 minutes** (configurable via auth_access_token_expire_minutes). **AI delegate** access tokens are PAT-style and live for LONG_LIVED_TOKEN_EXPIRE_DAYS (~3 months / 90 days); operators rotate them on the cadence that fits their key-management hygiene, and revocation is immediate via deactivating the delegate rather than waiting for expiry.
- Refresh tokens on the login/refresh route path currently live auth_refresh_token_expire_days (default **7 days**). The remember_me request field is accepted by the schema but is not currently threaded into token creation.

- Refresh rotation is **atomic**: one DB transaction revokes the old refresh JTI AND persists the new access+refresh pair. Replay of a spent refresh JWT fails with 401 Refresh token already redeemed — the transaction rolls back cleanly.

Inventory-based revocation

user_active_tokens is the ground truth for "is this JWT still valid?". Every mint persists a row; every verify consults it via a 30-second LRU. Deactivation flips revoked_at=NOW() in one UPDATE and fans out a bus event so every cross-instance LRU evicts the matching entry on receipt.

No secrets leaked at rest

Delegate password hashes are bcrypt-derived from 32 bytes of cryptographically random data the operator never sees. An attacker with DB read rights cannot brute-force the hash.

Single-publisher invariant

UserService.deactivate_user is the SOLE publisher of admin.user_deactivated across the entire process. TokenManager and WebSocketAuthManager are pure subscribers/fallback readers: they evict + close on receipt or on DB scan, but never emit the event themselves. This holds for operator deactivation AND delegate deactivation (the route reuses UserService.deactivate_user).

MCP transport = Streamable HTTP

The MCP endpoint uses Model Context Protocol's Streamable HTTP transport. No cookies, no CSRF on MCP calls (bearer header bypass). Every tool call opens a sub-request inside the same bearer-authenticated HTTP connection.

Related reading

- docs/architecture.md — repository + SCD2 + bus layering.
- docs/operations.md — multi-instance trade coordinator and static-hash shard partitioning.
- docs/api.md — full REST surface.

API

Snapper provides a REST API and WebSocket interface for platform interaction. The API accepts JWT authentication either through HTTP-only cookies or an Authorization: Bearer <jwt> header. When both are present, Bearer auth wins. Cookie-authenticated mutating requests that use the CSRF guard require a valid X-CSRF-Token header; Bearer-authenticated requests skip CSRF because they are not ambient browser credentials. The auth token lifecycle routes (/api/auth/login, /api/auth/refresh, /api/auth/logout, /api/auth/ws_token) intentionally omit the CSRF guard.

Authentication

Browser sessions usually use HTTP-only cookies. After login, the server sets access_token, refresh_token, and csrf_token cookies automatically. API clients and MCP/AI delegates may send the access JWT as a Bearer token instead.

Roles and Permissions

Role	Access
viewer	Read-only market data, orders, positions, strategies, system status, backtests, notifications; can register and manage the caller's own notification devices
ai_delegate	Scoped automation principal for MCP and AI-review workflows; can read market/orders/positions/strategies/signals/backtests/system status and create/cancel/manage scoped orders/positions, but cannot manage users, settings, processes, credentials, scope grants, notifications, or paired execution
operator	Viewer permissions plus trade execution, process and strategy lifecycle management, backtest management, and paired-execution terminalization
admin	Full access including user management, system configuration, wallet/credential management, scope grant management, operator impersonation

Multi-tenant permissions (ADMIN only): read:wallet_credentials, manage:wallet_credentials, manage:scope_grants, impersonate:operator.

POST /api/auth/login

Authenticate and create a session. Sets access_token, refresh_token, and csrf_token cookies on the response.

Request: the body is a LoginRequest envelope (PayloadRequest) wrapping a LoginBody payload — the client stamps its own provenance on the outer envelope.

```
POST /api/auth/login
Content-Type: application/json

{
    "type": "login_request",
    "sequence_id": 1,
    "public_id": "<client-uuid7>",
    "timestamp": "2026-01-18T12:00:00Z",
    "session_id": "<client-session>",
    "topic": null,
    "payload": {
        "username": "admin",
        "password": "password123",
        "remember_me": false
    }
}
```

Response (200):


```
{
    "type": "login_response",
    "sequence_id": 1,
    "public_id": "<uuid7>",
    "timestamp": "2026-01-18T12:00:00Z",
    "session_id": "<server-session>",
    "topic": null,
    "payload": {
        "type": "login",
        "sequence_id": 1,
        "public_id": "<uuid7>",
        "timestamp": "2026-01-18T12:00:00Z",
        "session_id": "<server-session>",
        "topic": null,
        "message": "Login successful",
        "expires_in": 900,
        "user": {
            "type": "user_profile",
            "sequence_id": 1,
            "public_id": "<uuid7>",
            "timestamp": "2026-01-18T12:00:00Z",
            "session_id": "<server-session>",
            "topic": null,
            "username": "admin",
            "email": "admin@example.com",
            "role": "admin",
            "is_active": true,
            "created_at": "2026-01-10T08:00:00Z",
            "operator_public_ids": [],
            "primary_operator_public_id": null,
            "active_wallet_public_id": null,
            "default_language": null
        },
        "access_token": null,
```

```
        "refresh_token": null
    }
}
```

The login handler returns the raw UserProfile from authenticate_user() and overlays principal.active_wallet_public_id onto the returned user via model_copy. On the initial login path the freshly-built principal has no wallet selected yet, so active_wallet_public_id lands as null; the operator-membership fields are not enriched on this surface either (operator_public_ids = [], primary_operator_public_id = null). Call GET /api/auth/me after login to get a fully-enriched profile with operator memberships resolved.

The outer envelope (LoginResponse) and the inner payload (LoginData) both carry the standard provenance fields (type, sequence_id, public_id, timestamp, session_id, topic). access_token and refresh_token are null in the cookie flow; they are populated only when the caller passes ?return_tokens=true for headless integrations.

Cookies set:

Cookie	HttpOnly	Secure	SameSite	Path	Max-Age	Description
access_token	Yes	Yes (prod)	Lax/Strict	/	session	JWT access token (15 min)
refresh_token	Yes	Yes (prod)	Lax/Strict	/api/auth	7 days	JWT refresh token
csrf_token	No	Yes (prod)	Lax/Strict	/	session	CSRF protection token

POST /api/auth/refresh

Refresh session tokens. The refresh_token cookie is sent automatically by the browser. Returns new tokens in cookies plus a WebSocket authentication token in the response body. Long-running clients that already hold a valid access token and only need a fresh ws_token should call POST /api/auth/ws_token instead — that route does not rotate the refresh-token pair.

Request:

```
POST /api/auth/refresh
```

Response (200):

```
{
    "type": "refresh_response",
    "sequence_id": 2,
    "public_id": "<uuid7>",
    "timestamp": "2026-01-18T12:00:00Z",
```

```
    "session_id": "<server-session>",
    "topic": null,
    "payload": {
        "type": "refresh",
        "sequence_id": 2,
        "public_id": "<uuid7>",
        "timestamp": "2026-01-18T12:00:00Z",
        "session_id": "<server-session>",
        "topic": null,
        "message": "session refreshed",
        "ws_token": "eyJhbGciOi...",
        "ws_token_exp": "2026-01-18T12:30:00Z",
        "csrf_token": "abc123def456...",
        "user": {
            "type": "user_profile",
            "sequence_id": 2,
            "public_id": "<uuid7>",
            "timestamp": "2026-01-18T12:00:00Z",
            "session_id": "<server-session>",
            "topic": null,
            "username": "admin",
            "email": "admin@example.com",
            "role": "admin",
            "is_active": true,
            "created_at": "2026-01-10T08:00:00Z",
            "operator_public_ids": [],
            "primary_operator_public_id": null,
            "active_wallet_public_id": null,
            "default_language": null
        },
        "access_token": null,
        "refresh_token": null
    }
}
```


The refresh handler returns the UserProfile from get_user_by_id() and overlays the principal's active_wallet_public_id onto it — that value is seeded from token_data.active_wallet_public_id (and may be swapped via an optional RefreshTokenPayload wallet hint), so the refresh response generally carries the active wallet (it is only null when the refresh JWT itself carries no wallet claim and no hint is supplied). Operator memberships are still NOT resolved here (operator_public_ids = [], primary_operator_public_id = null); use GET /api/auth/me for the enriched view.

access_token and refresh_token are populated only when ?return_tokens=true is set (headless integrations); the cookie flow leaves them null and writes the rotated JWTs as Set-Cookie headers. The RefreshTokenRequest body (optional) carries active_wallet_public_id / clear_active_wallet to atomically swap the caller's active wallet during the rotation; an Authorization: Bearer <refresh-jwt> header is read first, with the refresh_token cookie used as fallback for browser callers.

POST /api/auth/ws_token

Mint a one-shot WebSocket authentication token without rotating the caller's refresh-token pair. Authenticates via the access bearer (header or cookie) and returns a fresh ws_token bound to the access JWT's session. Per-source-IP rate-limited so a reconnect storm cannot exhaust the token store.

Use this route when a long-running client (e.g. a push-wakeup monitor) needs to mint successive ws_tokens on its own cadence without rotating the refresh-token pair shared with sibling processes.

Request:

```
POST /api/auth/ws_token
Authorization: Bearer <access_token>
```

Response (200):

```
{
    "type": "ws_token_response",
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
    "session_id": "<server-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:15:00Z",
    "topic": null,
    "payload": {
        "type": "ws_token",
        "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5c",
        "session_id": "<server-session>",
        "sequence_id": 1,
        "timestamp": "2026-01-18T12:15:00Z",
        "topic": null,
        "message": "ws_token issued",
        "ws_token": "eyJhbGciOi...",
        "ws_token_exp": "2026-01-18T12:30:00Z",
        "expires_in": 900
    }
}
```

expires_in mirrors ws_token_exp as relative seconds for clients that prefer relative-deadline math.

Errors:

- 401 Unauthorized — access bearer is absent, expired, or invalid.
- 429 Too Many Requests — per-source-IP minute budget exhausted.

POST /api/auth/logout

Logout and invalidate the current session. Clears all authentication cookies and blacklists tokens.

Request:

```
POST /api/auth/logout
```

Response (200):

```
{
    "type": "message",
    "public_id": "<uuid7>",
    "session_id": "<server-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "topic": null,
    "payload": "Logged out successfully"
}
```

GET /api/auth/me

Get the currently authenticated user's profile.

Request:

```
GET /api/auth/me
```

Response (200):

Handler returns a UserResponse envelope (PayloadResponse[user_response, UserProfile]) wrapping the full UserProfile payload, which carries the multi-tenant trio:

```
{
    "type": "user_response",
    "sequence_id": 3,
    "public_id": "<uuid7>",
    "timestamp": "2026-01-18T12:00:00Z",
    "session_id": "<server-session>",
    "topic": null,
    "payload": {
        "type": "user_profile",
        "sequence_id": 3,
        "public_id": "<uuid7>",
        "timestamp": "2026-01-18T12:00:00Z",
        "session_id": "<server-session>",
        "topic": null,
        "username": "admin",
        "email": "admin@example.com",
        "role": "admin",
        "is_active": true,
        "created_at": "2026-01-10T08:00:00Z",
        "operator_public_ids": ["019d6ca4-..."],
        "primary_operator_public_id": "019d6ca4-...",
        "active_wallet_public_id": "019d7e9a-...",
```

```
        "default_language": "pl"
    }
}
```

Only /api/auth/me enriches the full UserProfile. The operator_public_ids / primary_operator_public_id fields are populated from user_operator_memberships (ADMIN receives every active operator; OPERATOR/VIEWER receive only their explicit memberships); active_wallet_public_id reflects the active wallet selection. These fields power the frontend OperatorPicker without a second round trip. The login / refresh responses return the UserProfile from authenticate_user() / get_user_by_id() with the active wallet overlaid from the request principal — operator memberships are NOT resolved on those two surfaces (operator_public_ids = [], primary_operator_public_id = null). Active-wallet semantics differ between the two: login lands active_wallet_public_id = null (no wallet selected at first auth); refresh propagates whatever wallet the refresh JWT carries (or the request hint). Clients should call /api/auth/me to get a fully-resolved profile.

POST /api/auth/me/update

Update the authenticated caller's self-service preferences. Currently exposes default_language only; additional preference fields may be added later without changing the endpoint contract. Mirrors the admin POST /api/auth/users/{user_id}/update shape (codebase convention: POST + verb, no REST PATCH).

Request:

```
POST /api/auth/me/update
Content-Type: application/json
X-CSRF-Token: <token>
```

```
{
    "type": "update_auth_me_request",
    "payload": {
        "default_language": "pl"
    }
}
```

default_language is validated against the union of supported client codes (iOS-canonical + frontend-canonical forms accepted — e.g. "zh-Hans", "zh", "pt-BR", "pt", "nb", "no"). null clears the preference.

Response (200):

Returns the same UserResponse envelope shape as GET /api/auth/me, with the updated default_language echoed in payload.

Errors:

- 404 — caller's user row was not found (rare; only surfaces if an admin concurrently deactivates the user between auth-dep resolution and the update commit).
- 422 — default_language not in the supported-language allowlist.

GET /api/auth/users

List users. Requires manage:users.

Query parameters:

Parameter	Type	Description
include_inactive	bool	Include deactivated users (default false)
as_of	datetime	Optional point-in-time query timestamp

Returns UserListResponse with payload and count.

POST /api/auth/users

Create a user. Requires manage:users and CSRF for cookie auth. Body is CreateUserRequest wrapping username, password, optional email, role, and is_active.

POST /api/auth/users/{user_id}/update

Update email, role, and active state for an existing user. Requires manage:users and CSRF for cookie auth. Returns UserResponse.

POST /api/auth/users/{user_id}/deactivate

Deactivate a user through the canonical kill-switch flow. Requires manage:users and CSRF for cookie auth. The path segment is resolved as the target username; self-deactivation returns 400, unknown active users return 404. The service also revokes active sessions and emits admin.user_deactivated for immediate verify-cache eviction and WebSocket close fanout; auth listeners also poll the committed users.is_active=False row as broker-outage fallback.

POST /api/auth/users/{user_id}/change-password

Change a password. Users may change their own password; admins may change any password. Requires CSRF for cookie auth and is account-rate limited. Body is ChangePasswordRequest with current and new password.

POST /api/auth/users/{user_id}/admin-reset-password

Admin password reset without the current password. Requires manage:users, CSRF for cookie auth, and is account-rate limited. Body is AdminResetPasswordRequest.

CSRF Protection

The csrf_token cookie is readable by JavaScript (not HttpOnly). For cookie-authenticated mutating requests outside /api/auth/login, /api/auth/refresh, /api/auth/logout, and /api/auth/ws_token, include its value as a header:

```
X-CSRF-Token: <value from csrf_token cookie>
```

New CSRF tokens are issued on login and refresh. There is no separate endpoint for obtaining CSRF tokens.

Bearer-authenticated requests do not require X-CSRF-Token. This is intentional: the request is authorized by the explicit Authorization: Bearer ... header rather than an ambient browser cookie.

REST Endpoints

REST endpoints return the same Data schemas used by WebSocket messages (OrderData, SignalData, ExecutionData, PositionData, CandleData from messaging.schemas.data). This means the wire format is identical whether data arrives via REST or the WebSocket feed.

REST envelopes — PayloadResponse / PayloadListResponse

REST responses are JSON objects with a typed envelope plus a payload. Two shapes ship from src/snapper/api/schemas/base.py:

- PayloadResponse[T, P] — singleton response, fields: type (Literal discriminator), envelope provenance (public_id, session_id, sequence_id, timestamp, topic) and payload: P.
- PayloadListResponse[T, P] — list response, same envelope plus payload: list[P] and count: int (always equal to len(payload)).

Per-item provenance is preserved on items that originate from a bitemporal DB projection (the item carries its own public_id/session_id/sequence_id); minted results share the envelope's provenance. Each item's data-type shape is identical to what the WebSocket feed delivers for the same domain object — only the wrapping differs.

OpenAPI request-body schemas for routes that use the json_body dependency are patched into FastAPI's generated OpenAPI from openapi_extra metadata. Optional request bodies, such as refresh and delegate deactivate, stay optional in the generated schema. The mounted /api/mcp Streamable HTTP sub-app is intentionally outside FastAPI's OpenAPI route list; its tool contract is documented in ai-integration.md.

Provenance on Reads vs. Mutations

- **GET reads** — No client provenance is expected. When telemetry recording is enabled (disabled by default), the server records the request in the telemetry table for observability; otherwise no provenance is recorded for reads. Either way, it does not stamp provenance onto the response items beyond what was stored at write time.
- **Mutations (POST)** — All mutations use command-style POST with a PayloadRequest envelope carrying provenance (public_id, session_id, sequence_id, timestamp) and domain intent in payload. The server-side ClientProvenanceMiddleware extracts these fields, emits a structured info log, and runs a per-session GapDetector to warn on sequence gaps. Gaps are logged as warnings but never reject requests.

ClientProvenanceMiddleware

ClientProvenanceMiddleware is an ASGI middleware that processes every mutation request:

1. Intercepts the request body transparently (replays chunks to downstream handlers).
2. Extracts session_id, sequence_id, and public_id from the JSON body when present.
3. Emits a structured log with client_session_id, client_sequence_id, client_public_id, and the request path.
4. Runs a per-session GapDetector to detect sequence gaps from the client.
5. Records the mutation in the control table via a finally block, so the row is persisted whether the request succeeded or failed.

The control recording is non-blocking: any DB failure is logged and swallowed so the response already sent to the client is never invalidated. The control row includes the redacted request payload, HTTP method, path, outcome (ok, error, or exception), and server-side provenance (session_id and sequence_id from the middleware's own SequenceTracker).

All data endpoints support bitemporal querying via the optional as_of parameter (UTC datetime). When provided, the query returns data as it was known at that point in time. When omitted, the current time is used.

GET /api/health

Public health check endpoint. No authentication required.

status reflects the health of enabled long-running CORE processes: "healthy" when all are running, "error" if any are missing. In SERVER_API_ONLY mode the status is always "healthy" (no processes are started by design).

Response (200):


```
{
    "type": "health_check_response",
    "sequence_id": 1,
    "public_id": "<uuid7>",
    "timestamp": "2026-01-18T12:00:00Z",
    "session_id": "<server-session>",
    "topic": null,
    "payload": {
        "type": "health_check",
        "sequence_id": 1,
        "public_id": "<uuid7>",
        "timestamp": "2026-01-18T12:00:00Z",
        "session_id": "<server-session>",
        "topic": null,
        "status": "healthy",
        "version": "0.1.0",
        "connections": {
            "active_connections": 5,
            "zmq_subscribers": 12,
            "subscriber_tasks": 12,
            "active_topics": 8,
            "active_clients": 3
        },
        "topics": {
            "active": 3
        },
        "gap_detection": {
            "bridge": {
                "gaps_detected": 0,
                "session_resets": 0,
                "duplicates": 0,
                "mid_stream_joins": 0,
                "rejected_unstamped": 0
            },
            "rest_clients": {}
        }
    }
}
```


The outer health_check_response envelope carries server-side provenance; the inner health_check payload holds the runtime snapshot. gap_detection provides observability into sequence gap detection across the ZMQ bridge and per-session REST client detectors.

GET /api/health/egress

Operator egress route snapshot. Requires read:system_status permission and uses the same CSRF guard as the detailed monitoring health routes.

The endpoint returns the API process's local egress snapshot merged with the latest system.egress.snapshot frames from other pool-bearing processes such as snapper-feed. Missing remote snapshots are not an error: the payload still includes the API process. Stale remote snapshots remain visible and are flagged in containers.

Response (200):


```
{
    "type": "egress_health_response",
    "sequence_id": 1,
    "public_id": "<uuid7>",
    "timestamp": "2026-01-18T12:00:00Z",
    "session_id": "<server-session>",
    "topic": null,
    "payload": {
        "type": "egress_health",
        "sequence_id": 1,
        "public_id": "<uuid7>",
        "timestamp": "2026-01-18T12:00:00Z",
        "session_id": "<server-session>",
        "topic": null,
        "enabled": true,
        "on_all_quarantined": "wait",
        "private_fallback_route_id": "pl",
        "private_on_fallback": false,
        "containers": [
            {
                "container": "api:coord-0@snapper",
                "last_seen_age_seconds": 0.0,
                "stale": false,
                "route_count": 1
            },
            {
                "container": "publisher:kraken@snapper-feed",
                "last_seen_age_seconds": 1.4,
                "stale": false,
                "route_count": 1
            }
        ],
        "routes": [
            {
                "id": "default",
                "kind": "direct",
                "proxy_url": null,
                "region": "host",
                "exit_ip": "198.51.100.11",
                "provider": "isp",
                "priority": 100,
                "allowed_exchanges": [],
                "enabled": true,
                "quarantined": false,
                "quarantine_seconds_remaining": null,
```

```
                "in_use_count": 2,
                "active_reservations": [
                    {
                        "exchange": "kraken",
                        "traffic_class": "private",
                        "container": "api:coord-0@snapper"
                    },
                    {
                        "exchange": "kraken",
                        "traffic_class": "public",
                        "container": "publisher:kraken@snapper-feed"
                    }
                ],
                "connections": [
                    {
                        "host": "ws-auth.kraken.com",
                        "kind": "ws",
                        "exchange": "kraken",
                        "traffic_class": "private",
                        "container": "api:coord-0@snapper",
                        "count": 1,
                        "last_seen_at": null
                    },
                    {
                        "host": "api.kraken.com",
                        "kind": "rest",
                        "exchange": "kraken",
                        "traffic_class": "public",
                        "container": "publisher:kraken@snapper-feed",
                        "count": 0,
                        "last_seen_at": "2026-01-18T12:00:01Z"
                    }
                ],
                "transfer": null
            }
        ]
    }
}
```


region, exit_ip, and provider are optional operator metadata copied from the egress_pool.routes[] entry. proxy_url is present so the API can join sidecar transfer samples to SOCKS5 routes by listener port. active_reservations lists the unique (container, exchange, traffic_class) tuples currently reserved on the route. connections lists target hostnames by (container, host, kind, exchange, traffic_class): WebSocket rows carry currently open counts, while REST rows retain the capped last-seen host with count zero after the short reservation releases. Only hostnames are reported; URL paths, queries, headers, bodies, and credentials are never included.

SOCKS5 routes may include transfer when the API has a matching system.egress.transfer sample from snapper-egress. The nested object contains interface, socks5_listen_port, cumulative rx_bytes and tx_bytes, current rx_rate_bytes_per_second and tx_rate_bytes_per_second when a reliable delta exists, latest_handshake_at, counter_reset, sampled_at, sample_age_seconds, and stale. Direct routes, missing samples, and ambiguous port joins return transfer: null.

containers lists each reporting process, the age of its latest snapshot as observed by the API process, whether that snapshot is stale, and how many routes it reported. private_on_fallback is true when any private reservation is active on a non-direct route.

GET /api/candles

Fetch OHLCV candlestick data. Requires read:market_data permission.

Request:

```
GET /api/candles?instrument=BTC-USD&exchange=kraken&timeframe=1h&limit=100
```

Parameters:

Parameter	Type	Required	Description
instrument	string	yes	Instrument symbol (e.g., BTC-USD)
exchange	string	yes	Exchange name (kraken, kraken_futures, kraken_equities, walutomat, polygon)
timeframe	string	yes	Candle timeframe (e.g., 1m, 5m, 15m, 30m, 1h, 4h, 1d)
limit	int	no	Number of candles, max 1000 (default 100)
as_of	datetime	no	Point-in-time query, UTC (default: current time)
start	datetime	no	Market-time window start (open_at, UTC); pair with end
end	datetime	no	Market-time window end (open_at, UTC); pair with start

Returns 200 OK with an empty payload array when no candles match the query (unknown instrument, no warm-cache rows, etc.) — the CandleListResponse envelope is the canonical shape for empty results too.

When both start and end are supplied the route reads a market-time range by open_at (ascending, capped at limit), bypassing the cache and the as_of write-time routing. This navigates the full persisted history — including bulk-backfilled corpora whose database write time is unrelated to their market time — and powers the market view's time-travel scrubber. Supplying only one of the pair, or start >= end, returns 400.

Response (200):


```
{
    "type": "candle_list",
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
    "session_id": "<server-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "topic": null,
    "payload": [
```

```
        {
            "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
            "type": "candle",
            "timestamp": "2026-01-18T12:00:00Z",
            "instrument": "BTC-USD",
            "exchange": "kraken",
            "timeframe": "1h",
            "open_at": "2026-01-18T11:00:00Z",
            "open": 42000.0,
            "high": 42500.0,
            "low": 41800.0,
            "close": 42300.0,
            "volume": 1234.56,
            "vwap": 42150.0,
            "trades": 5678,
            "complete": true
        }
    ],
    "count": 1
}
```


The complete flag marks a provisional intra-minute update of a still-forming bar when false, and the final bar for its window when true (default true).

GET /api/candles/db

Explicit DB-only candle read for operators. Parameters and response shape match GET /api/candles, but the route bypasses the in-process market cache unconditionally so incident response can verify persisted candles rows directly.

GET /api/candles/cache

Explicit cache-only candle diagnostic read. Parameters match GET /api/candles except as_of is not accepted. For cache-eligible timeframes (1m, 5m, 15m, 30m) the route returns cache-shaped payload data with diagnostic fields such as is_warm, source, and sample_count; cold cache returns an empty payload with is_warm=false instead of silently falling back to DB. Long frames (1h, 4h, 1d) fall through to persisted rows because the cache does not serve them.

GET /api/orders

Fetch orders with optional filtering. Requires read:orders permission.

Request:

```
GET /api/orders?symbol=BTC-USD&exchange=kraken&limit=100&offset=0
```

Parameters:

Parameter	Type	Required	Description
symbol	string	no	Filter by instrument symbol
exchange	string	no	Filter by exchange (paper, kraken, kraken_futures, walutomat)
limit	int	no	Number of orders, 1-1000 (default 100)
offset	int	no	Number of orders to skip (default 0)
as_of	datetime	no	Point-in-time query, UTC (default: current time)
operator_public_id	string	no	Scope to a single operator (403 if foreign)
wallet_public_id	string	no	Scope to a single wallet (403 if inaccessible)

Response (200):


```
{
    "type": "order_list",
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
    "session_id": "<server-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "topic": null,
    "payload": [
        {
            "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
            "type": "order",
            "timestamp": "2026-01-18T12:00:00Z",
            "instrument": "BTC-USD",
            "exchange": "kraken",
            "client_order_id": "signal-a1b2c3d4",
            "exchange_order_id": "KRAKEN-456",
            "created_at": "2026-01-18T12:00:00Z",
            "updated_at": "2026-01-18T12:01:00Z",
            "side": "buy",
            "order_type": "limit",
            "price": 42000.0,
            "size": 0.1,
            "filled_size": 0.1,
            "average_price": 42000.0,
            "status": "filled",
            "time_in_force": "GTC",
            "mode": "live",
            "error": null
        }
    ],
    "count": 1
}
```

POST /api/orders

Create a manual order via a manual_once execution plan. Requires create:orders permission.

Request:

```
POST /api/orders
Content-Type: application/json
X-CSRF-Token: <csrf_token>

{
    "type": "create_order_command",
    "session_id": "ui",
    "sequence_id": 0,
    "public_id": "<uuid7>",
    "timestamp": "2026-04-10T12:00:00Z",
    "payload": {
        "instrument": "BTC-USD",
        "instrument_public_id": "<uuid>",
        "exchange": "kraken",
        "mode": "live",
        "side": "buy",
        "order_type": "limit",
        "quantity": 0.5,
        "price": 50000.0,
        "time_in_force": "GTC",
        "post_only": false,
        "leverage": null,
        "reduce_only": false,
        "wallet_public_id": "<uuid>",
        "idempotency_key": "<uuid7>",
        "ai_review_public_id": null
    }
}
```

Payload fields:

Field	Type	Required	Description
instrument	string	yes	Native symbol (e.g., BTC-USD)
instrument_public_id	string	yes	Instrument UUID
exchange	string	yes	Exchange name
mode	string	no	live (default) or paper
side	string	yes	buy or sell
order_type	string	yes	market, limit, stop, stop_limit
quantity	float	yes	Order size (must be > 0)
price	float	cond	Required for limit and stop_limit

Field	Type	Required	Description
stop_price	float	cond	Required for stop and stop_limit
time_in_force	string	no	Time-in-force policy (GTC default)
post_only	boolean	no	Maker-only order flag (false default)
leverage	int	no	Optional leverage multiplier
reduce_only	boolean	no	Reduce-only flag for closing exposure (false default)
wallet_public_id	string	yes	Target wallet UUID
operator_public_id	string	no	Operator identity
idempotency_key	string	no	Dedup key (409 on duplicate)
ai_review_public_id	string	no	Approved ai_reviews row cited for AI-mediated manual orders

Response (200): ExecutionPlanResponse envelope with plan details.

Errors: 403 (disabled/forbidden), 409 (idempotency conflict), 422 (validation).

POST /api/orders/{plan_public_id}/cancel

Cancel an active execution plan by its plan public id. Requires cancel:orders permission and caller access to the plan's wallet.

The route transitions the plan to cancel_requested, hydrates the venue-assigned exchange_order_id from the active orders row if available, and inserts a cancel TradeCommand so the outbox dispatcher can publish OrderCancelData on the orders.commands.{ex}.{instr}.cancel topic. If the cancel TradeCommand insert fails the plan is transitioned to failed with last_error, and HTTP 500 is returned — PlanExecutorService will re-emit the cancel on the next startup (idempotent via has_pending_cancel_command).

Request:

```
POST /api/orders/<plan_public_id>/cancel
Content-Type: application/json
X-CSRF-Token: <csrf_token>

{
    "type": "cancel_order_command",
    "session_id": "ui",
    "sequence_id": 0,
    "public_id": "<uuid7>",
    "timestamp": "2026-04-10T12:01:00Z",
    "payload": {
```

```
        "reason": "changed mind"
    }
}
```

Response (200): ExecutionPlanResponse with status cancel_requested.

Errors: 403 (wallet not accessible), 404 (not found), 409 (already terminal or concurrent change), 422 (caps violation), 500 (cancel command insert failed).

POST /api/orders/by-client-order-id/{client_order_id}/cancel

UI convenience route that cancels by the child order's client_order_id (the value displayed on the Orders table). Resolves the owning plan via the trade_commands table and delegates to the shared cancel flow described above.

Requires cancel:orders permission. Same response shape and error codes as the plan-id route, plus 404 when no plan-linked trade_commands row is found for the given client_order_id.

Request:

```
POST /api/orders/by-client-order-id/<client_order_id>/cancel
Content-Type: application/json
X-CSRF-Token: <csrf_token>

{
    "type": "cancel_order_command",
    "session_id": "ui",
    "sequence_id": 0,
    "public_id": "<uuid7>",
    "timestamp": "2026-04-11T12:00:00Z",
    "payload": {
        "reason": "cancelled from Orders table"
    }
}
```

Response (200): ExecutionPlanResponse with status cancel_requested.

Errors: 403 (wallet not accessible), 404 (no linked plan), 409 (already terminal), 422 (caps violation), 500 (cancel command insert failed).

GET /api/instrument-capabilities

Fetch instrument order capability matrix. Requires read:market_data.

Parameters:

Parameter	Type	Required	Description
exchange	string	no	Filter by exchange

Parameter	Type	Required	Description
instrument_public_id	string	no	Filter by instrument
as_of	datetime	no	Point-in-time query

Response (200): InstrumentCapabilityListResponse with capability flags.

GET /api/venue-fee-schedules

Fetch venue fee schedules. Requires read:market_data.

Parameters:

Parameter	Type	Required	Description
exchange	string	no	Filter by exchange
as_of	datetime	no	Point-in-time query

Response (200): VenueFeeScheduleListResponse with fee tiers.

GET /api/signals

Fetch trading signals with optional filtering. Requires read:market_data permission.

Request:

```
GET /api/signals?instrument=BTC-
USD&strategy=rsi_btc_1h&exchange=paper&hours=24&limit=100
```

Parameters:

Parameter	Type	Required	Description
instrument	string	no	Filter by instrument symbol
strategy	string	no	Filter by strategy name
exchange	string	no	Filter by exchange (paper, kraken, kraken_futures, walutomat)
hours	int	no	Hours of history, max 168 (default 24)
limit	int	no	Number of signals, max 1000 (default 100)
as_of	datetime	no	Point-in-time query, UTC (default: current time)
operator_public_id	string	no	Scope to a single operator (403 if foreign)
wallet_public_id	string	no	Scope to a single wallet (403 if inaccessible)

Response (200):


```
{
    "type": "signal_list",
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
    "session_id": "<server-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "topic": null,
    "payload": [
        {
            "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
            "type": "signal",
            "timestamp": "2026-01-18T12:00:00Z",
            "instrument": "BTC-USD",
            "exchange": "paper",
            "side": "buy",
            "strength": 0.85,
            "reason": "RSI 28.5 <= 30",
            "strategy_name": "rsi_btc_1h",
            "price": 42000.0,
            "fired_at": "2026-01-18T12:00:00Z"
        }
    ],
    "count": 1
}
```


GET /api/executions

Fetch order fills/executions. Requires read:orders permission.

Request:

```
GET /api/executions?limit=100
```

Parameters:

Parameter	Type	Required	Description
limit	int	no	Number of executions, max 1000 (default 100)
as_of	datetime	no	Point-in-time query, UTC (default: current time)
operator_public_id	string	no	Scope to a single operator (403 if foreign)
wallet_public_id	string	no	Scope to a single wallet (403 if inaccessible)

Response (200):


```
{
    "type": "execution_list",
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
    "session_id": "<server-session>",
    "sequence_id": 1,
```

```
    "timestamp": "2026-01-18T12:01:00Z",
    "topic": null,
    "payload": [
        {
            "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
            "type": "execution",
            "timestamp": "2026-01-18T12:01:00Z",
            "trade_id": "TTRAD-456",
            "exchange_order_id": "KRAKEN-456",
            "client_order_id": "signal-a1b2c3d4",
            "instrument": "BTC-USD",
            "exchange": "kraken",
            "side": "buy",
            "size": 0.1,
            "price": 42000.0,
            "last_size": 0.1,
            "last_price": 42000.0,
            "fee": 0.001,
            "fee_asset": "USD",
            "status": "filled",
            "executed_at": "2026-01-18T12:00:59Z"
        }
    ],
    "count": 1
}
```


GET /api/positions

Fetch current portfolio positions. Requires read:positions permission.

Request:

```
GET /api/positions
```

Parameters:

Parameter	Type	Required	Description
as_of	datetime	no	Point-in-time query, UTC (default: current time)
operator_public_id	string	no	Scope to a single operator (403 if foreign)
wallet_public_id	string	no	Scope to a single wallet (403 if inaccessible)

Response (200):


```
{
    "type": "position_list",
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
    "session_id": "<server-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "topic": null,
    "payload": [
```

```
        {
            "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
            "type": "position",
            "timestamp": "2026-01-18T12:00:00Z",
            "instrument": "BTC-USD",
            "exchange": "kraken",
            "quantity": 0.5,
            "average_price": 41500.0,
            "unrealized_pnl": 250.0,
            "realized_pnl": 100.0,
            "mode": "live",
            "position_cycle_public_id": "019e1a2b-4d5e-7f6a-8b9c-0d1e2f3a4b5c"
        }
    ],
    "count": 1
}
```


The position_cycle_public_id field is null when no open position cycle exists for the position (e.g. flat positions or positions without cycle tracking).

POST /api/execution-plans

Create a bracket (SL/TP) execution plan on an open position cycle. Requires create:orders permission.

Request:

```
POST /api/execution-plans
Content-Type: application/json
X-CSRF-Token: <csrf_token>
```

Body (BracketCreateCommand envelope):


```
{
    "type": "create_bracket_command",
    "public_id": "019e1a2b-0000-7000-8000-000000000001",
    "session_id": "ui-session-1",
    "sequence_id": 1,
    "timestamp": "2026-04-12T18:00:00Z",
    "payload": {
        "position_cycle_public_id": "019e1a2b-4d5e-7f6a-8b9c-0d1e2f3a4b5c",
        "sl_price": 48000.0,
        "tp_price": 55000.0
    }
}
```


At least one of sl_price or tp_price is required.

Response (200): ExecutionPlanResponse wrapping the new armed bracket plan.

Errors: 422 (invalid params, missing capability), 409 (cycle not open, duplicate), 403 (wallet inaccessible), 503 (executor unavailable).

POST /api/execution-plans/{plan_public_id}/cancel

Cancel a bracket execution plan. Armed brackets transition to cancelled directly. Active brackets transition to cancel_requested with cancel TradeCommands emitted. Requires cancel:orders permission.

Request:

```
POST /api/execution-plans/{plan_public_id}/cancel
Content-Type: application/json
X-CSRF-Token: <csrf_token>
```

Response (200): ExecutionPlanResponse wrapping the updated plan.

Errors: 404 (not found), 409 (already terminal or concurrent cancel).

GET /api/execution-plans/{plan_public_id}

Retrieve a single execution plan by public_id. Requires read:orders permission.

Response (200): ExecutionPlanResponse wrapping the plan.

GET /api/execution-plans/{plan_public_id}/decisions

List decision audit rows for an execution plan. Requires read:orders permission.

Response (200): List of decision rows with action, reason, and timestamp.

POST /api/trailing-stops

Create a trailing stop execution plan on an open position cycle. The trailing stop ratchets the stop price as the market moves favorably and triggers a reduce_only market close on breach.

Request body:

- position_cycle_public_id (string, required): Target position cycle.
- trailing_pct (float, required): Trailing distance as percentage (0 < x < 100).
- min_lock_pct (float, optional, default 0): Minimum profit % before trailing activates.
- idempotency_key (string, optional): Dedup key for retries.

Permission: create:orders. CSRF token required.

Response (200): ExecutionPlanResponse with plan_type: "trailing_stop", status: "armed".

Response (409): Cycle not open or duplicate trailing stop on same cycle.

Response (422): Invalid params, missing capability, or no open position.

POST /api/trailing-stops/{plan_public_id}/cancel

Cancel a trailing stop. Armed stops transition directly to cancelled. Active stops (with in-flight child orders) transition to cancel_requested.

Permission: cancel:orders. CSRF token required.

GET /api/trailing-stops/{plan_public_id}

Retrieve a trailing stop plan by public_id. Only returns plans with plan_type: "trailing_stop".

Permission: read:orders.

GET /api/trailing-stops/{plan_public_id}/decisions

List decision audit rows for a trailing stop plan.

Permission: read:orders.

GET /api/trailing-stops/by-cycle/{cycle_public_id}

Get live trailing stop state for a position cycle. Returns peak_price and current_stop from the evaluator's in-memory state.

Permission: read:orders.

Response (200): TrailingStopStateResponse with live state, or { "type": "message", "payload": "none" } if no active trailing stop.

Response (404): Position cycle not found.

Paired Execution

The paired-execution operator surface exposes halted or exposed multi-leg groups and the manual attestation used after an operator resolves a manual_intervention group at the venue. Non-admin callers are filtered to their accessible wallets; admin is unscoped.

GET /api/paired-execution/incidents

List halted or currently exposed paired-execution scopes visible to the caller. Each incident is keyed by (wallet_public_id, strategy_id, group_key) and may contain an active durable halt, exposed groups, or both. halt_missing: true marks the anomalous window where an exposed group exists before the scanner has restored the durable halt row.

Permission: read:positions.

Response (200): PairedExecutionIncidentListResponse with paired_execution_incident payload items. Each item includes the scope, optional halt, halt_missing, and groups; each group contains per-leg signed exposure with open_qty = filled_signed_qty - compensated_signed_qty.

Response (503): Paired-execution tables are unavailable for the active repository backend.

POST /api/paired-execution/groups/{group_public_id}/terminalize

Attest that a paired-execution group in manual_intervention (or a compensating group reopened onto a manual leg) has been resolved at the venue. The repository completes the group; the guard scanner clears the scope's durable halt and in-memory mirrors within one scan cycle.

Permission: manage:paired_execution. CSRF token required for cookie-authenticated requests.

Response (200): PairedGroupTerminalizeResponse carrying the completed group projection and its legs' true accounting.

Response (404): No current active group exists with that id, or the group belongs to a wallet outside the caller's accessible set.

Response (409): The group is not currently attestable because its status is not manual, it has no legs or a held original command, or a sibling leg still has automation in flight.

Response (503): Paired-execution tables are unavailable for the active repository backend.

GET /api/position-cycles/open

List all open position cycles with age information. Admin-only endpoint for diagnosing orphaned cycles (open cycles without a matching trading engine).

Query parameters:

- min_age_hours (float, optional, default 0): Only return cycles open longer than this.

Permission: manage:users (admin only).

Response (200): PositionCycleListResponse envelope whose payload items carry cycle_public_id, shard_key, instrument_public_id, exchange, mode, wallet_public_id, direction, max_qty (per-cycle peak, not lifetime), opened_at, and age_hours.

POST /api/position-cycles/close-orphan

Close a specific orphaned position cycle. The operator should verify via GET /open that the cycle is genuinely orphaned before calling this.

Query parameters:

- cycle_public_id (string, required): Public ID of the cycle to close.

Permission: manage:users (admin only). CSRF token required.

Response (200): OrphanSweepResponse envelope with payload { "closed_count": 1, "closed_cycle_ids": ["<id>"] }.

Response (404): No active open cycle found for the given public_id.

POST /api/position-cycles/sweep-orphans

Bulk-close all open position cycles older than min_age_hours. Default threshold is 72 hours (3 days). Review via GET /open?min_age_hours=72 before running.

Query parameters:

- min_age_hours (float, optional, default 72, minimum 1): Minimum age threshold.

Permission: manage:users (admin only). CSRF token required.

Response (200): OrphanSweepResponse envelope with payload { "closed_count": N, "closed_cycle_ids": [...] }.

Response (400): min_age_hours must be at least 1.

GET /api/exchanges

List distinct exchange names from active symbol aliases. Requires read:market_data permission.

Request:

```
GET /api/exchanges
```

Response (200):


```
{
    "type": "exchange_list",
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
    "session_id": "<server-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "topic": null,
```

```
    "payload": ["kraken", "polygon", "walutomat"],
    "count": 3
}
```


GET /api/exchanges/{exchange}/instruments

List distinct native symbols available on a given exchange. Requires read:market_data permission.

Request:

```
GET /api/exchanges/kraken/instruments
```

Response (200):


```
{
    "type": "instrument_list",
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
    "session_id": "<server-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "topic": null,
    "payload": ["BTC-USD", "ETH-USD", "SOL-USD"],
    "count": 3
}
```


GET /api/exchanges/{exchange}/instruments/detail

List capability-aware instrument rows for an exchange. Requires read:market_data. Each row includes the native symbol plus trade and market-data capability flags, instrument kind, and expiry metadata so clients can show market-data-only instruments without a second request.

GET /api/instruments/{exchange}/{native_symbol}/related

Return related instruments for a concrete exchange/native-symbol pair. Requires read:market_data. Used by cross-asset and continuous-contract UI flows to navigate from an instrument to its configured underlying, front month, and sibling contracts.

GET /api/status

System-wide status including trader process, backtests, and active strategies. Requires read:system_status permission.

Request:

```
GET /api/status
```

Response payload excerpt (200):

The actual response is a SystemStatusResponse envelope with provenance fields and this object under payload.


```
{
    "trader": {
        "status": "running",
        "pid": null,
        "started_at": null,
        "command": null,
        "exit_code": null,
        "error": null
    },
    "backtests": {},
    "strategies": [
        {
            "strategy_name": "rsi_btc_1h",
            "status": "running",
            "details": {},
            "signals_generated": 42,
            "trades_executed": 5,
            "last_signal": "buy",
            "last_signal_time": "2026-01-18T11:45:00Z",
            "pnl": 150.25,
            "pid": 12345,
            "uptime": "2h 15m"
        }
    ]
}
```


GET /api/ws/stats

WebSocket and ZMQ bridge statistics. Requires read:system_status permission.

Request:

```
GET /api/ws/stats
```

Response payload excerpt (200):

The actual response is a WsStatsResponse envelope with provenance fields and this object under payload.


```
{
    "websocket": {
        "active_connections": 5,
        "topic_subscribers": {
            "market.kraken.BTC-USD.candles.1h": 3,
            "signals.paper.BTC-USD.rsi_btc_1h": 2
        },
        "client_count": 5
```

```
    },
    "zmq_bridge": {
        "active_topics": 12,
        "subscriber_tasks": 12,
        "available_topics": ["market.", "signals.", "system.heartbeats.",
"admin.", "orders.commands.", "orders.events.", "accruals.", "backtest.",
"alerts.", "plans.decisions.", "ai_reviews.", "processes.events.summary.",
"processes.events.configured.", "processes.events.runs.",
"strategies.events.list."]
    },
    "connections": {
        "active_connections": 5,
        "zmq_subscribers": 12,
        "subscriber_tasks": 12,
        "active_topics": 8,
        "active_clients": 5
    },
    "topics": {
        "market.kraken.BTC-USD.candles.1h": {
            "active_subscribers": 3,
            "received": 1200,
            "forwarded": 1180,
            "throttled": 20,
            "dropped": 0,
            "timeout": 0,
            "errors": 0,
            "invalid_messages": 0,
            "last_message_ts": 1737208800.0,
            "throttle_ms": 100,
            "pattern": "market."
        }
    },
    "subscriptions": {
        "per_topic": {
            "market.kraken.BTC-USD.candles.1h": 3
        },
        "per_client": {
            "140234567890": ["market.kraken.BTC-USD.candles.1h"]
        }
    },
    "config": {
        "broker_xpub": "tcp://127.0.0.1:7501",
        "heartbeat_interval_ms": 15000
    }
}
```


GET /api/zmq/health

ZMQ bridge health check. Requires read:system_status permission.

Request:

```
GET /api/zmq/health
```

Response payload excerpt (200):

The actual response is a ZmqHealthResponse envelope with provenance fields and this object under payload.


```
{
    "status": "healthy",
    "timestamp": "2026-01-18T12:00:00Z",
    "components": {
        "zmq_context": "ok",
        "websocket_manager": "ok",
        "active_connections": 5
    },
    "config": {
        "available_topics": ["market.", "signals.", "system.heartbeats.",
"admin.", "orders.commands.", "orders.events.", "accruals.", "backtest.",
"alerts.", "plans.decisions.", "ai_reviews.", "processes.events.summary.",
"processes.events.configured.", "processes.events.runs.",
"strategies.events.list."]
    },
    "connections": {
        "active_connections": 5,
        "zmq_subscribers": 12,
        "subscriber_tasks": 12,
        "active_topics": 8,
        "active_clients": 3
    },
    "message_stats": {},
    "errors": []
}
```


GET /api/market/cache/health

Diagnostic snapshot of the in-process market cache, configured pair stats, and persist-policy universe. Requires read:market_data. Returns counts for cached instruments, cached pairs, and persisted instrument universe size.

GET /api/market/cache/stats/configured

Return cached Pearson and cointegration stats for every configured market-stats pair. Requires read:market_data. Configured-but-cold pairs return placeholders with is_warm=false so dashboards can render stable "computing" rows.

GET /api/market/cache/stats/{exchange_a}/{symbol_a}/{exchange_b}/{symbol_b}

Return cached stats for one configured pair. Requires read:market_data. Unknown exchanges return 400; unconfigured pairs return 404; configured-but-not-yet-computed pairs return 200 with is_warm=false.

GET /api/market/coverage

Per-exchange market-data coverage over active instruments. Requires read:system_status.

Query parameters:

Parameter	Type	Description
tick_window_seconds	int	Freshness window for ticks (default 600)
candle_window_seconds	int	Freshness window for candles (default 1800)

The payload includes one row per exchange with instrument count, fresh_ticks, fresh_candles, gated_off, and dark counts.

GET /api/market/feed-health

Current per-symbol feed-health rows. Requires read:system_status.

Query parameters:

Parameter	Type	Description
exchange	string	Optional lowercase exchange filter
fresh_within_seconds	int	Optional staleness filter; older snapshots are dropped

Process Management

Process endpoints manage background services (feeds, strategies, executors, brokers). Most require manage:processes permission (operator/admin). The summary endpoint requires only read:system_status (viewer+).

GET /api/processes/available

List registered process templates that can be instantiated. Requires manage:processes permission.

Response (200):


```
{
    "type": "available_processes",
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
    "session_id": "<server-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "topic": null,
```

```
    "payload": [
        {
            "type": "available_process",
            "sequence_id": 1,
            "public_id": "<uuid7>",
            "timestamp": "2026-01-18T12:00:00Z",
            "session_id": "<server-session>",
            "topic": null,
            "name": "zmq_broker",
            "class_path":
"snapper.messaging.infrastructure.broker.ZmqBrokerProcess",
            "method": "start",
            "description": "ZeroMQ XPUB/XSUB message broker",
            "lifecycle": "long_running",
            "role": "core",
            "tags": ["infrastructure"],
            "parameters_schema": null
        }
    ],
    "count": 1
}
```


GET /api/processes/configured

List configured process instances with runtime state. Requires manage:processes permission.

Response (200):


```
{
    "type": "configured_processes",
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
    "session_id": "<server-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "topic": null,
    "payload": [
        {
            "type": "configured_process",
            "name": "zmq_broker",
            "enabled": true,
            "running": true,
            "mode": "thread",
            "class_path":
"snapper.messaging.infrastructure.broker.ZmqBrokerProcess",
            "method": "start",
            "parameters": {},
            "note": null,
            "lifecycle": "long_running",
            "role": "core",
            "tags": ["infrastructure"],
            "parameters_schema": null,
            "is_one_shot": false,
            "active_public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
            "kind": "instance",
            "wallet_public_id": null,
            "parent_template": null,
```

```
            "coordinator": "coord-0",
            "managed_remotely": false
        }
    ],
    "count": 1
}
```

GET /api/processes/summary

Lightweight process category counts for the overview dashboard. Requires read:system_status permission.

Response (200):

```
{
    "type": "process_summary_response",
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
    "session_id": "<server-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "topic": null,
    "payload": {
        "type": "process_summary",
        "coordinator": "coord-0",
        "feeds": { "running": 2, "total": 3 },
        "strategies": { "running": 1, "total": 2 },
        "executors": { "running": 1, "total": 1 },
        "brokers": { "running": 1, "total": 1 },
        "processes": []
    }
}
```

POST /api/processes

Create a new process configuration from a registered template. Requires manage:processes permission. Returns 201 on success.

Request:

```
POST /api/processes
Content-Type: application/json
X-CSRF-Token: <csrf_token>

{
    "type": "process_create_request",
    "public_id": "<uuid7>",
    "session_id": "<client-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "payload": {
        "name": "kraken_feed_btc",
        "template": "kraken_feed_publisher",
        "enabled": true,
```

```
        "mode": "thread",
        "parameters": { "symbols": ["BTC-USD"] },
        "note": "Kraken BTC feed"
    }
}
```

Response (201):

```
{
    "type": "process_create_response",
    "public_id": "<uuid7>",
    "session_id": "<server-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "topic": null,
    "payload": {
        "type": "process_create",
        "public_id": "<uuid7>",
        "session_id": "<server-session>",
        "sequence_id": 1,
        "timestamp": "2026-01-18T12:00:00Z",
        "topic": null,
        "status": "created",
        "process": {
            "name": "kraken_feed_btc",
            "template": "kraken_feed_publisher"
        }
    }
}
```

GET /api/processes/schema/{name}

Get the configuration schema and defaults for a registered process template. Requires manage:processes permission.

Response (200):

```
{
    "type": "process_schema_response",
    "public_id": "<uuid7>",
    "session_id": "<server-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "topic": null,
    "payload": {
        "type": "process_schema",
        "public_id": "<uuid7>",
        "session_id": "<server-session>",
        "sequence_id": 1,
        "timestamp": "2026-01-18T12:00:00Z",
        "topic": null,
        "name": "kraken_feed_publisher",
        "description": "Kraken market data feed publisher",
        "class_path":
"snapper.messaging.publishers.kraken.KrakenMarketDataPublisher",
        "method": "start",
```

```
        "default_enabled": true,
        "default_mode": "thread",
        "default_parameters": {},
        "lifecycle": "long_running"
    }
}
```

POST /api/processes/{name}/start

Start a configured process. Requires manage:processes permission.

Request:

```
POST /api/processes/zmq_broker/start
Content-Type: application/json
X-CSRF-Token: <csrf_token>

{
    "type": "process_start_request",
    "public_id": "<uuid7>",
    "session_id": "<client-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "payload": {
        "mode": "process"
    }
}
```

All payload fields are optional. mode overrides the stored execution mode for this run only; parameters supplies run-only constructor overrides. Neither field persists to Settings, and omitting either value uses the stored configuration. Strategy processes reject start-time parameters so their operator/wallet/output scope check always runs against persisted launch parameters. operator_public_id and wallet_public_id can never be overridden at start time. Executor templates cannot be started directly; start the generated executor_<exchange>_w<wallet_short> instance instead.

Response (200):

```
{
    "type": "process_start_response",
    "public_id": "<uuid7>",
    "session_id": "<server-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "topic": null,
    "payload": {
        "type": "process_start",
        "public_id": "<uuid7>",
        "session_id": "<server-session>",
        "sequence_id": 1,
        "timestamp": "2026-01-18T12:00:00Z",
        "topic": null,
        "status": "success",
        "name": "zmq_broker",
```

```
        "process_public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
        "message": "Process started"
    }
}
```


POST /api/processes/{name}/stop

Stop a running process. Requires manage:processes permission.

Request:

```
POST /api/processes/zmq_broker/stop
X-CSRF-Token: <csrf_token>
```

Response (200):


```
{
    "type": "process_stop_response",
    "public_id": "<uuid7>",
    "session_id": "<server-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "topic": null,
    "payload": {
        "type": "process_stop",
        "public_id": "<uuid7>",
        "session_id": "<server-session>",
        "sequence_id": 1,
        "timestamp": "2026-01-18T12:00:00Z",
        "topic": null,
        "status": "success",
        "name": "zmq_broker",
        "message": "Process stopped"
    }
}
```


GET /api/processes/runs

List historical process runs. Requires manage:processes permission.

Parameters:

Parameter	Type	Required	Description
limit	int	no	Number of runs to return (default 50)
name	string	no	Filter by process name

Response (200):

```
{
    "type": "process_runs",
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
    "session_id": "<server-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T18:00:00Z",
    "topic": null,
    "payload": [
        {
            "type": "process_run",
            "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
            "process_name": "zmq_broker",
            "status": "succeeded",
            "role": "core",
            "lifecycle": "long_running",
            "parameters": {},
            "result": null,
            "error": null,
            "tags": ["infrastructure"],
            "started_at": "2026-01-18T10:00:00Z",
            "completed_at": "2026-01-18T18:00:00Z"
        }
    ],
    "count": 1
}
```


Strategies

GET /api/strategies

List configured strategy processes with lightweight status. Requires read:strategies permission (viewer+).

Response (200):


```
{
    "type": "strategy_list",
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
    "session_id": "<server-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "topic": null,
    "payload": [
        {
            "type": "strategy_process",
            "name": "strategy_rsi_btc_1h",
            "running": true,
            "enabled": true,
            "mode": "process"
        }
    ],
    "count": 1
}
```

Settings

Settings-management endpoints require configure:system permission (admin only). GET /api/settings/features is the public exception.

GET /api/settings

List all settings, optionally filtered by category.

Parameters:

Parameter	Type	Required	Description
category	string	no	Filter by setting category
as_of	datetime	no	Point-in-time timestamp for temporal setting reads

Response (200):


```
{
    "type": "setting_list",
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
    "session_id": "<server-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "topic": null,
    "payload": [
        {
            "type": "setting_read",
            "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5c",
            "session_id": "<server-session>",
            "sequence_id": 1,
            "timestamp": "2026-01-18T12:00:00Z",
            "topic": null,
            "key": "polygon_api_key",
            "value": "IvLG...",
            "category": "api",
            "description": "Polygon.io market-data API key",
            "updated_at": "2026-01-15T10:00:00Z",
            "updated_by": "admin"
        }
    ],
    "count": 1
}
```


Note: per-wallet trading credentials (kraken, walutomat, kraken_futures) are NOT exposed through the settings endpoints. They live in the wallet_credentials table and are managed via the dedicated /api/wallets/{wallet_public_id}/credentials* routes (src/snapper/server/

credential_routes.py — GET for the active summaries, POST for create, and POST rotate for replacing an existing credential row). Seed files (dev.toml / prod.toml — see Configuration / Wallet Credentials) remain the bootstrap source for dev/prod parity.

GET /api/settings/features

Public feature-flag projection used by the frontend before auth-gated navigation renders. No authentication required. Currently exposes ai_integration_enabled; disabled AI integration also makes /api/mcp and /api/ai-delegates/* return the shared feature_disabled envelope.

GET /api/settings/categories

List distinct setting category names.

Parameters:

Parameter	Type	Required	Description
as_of	datetime	no	Point-in-time timestamp for temporal category reads

Response (200):


```
{
    "type": "setting_categories",
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
    "session_id": "<server-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "topic": null,
    "payload": ["exchanges", "strategy", "system"],
    "count": 3
}
```


GET /api/settings/push-beta/users

Return the active push-beta gate configuration. Requires configure:system. If the underlying push_beta_config setting is absent or malformed, the endpoint returns the default disabled gate with an empty allowlist so admins can see the effective routing state.

POST /api/settings/push-beta/users

Replace the push-beta gate configuration in one call. Requires configure:system and CSRF for cookie auth. The request body is UpdatePushBetaUsersCommand; user_public_ids becomes the complete allowlist after the write, so callers should read, edit locally, then submit the full desired set.

Request:

```
POST /api/settings/push-beta/users
Content-Type: application/json
X-CSRF-Token: <csrf_token>

{
    "type": "update_push_beta_users_command",
    "public_id": "<uuid7>",
    "session_id": "<client-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "payload": {
        "enabled": true,
        "user_public_ids": ["019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b"]
    }
}
```

Response (200):

```
{
    "type": "push_beta_config_response",
    "public_id": "<uuid7>",
    "session_id": "<server-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "topic": null,
    "payload": {
        "type": "push_beta_config_read",
        "public_id": "<uuid7>",
        "session_id": "<server-session>",
        "sequence_id": 1,
        "timestamp": "2026-01-18T12:00:00Z",
        "topic": null,
        "enabled": true,
        "user_public_ids": ["019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b"]
    }
}
```

POST /api/settings/{key}/set

Set (create or update) a setting value.

Request:

```
POST /api/settings/polygon_api_key/set
Content-Type: application/json
X-CSRF-Token: <csrf_token>

{
    "type": "setting_update",
    "public_id": "<uuid7>",
    "session_id": "<client-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "payload": {
        "value": "new-api-key-value",
        "category": "api",
        "description": "Polygon.io market-data API key"
    }
}
```

Payload fields:

Field	Type	Required	Description
value	string	yes	Setting value
category	string	no	Setting category (default: system)
description	string	no	Human-readable description

Response (200):


```
{
    "type": "setting_response",
    "public_id": "<uuid7>",
    "session_id": "<server-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "topic": null,
    "payload": {
        "type": "setting_read",
        "public_id": "<uuid7>",
        "session_id": "<server-session>",
        "sequence_id": 1,
        "timestamp": "2026-01-18T12:00:00Z",
        "topic": null,
        "key": "polygon_api_key",
        "value": "new-api-key-value",
        "category": "api",
        "description": "Polygon.io market-data API key",
        "updated_at": "2026-01-18T12:00:00Z",
        "updated_by": "admin"
    }
}
```

POST /api/settings/{key}/remove

Soft-delete a setting by key (sets known_to to current time).

Request:

```
POST /api/settings/polygon_api_key/remove
Content-Type: application/json
X-CSRF-Token: <csrf_token>

{
    "type": "remove_setting_request",
    "public_id": "<uuid7>",
    "session_id": "<client-session>",
    "sequence_id": 2,
    "timestamp": "2026-01-18T12:00:01Z",
    "payload": {}
}
```

Response (200):

```
{
    "type": "message",
    "public_id": "<uuid7>",
    "session_id": "<server-session>",
    "sequence_id": 2,
    "timestamp": "2026-01-18T12:00:01Z",
    "topic": null,
    "payload": "Setting 'polygon_api_key' deleted successfully"
}
```

AI Delegates

AI delegate management is gated by ai_integration_enabled, the same feature flag used by /api/mcp. When disabled, /api/ai-delegates/* returns the shared feature_disabled envelope. Delegate tokens are bearer-only automation credentials for MCP-compatible clients and are returned exactly once, on creation.

POST /api/ai-delegates

Create an AI delegate owned by the authenticated operator and mint a long-lived access JWT.

Permission: operator role or higher. CSRF token required for cookie-authenticated requests.

Body (DelegateCreateRequest envelope):

```
{
    "type": "delegate_create_request",
    "public_id": "<client-uuid7>",
```

```
    "session_id": "<client-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "payload": {
        "label": "research-agent",
        "operator_public_id": null,
        "caps": {
            "max_order_quantity_per_instrument": null,
            "max_open_orders": 2,
            "max_daily_notional_usd": 1000.0,
            "max_cancels_per_minute": 5
        }
    }
}
```

operator_public_id must be one of the caller's operator memberships; null uses the caller's primary operator. All caps are optional; null means the delegate inherits the Snapper-wide fallback for that cap.

Response (200): DelegateCreatedResponse with the delegate projection and one-shot access_token. List and detail endpoints never re-serve the token.

Response (409): The label cannot produce a unique delegate username, or the owner has reached the per-owner delegate cap.

Response (422): The requested operator binding is outside the caller's claim set.

GET /api/ai-delegates

List the caller's active delegates. Deactivated delegates are omitted.

Permission: operator role or higher.

Response (200): DelegateListResponse with DelegateRead payload items.

GET /api/ai-delegates/{delegate_public_id}

Fetch one active delegate owned by the caller.

Permission: operator role or higher.

Response (200): DelegateResponse.

Response (404): The delegate does not exist or is not owned by the caller.

PATCH /api/ai-delegates/{delegate_public_id}

Replace a delegate's trading caps while preserving username and label immutability. The write is SCD2-preserved so cap history remains auditable.

Permission: operator role or higher. CSRF token required for cookie-authenticated requests.

Body (DelegateCapsUpdateRequest envelope):

```
{
    "type": "delegate_caps_update_request",
    "public_id": "<client-uuid7>",
    "session_id": "<client-session>",
    "sequence_id": 2,
    "timestamp": "2026-01-18T12:01:00Z",
    "payload": {
        "caps": {
            "max_order_quantity_per_instrument": null,
            "max_open_orders": 1,
            "max_daily_notional_usd": 500.0,
            "max_cancels_per_minute": 3
        }
    }
}
```

Response (200): DelegateResponse with the updated caps.

Response (404): The delegate does not exist or is not owned by the caller.

POST /api/ai-delegates/{delegate_public_id}/deactivate

Deactivate a delegate using the shared user kill-switch flow. The server closes the active delegate row, revokes active tokens, and publishes the same deactivation event used by admin user deactivation.

Permission: operator role or higher. CSRF token required for cookie-authenticated requests.

Body: Optional DelegateDeactivateRequest envelope with payload.reason for audit context.

Response (200): DelegateResponse with is_active: false.

Response (404): The delegate does not exist or is not owned by the caller.

Underlying Assets

GET /api/underlyings

List all underlying assets with instrument counts.

```
GET /api/underlyings?as_of=2026-06-20T12:00:00Z
```

Returns PayloadListResponse with UnderlyingAssetData items (ticker, name, asset_class, sector, instrument_count).

GET /api/underlyings/{ticker}/instruments

List instruments mapped to an underlying asset.

```
GET /api/underlyings/SPX/instruments?relationship_type=derivative
```

Query parameters:

- relationship_type (optional): Filter by exact, derivative, or proxy
- as_of (optional): Point-in-time query timestamp

GET /api/underlyings/{ticker}/front-month

Return the front-month (nearest non-expired) futures contract for an underlying.

```
GET /api/underlyings/SPX/front-month?exchange=kraken_equities&contract_family=ES
```

Query parameters:

- exchange (optional): Filter by exchange
- contract_family (optional): Filter by product root (e.g., ES vs MES)
- as_of (optional): Point-in-time query timestamp

Returns PayloadResponse with FrontMonthData (instrument_public_id, native_symbol, exchange, expiry_at, relationship_type, contract_family). Returns 404 if no active futures contracts exist.

GET /api/underlyings/{ticker}/contracts

List all futures contracts for an underlying asset.

```
GET /api/underlyings/SPX/contracts?include_expired=true&contract_family=ES
```

Query parameters:

- exchange (optional): Filter by exchange
- contract_family (optional): Filter by product root
- include_expired (optional, default false): Include expired contracts
- as_of (optional): Point-in-time query timestamp

Returns PayloadListResponse with ContractData items. Each item includes is_front_month (true for nearest non-expired within same contract family).

GET /api/underlyings/{ticker}/continuous

Build a continuous futures series for an underlying from active contract metadata and candle rows.

```
GET /api/underlyings/SPX/continuous?
exchange=kraken_equities&contract_family=ES&timeframe=1d&start=2026-01-01T00:00:00Z&end=20
```

Query parameters:

- exchange (required): Exchange for the contract ladder
- contract_family (required): Product root, e.g. ES vs MES
- timeframe (required): Candle timeframe to load
- start (required): Inclusive UTC start timestamp
- end (required): Exclusive UTC end timestamp
- method (optional, default panama): Adjustment method, unadjusted, ratio, or panama
- rollover_days_before (optional, default 0): Roll contracts this many days before expiry, range 0..365
- as_of (optional): Point-in-time query timestamp

Returns a continuous-series response with the selected contract windows and candle points. A successful response may be continuous_full or continuous_partial; partial responses include failed_roll and message fields describing the unavailable roll window. Returns 400 for invalid parameters and 404 when the underlying cannot be resolved.

Multi-Tenant (Wallets, Operators, Scope Grants, Credentials)

ADMIN principals see the full catalogue; AI_DELEGATE, VIEWER, and OPERATOR principals see only the subset covered by their operator memberships and active scope grants.

GET /api/wallets

List wallets accessible to the current principal. ADMIN sees all; OPERATOR/VIEWER sees only wallets covered by at least one active scope grant from their operator set.

GET /api/operators

List operators accessible to the current principal. ADMIN sees all; OPERATOR/VIEWER sees only operators in principal.operator_public_ids.

GET /api/scope-grants

List active scope grants on a given wallet. Non-ADMIN callers must have visibility into the target wallet; otherwise 403. Required query parameter: wallet_public_id.

POST /api/scope-grants

Create a new scope grant. Requires manage:scope_grants (ADMIN only). Returns 409 on overlap conflict. Body fields: operator_public_id, wallet_public_id, scope_kind (underlying or instrument), underlying_public_id or instrument_public_id, optional note.

POST /api/scope-grants/handover

Atomic SCD2 close + insert transfer of a scope grant to a different operator. Requires manage:scope_grants. Body: from_grant_public_id, to_operator_public_id, optional reason. Returns 404 if source grant missing, 400 on self-handover, 409 on cross-scope overlap.

POST /api/scope-grants/{grant_public_id}/revoke

Atomic SCD2 close (no replacement row) of an active scope grant + emits admin.scope_revoked on the bus for live AI_DELEGATE subscription revalidation. Requires manage:scope_grants (ADMIN only). Body: optional reason (flows to event payload, NOT to the closed row). Returns 404 if the grant does not exist or is already closed (double-revoke).

POST /api/wallets

Create a new wallet. Requires manage:wallet_credentials (ADMIN only). Returns 409 if (label, is_paper) active-unique index is violated. Body: label, optional description, is_paper (default false).

GET /api/wallets/{wallet_public_id}/credentials

List active credentials on a wallet as summaries (no encrypted payload on the wire). Requires read:wallet_credentials (ADMIN only).

POST /api/wallets/{wallet_public_id}/credentials

Create a new wallet credential. Requires manage:wallet_credentials. Plaintext credential_payload is Fernet-encrypted server-side before DB insert. Body: exchange, credential_type (api_key_secret, rsa_pem, oauth, paper), credential_payload (dict), optional label. Returns 409 if (wallet, exchange) already exists.

POST /api/wallets/{wallet_public_id}/credentials/{credential_public_id}/rotate

SCD2 close + insert rotation. Old credential row closed, new row inserted with updated encrypted payload. Requires manage:wallet_credentials. Returns 404 if credential not found. Validates credential_payload against the existing credential type before encrypting.

WebSocket

Connection and Authentication

The WebSocket endpoint is at /api/ws. Authentication is message-based, not query-parameter-based, but the upgrade must already carry an access JWT via Authorization: Bearer <access_token> or, for browser clients, the access_token cookie. Bearer auth is checked first; the cookie is the fallback. Missing or invalid upgrade auth is rejected with close code 4401 before the auth_required challenge.

Connection flow:

1. Client connects to ws://host:port/api/ws (or wss:// over TLS) with an access bearer token or active auth session cookie
2. Server validates the origin header and bearer/cookie auth
3. Server sends auth_required message
4. Client sends authenticate message with a WebSocket token
5. Server validates the token and sends auth_ok
6. Server sends auth_complete with available topics for the user's role
7. Client can now subscribe to topics

Obtaining a WebSocket token:

Call POST /api/auth/ws_token with an access token (Bearer header or cookie auth). The route returns a short-lived, one-shot ws_token without rotating the refresh-token pair. POST /api/auth/refresh can also return ws_token during session rotation, but long-running clients should prefer /api/auth/ws_token.

Client-originated control frames (authenticate, reauth, subscribe, unsubscribe, get_subscriptions, ping) inherit StrictDataSchema; clients must stamp public_id, session_id, sequence_id, and timestamp before sending. topic defaults to null and can be omitted on client frames.

Authentication Messages

Server sends after connection:

```
{
    "type": "auth_required",
    "public_id": "019e1a2b-0000-7000-8000-000000000901",
    "session_id": "<server-ws-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "topic": null,
    "timeout": 10
}
```

Client sends to authenticate:

```
{
    "type": "authenticate",
    "public_id": "019e1a2b-0000-7000-8000-000000000a01",
    "session_id": "<client-ws-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "ws_token": "eyJhbGciOi..."
}
```

Server sends on success:

```
{
    "type": "auth_ok",
    "public_id": "019e1a2b-0000-7000-8000-000000000902",
    "session_id": "<server-ws-session>",
    "sequence_id": 2,
    "timestamp": "2026-01-18T12:00:00Z",
    "topic": null,
    "exp": "2026-01-18T12:30:00Z"
}
```

Server sends session info:

```
{
    "type": "auth_complete",
    "public_id": "019e1a2b-0000-7000-8000-000000000903",
    "session_id": "<server-ws-session>",
    "sequence_id": 3,
    "timestamp": "2026-01-18T12:00:00Z",
    "topic": null,
    "available_topics": [
        "ai_reviews.",
```

```
        "alerts.",
        "backtest.",
        "market.",
        "orders.commands.",
        "orders.events.",
        "plans.decisions.",
        "processes.events.configured.",
        "processes.events.runs.",
        "processes.events.summary.",
        "signals.",
        "strategies.events.list.",
        "system.heartbeats."
    ],
    "user_role": "operator",
    "session_expires_at": "2026-01-18T12:15:00Z",
    "ws_token_exp": "2026-01-18T12:30:00Z"
}
```

available_topics contains allowed registry roots for the user's role. Subscriptions may use a registry root such as market. or a full concrete topic such as market.kraken.BTC-USD.candles.1h. Intermediate prefixes such as market.kraken. are rejected. Backtests also allow scoped prefixes: backtest., backtest.{wallet_public_id}., and backtest.{wallet_public_id}.{run_public_id}., subject to RBAC and wallet scope.

Server sends on failure:

```
{
    "type": "auth_failed",
    "public_id": "019e1a2b-0000-7000-8000-000000000904",
    "session_id": "<server-ws-session>",
    "sequence_id": 4,
    "timestamp": "2026-01-18T12:00:00Z",
    "topic": null,
    "reason": "Invalid token"
}
```

Reauthentication

Before the WebSocket token expires, the server sends a reauth_required message. Clients may also refresh proactively before that deadline. In both cases they obtain a new one-shot ws_token via POST /api/auth/ws_token and send it as a reauth message. Use POST /api/auth/refresh only when the HTTP access/refresh token session itself needs rotation.

Server warning:

```
{
    "type": "reauth_required",
    "public_id": "019e1a2b-0000-7000-8000-000000000905",
    "session_id": "<server-ws-session>",
    "sequence_id": 5,
    "timestamp": "2026-01-18T12:28:00Z",
```

```
    "topic": null,
    "deadline": "2026-01-18T12:29:00Z"
}
```


Client sends new token:


```
{
    "type": "reauth",
    "public_id": "019e1a2b-0000-7000-8000-000000000a02",
    "session_id": "<client-ws-session>",
    "sequence_id": 2,
    "timestamp": "2026-01-18T12:28:30Z",
    "ws_token": "eyJhbGciOi...(new token)..."
}
```


Server confirms:


```
{
    "type": "reauth_ok",
    "public_id": "019e1a2b-0000-7000-8000-000000000906",
    "session_id": "<server-ws-session>",
    "sequence_id": 6,
    "timestamp": "2026-01-18T12:28:30Z",
    "topic": null,
    "exp": "2026-01-18T13:00:00Z"
}
```


If the client does not reauthenticate in time:


```
{
    "type": "auth_expired",
    "public_id": "019e1a2b-0000-7000-8000-000000000907",
    "session_id": "<server-ws-session>",
    "sequence_id": 7,
    "timestamp": "2026-01-18T12:30:00Z",
    "topic": null
}
```


Subscription Management

Subscribe to topics:


```
{
    "type": "subscribe",
    "public_id": "019e1a2b-0000-7000-8000-000000000a03",
    "session_id": "<client-ws-session>",
    "sequence_id": 3,
    "timestamp": "2026-01-18T12:00:01Z",
    "topics": ["market.kraken.BTC-USD.candles.1h", "signals.paper.BTC-
USD.rsi_btc_1h"]
}
```


Server confirms subscription:

```
{
    "type": "subscription_success",
    "public_id": "019e1a2b-0000-7000-8000-000000000908",
    "session_id": "<server-ws-session>",
    "sequence_id": 8,
    "timestamp": "2026-01-18T12:00:01Z",
    "topic": null,
    "action": "subscribe",
    "status": "subscribed",
    "topics": ["market.kraken.BTC-USD.candles.1h", "signals.paper.BTC-
USD.rsi_btc_1h"],
    "denied_topics": [],
    "active_subscriptions": ["market.kraken.BTC-USD.candles.1h",
"signals.paper.BTC-USD.rsi_btc_1h"],
    "message": null
}
```


Unsubscribe:


```
{
    "type": "unsubscribe",
    "public_id": "019e1a2b-0000-7000-8000-000000000a04",
    "session_id": "<client-ws-session>",
    "sequence_id": 4,
    "timestamp": "2026-01-18T12:00:02Z",
    "topics": ["market.kraken.BTC-USD.candles.1h"]
}
```


List active subscriptions:


```
{
    "type": "get_subscriptions",
    "public_id": "019e1a2b-0000-7000-8000-000000000a05",
    "session_id": "<client-ws-session>",
    "sequence_id": 5,
    "timestamp": "2026-01-18T12:00:02Z"
}
```


Response:


```
{
    "type": "subscriptions_list",
    "public_id": "019e1a2b-0000-7000-8000-000000000909",
    "session_id": "<server-ws-session>",
    "sequence_id": 9,
    "timestamp": "2026-01-18T12:00:02Z",
    "topic": null,
    "subscriptions": ["signals.paper.BTC-USD.rsi_btc_1h"],
    "available_topics": [
        "ai_reviews.",
        "alerts.",
        "backtest.",
        "market.",
        "orders.commands.",
        "orders.events.",
        "plans.decisions.",
```

```
        "processes.events.configured.",
        "processes.events.runs.",
        "processes.events.summary.",
        "signals.",
        "strategies.events.list.",
        "system.heartbeats."
    ],
    "total_available": 13
}
```


Ping/Pong

Client sends:


```
{
    "type": "ping",
    "public_id": "019e1a2b-0000-7000-8000-000000000a06",
    "session_id": "<client-ws-session>",
    "sequence_id": 6,
    "timestamp": "2026-01-18T12:00:30Z"
}
```


Server responds:


```
{
    "type": "pong",
    "public_id": "019e1a2b-0000-7000-8000-00000000090a",
    "session_id": "<server-ws-session>",
    "sequence_id": 10,
    "timestamp": "2026-01-18T12:00:30Z",
    "topic": null,
    "active_connections": 5
}
```


Server Data Messages

Data messages are flat JSON objects forwarded directly from the ZMQ bus. There is no "data" wrapper. Messages with malformed JSON are dropped by the bridge before forwarding and are counted in the invalid_messages metric for the topic.

All data messages carry session_id and sequence_id provenance fields (see REST section above). All WebSocket control messages (auth, subscribe, ping/pong, errors) inherit from StrictDataSchema, so they also carry the same provenance envelope. Clients can use these fields to detect gaps without server-side replay support.

Candle


```
{
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
    "type": "candle",
```

```
    "timestamp": "2026-01-18T11:00:00Z",
    "session_id": "019e0000-0000-7000-0000-000000000001",
    "sequence_id": 42,
    "instrument": "BTC-USD",
    "exchange": "kraken",
    "timeframe": "1h",
    "open_at": "2026-01-18T11:00:00Z",
    "open": 42000.0,
    "high": 42500.0,
    "low": 41800.0,
    "close": 42300.0,
    "volume": 1234.56,
    "vwap": 42150.0,
    "trades": 5678,
    "complete": true
}
```


The complete flag marks a provisional intra-minute update of a still-forming bar when false, and the final bar for its window when true (default true).

Tick


```
{
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
    "type": "tick",
    "timestamp": "2026-01-18T12:00:00Z",
    "instrument": "BTC-USD",
    "exchange": "kraken",
    "volume": 1234.5,
    "bid": 42000.0,
    "ask": 42001.0,
    "last": 42000.5
}
```


Trade


```
{
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
    "type": "trade",
    "timestamp": "2026-01-18T12:00:00Z",
    "instrument": "BTC-USD",
    "exchange": "kraken",
    "executed_at": "2026-01-18T12:00:00Z",
    "price": 42000.5,
    "volume": 0.25,
    "side": "buy"
}
```


Signal


```
{
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
    "type": "signal",
```

```
    "timestamp": "2026-01-18T12:00:00Z",
    "session_id": "019e0000-0000-7000-0000-000000000001",
    "sequence_id": 7,
    "instrument": "BTC-USD",
    "exchange": "paper",
    "side": "buy",
    "strength": 0.85,
    "reason": "RSI 28.5 <= 30",
    "price": 42000.0,
    "strategy_name": "rsi_btc_1h",
    "fired_at": "2026-01-18T12:00:00Z"
}
```


Execution


```
{
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
    "type": "execution",
    "timestamp": "2026-01-18T12:01:00Z",
    "trade_id": "TTRAD-456",
    "exchange_order_id": "KRAKEN-456",
    "client_order_id": "signal-a1b2c3d4",
    "instrument": "BTC-USD",
    "exchange": "kraken",
    "side": "buy",
    "size": 0.1,
    "price": 42000.0,
    "fee": 0.001,
    "fee_asset": "USD",
    "status": "filled",
    "executed_at": "2026-01-18T12:00:59Z"
}
```


Order


```
{
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
    "type": "order",
    "timestamp": "2026-01-18T12:00:00Z",
    "exchange_order_id": "KRAKEN-456",
    "client_order_id": "signal-a1b2c3d4",
    "instrument": "BTC-USD",
    "exchange": "kraken",
    "side": "buy",
    "status": "submitted",
    "order_type": "limit",
    "size": 0.1,
    "filled_size": 0.0,
    "price": 42000.0,
    "average_price": null,
    "reason": null,
    "time_in_force": "GTC",
    "error": null,
```

```
    "created_at": "2026-01-18T12:00:00Z",
    "updated_at": null
}
```


Heartbeat


```
{
    "public_id": "019e1a2b-3c4d-7e5f-8a9b-0c1d2e3f4a5b",
    "type": "heartbeat",
    "timestamp": "2026-01-18T12:00:00Z",
    "session_id": "019e1a2b-0000-7000-8000-000000000001",
    "sequence_id": 42,
    "component": "zmq_broker",
    "sequence": 42,
    "status": "healthy",
    "lag_ms": 5,
    "meta": {}
}
```


Error


```
{
    "type": "error",
    "message": "Topic 'invalid.topic' does not exist",
    "timestamp": "2026-01-18T12:00:00Z"
}
```


Available Topic Patterns

Topics use dot-separated hierarchical names. Subscribe using a full topic string or one of the registry roots returned in available_topics. Intermediate prefixes are rejected; use market., not market.kraken.. Backtest subscriptions are the exception and support the scoped prefixes documented in the WebSocket auth section above.

Market Data

- market.{exchange}.{instrument}.candles.{timeframe} -- OHLCV candles
- market.{exchange}.{instrument}.ticks -- Price ticks
- market.{exchange}.{instrument}.trades -- Individual trades
- market.paper.{source_exchange}.{instrument}.candles.{timeframe} -- Paper candles sourced from a real exchange
- market.paper.{source_exchange}.{instrument}.ticks -- Paper ticks sourced from a real exchange

Signals

- signals.{exchange}.{instrument}.live -- Live trading signals

- signals.paper.{instrument}.{strategy_name} -- Paper trading signals

Order Events

- orders.events.{exchange}.{instrument}.submitted -- Order submitted
- orders.events.{exchange}.{instrument}.accepted -- Order accepted
- orders.events.{exchange}.{instrument}.rejected -- Order rejected
- orders.events.{exchange}.{instrument}.executed -- Order fill
- orders.events.{exchange}.{instrument}.cancelled -- Order cancelled
- orders.events.{exchange}.{instrument}.expired -- Order expired
- orders.events.{exchange}.{instrument}.replaced -- Order replaced
- orders.events.{exchange}.{instrument}.unknown -- Ambiguous venue submit outcome (non-terminal; resolved to accepted or rejected via venue verification)

Order Commands

- orders.commands.{exchange}.{instrument}.submit -- Submit order
- orders.commands.{exchange}.{instrument}.cancel -- Cancel order
- orders.commands.{exchange}.{instrument}.replace -- Replace order

System

- system.heartbeats -- Global heartbeat
- system.heartbeats.strategy.{name} -- Strategy heartbeat
- system.heartbeats.executor.{exchange} -- Single-wallet executor heartbeat
- system.heartbeats.executor.{exchange}.{wallet_short} -- Per-wallet executor heartbeat; wallet_short is exactly 12 lowercase hex characters
- system.heartbeats.feed.{exchange} -- Live feed heartbeat
- system.heartbeats.feed.paper.{source} -- Paper replay feed heartbeat
- system.egress.snapshot -- Egress pool route snapshots
- system.egress.transfer -- WireGuard transfer samples
- admin.{resource} -- Administrative events (admin only)
- processes.events.summary.{coord_slug} -- Process summary snapshots
- processes.events.configured.{coord_slug} -- Configured process-name snapshots
- processes.events.runs.{process_name} -- Per-process lifecycle transitions
- strategies.events.list.{coord_slug} -- Strategy class-path snapshots

Alerts and Reviews

- alerts.{user_public_id}.{alert_type} -- Notification stream
- plans.decisions.{plan_public_id} -- Execution-plan decision events

- ai_reviews.{user_public_id}.{strategy_public_id}.{suffix} -- AI delegate review frames
- accruals.{exchange}.{instrument}.{accrual_type} -- Funding, rollover, and borrow accruals
- backtest.{wallet_public_id}.{run_public_id}.{event} -- Backtest lifecycle and progress frames

Error Handling

HTTP Status	Description
401	Missing or invalid authentication
403	Insufficient permissions or invalid CSRF token
404	Resource not found
422	Request validation error
429	Rate limit exceeded (Retry-After header included)
500	Internal server error

Usage Example (Python)

```python
import httpx

BASE_URL = "http://localhost:8000/api"

async def main():
    async with httpx.AsyncClient() as client:
        # Login: LoginRequest envelope around LoginBody payload
        login_resp = await client.post(
            f"{BASE_URL}/auth/login",
            json={
                "type": "login_request",
                "sequence_id": 1,
                "public_id": "<client-uuid7>",
                "timestamp": "2026-01-18T12:00:00Z",
                "session_id": "<client-session>",
                "topic": None,
                "payload": {
                    "username": "admin",
                    "password": "password123",
                    "remember_me": False,
                },
            },
        )

        ws_token_resp = await client.post(f"{BASE_URL}/auth/ws_token")
        # WsTokenResponse envelope: ws_token lives under .payload
```

```
        ws_token = ws_token_resp.json()["payload"]["ws_token"]

        candles_resp = await client.get(
            f"{BASE_URL}/candles",
            params={
                "instrument": "BTC-USD",
                "exchange": "kraken",
                "timeframe": "1h",
            },
        )
        candles = candles_resp.json()
        print(candles)
```

Usage Example (JavaScript)

This example is intentionally minimal. The shipped frontend client reuses cached WS tickets, schedules proactive reauthentication before expiry, and sends heartbeat pings every 5 seconds.

```
async function connect() {
    async function mintWsToken() {
        const wsTokenResp = await fetch("/api/auth/ws_token", {
            method: "POST",
            credentials: "include",
        });
        // WsTokenResponse envelope wraps the data in `.payload`.
        const { payload } = await wsTokenResp.json();
        return payload.ws_token;
    }

    let wsToken = await mintWsToken();

    const protocol = location.protocol === "https:" ? "wss:" : "ws:";
    const ws = new WebSocket(`${protocol}//${location.host}/api/ws`);
    const clientSessionId = crypto.randomUUID();
    let sequenceId = 0;

    const controlFrame = (type, body = {}) => ({
        type,
        public_id: crypto.randomUUID(),
        session_id: clientSessionId,
        sequence_id: ++sequenceId,
        timestamp: new Date().toISOString(),
        ...body,
    });

    ws.onopen = () => {
        console.log("Connected, waiting for auth_required...");
    };

    ws.onmessage = async (event) => {
        const msg = JSON.parse(event.data);

        if (msg.type === "auth_required") {
            ws.send(JSON.stringify(controlFrame("authenticate", {
                ws_token: wsToken,
```

```
            })));
        }

        if (msg.type === "auth_complete") {
            console.log("Authenticated. Topics:", msg.available_topics);
            ws.send(JSON.stringify(controlFrame("subscribe", {
                topics: ["market.kraken.BTC-USD.candles.1h"],
            })));
        }

        if (msg.type === "candle" || msg.type === "tick") {
            console.log("Data:", msg);
        }

        if (msg.type === "reauth_required") {
            wsToken = await mintWsToken();
            ws.send(JSON.stringify(controlFrame("reauth", {
                ws_token: wsToken,
            })));
        }
    };

    setInterval(() => {
        if (ws.readyState === WebSocket.OPEN) {
            ws.send(JSON.stringify(controlFrame("ping")));
        }
    }, 5000);
}
```

Backtests

Backtest endpoints manage strategy backtesting runs. Read endpoints require read:backtests permission (viewer+). Mutation endpoints require manage:backtests (operator+). Every read and mutation is wallet-scoped and fails with 400 when the caller has no active wallet selected.

POST /api/backtests

Create and launch a new backtest run. Requires an active wallet selection. The route accepts a BacktestCreateCommand envelope wrapping a BacktestCreateBody payload, creates a pending run row, and starts a one-shot BacktestRunnerProcess.

Request body:

```
{
    "type": "backtest_create_command",
    "sequence_id": 1,
    "public_id": "<client-uuid7>",
    "timestamp": "2026-01-18T12:00:00Z",
    "session_id": "<client-session>",
    "topic": null,
    "payload": {
        "strategy_class": "RSIReversion",
```

```
        "instrument_public_id": "<instrument-public-id>",
        "exchange": "kraken",
        "timeframe": "1h",
        "start_date": "2026-01-01T00:00:00Z",
        "end_date": "2026-06-01T00:00:00Z",
        "initial_cash": 10000.0,
        "strategy_params": {"period": 14, "upper": 70.0, "lower": 30.0},
        "execution_mode": "direct_db",
        "fill_model": "market",
        "slippage_bps": 0.0,
        "commission_bps": 0.0,
        "target_execution_exchange": null
    }
}
```

Response: BacktestRunResponse with the created run details.

GET /api/backtests

List backtest runs with optional filters.

Query parameters:

Parameter	Type	Description
strategy	string	Filter by strategy name
status	string	Filter by status (pending, running, completed, failed, cancel_requested, cancelled)
config_hash	string	Pairing-stable SHA-256 config hash used by comparison auto-pairing
limit	int	Page size (1-100, default 20)
offset	int	Page offset (default 0)
as_of	datetime	Temporal query timestamp

GET /api/backtests/strategy-classes

List registered strategy-class identifiers accepted by BacktestCreateBody.strategy_class. Used by the UI to populate the create-backtest strategy selector.

POST /api/backtests/compare

Create or return an idempotent existing comparison row for two terminal runs. Manual mode supplies run_a_public_id and run_b_public_id; auto mode supplies config_hash and optionally anchor_run_public_id. The route accepts a BacktestCompareRequest envelope wrapping BacktestCompareBody. Requires an active wallet and read:backtests. See backtesting.md for pairing semantics.

GET /api/backtests/compare

List recent comparison rows for the caller's active wallet.

Query parameters: limit, offset, and as_of.

GET /api/backtests/compare/{comparison_public_id}

Fetch comparison metadata plus recomputed metrics, equity, trades, and signals diffs from the current artifact rows.

GET /api/backtests/{run_id}

Get backtest run detail by public ID.

POST /api/backtests/{run_id}/cancel

Cancel a pending or running backtest. The route accepts a BacktestCancelCommand envelope wrapping BacktestCancelBody (reason is optional), sets status to cancel_requested via SCD2 close-and-insert, and returns 409 if the run is already in a terminal state.

POST /api/backtests/{run_id}/rerun

Create a new backtest run with the same configuration as the original.

GET /api/backtests/{run_id}/trades

Get paginated trades for a backtest run.

GET /api/backtests/{run_id}/signals

Get paginated signals for a backtest run.

GET /api/backtests/{run_id}/events

Get lifecycle events for a backtest run (started, failed, etc.).

GET /api/backtests/{run_id}/equity

Get paginated equity curve points for a run.

Query parameters: limit (1-20000, default 5000), after, and as_of.

AI Reviews

Endpoints for the human-in-the-loop review queue. Strategies emit ai_reviews.* requests via the create_ai_review_and_await() primitive (see strategies.md); the selected, registered AI delegate replies via these routes.

POST /api/ai-reviews/{review_public_id}/decision

REST mirror of the submit_ai_review_decision MCP tool. Body is AiReviewDecisionCommand, an envelope wrapping an inner AiReviewDecisionRequest:

```
{
    "type": "ai_review_decision_command",
    "sequence_id": 1,
    "public_id": "<uuid7>",
    "timestamp": "2026-01-18T12:00:00Z",
    "session_id": "<client-session>",
    "topic": null,
    "payload": {
        "decision": "approve",
        "rationale": "Risk profile within bounds."
    }
}
```

decision is "approve" or "reject" (validated against AiReviewDecisionEnum; unknown values yield 422 with error_code='invalid_decision'). rationale is optional free text, ≤ 4096 chars per the ai_reviews.rationale column constraint. Requires Permission.CREATE_ORDERS, but that permission is not sufficient by itself: the caller must resolve to a registered ai_delegates row and have a live scope grant for the review wallet and instrument. The dispatch fans out on bus.ai_review_decision and emits ai_reviews.{user}.{strategy}.decision_ack on the WS surface.

Responses use AiReviewDecisionResponse — the canonical envelope shared with the MCP CallToolResult payload:

```
{
    "success": true,
    "error_code": null,
    "message": "Decision recorded.",
    "details": { "...": "..." }
}
```

Error status codes: 404 (review not found), 403 (caller not authorized for this review), 409 (already resolved by peer), 410 (deadline elapsed), 422 (invalid decision).

GET /api/ai-reviews/pending

List pending CONSULT reviews where the caller is the selected AI delegate. Used by the bridge for catch-up after WS reconnect and by operator dashboards. Snapshot is bounded by limit and ordered by fanout_after ASC (oldest first). The route first requires Permission.READ_SIGNALS (callers without it receive 403); callers that pass that gate but are not registered as an AI delegate principal receive 422.

Query parameters:

Parameter	Type	Description
limit	int	Page size, 1..500 (default 100)
wallet_public_id	string	Optional wallet filter

Response: PendingReviewListResponse (items + count) where each PendingReviewSummaryItem carries the resolved instrument ticker plus the raw signal_envelope payload (thesis, side, news anchors) so the AI delegate inbox can render a meaningful row without a follow-up read.

Devices

APNs device registration for the iOS app (push notifications for order / position / system events) plus per-device alert preferences. All rows are bound to user_public_id and follow the SCD2 close-and-insert convention.

POST /api/devices

Register or refresh the caller's APNs device token. Body is RegisterDeviceCommand. Same-token re-registrations collapse onto the existing SCD2 row via upsert_notification_device and return the stable public_id reused across versions. Returns NotificationDeviceResponse.

GET /api/devices

List the caller's active devices, newest registered first. Returns NotificationDeviceListResponse (payload + count — PayloadListResponse base); rows whose active version has token_status='user_unregistered' (tombstones) are excluded.

DELETE /api/devices/{device_public_id}

Soft-delete a device via SCD2 close + tombstone successor: the active row is closed and a successor with token_status='user_unregistered' is inserted so an as_of query after the close sees an inactive row rather than a gap. Returns MessageResponse. Marking a nonexistent or not-owned device inactive returns 404.

PATCH /api/devices/{device_public_id}/prefs

Upsert a per-(device, alert_type, scope) preference for the caller. Body is UpdateDevicePrefCommand. The composite scope key is (device_public_id, alert_type, operator_public_id, wallet_public_id) — three null-permutations map to the three partial unique indexes on device_alert_prefs. Returns DeviceAlertPrefResponse.

POST /api/devices/{device_public_id}/prefs/{pref_public_id}/revoke

Close one device-scoped alert preference (SCD2 in place). Backend convention is POST + envelope (not DELETE) so every write carries client-side provenance for the gap detector — same as cancel_order / revoke_scope_grant. The pref row is closed by stamping known_to at the revoke timestamp; no successor is inserted, which frees the slot in the partial unique index. Body is RevokeDevicePrefCommand; returns RevokeDevicePrefResponse.

GET /api/devices/{device_public_id}/prefs

List active per-(alert_type, scope) prefs for a caller-owned device. Returns DeviceAlertPrefListResponse. Tombstoned device prefs are filtered out server-side via the join to notification_devices on token_status='active'.

Alerts

Real-time alert feed for the dashboard + iOS notification panel. Alert events are minted by various subsystems and surfaced via WS on alerts.* (see messaging.md for the canonical AlertType enumeration) plus this REST catch-up tail.

GET /api/alerts/history

Keyset-paginated page of the caller's active alert_events. Order: (timestamp DESC, public_id DESC).

Query parameters:

Parameter	Type	Description
limit	int	Page size, 1..200 (default 50)
before	string	Opaque cursor token from a previous page's next_cursor; None or malformed values map to page 1 (max 160 chars)

The before cursor is decoded server-side back into the internal AlertListCursor and applied as a keyset predicate against Repository.list_recent_alerts_for_user. Returns AlertHistoryResponse.

GET /api/alerts/{alert_public_id}

Fetch a single active alert_events row if owned by the caller. Returns AlertEventResponse. Ownership is enforced server-side via user_public_id equality; 404 is returned for unknown IDs **and** for IDs owned by other users, so the endpoint never leaks the existence of foreign alert events.

Alert Defaults

Per-user fallback preferences that apply when no per-device device_alert_prefs row matches.

GET /api/alert_defaults

List the caller's active user-level fallback prefs. Returns UserAlertDefaultListResponse (payload + count — PayloadListResponse base). An empty list is the legitimate "no overrides" state — clients should fall through to the in-app default UI; the route does **not** auto-create rows on first read.

PATCH /api/alert_defaults

Upsert the caller's user-level fallback pref for a given alert_type via SCD2 close-and-insert. Body is UpdateUserAlertDefaultCommand.

Metrics

Observability endpoints surfaced for the operations dashboard and the iOS Home/system-status surface. All routes require Permission.READ_SYSTEM_STATUS. See observability.md for the underlying retention / system-metrics pipeline and snapshot schemas.

GET /api/metrics/notifications

Current notify-sidecar outbox counters (per-status row counts — *not* a rolling window). Returns NotificationMetricsResponse.

GET /api/metrics/system

Most recent SystemMetricsSnapshot from the in-memory ring buffer (CPU, memory, GC, file descriptors, plus the process connection count under process.num_connections). Returns SystemMetricsResponse.

GET /api/metrics/system/history

Bounded slice of the in-memory system-metrics history (no cursor / page tokens — the caller filters with since / until + limit). Returns SystemMetricsHistoryResponse.

Query parameters:

Parameter	Type	Description
since	datetime	Lower bound (inclusive)
until	datetime	Upper bound (inclusive)
limit	int	Page size, 1..100000 (default 720 — roughly the last hour at the default 5-second cadence)

Ring-buffer depth is capped by SYSTEM_METRICS_HISTORY_CAP (default 17280, ~24 h at 5 s cadence — see configuration.md).

POST /api/metrics/system/tracemalloc/start

Arm Python tracemalloc with an auto-stop deadline. Body-less; duration_s is supplied as a query parameter. Returns TracemallocStateResponse.

POST /api/metrics/system/tracemalloc/stop

Disarm tracemalloc and cancel any pending auto-stop deadline. Returns TracemallocStateResponse.

GET /api/metrics/retention

Most recent retention-scheduler run summary. Returns RetentionRunResponse whose payload is a RetentionRunData with top-level fields run_started_at, run_completed_at, dry_run, and results: list[RetentionPolicyResult]. Each per-policy result inside results carries archived_rows, purged_rows, files_written, and the window boundaries (day_start, day_end).

GET /api/metrics/db/tables

Most recent per-table row-count snapshot. Refreshed on the cadence configured by DB_METRICS_INTERVAL_SECONDS. Returns DbStatsResponse.

GET /api/metrics/rest-rate

Per-exchange REST call rates + venue rate-limit utilization (rps_1s / rps_10s / rps_60s rolling windows, plus limit_rps + utilization when the venue's documented cap is known). Read from the in-process RestCallTracker snapshot. Returns RestRateResponse.

AI Delegates

AI delegate management is feature-gated by ai_integration_enabled; when disabled, these routes return a 503 feature_disabled envelope matching /api/mcp.

Route	Description
POST /api/ai-delegates	Create a delegate and return its 90-day access JWT once
GET /api/ai-delegates	List active delegates owned by the caller
GET /api/ai-delegates/{delegate_public_id}	Fetch one owned delegate; 404 also covers foreign IDs
PATCH /api/ai-delegates/{delegate_public_id}	SCD2 close+insert new caps for an owned delegate
POST /api/ai-delegates/{delegate_public_id}/deactivate	Deactivate via the shared kill-switch flow and revoke active tokens

The routes are documented end-to-end in ai-integration.md alongside the Claude Code / Claude Desktop / Cursor / Windsurf wire-up instructions. They follow the same PayloadResponse envelope convention used by the auth routes above. All delegate-management routes require OPERATOR or higher; mutating routes also require CSRF for cookie-authenticated requests. On create, omitted operator_public_id binds to the caller's

primary_operator_public_id and returns 422 when no primary exists. Non-admin callers that supply operator_public_id must choose one of their authenticated operators; ADMIN callers may bind explicitly using the admin operator bypass.

Paired Execution (operator surface)

Operator endpoints for the paired-execution guard (dark behind PAIRED_EXECUTION_GUARD_ENABLED; usable regardless for reading state). Both narrow to the SQL repository (503 otherwise). The operator workflow — triage, manual venue resolution, attestation — is documented in paired-execution.md.

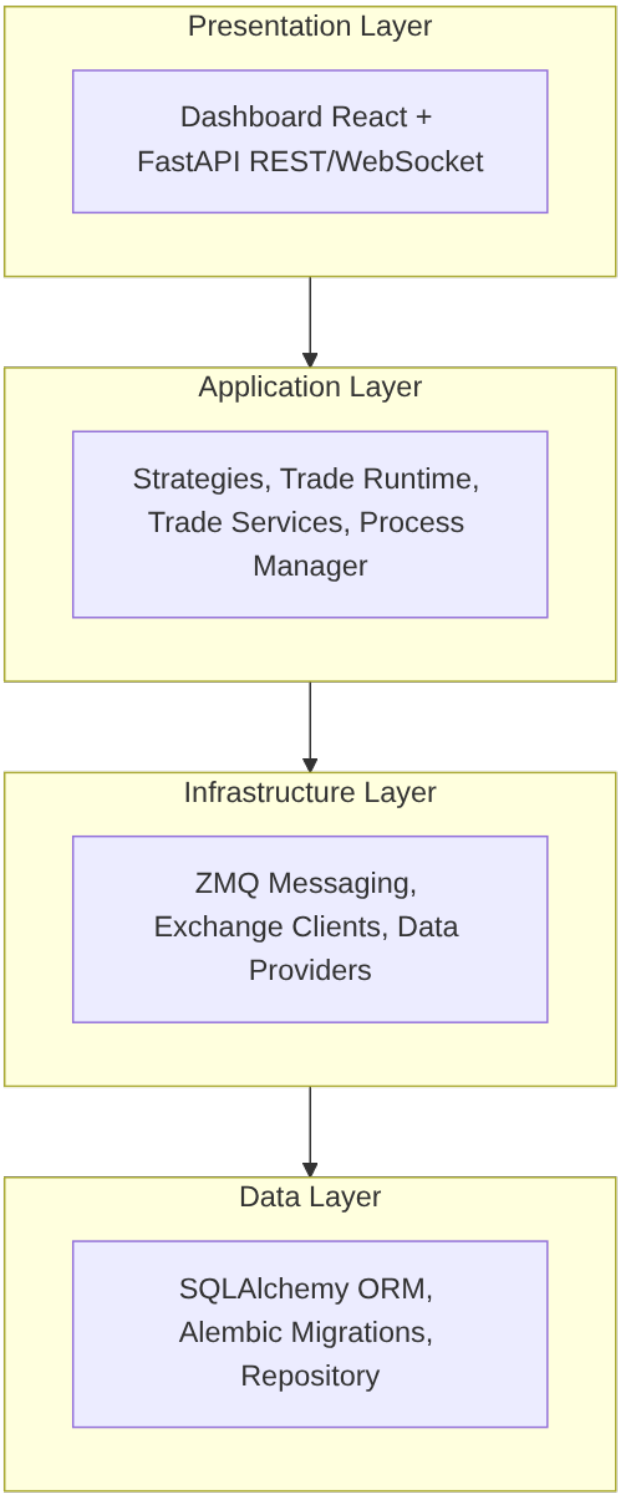
Route	Description
GET /api/paired-execution/incidents	Per-scope incidents: active halts ∪ exposed groups with per-leg signed exposure (READ_POSITIONS, wallet-scoped)
POST /api/paired-execution/groups/{group_public_id}/terminalize	Attest a manual_intervention group resolved at the venue (MANAGE_PAIRED_EXECUTION, CSRF; 404 also covers out-of-scope wallets, 409 when not attestable)

Architecture

Snapper is a trading platform built with a layered architecture using asynchronous processing and ZeroMQ messaging.

The trading path is facts-canonical: Order and Execution are the canonical business facts, while Position, Balance, and equity are derived projections that can be rebuilt from facts plus checkpoints.

System Layers



Components

Core (src/snapper/core/)

Fundamental types and aliases used throughout the application:

- TradeSide — Trade direction (buy, sell)
- OrderType — Order type (market, limit, stop, stop_limit)
- OrderStatus — Order status in lifecycle
- ExecutionMode — Execution mode (live, paper)
- OrderExchange — Order-capable exchanges (paper, kraken, kraken_futures, walutomat)

Config (src/snapper/config/)

Two-layer configuration:

1. **Bootstrap Settings** (bootstrap.py)

   Settings from environment variables required before database connection:

   - Database URL
   - Deployment environment (SNAPPER_ENV)
   - Master password for encryption
   - HTTP and ZMQ server endpoints

2. **App Settings** (app.py)

   Settings from database (encrypted):

   - Exchange API keys
   - Trading parameters
   - Authentication configuration

   Production-like SNAPPER_ENV values (production, prod, staging) fail fast on placeholder secrets: bootstrap refuses the default MASTER_PASSWORD at load time, and AppSettings raises on the placeholder auth_secret_key / csrf_secret_key DB defaults instead of silently signing tokens with development values.

Data (src/snapper/data/)

Persistence layer with SQLAlchemy:

- **ORM Models** (models.py):
  - Instrument — Financial instruments (natural key: symbol_public_id + exchange). Instrument.public_id is the stable identity used by fact tables (orders, executions, positions, signals, candles). The versioned business attributes symbol_public_id and exchange are resolved via ensure_instrument()
  - Candle — OHLCV candles

- Trade — Transactions
- Order — Canonical order lifecycle facts (includes mode: "live" or "paper")
- Execution — Canonical confirmed fills
- Position — Derived position projection (includes mode: "live" or "paper")
- TradeCommand — Durable trade intent written by the engine before execution
- VenueEvent — Durable venue observations and acknowledgements persisted by executors
- TradeProjectionCheckpoint — Materialized position/balance snapshot for fast recovery
- PairedExecutionGroup, PairedExecutionLeg, PairedExecutionHalt — durable arming, compensation, and halt state for multi-leg paired execution
- Signal — Signal events
- User — System users
- Setting — Settings (encrypted)
- Symbol — Symbol identity with versioned attributes (native_symbol, base, quote, asset_type) via SCD Type 2. Archive symbols for filesystem paths are derived from the anchor row (first version per public_id)
- SymbolAlias — Exchange-specific symbol aliases (one row per (symbol_public_id, exchange, channel))
- SymbolExchangeCapability — Exchange-specific symbol capabilities
- ProcessRun — Background process execution records
- InstrumentSpec — Instrument trading specifications (tick_size, margins, expiry_at, instrument_kind)
- UnderlyingAsset — Canonical asset identity (e.g., S&P 500, Gold) linking instruments across exchanges
- InstrumentUnderlyingMapping — Temporal link from instrument to underlying (exact/derivative/proxy)
- MarketSnapshot — Real-time market data snapshots
- InstrumentFeedHealth — Current per-symbol feed-health snapshot keyed by coordinator, exchange, native symbol, stream kind, and timeframe
- UserLoginEvent — Authentication event log
- Control — Always-on audit for commands, auth events, subscribe/unsubscribe, REST mutations. Includes redacted payload, outcome, and client causation linkage (client_session_id, client_public_id)
- Telemetry — Toggleable high-volume table for pings, heartbeats, pongs, and GET read requests. Recording gated by TELEMETRY_RECORDING_ENABLED setting

Most ORM models carry provenance via TemporalMixin (the few exceptions — UserActiveToken, AiDelegate, AiReview, AiReviewEvent, InstrumentFeedHealth — opt out because they manage their own lifecycle / revocation semantics):

- session_id (str, required) — producer session identity
- sequence_id (int, required) — per-table monotonic counter for gap detection

StrictDataSchema (Pydantic base) requires these fields at construction. Producers must obtain values from a SequenceTracker before creating the event, ensuring every payload is complete and identifiable from birth.

All ORM models use a dual-key identity pattern:

- id (INTEGER, internal PK) — row/version identifier for SCD2 close operations: SELECT → lock → close by id → INSERT new version. Never exposed outside the repository layer.
- public_id (UUID7 string) — logical entity identifier for domain joins and external references. Stable across SCD2 versions.
- timestamp (DateTime, system time / known_from)
- known_to (DateTime NOT NULL, default KNOWN_TO_MAX = 9999-12-31T23:59:59 UTC, active rows have known_to == KNOWN_TO_MAX; query pattern: WHERE timestamp <= :t AND known_to > :t)

No hard foreign keys between temporal entities. All domain joins use child.instrument_public_id == Instrument.public_id (or Execution.order_public_id == Order.public_id) with temporal filters. This enables future archival of closed SCD2 rows without breaking referential integrity.

- **SQLAlchemyRepository** (repository.py) — Async CRUD for SQLite/PostgreSQL
- **DatabaseRepository** (repository.py) — Sync access for scripts, archiver, and background updaters
- **CandleCacheArchiver** (archiver.py) — Exports active candle data to polygon-compatible CSV cache files per exchange/archive_symbol
- **EventArchiver** (archiver.py) — Exports append-only event rows (Tick, Trade, Signal, Execution, Telemetry, Control) to per-day CSV archive files with full temporal metadata, merge/dedup, and optional purge
- **CandleAuditArchiver** (archiver.py) — Exports all candle SCD2 versions (closed + active) with full temporal metadata, grouped by open_at date
- **StateArchiver** (archiver.py) — Generic archiver for state-SCD2 tables (Order, Position, Instrument, Setting, Symbol, etc.) with closed_only filtering and Symbol anchor row protection
- **Archive symbol resolution** (archive_symbols.py) — Stable filesystem-safe symbol naming derived from Symbol anchor rows, with seniority-based collision handling

Strategies (src/snapper/strategies/)

Trading strategies framework:

- **BaseStrategy** (base.py) — Base class for all strategies
- **StrategySignal** — Trading signal structure
- **StrategyConfig** — Strategy configuration

Built-in strategies:

- RSIReversion (rsi.py) — Mean reversion on RSI
- MACDCrossover (macd.py) — MACD crossover
- CointegrationPairs (cointegration.py) — Pairs trading

Indicators (src/snapper/indicators/)

Technical indicators:

- rsi — Relative Strength Index (Python implementation)
- macd — Moving Average Convergence Divergence
- ta_lib_adapter — TA-Lib library adapter

Messaging (src/snapper/messaging/)

ZeroMQ pub/sub architecture:

Components:

- **Broker** (infrastructure/broker.py) — XPUB/XSUB proxy
- **Publishers** (publishers/) — Market data publication. When multiple timeframes are configured, a publisher subscribes to the venue's 1m stream only and a CandleAggregator (publishers/candle_aggregator.py) synthesizes the higher timeframes (5m/15m/30m/1h/4h/1d) client-side at UTC boundaries, publishing them on the same market.*.candles.{timeframe} topic family. Synthesized bars are also persisted to the candles table (tagged source='synthesized' with a complete trustworthy-boundary flag) when a higher timeframe is configured for persistence — off by default — under the same persist policy as 1m. Native 1m persistence runs through a NativeCandleFinalizer (publishers/native_candle_finalizer.py) — the persistence-side peer of CandleAggregator and TradeCandleBuilder: by default the DB stores only the final complete=true bar per native 1m window (eliminating intra-minute SCD2 churn), while the default-off persist_intermediate_candles setting restores per-frame persistence (ZMQ publishes every frame regardless). The candle

source provenance vocabulary is native | calculated | synthesized (the ck_candle_source CHECK constraint on the candles/shadow_candles tables): native for venue OHLC bars, calculated for trade-built 1m bars (see the Kraken Spot sub-bullet below), and synthesized for aggregator-derived higher timeframes. The single-source read cutover is gated by the default-off candle_single_source setting (when ON, /api/candles serves 5m/15m/30m from the persisted plane instead of on-read derivation, after verify-candle-coverage confirms the plane is populated) (see docs/messaging.md "Higher-Timeframe Candle Synthesis"). An optional, default-off candle_forward_fill setting enables a time-driven flush that seals a wall-clock-ended window with no later 1m and forward-fills empty windows with a flat carried-close bar — for instruments whose warmup corpus is contiguous at the timeframe (24/7 crypto), never for session-based equities. The flag is gated per venue by the publisher's _supports_forward_fill() (only kraken spot), so it is forced off (with a warning) elsewhere and in paper replay. The paper publisher synthesizes from replayed 1m too, anchoring the aggregator's live epoch at the replay start so historical higher-TF windows are emitted rather than suppressed.

    - **Trade-built Spot 1m** — The Kraken Spot publisher (publishers/kraken.py) can build live 1m bars from the public trade feed instead of the native OHLC channel. A TradeCandleBuilder (infrastructure/exchanges/_trade_candle_builder.py) folds each TradeUpdate into its (symbol, minute) bucket and emits completed minutes once they age past trade_built_finalize_grace_seconds (default 12). This path is gated by the spot_candle_source setting (default native; set trade_built to enable), which tags persisted rows source='calculated' and disables the native candle-only liveness watchdog. The default-off spot_trade_built_shadow_enabled setting runs a parallel A/B writer that persists trade-built 1m bars to the separate shadow_candles table (source='calculated') while leaving the live native plane untouched — for comparing the two sources before any cutover.
    - **Executors** (executors/) — Per-wallet order execution on exchanges. One executor process per (exchange, wallet) pair is spawned at boot from active wallet_credentials rows; each instance filters incoming commands by wallet_public_id and loads its credentials via CredentialResolver during startup. Persisted executor_<exchange> rows are templates only; runtime instances are named executor_<exchange>_w<wallet_short> and bare templates are not started as live executors.
    - **Schemas** (schemas/) — Pydantic message models
    - **Topics** (topics/) — ZMQ topic definitions

Message types (in messaging.schemas.data):

- TickData — Price tick
- CandleData — OHLCV candle
- SignalData — Trading signal
- TradeData — Trade execution

- OrderData — Order status
- ExecutionData — Order execution
- OrderRequestData — Order request
- HeartbeatData — Component heartbeat

Every Data class carries public_id: str (UUID7), type: Literal[...], and timestamp: datetime.

Venue reconciliation (executors)

Executors can periodically poll exchange REST APIs to reconcile local pending-order state against the exchange's authoritative view. This detects two failure modes that WebSocket streams alone miss: **fill gaps** (exchange has more filled quantity than the executor observed) and **disappeared orders** (pending orders absent from the open-orders snapshot). The same cycle also works the durable command plane in both directions: **ghost orders** (open venue orders no executor is tracking) are adopted, and **dispatched commands** with no venue evidence are verified against venue truth (see below).

Always-on — runs alongside the executor's order handler, heartbeat, and private fill-stream tasks, all of which run as supervised loops (no feature flag; the defensive polling is cheap enough that gating it provided no value). Reconciliation cycles are serialized on an executor-level lock: the periodic 60s cycle and the fill-stream supervisor's post-reconnect heal cannot run concurrently and double-emit the same corrective. Each cycle — including lock acquisition — is bounded by a 300s timeout, so a hung venue call cannot hold the lock forever and wedge both loops; a cancelled cycle is safely recomputed next period because correctives carry stable synthetic ids.

_reconcile_fill_gap (in messaging/executors/base.py) emits a corrective ExecutionUpdate covering the observed cum-qty delta, stamping the synthetic execution with the deterministic exec_id=recon-<exchange_oid>-c<venue_cum> — re-emitting the same gap (after a failed publish, or again at the next startup recovery) dedupes at every consumer instead of double-applying, while a gap whose venue cumulative advanced gets a fresh id. Correctives carry venue-true fees instead of fabricating fee=0: the snapshot's order-level running commission, or the venue fills-summary total, rides the frame's cumulative cum_fee. When a venue-implemented fee source is transiently unusable (failed lookup, partial fills page, no usable rows yet) the corrective is deferred rather than freezing a fee-less emission under its stable exec id; the deferral is bounded at five recon cycles, after which the corrective emits fee-less with a CRITICAL manual-reconcile signal. A venue reporting neither a snapshot-level running commission nor a per-order fills summary emits fee-less as before.

_reconcile_disappeared_order (in messaging/executors/base.py) calls get_order() on orders missing from the open-orders snapshot to determine their actual terminal status (CLOSED / CANCELED / EXPIRED), emits any residual fill gap first, then publishes the terminal ExecutionUpdate. Walutomat's polling client never guesses filled-vs-canceled for

a disappeared order whose final-state query fails: the order stays tracked and the query retries every poll cycle, escalating to a single CRITICAL log after ten consecutive failures while still retrying — no terminal state is ever fabricated.

Ghost-order adoption and dispatched-command verification

A DISPATCHED trade-command row is durable pre-send intent, so the recon cycle also works the durable command plane.

Ghost adoption: an open venue order absent from the executor's pending set is attributed via a strict create/submit command lookup by client order id (cancels share the original order's cid; multiple active matches raise instead of guessing) and adopted with the original request rebuilt from the command row (application/trade/command_request.py, shared with the outbox publish path) so shard attribution survives adoption. Foreign/manual orders and other wallets' orders are never touched. Orders whose cid was already pending when the open-orders snapshot was taken never adopt from that snapshot (a stale snapshot would double-count a corrective). Adoption seeds the fill watermarks from the durable venue-event rows fail-closed — the engine books corrective deltas, so an untrusted zero watermark would double-book — and repairs the durable orders row so coordinator restart recovery re-arms engine intent.

Dispatched verification: active dispatched create/submit commands past a minimum age whose cid has no resolving venue evidence (a lone unknown-submit row does not resolve — parked entries lost to an executor restart land exactly here) are verified by client id under a per-cycle fairness cap with index rotation. A found order is adopted. Venue-verified absence twice in a row rejects only under publish-success-before-terminal semantics (the durable order_rejected row is written only after a confirmed REJECTED publish — a premature terminal row would exempt the command from the sweep while the engine guard stayed held), only while the command is young enough that the venue's bounded closed-order lookback keeps absence authoritative (older commands escalate WARN-only), and never when the dispatch TTL is disabled — a frame can then legitimately be in flight at any age, so absence may never auto-reject.

False-rejection heal: a command found OPEN on the venue after a durable REJECTED is adopted and its rejection durably restored to ACCEPTED through a retried restore queue; the coordinator's reconciliation loop runs a restart-proof resurrection pass for REJECTED rows whose live venue evidence postdates the rejection, and the next fold cycle re-derives the true state from the full event history.

Adopted-order intent re-arm: every adoption-shaped ACCEPTED publish (ghost adoption, ambiguous-submit verification, false-reject heal, startup recovery of a venue-verified-open row) carries reason="adopted" (ORDER_STATUS_REASON_ADOPTED, a shared wire constant), and the running coordinator re-arms a RELEASED engine's in-flight guard on exactly those frames — an engine that consumed a false REJECTED, or whose lazy timeout valve cleared, would otherwise emit a new order on the next signal while the adopted one still works the book. The re-arm never clobbers a newer live intent, skips

frames whose registered shard mismatches the routed engine, and refuses cids already retired by an honest terminal (FILLED fill, cancelled/expired confirm); a refused or stale adopted frame also suppresses the TradeService shadow-write so it cannot regress a FILLED projection back to ACCEPTED. A failed adopted publish is retried each recon cycle because that frame is the running engine's only re-arm signal.

Market orders without price — venue-fills VWAP heal

If a market order's exchange snapshot reports price=None (Kraken Futures snapshots carry only limitPrice), the fill-gap path first asks the venue for the order's own fills via ExchangeClientBase.get_order_fill_summary (venues with a real per-order fills source set the supports_fill_summary capability flag; the same aggregate also supplies corrective fees) and uses the venue-true VWAP — but only when the fills page covers the **whole** filled quantity within a relative 1e-6 tolerance; a partial page is refused rather than silently skewing VWAP. The CCXT snapshot builder additionally backfills price from the executed average when the venue reports one. Only when the venue reports neither a price, nor an executed average, nor whole-coverage per-order fills does the path log an ERROR ("no price on market order, skipping corrective fill") and skip the corrective emission — recovery then depends on a later snapshot populating price, or manual reconciliation against the exchange's fill history. Operators seeing this ERROR should not patch Snapper to approximate from ticks; the skip is an intentional trade-off (predictable behaviour + observable gap) over silent approximation drift.

Private fill-stream supervision

The private execution WebSocket stream is supervised, not spawn-once. _supervise_execution_stream (in messaging/executors/base.py) treats ANY termination of the stream handler — venue SDK reconnect-budget exhaustion, a raised error, or a clean generator return — as death and respawns it with capped jittered exponential backoff (1 s → 60 s, reset after 300 s of healthy runtime), never abandoning. Previously the stream was started once: after an outage exceeding the SDK's internal reconnect budget (~2 min futures, ~5 min spot), fills went permanently dark while the process kept reporting RUNNING.

Death is detectable because the venue clients raise instead of exiting cleanly: the spot executions generator raises ConnectionError when its consume loop ends on a terminal SDK error (closing the poisoned client), ensure-connected on both venues rebuilds a slot whose client carries a terminal SDK error instead of returning it as connected, and subscribe sends are bounded by a 5 s timeout so a wedged socket cannot hang an attempt.

Resubscribe is idempotent: subscription is snapshot-free at the source (spot subscribes with order/trade snapshots disabled; futures drops fills_snapshot frames) because startup recovery and the recon loop own snapshot-shaped state — a venue snapshot replayed on

top of recovered watermarks would inflate cumulatives above venue truth. Before each post-death re-entry the supervisor runs a best-effort reconciliation pass so the dark-window fill gap is healed before the stream re-attaches.

Per-order fill booking is atomic: each tracked order carries a fill_lock serializing the dedupe gate, delta build, durable venue-event write, publish, and watermark advance across the stream task, recon correctives, orphan flush, and cancel handling. Two cumulative watermarks are kept per order: last_seen_cum_qty (committed — advances only on publish success, so ZMQ deltas anchor to what the engine actually received) and last_recorded_cum_qty (durable — advances on venue-event write success, so additive checkpoint replay of the durable rows sums to venue truth whichever prior step failed). The same dual-plane pattern covers fees: per-currency signed fee watermarks (last_recorded_fee durable, last_published_fee published) anchor fee attribution — cumulative cum_fee frames set their currency's entry, per-fill fee frames add — so successive gap correctives attribute exactly the not-yet-attributed remainder and never re-charge fees that live per-fill frames already booked.

Executor loop supervision and escalation

The same supervisor pattern covers every executor loop, not just the fill stream: a generic _supervise_loop (in messaging/executors/base.py) wraps the order handler, the heartbeat loop, and the reconciliation loop with the identical capped jittered exponential backoff (1 s → 60 s, reset after 300 s of healthy runtime). Before respawning the order handler the supervisor rebuilds its SUB socket — re-entering on a poisoned socket would just die again. Previously a died order-handler task was a permanent invisible order-execution outage while the process kept reporting RUNNING.

A _supervise_loop-wrapped loop that keeps dying for 1500 s without a 300 s healthy run escalates: the supervisor raises ExecutorTaskDeadError out of start(), sibling tasks are cancelled, ZMQ sockets are closed, and the process launcher rebuilds a fresh service instance — safe because startup recovery emits corrective fills. The fill stream's dedicated supervisor backs off indefinitely and never escalates. The ceiling deliberately exceeds the launcher's 1200 s healthy-uptime budget reset, so escalation-driven restarts retry slowly forever instead of exhausting the restart budget; only fast startup crash-loops exhaust it.

Executor heartbeats report honest status derived from these supervision seams instead of a hardcoded HEALTHY: active death streaks, a reconciliation progress clock (recon swallows its per-cycle failures by design, so it can fail forever without dying — only progress age shows that), the age of the order command currently in flight (a command wedged inside its venue call never dies and never progresses), an unhealed accept-event backlog, and parked ambiguous submits all degrade the published status to WARNING or ERROR. The computation is crash-proofed: a raising status read degrades to WARNING

with the failure in the reasons, never heartbeat absence or false HEALTHY. Restart-budget exhaustion at the launcher is paged through the same heartbeat pipeline (see "Restart watchdog and parking" under Processes).

Application (src/snapper/application/)

Business logic:

- **Engine** (engine/) — TraderCoordinator is the multi-symbol trade runtime coordinator; TradingEngineService is the per-instrument engine that turns signals into TradeCommand rows and order requests
- **Trade** (trade/) — Trade-domain services: trade_service.py (in-memory command and position read model, fill deduplication, funding accrual application, and reconciliation halt feedback), balance_service.py (cash/equity/exposure projection), outbox.py (outbox-driven publishing with wake-up + polling fallback), reconciler.py (trade-command lifecycle fold, evidence-scoped stale-command scan, and reconciliation failure feedback), command_request.py (rebuilds the original order request from a durable command row — shared by the outbox publish path and the executor's adoption sweeps)
- **Process Manager** (process_manager/) — Process management
- **Services** (services/) — Application services, including market_cache.py (in-process 1m candle cache and pair-stat snapshots) and market_persist_policy.py (runtime policy deciding which market-data streams are persisted to DB versus cache-only)
- **Updaters** (updaters/) — Data updates (symbols, historical)
- **Risk** (risk/) — Risk defaults and sizing models
- **Portfolio** (portfolio/) — Signed long/short portfolio accounting

Infrastructure (src/snapper/infrastructure/)

External integrations:

- **Exchanges** (exchanges/) — Exchange clients (Kraken, Walutomat). kraken_sdk_patches.py is a version-coupled patch layer over python-kraken-sdk (3.2.x) that the Kraken venue implementations install at import time, so every process that builds a Kraken WS client — executors as well as publishers — is covered. Among its fixes is WS teardown hardening: the stock SDK reconnect backoff sleeps uninterruptibly (up to ~3 min), so a venue-bounded close() firing during it leaked one aiohttp ClientSession per rebuild cycle and the abandoned reconnect later spawned orphaned child tasks. The hardened reconnect polls the stop flag during backoff and reaps its children on every exit path, and force_close_ws_client() finishes teardown directly — cancel run tasks, bounded drain, close the session — whenever a venue's bounded close() times out or raises.
- **Market Data** (market_data/) — WebSocket feeds

- **Symbols** (symbols/) — Symbol mapping
- **Security** (security/) — Settings encryption
- **Logging** (logging/) — Logging configuration

Server (src/snapper/server/)

FastAPI application:

- **App Factory** (app.py) — create_app() with lifespan management
- **REST API** — HTTP endpoints
- **WebSocket** — Real-time streaming via ZMQ bridge with per-frame scope filters for ai_reviews.*, orders.events.*, and alerts.*
- **Static Files** — Frontend dashboard
- **ClientProvenanceMiddleware** (provenance_middleware.py) — Extracts client provenance from mutation requests, runs per-session gap detection, records mutations to control table and GET reads to telemetry table

API (src/snapper/api/)

Shared API schemas and WebSocket auth helpers (not route definitions):

- **Schemas** (schemas/) — Pydantic request/response models (health, process, settings).

  Schema class hierarchy:

  ```
  text
  BaseModel
  ├── PartialBody           — extra="ignore", strict=True (GapEnvelope,
  WsTokenPayload)
  ├── StrictBody            — extra="forbid", strict=True (request bodies,
  nested DTOs)
  │   └── StrictDataSchema  — + provenance (all event payloads, REST/WS/ZMQ)
  │       ├── PayloadRequest[T]
  │       ├── PayloadResponse[T]
  │       └── PayloadListResponse[T]
  ├── ExchangeResponse      — extra="allow", strict=True (exchange API
  parsing)
  ├── ExchangeRequest       — extra="forbid", strict=True (outgoing exchange
  requests)
  └── BaseSettings          — BootstrapSettingsLoader (env var config)
  ```

- **Auth** (auth/) — WebSocket token service and schemas

Route modules live closer to their domains: server/app.py (assembly and data endpoints), server/process_routes.py, server/strategy_routes.py, config/settings_routes.py, and auth/routes.py.

REST endpoints return the same Data schemas used by WebSocket messaging (messaging.schemas.data): OrderData, SignalData, ExecutionData, PositionData, CandleData. Old REST-specific schema modules were removed.

Auth (src/snapper/auth/)

Authentication system:

- JWT tokens with access/refresh via HTTP-only cookies
- CSRF protection
- Role-based access (viewer, operator, admin, ai_delegate). The ai_delegate role is the principal type minted by the AI Delegates flow (see ai-integration.md) and is gated by its own permission matrix separate from human operators.
- WebSocket authentication

MCP (src/snapper/mcp/)

Model Context Protocol server mounted at /api/mcp as a Starlette sub-application. Exposes read/query tools plus permission-gated order actions (submit_manual_order, cancel_order) in tools.py, consumed by AI delegates over Streamable HTTP. The mount re-applies its own middleware stack: feature-flag gating, bearer auth (auth.py), and per-principal rate limiting (rate_limiting.py); each tool handler runs output sanitization (output_sanitizer.py) and returns a structured error envelope (error_envelope.py). See ai-integration.md for the delegate flow and the @mateusz-klatt/snapper-mcp client plugin.

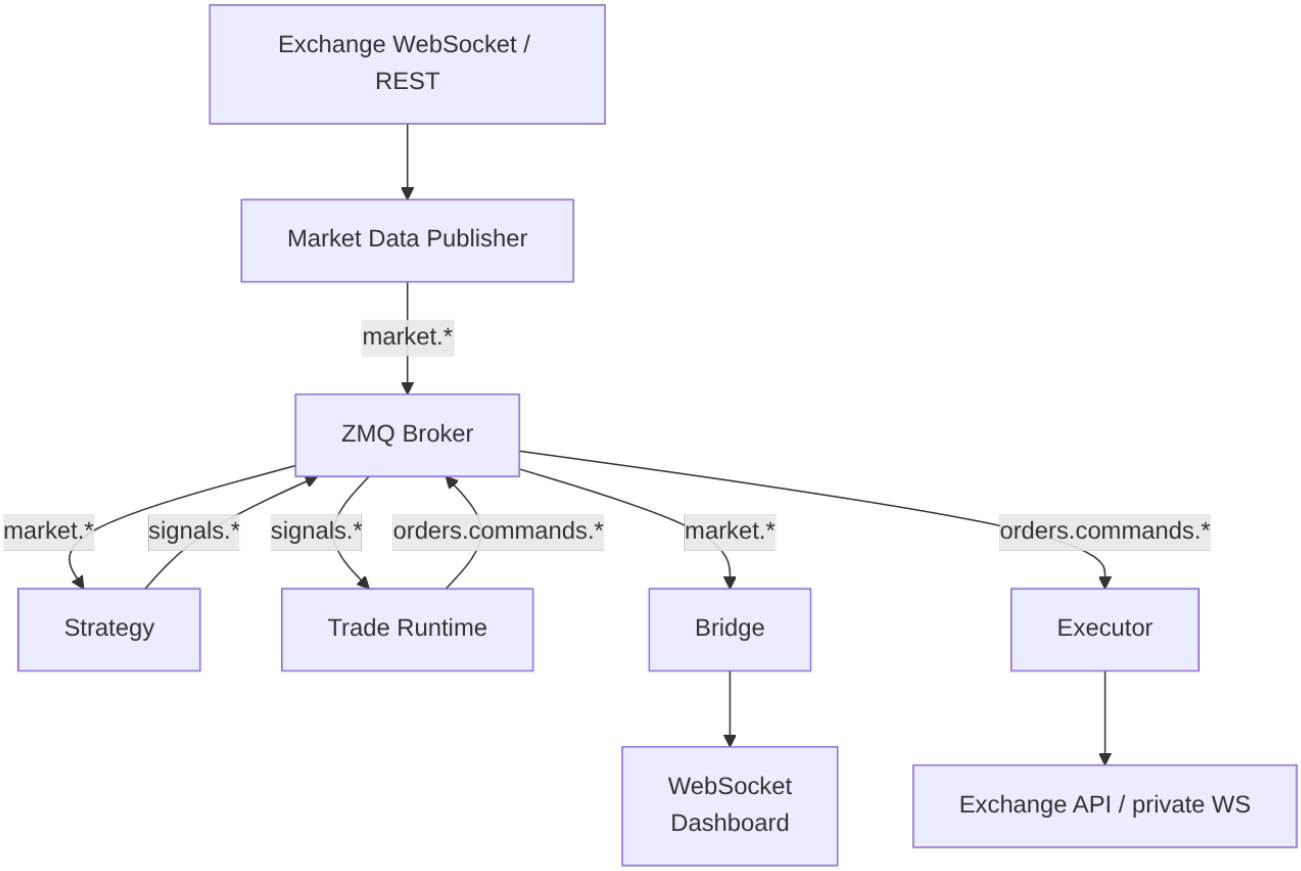
CLI (src/snapper/cli/)

Command-line interface (Typer):

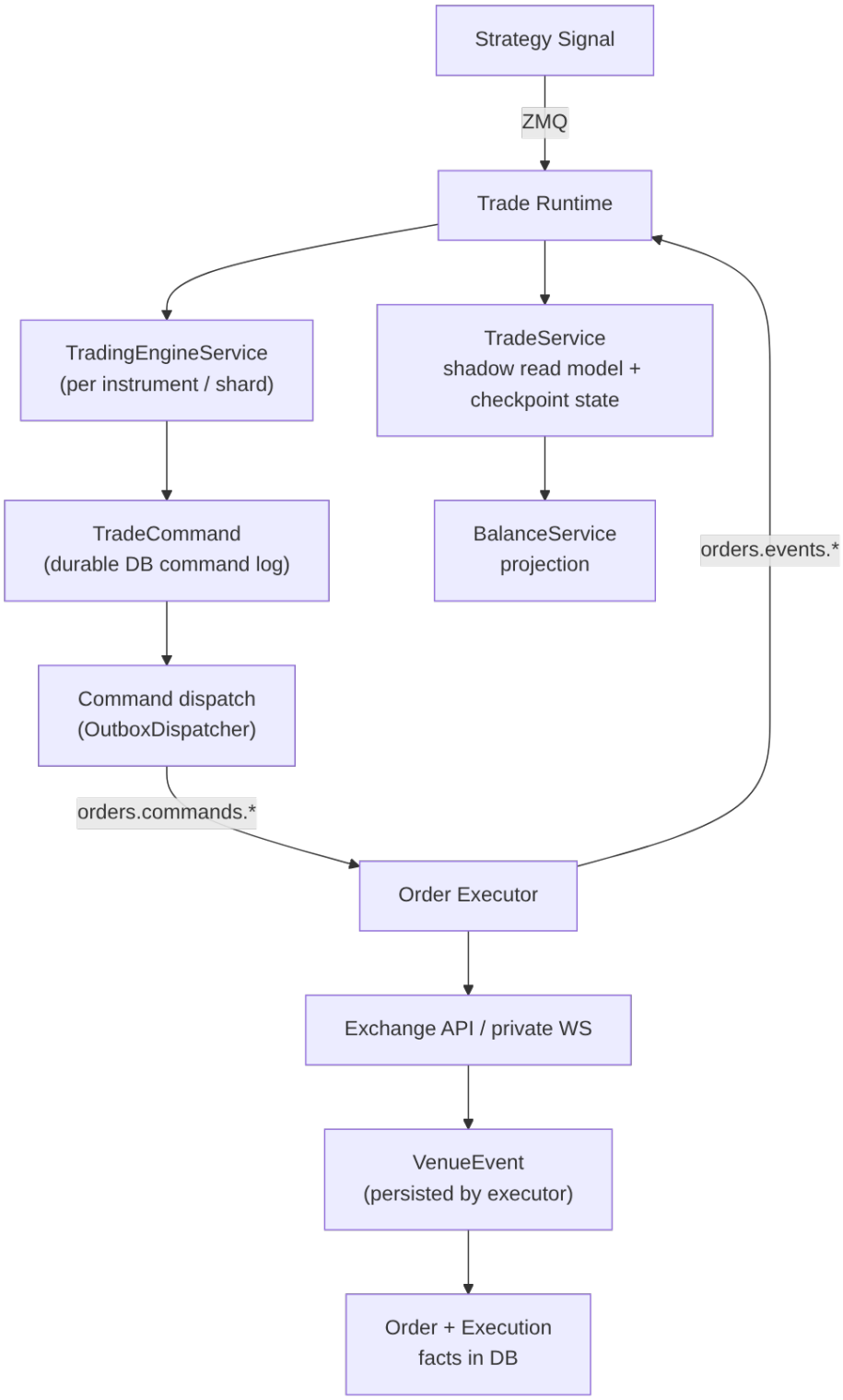
- Server and infrastructure management
- Database operations
- User management
- Market data updates

Data Flow

Market Data and Signal Flow



Order Flow



Database

SQLite for development, PostgreSQL for production.

Schema

```
-- Market data (joined to instruments via instrument_public_id)
instruments         -- Financial instruments (logical key: symbol_public_id +
exchange)
candles             -- OHLCV data
ticks               -- Real-time price snapshots
trades              -- Transaction history
market_snapshots    -- Real-time market data (SCD2 per instrument, one active
row each)
instrument_feed_health -- Current per-symbol feed health snapshots

-- Trading (joined to instruments via instrument_public_id)
orders              -- Order history (mode: live/paper)
executions          -- Order executions (joined to orders via order_public_id)
positions           -- Portfolio positions (mode: live/paper)
position_cycles     -- Open/closed cycles tracking per-cycle peak qty
signals             -- Strategy signals
execution_plans     -- Plan envelopes (brackets, trailing stops, manual)
execution_plan_checkpoints -- Materialized plan state snapshots
execution_plan_decisions   -- Per-tick plan decisions (audit trail)
execution_plan_decision_outbox -- Durable retry state for plans.decisions fanout

-- Trade runtime (durable command path)
trade_commands              -- Durable trade intent (engine writes before
execution)
venue_events                -- Durable venue observations and acknowledgements
from executors
trade_projection_checkpoints -- Materialized position/balance snapshots
paired_execution_groups     -- Multi-leg group FSM for arming and compensation
paired_execution_legs       -- Per-leg command binding and signed exposure
accounting
paired_execution_halts      -- Durable per-scope halt projection for broken/
exposed groups

-- Multi-tenant
wallets                     -- Trading accounts (label, is_paper)
wallet_credentials          -- Per-wallet exchange credentials (Fernet-
encrypted)
wallet_operator_scope_grants -- Per-wallet operator scope grants
operators                   -- Operator identities (multi-tenant trade actors)
user_operator_memberships   -- Operator memberships per user
user_trading_caps           -- Per-user trading-caps overrides

-- Symbol management
symbols              -- Symbol identity + versioned attributes (SCD2)
symbol_aliases       -- Exchange-specific symbol mappings
symbol_exchange_capabilities -- Exchange-specific capabilities
underlying_assets    -- Underlying-asset definitions (YAML-sourced)
instrument_underlying_mappings -- Active instrument → underlying mappings
continuous_contract_configs -- Continuous-contract series config

-- Instrument specifications
instrument_specs              -- Trading specifications (tick_size, lot_size,
limits)
instrument_order_capabilities -- Per-instrument order-side capability flags
venue_fee_schedules           -- Maker/taker fee schedules per venue

-- Funding + accruals
```

```
funding_rates       -- Per-instrument funding rate history (perps)
accrual_ledger      -- Funding / rollover / borrow accrual entries

-- Backtest
backtest_runs           -- One row per backtest run
backtest_events         -- Lifecycle events (started/failed/completed/...)
backtest_signals        -- Strategy signals emitted during a run
backtest_trades         -- Trades minted by the backtest engine
backtest_equity_points  -- Equity curve samples
backtest_results        -- Final per-run metrics
backtest_comparisons    -- Pairing-grouping fingerprint anchors

-- AI Reviews + Delegates
ai_delegates        -- AI delegate principals (PAT-style auth)
ai_reviews          -- Human-in-the-loop review queue rows
ai_review_events    -- Per-review lifecycle audit trail

-- Notifications
alert_events        -- APNs alert payloads
alert_deliveries    -- Per-device delivery attempts
device_alert_prefs  -- Per-(device, alert_type, scope) preferences
notification_devices -- Registered APNs device tokens
user_alert_defaults -- Per-user fallback alert preferences

-- System
users               -- User accounts
user_active_tokens  -- Active JWT inventory (revocation)
settings            -- Settings (encrypted)
user_login_events   -- Authentication event log
process_runs        -- Background process execution records
control             -- Audit log (commands, auth events, mutations)
telemetry           -- High-volume operational events (toggleable)
```

Migrations

Alembic manages migrations:

```
snapper db-upgrade    # Apply migrations
snapper db-downgrade  # Rollback
```

Trade Runtime

The trade runtime always uses the durable (outbox-driven) dispatch path. TradingEngineService writes TradeCommand rows to the database; OutboxDispatcher polls the table and publishes undispatched commands to ZMQ. Executor VenueEvent writes are fail-closed for pre-acceptance events — a failed persist raises before the executor acknowledges the venue event. Once the venue has accepted an order (returned an order id), a failed order_accepted persist no longer aborts the flow: it is logged CRITICAL, the entry is flagged accept_event_pending and queued, ACCEPTED is

still published (the venue state is the truth), and the executor's recon loop retries the durable write each cycle until it sticks — probing for an existing row first, so a timeout after a committed write cannot insert a duplicate accept event.

trade_commands rows are a fold of the venue_events truth plane. The coordinator's per-exchange ReconciliationLoop folds each active command's venue events into a durable status advance through a lifecycle CAS (advance_trade_command_lifecycle): the first accept supplies acked_at and the exchange order id, the max cumulative fill decides partial-vs-filled, and the last terminal-class event maps to the terminal status. The fold is rank-monotonic — a command status never regresses, so duplicate and out-of-order events collapse idempotently. A later accept or fill supersedes an earlier REJECTED (rejections are legally retried by the outbox, and a resurrection pass restores durably-REJECTED rows whose live venue evidence postdates the rejection); other terminals are never resurrected. The fold covers create/submit commands past the outbox's territory (created rows stay the outbox's), and stale-command reporting is scoped by evidence: an aged active command with zero venue evidence WARNs, unknown-only evidence reports INFO, real evidence is silent — the fold advances it. Cancels keep the legacy stale WARN because they share the original order's cid. Coordinator-side reconciliation scan failures are recorded only against known or last-seen real shard keys owned by that coordinator. Executor heartbeat venue-health metadata can also halt known real shard keys for an exchange/wallet scope and drop new cold-path signals for that scope.

Four order-safety layers sit on the dispatch path:

- **Dispatch max-age TTL** (TRADE_COMMAND_DISPATCH_TTL_S, default 30 s): the outbox CAS-expires a stale CREATED create/submit command to terminal EXPIRED instead of publishing it (cancels and replaces are exempt), and the engine releases its in-flight intent via a synthetic expired event — an outage backlog cannot fire MARKET orders priced off old signals. The executor-side stale check is venue-truth-backed: a found order is adopted, an unverifiable venue drops the frame silently (recon resolves it), and only venue-verified absence rejects.
- **Duplicate-submit guard**: before placing, the executor drops outbox replays of an already-evidenced client_order_id (live pending entry, unhealed-accept queue, or a durable venue-event evidence probe) — fail-closed on probe failure, publishing nothing. A replay whose evidence includes a breaker-open event instead reruns the breaker disposition (below) so a crash between the evidence write and the REJECTED publish cannot strand engine intent.
- **UNKNOWN submit state**: when a venue submit outcome is ambiguous (timeout, connection drop mid-send), the executor parks the order as non-terminal OrderEventEnum.UNKNOWN instead of fabricating a REJECTED — a false reject would let the engine re-emit and double the position. It then verifies against venue truth by client id (find_order_by_client_id: could-not-check never counts as venue-says-no); a found order is adopted, two consecutive authoritative absences reject safely, anything else stays parked. The recon loop verifies parked entries each cycle under a

per-cycle fairness cap with index-based rotation (a large parked set rotates through verification rounds instead of starving its tail or blowing the cycle's time budget), and the engine holds the in-flight guard until resolution. An order_unknown safety-critical alert (user-scoped, with admin fan-out for strategy orders) fires while an order is parked.

- **Breaker-open disposition**: a submit refused by the venue circuit breaker (typed CircuitBreakerOpenError) is authoritative not-submitted but NOT a venue rejection, and it must not blind-retry — the breaker may have closed, and a late redispatched frame would place an order the engine no longer tracks. The executor writes a probe-guarded durable order_breaker_open venue event (which counts as duplicate-submit evidence), CAS-fails the command row to terminal FAILED so the outbox can never re-fetch it, and only then publishes REJECTED with reason circuit_breaker_open so the engine releases intent. Any step failing parks the entry and the recon loop reruns the sequence until it completes.

The executor also gates order-type vocabulary before any venue send. The durable command plane (trade_commands, OrderRequestData) speaks the CORE vocabulary (market/limit/stop/stop_limit); one shared venue-request builder (_exchange_order_request_from_core in messaging/executors/base.py, used by both the submit path and ghost-adoption row repair) translates it to the venue wire vocabulary (stop-loss/stop-loss-limit for the stop types). A command whose order type has no wire mapping, or a stop-typed command without a stop_price trigger, raises inside the builder — before the network call, so the order is provably not placed — and the submit path's definitive-reject branch publishes REJECTED and writes the durable order_rejected venue event instead of letting a half-formed frame reach the venue.

Spot (Kraken) create_order is additionally excluded from blind network retry — an ambiguous network failure may have placed the order, and Kraken's cl_ord_id dedupe covers only open orders, so a blind retry of a MARKET order could double-place. The 429 rate-limit retry is kept (an exhausted 429 is a definitive venue-side rejection).

Executor startup recovery emits corrective fills instead of silently re-baselining: the old recovery seeded fill tracking from the venue's current cumulative, so every fill that landed while the executor was down became invisible to projections forever. Recovery now reads both truth planes per order — the durable venue_events fill rows and the executions log (rows exist only for successfully published fills) — seeds the committed/durable watermarks from what each plane proves, republishes recorded-but-unpublished fills under their original exec ids (idempotent — the engine, checkpoint replay, and the executions insert all dedupe by exec id), and emits the remaining venue-ahead gap as a recon corrective with a deterministic id, so repeated restarts and publish retries converge instead of double-applying. Orders that went terminal during the downtime get their fill gap healed before the terminal event is projected; orders the venue cannot verify at startup are parked with DB-derived seeds and retried each recon cycle. Fills without a venue exec id are never republished — they dedupe on an identity-shaped fallback key

shared by the live engine and checkpoint replay, count toward the committed seed as already shown, and the durable rows heal a coordinator that missed them at its next restart. This class is shrinking: Walutomat's cumulative polls now stamp deterministic wal-{oid}-c{basis_units} exec ids on every emission (the disappeared-order terminal upgrade gets a -t suffix so a same-cumulative status upgrade is never identity-dropped), so only rows recorded before that carry no id. Operator notes are in docs/operations.md "Recovery-time corrective fills".

The TraderCoordinator class acts as the trade runtime coordinator and integrates TradeService (command lifecycle) and BalanceService (balance tracking). It spawns the outbox dispatcher (when a SQLAlchemyRepository is wired) plus per-exchange reconciliation loops that drive the trade-command lifecycle fold and feed the circuit breaker. Canonical Order and Execution rows are persisted on the exchange/executor path.

Multiple TraderCoordinator processes can run against the same DB + broker with deterministic SHA-256 shard partitioning via snapper.core.partitioning.ShardOwnership. Each coordinator owns ~1/N of the shard_key set; signals, venue events, recovery, outbox dispatch, and reconciliation all filter by ownership so no two instances process the same shard. Default --instance-count 1 is byte-identical to single-coordinator behavior. Scaling to N≥2 requires PostgreSQL: the coordinator fails fast (ValueError) at startup when instance_count > 1 on a SQLite backend, because SELECT ... FOR UPDATE row locking — which the money-path DALs depend on — is a no-op there. See docs/operations.md for the systemd template recipe and scale-up / scale-down / crash-recovery procedures.

Checkpoint recovery resolves wallet-aware shard keys from wallet_credentials at the checkpoint's checkpoint_at timestamp before falling back to the boot-time wallet_short cache. This keeps rolling restarts and credential rotations from recovering old .w{wallet_short} checkpoints into an empty or stale wallet scope.

Recovery is also **gap-aware**. The executor commits a fill_observed venue event BEFORE publishing the fill and inserts the executions row only after a successful publish, so a fill recorded but never consumed (publish/insert never completed, and not absorbed by a later cumulative fill) can be stepped over by the scalar last_venue_event_id watermark and dropped. Recovery detects this per shard by **cumulative coverage** — recorded gross fill quantity (deduped by venue identity) exceeding the consumed executions total for the shard's orders — which is exact in both the absorption case (equal totals, no gap) and the true-drop case. A gapped, non-funding shard is rebuilt chronologically from its durable venue_events: a checkpoint shard overlays only the fill-derived fields (position, entry, cash, realized PnL, turnover, dedup set, watermark), leaving command identity and peak_equity from the checkpoint untouched; a shard with no checkpoint is rebuilt from scratch. A dedicated venue-plane pass discovers shards with recorded fills that the

execution-replay pass never visits (it early-returns on an empty executions table). Funding (futures) shards are left to status-quo recovery — a from-scratch replay cannot reconstruct pre-checkpoint accruals — so the rebuild is spot-scoped.

The coordinator also spawns the **paired-execution guard scanner** (PairedExecutionGuardScanner) as a background task: a DB-only liveness loop that assembles and arms multi-leg signal groups behind an arming barrier (no grouped command reaches a venue until every sibling leg is durably registered), breaks groups that miss their assembly/fill deadlines, cancels and reduce-only-flattens exposed legs, projects durable per-scope halts, and completes settled groups. The guard is dark by default (PAIRED_EXECUTION_GUARD_ENABLED false): live multi-leg emission is refused fail-closed at the strategy layer, while the scanner's compensation backstop always runs. Operator surface: GET /api/paired-execution/incidents and POST /api/paired-execution/groups/{id}/terminalize — runbook in docs/paired-execution.md.

Bitemporal Model

Most entity tables inherit TemporalMixin which provides id, public_id, timestamp (bus-time / known_from), and known_to columns. A small number of tables that manage their own lifecycle (notably user_active_tokens, ai_delegates, ai_reviews, ai_review_events, and instrument_feed_health) opt out.

Key concepts:

- **SCD Type 2** — No in-place UPDATE or DELETE. Mutations use a close+insert pattern: the current row is closed (known_to set to now) and a new row is inserted with the updated values and known_to = KNOWN_TO_MAX.
- **KNOWN_TO_MAX** — Sentinel value (9999-12-31T23:59:59 UTC) marking the active version of a row.
- **Point-in-time queries** — REST endpoints accept an as_of query parameter to retrieve the state of data at a specific moment.
- **Helpers**:
  - where_active(model, at) — SQLAlchemy filter for temporal queries. at: datetime is required — no silent default-to-now
  - where_active_now(model) — convenience for operations that genuinely mean "current state" (auth login, heartbeat). Not a shortcut for skipping as_of threading
  - close_and_insert() / close_and_insert_sync() — Repository helpers that atomically close the current row and insert a new version

Batch atomicity guarantees

True SCD2 batch writes (upsert_candles, upsert_market_snapshots) and single-row close+insert methods (revise_*, close_and_insert) commit as all-or-nothing transactions. The pattern is:

```
async with self.session() as s:          # opens a transaction
    for row in rows:
        # SELECT ... FOR UPDATE the active version
        # UPDATE close (set known_to = bus_time)
        # s.add(new row)
    await s.commit()                     # single commit
```

If any row in the loop raises (IntegrityError, serialization failure, connection drop, await cancellation), the transaction aborts and **no** close/insert pair from this batch lands in the DB. The batch must be retried in full; there is no partial-progress mode.

upsert_candles batch-loads existing versions for unique (instrument_public_id, timeframe, open_at) keys before closing and inserting rows. If the incoming batch contains duplicate candle natural keys, those rows keep the sequential close+insert path because an earlier row in that group can create the version a later row must close. Lookup batches are chunked to stay below SQLite parameter limits.

Append-only batch writes do NOT use the SCD2 close-insert pattern and split into two shapes:

- upsert_trades delegates to the dialect-aware _upsert_batch. On native dialects (SQLite / Postgres) it uses INSERT ... ON CONFLICT DO NOTHING in a single s.execute — atomic per-batch, and duplicates are silently skipped. On the fallback row-by-row path it wraps each row in a begin_nested() SAVEPOINT; successful rows stay in the outer transaction and are committed at the final await s.commit(), while rows that hit IntegrityError are skipped. This trades all-or-nothing for conflict tolerance.
- upsert_ticks issues a Core bulk insert via session.execute(insert(Tick), list(rows)). The function has two branches: when the caller passes a session, commit is external (caller-managed); otherwise it opens its own session and commits internally. Atomic per-batch on every dialect — it is not routed through _upsert_batch and has no duplicate-skipping behaviour.

Serialization on the same natural key: on **PostgreSQL**, row-level SELECT ... FOR UPDATE serializes concurrent transactions via MVCC row locks — a second session waiting on a locked active row observes the first session's close+insert only after the first commits; both batches complete without error and produce a clean SCD2 chain.

On **SQLite** (aiosqlite), SELECT ... FOR UPDATE is a no-op and the default transaction mode does not acquire a write lock at SELECT time — two concurrent tasks can both read the same active row before either writes. The first writer to COMMIT wins; the second writer's INSERT then collides on the public_id UNIQUE index and raises sqlalchemy.exc.IntegrityError. The final DB state is still a valid SCD2 chain (exactly one

active row, carried-forward public_id preserved, no orphan inserts) — one writer is simply reported the conflict via exception rather than implicitly serialized. Callers on SQLite that race on the same natural key must either retry the failed batch or rely on the single-publisher-per-exchange invariant to avoid the contention entirely.

At-scale implication: under heavy multi-publisher contention on the same natural key, batches serialize strictly. If a long-running batch holds the lock for many milliseconds per row, throughput falls linearly in contention. The current single-publisher-per-exchange shape makes this a non-issue; revisit if multi-publisher contention on the same natural key becomes a real workload.

Non-atomic fallback (_upsert_batch row-by-row path): the dialect-agnostic fallback (_upsert_batch in data/repository.py) wraps each row in a begin_nested() SAVEPOINT and continues on IntegrityError. Successful rows are NOT committed immediately — they are held in the outer transaction and finalised only by the final await s.commit(). This trades all-or-nothing for conflict tolerance — the caller sees inserted = N_successful, not all-or-nothing. Used only for append-only paths where a duplicate key is a genuine no-op. Never used for SCD2 close-and-insert.

Processes

The system manages processes through Process Manager:

- **Core** — Broker, Feed, Bridge (required — startup is aborted if any enabled long-running CORE process fails to start)
- **Strategy** — Trading strategies
- **Task** — One-time tasks
- **Backtest** — Backtesting processes

Processes can be:

- long_running — Run continuously (services)
- one_shot — Execute once (tasks)

Restart watchdog and parking

ProcessLauncherService reconciles died processes toward their desired state with backoff and two restart budgets (consecutive short-lived FAILED deaths, plus a lifetime backstop; both reset by healthy uptime). The watchdog drives native subprocesses and in-process async-task/thread processes through the same machinery — a died per-wallet executor task is respawned with the same backoff, budget, and stop-race guards as a native feed subprocess instead of being finalized FAILED and silently never restarted.

When a process exhausts its restart budget, a CORE market-data publisher escalates to a feed-container restart; any other process is parked. Parking is level-triggered and visible: /health reports ERROR while any process is parked (checked before every other branch, including the API-only early return and the TTL cache), and for per-wallet executor instances the launcher bursts synthetic ERROR heartbeats on the instance's own per-wallet heartbeat topic, re-bursting hourly while parked, so the existing critical-system-error alert pipeline pages the operator without any new alert type or topic family. A successful (re)start or a successful deliberate stop unparks the name; a failed stop or a failed manual restart keeps the marker.

Deployment Modes

By default, the FastAPI lifespan starts all enabled processes (all-in-one dev mode). Set SERVER_API_ONLY=true to skip process autostart -- the ZMQ-WebSocket bridge still starts, so the frontend receives live data from a separately-running engine. Useful when broker/strategies/executors run on different hosts.

Dedicated feed container (NYSE event-loop split): the market-data publishers (Kraken spot/equities/futures, Walutomat, paper) can be moved out of the FastAPI backend into a separate container so the API event loop stops sharing one CPU core with five WS publishers + their parse/persist work. Under NYSE open burst the shared loop saturated one core and starved the trade writer (drops); isolating each publisher onto its own process/core removes the contention.

PROCESS_AUTOSTART_PROFILE selects which registered processes a node autostarts (see snapper.core.types.ProcessAutostartProfileEnum):

- all (default) — every enabled process (single-container / dev).
- api — everything EXCEPT market-data publishers (the backend container: broker, executors, strategies, API).
- feed — ONLY market-data publishers (the feed container). Set by the feed-engine CLI command, which launches each publisher as its own OS subprocess (mode=PROCESS, uvloop) via ProcessLauncherService.start_feed_publishers. A market-data publisher is any registered process tagged both market-data and publisher.

A market-data publisher is filtered out of the backend's start_all_processes under the api profile, and get_core_health honours the same filter so a publisher that intentionally runs in the feed container is not reported as a missing CORE process on the backend.

The ZMQ broker stays in the backend (its tags are infrastructure, not publisher, so the api profile keeps it). Cross-container wiring uses two endpoint pairs because ZMQ bind rejects hostnames while connect resolves them:

- ZMQ_BROKER_BIND_XSUB / ZMQ_BROKER_BIND_XPUB — the routable interface the broker binds (tcp://0.0.0.0:7500 / :7501). Empty (default) falls back to the connect endpoints, so single-container behaviour is unchanged.
- ZMQ_BROKER_XSUB / ZMQ_BROKER_XPUB — the connect endpoints every process (backend's own components AND the feed container's publishers) dials. Both containers set these to the backend service name (tcp://snapper:7500 / :7501).

DB_POOL_SIZE / DB_MAX_OVERFLOW clamp each publisher subprocess's SQLAlchemy pool (PostgreSQL only). get_repository caches one engine per process, so splitting publishers across processes otherwise multiplies the default pool (5 + 10) per process toward Postgres max_connections. Keep count(publishers) * (DB_POOL_SIZE + DB_MAX_OVERFLOW) + backend pool under the server's max_connections.

Deploy notes (see docker-compose.yml snapper-feed service):

- Bring up the backend (api) and feed (feed) together. A partial docker compose up snapper with the api profile but no feed container stops all market-data ingest, so deploy both atomically.
- The broker's bound endpoint is seeded into its DB process-config parameters by registry sync on a fresh database. An existing deployment whose broker config already carries tcp://127.0.0.1:7500 must have that row re-synced (or the parameters cleared) so the broker picks up tcp://0.0.0.0:*; otherwise the feed container cannot reach it. In multi-instance deployments only coordinator instance 0 runs this replicated registry sync; non-zero instances skip it during FastAPI lifespan startup to avoid racing the same temporal Setting rows.
- **Egress routing (infrastructure/network/egress_*)** — Outbound exchange traffic is split into two classes by an EgressIdentity (egress_context.py) carried on a ContextVar: its traffic_class is public for market-data style traffic and private for authenticated executor traffic. Executor client lifecycles enter an egress_identity(..., traffic_class="private") scope (executors/base.py), so private traffic is direct-first and never traverses the public allow-list. Private idempotent REST reads and the next private WebSocket reconnect may use private_fallback_route_id when direct is unavailable; private REST mutations remain direct-only. Public/market-data traffic instead egresses through the per-venue snapper-egress SOCKS5/WireGuard routes, selected from the EgressPool by the exchange allow-list (egress_pool.py). This public routing is opt-in: by default feed publishers dial exchanges directly. Set feed_egress_enabled=true and restart snapper-feed to enable snapper-egress routing for public traffic, then confirm publisher connections use the configured snapper-egress SOCKS routes. See docs/snapper-egress.md for the sidecar deployment model. Each pool-bearing process publishes a read-only system.egress.snapshot frame on the heartbeat cadence; the API process subscribes to those frames and merges the

latest per-container snapshots into GET /api/health/egress for observability only. The snapper-egress sidecar also publishes system.egress.transfer samples from wg show <iface> dump; the API joins those onto SOCKS5 route rows by listener port without changing route selection.

Multi-instance deployment (AI-review fanout dedup): multi-worker uvicorn is a supported production topology. The shared ZMQ XPUB-XSUB broker delivers every internal bus event to every FastAPI worker's AiReviewService.start_bus_listener; each subscribed topic has its own dedup story so connected clients see exactly one external WS frame per source event regardless of worker count:

- bus.delegate_offline is idempotent at the DB layer — the handler runs a per-row atomic_dispatch_fanout_with_audit CAS UPDATE, so only one worker wins per affected review row even when every worker receives the bus event.
- bus.ai_review_decision is process-local — the handler resolves an asyncio.Future from the per-worker registry populated by the worker that ran create_review for the strategy await loop. Other workers stash the event in a bounded pending-resolution cache (1024-entry hard cap, 30s TTL); cache memory across the cluster is bounded by N × 1024 entries even at peak.
- bus.caps_violation_after_ai_approve is gated by deterministic partitioning. handle_caps_violation_bus_message short-circuits on ShardOwnership.owns(msg.review_public_id); exactly one worker (the SHA-256 owner of the review row) re-publishes the external ai_reviews.*.caps_violation frame. Other workers observe the bus event, log a debug skip, and return False. instance_count == 1 (default deployment) makes ShardOwnership.owns return True unconditionally so behaviour is byte-identical to single-worker.

The same partitioning primitive backs the Trade Runtime shard ownership; AI-review reuses it via the set_shard_ownership(ShardOwnership(coordinator_instance_id, coordinator_instance_count)) injection seam wired by the FastAPI lifespan after the publisher seam and before the bus listener starts.

Operational scaling: raise SNAPPER_COORDINATOR_INSTANCE_COUNT to N and start N FastAPI workers each with a distinct SNAPPER_COORDINATOR_INSTANCE_ID in [0, N) (env vars only — snapper server does not expose per-worker CLI flags; the --instance-id / --instance-count CLI flags exist on snapper trade-zmq for the Trade Runtime coordinator partitioning, not on the FastAPI server). Each worker subscribes to every bus topic; only the SHA-256 owner runs the caps-violation external fanout.

Execution Plans

The Execution Plans framework provides a unified control plane above the existing TradeCommand spine. All manual and algorithmic trading actions become ExecutionPlan instances evaluated by pluggable PlanEvaluator classes running inside PlanExecutorService.

Tables: execution_plans (plan state + lifecycle), execution_plan_checkpoints (high-churn evaluator state), execution_plan_decisions (tiered decision log), execution_plan_decision_outbox (durable retry state for the plans.decisions ZMQ fanout — written in the same transaction as each decision row the plan executor logs, drained with capped exponential backoff after publish failures; REST-surface decision rows are logged without an outbox entry), instrument_order_capabilities (capability matrix), venue_fee_schedules (fee tiers), position_cycles (one open→close lifetime per shard, brackets attach by position_cycle_public_id).

Plan types: manual_once (shipped), bracket (shipped — attaches to a position_cycles row), trailing_stop (shipped — stateful ratcheting stop with checkpoint persistence, separate /api/trailing-stops route module), peg, scheduler.

Manual order create flow: POST /api/orders creates a manual_once plan (pending), stamps child_client_order_id and native_instrument into plan.params, then inserts a create TradeCommand for the outbox dispatcher and transitions the plan to active. The command row stores the CORE order-type vocabulary (market/limit/stop/stop_limit) plus the stop_price trigger for stop types; translation to the venue wire vocabulary (stop-loss/stop-loss-limit) happens only at the executor's venue boundary. Cancel/flatten readers resolve the order type through core_order_type_from_plan_params, which still normalizes the legacy venue_order_type param carried by older plans. The OutboxDispatcher inside TraderCoordinator publishes the command as OrderRequestData on the orders.commands.{ex}.{instr}.submit topic.

Fill propagation: PlanExecutorService subscribes to orders.events. and market. on the broker XPUB and consumes every fill. Cumulative ExecutionData.size is compared against plan.filled_quantity with a small epsilon so duplicate or stale frames are dropped. Partial fills update filled_quantity and keep the plan active; full fills transition the plan to completed with completed_at stamped via SCD2.

Cancel flow: POST /api/orders/{plan_public_id}/cancel (plan id path) or POST /api/orders/by-client-order-id/{client_order_id}/cancel (UI convenience route) resolves the plan, verifies wallet scope, transitions the plan to cancel_requested, hydrates exchange_order_id from the active orders row, and inserts a cancel TradeCommand. On insert failure the plan is rolled back to failed and the route returns HTTP 500 — this is terminal (no retry). PlanExecutorService._recover_plans on the next service restart re-emits cancel commands only for plans still in cancel_requested state (deduplicated via has_pending_cancel_command); failed plans are not retried. The outbox dispatcher

branches on command_type and re-hydrates exchange_order_id one more time at publish time so a late venue ACK is picked up, then publishes OrderCancelData on the orders.commands.{ex}.{instr}.cancel topic.

Listen loop resilience: per-message exceptions (parse failures, handler errors) are caught inside the loop so a single bad frame cannot silently stop the service. Transient recv_multipart errors retry with a 100 ms backoff. asyncio.CancelledError still unwinds the loop for graceful shutdown. After _setup_subscriber() the service sleeps 500 ms for ZMQ XPUB/XSUB slow-joiner stabilization before recovery to avoid losing terminal events triggered by stranded-cancel re-emits.

Position cycles: the position_cycles table represents one flat→non-flat→flat lifetime per shard via the standard TemporalMixin SCD2 envelope. A partial unique index uq_pc_shard_open_active enforces at most one open cycle per shard_key at any instant. Five repository methods cover the lifecycle: insert_position_cycle, close_position_cycle (SCD2 close-and-insert), get_open_position_cycle (lookup by shard_key because paper shards embed strategy_tag), flip_position_cycle (TOCTOU-hardened atomic close+open with shard_key match assertion), and update_position_cycle_max_qty (monotonic — silent no-op when the new peak does not strictly exceed the existing one). The repository methods are called from the trader (not from TradeService, which stays sync and pure); ShardState carries trader-owned cache fields active_cycle_public_id and active_cycle_max_qty.

The trader's _sync_fill_to_trade_service samples old_qty before apply_venue_event and new_qty after, then delegates to _sync_position_cycle_on_fill. That helper classifies the transition via TradeService._detect_cycle_transition (returns Literal["open", "close", "flip", "scale_up"] | None with a 1e-12 epsilon to match the existing zero-snap) and issues the matching repository call. Four guards apply: degraded-identity fail-closed when engine.wallet_public_id is falsy, idempotency on open via a get_open_position_cycle pre-check, async instrument_public_id resolution via get_instrument_public_id_by_symbol, and a symmetric DB fallback on close/flip/scale_up paths so a degraded restart cannot reuse a stale cycle row from a prior life of the shard. When a flip's instrument resolution fails, the trader degrades to close-only — the old cycle is closed cleanly, the new leg is left uncovered with an audit-gap warning, and the cache is cleared so subsequent fail-soft paths handle themselves correctly.

_recover_engine_state runs _reconcile_position_cycles as a fourth recovery pass, after engine state is rebuilt and before the trading loop starts. It iterates self.engines (engines, not TradeService, because full replay rebuilds engine.position_qty but not the projection) and handles four cases: recovered flat with a stale open row → close the row; recovered non-flat with a matching-direction open row → hydrate the cache and bump max_qty if downtime scaled-up beyond the stored peak; recovered non-flat with an opposite-direction row → atomic flip via flip_position_cycle (or degrade to close-only on unresolved instrument); recovered non-flat with no row → bootstrap a synthetic cycle. Brackets attach to position_cycle_public_id, not to an order, so a flat reopen does not inherit stale

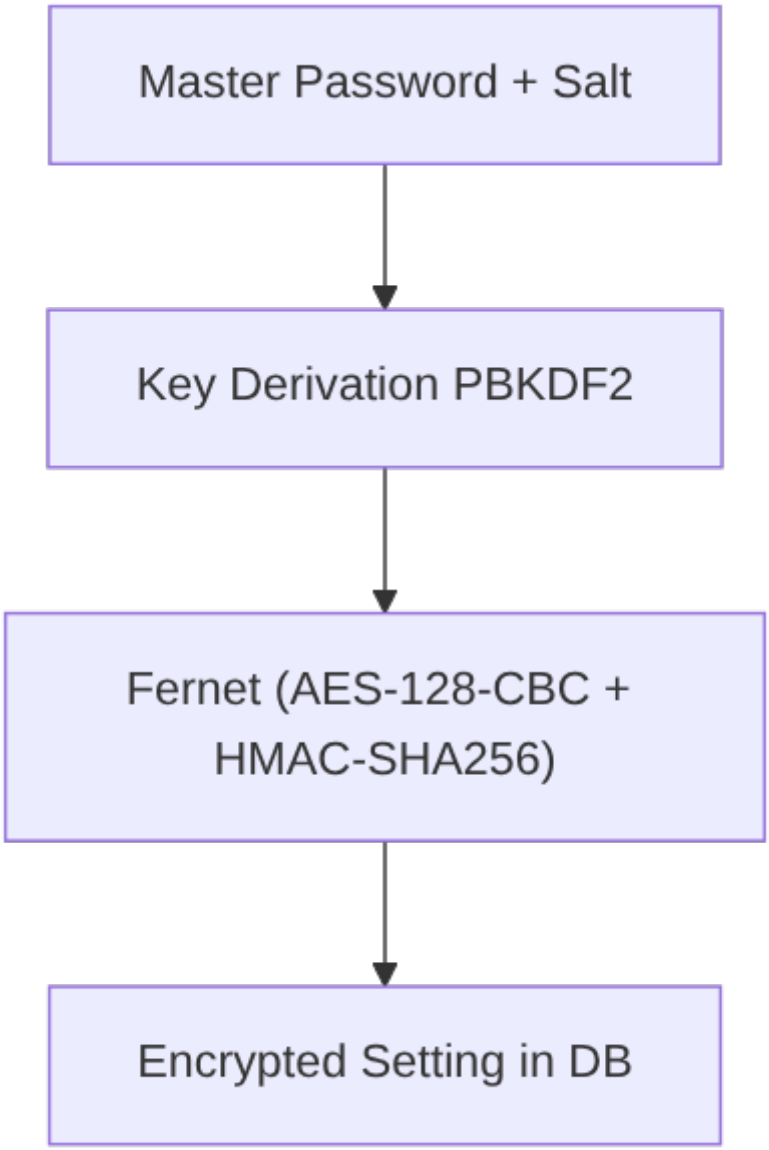
stop levels. Orphan cycles (open rows without a matching engine) can be detected via GET /api/position-cycles/open and closed individually with POST /api/position-cycles/close-orphan or in bulk with POST /api/position-cycles/sweep-orphans.

max_qty is per-cycle, not lifetime. Each position_cycles row tracks the peak absolute quantity reached within its own opened_at → closed_at window. A new cycle on the same shard starts a fresh accumulator at abs(opening_qty); it does not inherit or max against prior cycles' peaks. This matches the bracket-evaluator contract: a bracket attaching to cycle C1 is sized against C1's peak, not against all-time peak. A future UI surfacing max_qty must scope to the individual cycle row; summing or max-ing across cycles would conflate independent trading episodes.

Security

Settings Encryption

Sensitive data (API keys) are encrypted in the database:



Backtesting

The backtesting subsystem runs strategy simulations against historical candle data. It supports both direct repository-backed candle streaming and ZMQ replay through the same strategy processing contract.

Architecture:

- **BacktestConfig** (Pydantic): Strategy, instruments, date range, fill model
- **DirectDbEngine**: Candle-driven simulation loop with time-batched processing
- **ZmqReplayEngine**: Replay mode that publishes historical candles through ReplayPublisher on the same topic shape used by live feeds

- **ResultCollector**: In-memory artifact buffer (signals, trades, equity)
- **BacktestMetrics**: Pure-math metrics computation (Sharpe, Sortino, CAGR, etc.)
- **BacktestRunnerProcess**: ONE_SHOT RegisterableProcess orchestrating lifecycle
- **BacktestRepository**: Separate repo sharing session_factory, SCD2 status transitions

Lifecycle: pending -> running -> completed | failed | cancelled

Boot reconciliation: Orphaned runs (pending/running/cancel_requested) are swept to failed at server startup via reconcile_stale_runs(). This runs only on the normal startup path (skipped in SERVER_API_ONLY mode).

Storage: 7 bitemporal tables (BacktestRun, BacktestEvent, BacktestSignal, BacktestTrade, BacktestEquityPoint, BacktestResult, BacktestComparison) all with TemporalMixin.

Authentication

- JWT tokens via HTTP-only cookies (access: 15min, refresh: 7 days)
- CSRF tokens for mutating requests
- WebSocket tokens (one-time)
- Bcrypt password hashing

Kraken FCM market data (TradFi)

Kraken's FCM platform hosts index-futures, energy, metals, and yield contracts (collective name: "TradFi" in this project). Unlike crypto spot + perpetuals, FCM has **no public order API** — Snapper treats these instruments as observation-only.

Data sources

- REST instrument metadata: iapi.kraken.com/api/internal/markets/all/futures-contracts. Reverse-engineered from pro.kraken.com (requires Origin: https://pro.kraken.com + Referer: https://pro.kraken.com/ headers). Powers the kraken_equities_symbol_updater process.
- REST historical candles: iapi.kraken.com/api/internal/markets/{ws_symbol}/ticker/history. Same Origin/Referer requirement. Drives KrakenEquitiesExchangeClient.get_ohlcv() and the kraken_equities_aggregates_backfill process. Intervals {1, 5, 15, 30, 60, 1440} minutes accepted.
- WebSocket ticks + trades: wss://ws-equities.kraken.com (Kraken Spot WS v2 protocol with asset_class: futures_contract). Feed is delayed ~10 minutes per FCM policy. The outer envelope's delayed: true flag propagates onto every TickData.is_delayed=True so ZMQ subscribers can gate accordingly.

Capability model

Each TradFi symbol is seeded with a SymbolExchangeCapability(exchange="kraken_equities", can_trade=False, can_market_data=True) row. Every order-entry REST route (/api/orders, /api/execution-plans, /api/trailing-stops) calls snapper.server._capability_guard.require_tradable before emitting a TradeCommand — market-data-only submits fail with HTTP 422 error_code=instrument_market_data_only. The AST meta-test at tests/meta/test_order_entry_capability_guard.py enforces that every enumerated submit handler imports and calls the guard.

Frontend UX

GET /api/exchanges/{exchange}/instruments/detail exposes capability-aware rows (InstrumentDetailData — symbol + can_trade + can_market_data + instrument_kind + expiry_at). MarketData.tsx renders a "Market-data only" badge next to the page title when the currently-selected instrument is observation-only and marks every can_trade=false row in the instrument dropdown with the same pill. NewOrderModal.tsx consumes the same endpoint, shows an amber inline notice, suffixes option labels with — market-data only, and disables the submit button. Submit failures branch on APIError.details.error_code via errorMessages.ts.

Cross-asset strategy pattern (informational)

Strategies may subscribe TradFi candles via the normal market.kraken_equities.{symbol}.candles.{timeframe} topic and emit signals whose instrument targets a can_trade=True execution instrument (crypto spot, xStocks). See src/snapper/strategies/examples/tradfi_observe_crypto_execute.py for an illustrative EMA-crossover implementation + activation instructions in the module docstring. The backtest engines process multi-feed batches and route simulated fills to the signalled execution instrument. BacktestConfig.target_execution_exchange selects the fill venue when it differs from the source feed, and both direct-DB and ZMQ replay modes build topics for every configured exchange/instrument feed.

Operational runbook

See docs/operations.md "Kraken Equities (TradFi) market data" for enable-the-feed steps (SQL DB Setting toggles), backfill invocation, quarterly rotation cadence, and troubleshooting (iapi reachability checks, WS disconnect loops).

Backtesting

Snapper ships two backtest execution engines, both producing the same artifacts (signals, trades, equity points) so a strategy author can pick either path with confidence.

Execution modes

The execution_mode field on BacktestConfig selects which engine the runner instantiates:

- direct_db — DirectDbEngine reads candles directly from the repository and feeds them into the strategy in-process. Lowest overhead; recommended for fast iteration.
- zmq_replay — ZmqReplayEngine spins up a private XPUB/XSUB broker on OS-assigned local ports, attaches the strategy as a SUB, and publishes candles via a ReplayPublisher. Exercises the full live message-bus path, so any production wiring bug surfaces in the backtest.

Both engines call the same process_time_batch helper for fill simulation, signal recording, and equity sampling, so artifact parity is structural rather than aspirational.

Choosing between modes

Concern	direct_db	zmq_replay
Speed	fastest (no IPC)	slower (per-candle ZMQ hop)
Fidelity	bypasses message bus	mirrors live wire path
Concurrency	at most one active run (global uq_bt_single_running invariant — see below)	at most one active run (same DB-enforced invariant)
Cancel SLA	bounded by cancel_poll_ms	bounded by cancel_poll_ms

At-most-one-running invariant

Migration 0001_init includes a partial unique index uq_bt_single_running on backtest_runs(status) filtered to status='running' AND known_to=KNOWN_TO_MAX. This is a declarative DB guarantee — a second runner trying to transition pending → running receives IntegrityError at COMMIT, the runner catches it via is_single_running_conflict and transitions to failed with a clear error message. No engine, no broker, no ports consumed for the loser.

The constraint is dialect-aware (PostgreSQL via asyncpg + SQLite via aiosqlite); see snapper.data.backtest_conflict for detection details.

Cancellation

CancelProbe polls backtest_runs.status between time batches (Direct-DB) or per processed candle (ZMQ replay). Throttled by config.cancel_poll_ms (default 500 ms), each DB read bounded by probe_timeout_s (default 1.0 s). A stuck DB logs a warning and the next probe re-tries — cancel detection degrades gracefully under SQLite lock contention rather than hanging behind the driver's 30 s lock timeout.

End-to-end cancel SLA target: ≤ 5 s from the API call POST /api/backtests/{run_id}/cancel (which sets status to cancel_requested) to terminal status cancelled.

start_date warmup gating

Candles before config.start_date feed strategy indicators (so SMA/MACD warm up properly) but do **not** generate fills, signals, or equity points. The process_time_batch helper enforces this with a single if batch[0].open_at < config.start_date: return guard before fill simulation.

Both engines honour the gate identically; the parity test tests/integration/test_backtest_hardening.py::test_warmup_gating_parity_between_engines asserts field-for-field equality.

Signal and fill artifacts

process_time_batch is the single execution/fill path for both engines. For every timestamp batch it updates latest_closes for all candles first, feeds each candle through the strategy, drops pre-start_date signals, then records kept signals/trades and one equity point for the timestamp.

Each kept signal receives a fresh UUID7 public_id before any fill is simulated. When a trade is produced, backtest_trades.signal_public_id carries that same value so Direct-DB and ZMQ replay artifacts are FK-linkable in parity tests. The signal row records:

- signal_time from the candle batch timestamp (open_at);
- signal_type from StrategySignal.side;
- instrument from StrategySignal.instrument;
- price from the resolved target close when fill attribution succeeds, or from StrategySignal.price on the missing-target-close fallback;
- timestamp from the run's snapshot_as_of bus-time anchor.

The only current fill model is market. simulate_market_fill executes at the relevant close price adjusted by slippage_bps; commission is charged in basis points. Buy size is signal_strength * cash / cost_per_unit (default strength 1.0), and a sell flattens the current position quantity. Invalid close prices, zero/negative buy strength, no cash, or a sell with no position return None; the signal is still recorded, but no trade row is written.

When a fill exists, the trade row records:

- executed_at = fill.fill_at, which is the candle open_at timestamp passed into the fill model;
- instrument, side, quantity, price, fee, and pnl from the simulated fill (pnl is populated on sells from realized PnL delta);
- position_after from the portfolio after the fill;
- signal_public_id linking back to the triggering signal;
- timestamp from the same snapshot_as_of bus-time anchor.

ZMQ replay implementation notes

The replay path adds a few primitives the live system does not need:

- **Echo-ack handshake**: the publisher sends a sentinel WARMUP_PUBLIC_ID candle on every expected topic each round; the strategy mixin's _listen_loop ACKs each topic into state.acked_topics and only sets state.subscriber_ready when the set equals state.expected_topics. Up to WARMUP_MAX_RETRIES=5 rounds of WARMUP_READY_TIMEOUT_S=1.0 s each; failure raises BacktestReadinessTimeoutError.
- **Drain coordinator**: counts published vs processed candles. After the publisher streams the last candle it calls mark_done_publishing() and waits on drained.wait() bounded by DRAIN_TIMEOUT_S=10.0 s. Failure raises BacktestDrainTimeoutError with both counters in the message so an operator can tell publisher- stall from strategy-death.
- **Bounded cleanup**: the engine cancels publisher and listen tasks then asyncio.wait(timeout=2.0) — tasks that refuse to terminate are logged as leaks rather than silently kept alive.
- **Shielded DB writes**: the runner wraps every terminal-status DB write (cancelled, failed, fail-event insert) in asyncio.shield so a second cancel arriving mid-write does not interrupt the write coroutine.

Advanced metrics, live progress, comparison

The backtest surface ships three interlocking capabilities:

Advanced metrics

Eight additional metric columns on backtest_results — five promoted from the extra_metrics JSON blob (sortino_ratio, cagr, calmar_ratio, expectancy, avg_trade_pnl) plus three net-new metrics computed in snapper.application.backtest.metrics:

- **max_drawdown_duration_seconds** — longest peak-to-recovery window; unrecovered runs use the final suffix from last peak to end-of-curve. Degenerate input (fewer than two points, flat curve, strictly monotone) returns None + a metric_warning event.
- **exposure_ratio** — fraction of total run duration during which the portfolio held a non-zero position; leading-edge interval attribution. Zero-trade runs with non-zero duration return 0.0 (not degenerate).
- **turnover_ratio** — total notional traded divided by mean equity. Zero-trade returns 0.0; non-positive mean equity returns None + warning.

Read-side fallback at GET /api/backtests/{id} collapses older extra_metrics JSON into the typed slots using explicit is not None coalescing (never Python truthiness — preserves legitimate 0.0 values). The response strips the five promoted names from the emitted extra_metrics so mixed-vintage rows never surface the same metric twice.

Live progress (4-segment WS topic family)

Runner emits BacktestProgressData events on backtest.{wallet_public_id}.{run_public_id}.{event} where event ∈ {started, progress, milestone, completed, failed, cancelled}. The existing ZMQ→WS bridge forwards the envelope to every subscribed client.

- BacktestProgressEvent is the canonical Literal source of truth in snapper.messaging.schemas.data; topic validator, emitter, and frontend handlers all derive from it.
- BacktestProgressEmitter throttles progress at 250 ms, emits milestone events exactly once per 25/50/75 pct bucket (bypassing the throttle so milestones + progress at the same step do not collide), and fires started / terminal events once. Degrades gracefully when total_candles=None by disabling milestones and pinning progress_pct=0.0.
- Per-subscription RBAC: admins subscribe to any wallet prefix; non-admin roles subscribe only to prefixes matching their active_wallet_public_id. Foreign-wallet or bare backtest. subscriptions are denied at the handler before the bridge is touched.
- BacktestProgressData.milestone carries a @model_validator enforcing the cross-field invariant milestone is not None iff event == 'milestone' so a malformed payload cannot reach the frontend chip logic.

Config hash for auto-pair

Every new run persists a config_hash via compute_fingerprint(config, for_pairing=True) — SHA-256 over the 10 pairing-stable fields (strategy_class, instruments, start_date, end_date, initial_balance, strategy_params, timeframe, fill_model, slippage_bps, commission_bps), plus target_execution_exchange when it is set on the config (see "Cross-asset execution" → "Fingerprint + pairing" below — default-None runs keep their exact prior hash). execution_mode, snapshot_as_of, warmup_bars, and buffer_size are explicitly excluded so Direct-DB and ZMQ replay runs on the same config share a hash.

GET /api/backtests?config_hash={hash} returns sibling runs for the auto-pair UI; the repository query is wallet-scoped and indexed on (wallet_public_id, config_hash, timestamp).

Comparison

POST /api/backtests/compare creates (or returns existing) a comparison row; GET /api/backtests/compare/{id} returns the comparison metadata plus the diff **recomputed from current artifact rows** so metric-schema changes never stale a persisted diff. The diff surfaces four shapes:

- metrics_diff — per-name {run_a, run_b, delta, pct} with explicit is not None precedence.
- equity_overlay — full-outer-joined equity samples on point_time with nullable per-leg fields.
- trades_diff — multiset matching on (instrument, executed_at, side, quantized_qty, quantized_price) at 1e-8 precision so IEEE drift collapses but real ticks stay distinct. Common entries carry pnl_a/pnl_b/pnl_delta.
- signals_diff — multiset on (instrument, signal_time, signal_type).

Route ordering: compare endpoints are registered **before** /{run_id} so /compare is not captured as id="compare". Pair is normalised to lexical (min, max) before insert. Duplicate submit is idempotent via SELECT-then-INSERT with an IntegrityError rollback+re-SELECT race recovery path.

No-active-wallet fail-closed

Every backtest read endpoint (list, detail, trades, signals, events, equity, compare) returns **400 no active wallet selected** when principal.active_wallet_public_id is None. WS subscribes respond with a subscription_success frame whose status is denied (or partial when some requested topics succeeded), with the offending topics surfaced in denied_topics. The picker triggers a selectWalletAndRefresh on change that mints a new JWT carrying the chosen wallet claim before swapping the client scope, so REST and WS both authorise against the same wallet after the picker moves.

Cross-asset execution

Cross-asset strategies observe market data on one venue / instrument and emit signals that execute on a *different* venue / instrument (for example TradFiObserveCryptoExecute observes MNQU6-CME candles on kraken_equities and emits BUY/SELL signals on BTC-USD / kraken). The backtest engine supports this through a single config field.

TradFiObserveCryptoExecute is an illustrative, non-runnable example: it is **not** auto-registered, so BacktestConfig.validate_strategy_class rejects the config below as written. To run it, first copy the example module to the strategies package root and decorate it with @register_strategy, or substitute a registered strategy class (RSIReversion, MACDCrossover, or CointegrationPairs). The current registry is queryable at GET /api/backtests/strategy-classes, which returns the sorted StrategyFactory keys accepted as strategy_class values (see api.md):

```
BacktestConfig(
    strategy_class="TradFiObserveCryptoExecute",
    instruments={
        "kraken_equities": ["<MNQU6-CME-public-id>"],
        "kraken": ["<BTC-USD-public-id>"],
    },
    target_execution_exchange="kraken",
    start_date=..., end_date=...,
    wallet_public_id=...,
    initial_balance=10_000.0,
    strategy_params={"fast_period": 12, "slow_period": 26, "min_candles": 30},
)
```

Attribution semantics

When target_execution_exchange is not None, the batch processor substitutes at simulated-fill time:

- target_exchange = str(config.target_execution_exchange) — carries the *venue label* only. It is NOT validated against the feed populating latest_closes[target_instrument]; the caller is responsible for configuring instruments so the target symbol's feed matches the target venue.
- target_instrument = signal.instrument — comes directly from the strategy's StrategySignal, unchanged since pre-cross-asset.
- target_close = latest_closes[target_instrument] — the *price* at which the simulated fill executes. The source feed's close price is no longer used on cross-asset runs.

When target_execution_exchange is None (default), the fill is attributed to event.exchange + signal.instrument. The single-leg emitters (rsi.py, macd.py) always set signal.instrument == event.instrument, so for those strategies the recorded exchange / instrument / price are byte-identical with the pre-v1.2 path — every existing single-feed backtest keeps its exact fingerprint and result rows. CointegrationPairs does not follow

that convention: its on_candle returns a [primary, hedge] pair whose hedge leg targets the *partner* instrument, so that leg is attributed to event.exchange plus the partner symbol and fills at latest_closes[partner] (see "Multi-leg signal groups" below).

Multi-leg signal groups

A strategy callback may return a list[StrategySignal] (a leg group) instead of a single signal. The backtest path handles groups as follows:

- **Fail-closed group preflight** — _handle_candle_data runs the whole group through BaseStrategy._normalize_signal_group before returning anything: every element must be a StrategySignal, no two legs may share an instrument, and every leg must map to a configured output topic. A malformed group raises instead of emitting, so a backtest can never record half a spread.
- **Per-leg fan-out** — process_time_batch pairs each leg with the candle event that produced the group and processes the legs independently: each leg gets its own signal_public_id, fill simulation, and signal/trade rows.
- **Hedge-leg pricing** — the hedge leg fills at latest_closes[partner], the partner instrument's latest close from its own feed, not the observing candle's close. Because process_time_batch updates latest_closes for every candle in the timestamp batch before feeding the strategy, both legs price off same-timestamp data when both feeds emit. A partner symbol absent from latest_closes falls into the missing-target-close policy below.

CointegrationPairs additionally refuses to enter a one-sided position at the source: _build_paired_entry returns None (no signals at all) when the partner leg has no buffered price yet.

Missing-target-close policy

If signal.instrument has no entry in latest_closes (typically during the warmup overlap window where the strategy fires before the target feed has primed), the engine:

1. Skips simulate_market_fill + the trade row (no fill attempted).
2. Increments collector.cross_asset_blocked_fills by one, logged at DEBUG with reason='missing_target_close'.
3. Still records the signal row with price = float(signal.price) (source-close fallback) so downstream analytics see the strategy's intent.

At run end the counter surfaces through BacktestResultInsertRow.extra_metrics["cross_asset_blocked_fills"] *only when positive*. Single-feed runs keep extra_metrics == {} byte-identical with pre-cross-asset behaviour.

Scope limits

Two explicit non-goals in the current implementation:

- **Same-symbol multi-venue** — PortfolioTracker.positions is keyed by instrument only, so running BTC-USD on Kraken Spot + Kraken Futures simultaneously in one backtest is NOT supported. Cross-asset strategies in scope use distinct symbols across feeds.
- **Multi-target cross-asset** — a single target_execution_exchange per run means one strategy cannot emit signals for different target venues inside the same backtest.

Supported: cross-asset via REST

POST /api/backtests accepts an optional target_execution_exchange field on BacktestCreateBody. When set, simulated fills are attributed to that order-capable venue while candles still feed from exchange. When unset, the run stays single-exchange and byte-identical to pre-cross-asset behaviour. The same field round-trips through DB persistence + the rerun endpoint, and surfaces on BacktestRunData for frontend display. Allowed target values: paper / kraken / kraken_futures / walutomat.

Fingerprint + pairing

compute_fingerprint includes target_execution_exchange in the payload **only when non-None**, so default-None runs keep their exact pre-cross-asset hash in both the default and for_pairing=True paths. This preserves dedup cache validity and the auto-pair UI grouping logic in the _resolve_auto_pair resolver in backtest_routes.py. Explicit cross-asset runs (field set to a concrete venue like "kraken") generate distinct fingerprints so they never collide with single-venue baselines.

CLI

Snapper provides a command-line interface built on the Typer framework. All commands are available through the snapper command.

Usage

```
snapper [COMMAND] [OPTIONS]
```

Help for a specific command:

```
snapper [COMMAND] --help
```

Server and Infrastructure

server

Starts the FastAPI server with the web dashboard.

```
snapper server [OPTIONS]
```

Options:

Option	Type	Default	Description
--host	string	from env	Listen address
--port	int	from env	Server port
--reload / --no-reload	bool	from env (SERVER_RELOAD)	Auto-reload for development; explicit flag overrides the env-var setting

Environment:

Set SERVER_API_ONLY=true to skip process autostart while keeping the ZMQ-WebSocket bridge alive (useful when the engine runs as a separate process). Multi-worker uvicorn (multiple FastAPI processes sharing a broker) is supported — see the AI-review fanout dedup contract in docs/architecture.md (Deployment Modes); set SNAPPER_COORDINATOR_INSTANCE_ID + SNAPPER_COORDINATOR_INSTANCE_COUNT per worker (env vars only — snapper server does not expose per-worker CLI flags; the --

instance-id / --instance-count flags in the table below are on snapper trade-zmq) to enable the shared partitioning that prevents duplicate caps-violation WS frames under N>1.

Examples:

```
# Start with default settings (all-in-one dev mode)
snapper server

# Start on all interfaces
snapper server --host 0.0.0.0 --port 8000

# Development mode with auto-reload
snapper server --reload

# API-only mode (engine runs separately)
SERVER_API_ONLY=true snapper server
```

broker

Starts the ZeroMQ XPUB/XSUB broker for message routing.

```
snapper broker [OPTIONS]
```

Options:

Option	Type	Default	Description
--xsub	string	from env	XSUB endpoint (publishers)
--xpub	string	from env	XPUB endpoint (subscribers)

Example:

```
snapper broker --xsub tcp://127.0.0.1:7500 --xpub tcp://127.0.0.1:7501
```

trade-zmq

Starts the central trade runtime coordinator.

The process still runs as the trade-zmq command, but after the trade runtime redesign it also hosts TradeService, BalanceService, the outbox dispatcher, and the reconciliation loops. It writes TradeCommand rows, consumes orders.events.* to keep projections in sync, and persists checkpoints for recovery.

```
snapper trade-zmq [OPTIONS]
```

Options:

Option	Type	Default	Description
--signal-topics	string	signals.	Signal topics to subscribe
--instance-id	int	0 (or $SNAPPER_COORDINATOR_INSTANCE_ID)	Multi-instance partitioning: zero-based id for this coordinator in a multi-instance deployment. CLI flag > env var > default.
--instance-count	int	1 (or $SNAPPER_COORDINATOR_INSTANCE_COUNT)	Multi-instance partitioning: total coordinator instance count across the deployment. Must match across all coordinators.

Example:

```
snapper trade-zmq --signal-topics "signals.paper.BTC-
USD.rsi_btc_1h,signals.kraken.BTC-USD.live"
```

Multi-instance deployment:

```
# Coordinator 0 of 2
snapper trade-zmq --instance-id 0 --instance-count 2

# Coordinator 1 of 2 (separate process, same DB + broker)
snapper trade-zmq --instance-id 1 --instance-count 2
```

Each instance owns ~1/N of the shard_keys via deterministic SHA-256 hashing. Default --instance-count 1 is byte-identical to single-coordinator behavior (every shard owned by the single coordinator).

Multi-instance deployment requires PostgreSQL. With --instance-count greater than 1 on a SQLite backend the coordinator fails fast at startup with a ValueError: SQLite compiles SELECT ... FOR UPDATE to a plain SELECT, so cross-instance row claims cannot serialize. A single-instance SQLite coordinator is allowed but logs a startup warning about the same locking caveat.

See docs/operations.md for systemd template unit + scale-up/down/crash-recovery runbooks.

executor

Starts the standalone order execution service for a selected exchange. Process-managed deployments use per-wallet executor instances instead: bare executor_<exchange> process configs are templates, and the launcher expands active wallet_credentials rows into runnable executor_<exchange>_w<wallet_short> instances. Start/stop the generated per-wallet instance names in the process UI/API; starting a bare executor template is rejected.

```
snapper executor [OPTIONS]
```

Options:

Option	Type	Default	Description
-e, --exchange	string	kraken	Exchange (kraken, walutomat)

wallet_short is the last 12 lowercase hex characters of the wallet UUID7. Each per-wallet instance subscribes to the same exchange command prefix as the template, then filters by wallet_public_id before loading credentials or placing orders.

Examples:

```
# Kraken executor
snapper executor -e kraken

# Walutomat executor
snapper executor --exchange walutomat
```

feed

Starts the direct Kraken market data publisher helper via WebSocket. For the dedicated feed-container topology, use snapper feed-engine instead; it starts every enabled market-data publisher from the process registry as a supervised subprocess.

```
snapper feed [OPTIONS]
```

Options:

Option	Type	Default	Description
--symbols	string	BTC-USD	Comma-separated native symbols passed to the Kraken publisher. Symbols must be in native dash format (e.g. BTC-USD); slash-format WebSocket symbols are skipped as unknown.

Example:

```
snapper feed --symbols "BTC-USD,ETH-USD"
```

feed-engine

Runs the dedicated feed-container entrypoint. It syncs the process registry, starts enabled market-data publishers as OS subprocesses, and exits non-zero if any supervised publisher crashes so the orchestrator restarts the feed container. It does not start a broker; publisher subprocesses connect to the backend's configured ZMQ_BROKER_* endpoints. Per-process RSS/CPU summary events are emitted best-effort when the backend broker is reachable, but metrics-publisher setup failure does not abort feed startup.

```
snapper feed-engine
```

zmq-logger

Monitors and logs ZMQ message traffic.

```
snapper zmq-logger [OPTIONS]
```

Options:

Option	Type	Default	Description
--payload/--no-payload	bool	false	Log full payload
--file/--no-file	bool	true	Write to audit file
--audit-file	string	data/zmq_audit.jsonl	Audit file path
--max-length	int	200	Max payload preview length

Example:

```
snapper zmq-logger --payload --audit-file logs/zmq.jsonl
```

egress

Runs the snapper-egress sidecar from the unified Snapper image. Additional arguments after snapper egress are forwarded verbatim to the egress entrypoint's argparse parser. This is the compose dispatch path for ENTRYPOINT ["snapper"] plus command: ["egress"]; the module entrypoint python -m snapper.egress remains supported for bare-shell invocations. The Typer wrapper propagates the egress entrypoint's integer return code as the process exit code.

```
snapper egress [--instance-id snapper-egress-prod]
```

See snapper-egress.md for tunnel settings and verification steps.

Database

db-init

Initializes the database schema.

```
snapper db-init
```

db-upgrade

Runs Alembic migrations to a specified revision.

```
snapper db-upgrade [OPTIONS]
```

Options:

Option	Type	Default	Description
--revision	string	head	Target revision

Examples:

```
# Migrate to latest version
snapper db-upgrade

# Migrate to specific revision
snapper db-upgrade --revision abc123
```

db-downgrade

Rolls back Alembic migrations.

```
snapper db-downgrade [OPTIONS]
```

Options:

Option	Type	Default	Description
--revision	string	-1	Steps to roll back

Examples:

```
# Roll back one migration
snapper db-downgrade

# Roll back to specific revision
snapper db-downgrade --revision base
```

db-seed

Seeds the database with environment-specific data from TOML profiles. Automatically run by make migrate-dev and make migrate-prod.

```
snapper db-seed [OPTIONS]
```

Options:

Option	Type	Default	Description
--profile	string	dev	Seed profile name (e.g. dev, prod)

Users

init-admin

Creates the initial administrator user.

```
snapper init-admin [OPTIONS]
```

Options:

Option	Type	Default	Description
--username	string	admin	Username
--password	string	*(prompted, hidden, confirmed)*	Password — no default; the command prompts with hidden input and confirmation when the flag is omitted, so the value never lands in shell history.

Example:

```
# Prompts for password with hidden input + confirmation; no inline value
# means nothing lands in shell history
snapper init-admin --username admin
```

list-users

Displays a list of all system users.

```
snapper list-users
```

reset-password

Resets a user's password.

```
snapper reset-password USERNAME [OPTIONS]
```

Arguments:

- USERNAME — Username (required)

Options:

Option	Type	Default	Description
--new-password	string	(prompt)	New password

Example:

```
# Prompts for the new password with hidden input + confirmation; the secret
# never lands in shell history when --new-password is omitted
snapper reset-password john
```

Symbol Synchronization

update-kraken-symbols

Synchronizes symbol mappings from the Kraken API.

```
snapper update-kraken-symbols [OPTIONS]
```

Options:

Option	Type	Default	Description
-f, --force	bool	false	Force update

update-kraken-futures-symbols

Syncs Kraken Futures crypto symbol mappings from the exchange API.

```
snapper update-kraken-futures-symbols [OPTIONS]
```

Options:

Option	Type	Default	Description
-f, --force	bool	false	Force update

update-kraken-equities-symbols

Syncs Kraken Equities (FCM Futures) symbol mappings from the exchange API.

```
snapper update-kraken-equities-symbols [OPTIONS]
```

Options:

Option	Type	Default	Description
-f, --force	bool	false	Force update

update-walutomat-symbols

Synchronizes symbol mappings from the Walutomat API.

```
snapper update-walutomat-symbols [OPTIONS]
```

Options:

Option	Type	Default	Description
-f, --force	bool	false	Force update

update-polygon-symbols

Synchronizes symbol mappings from the Polygon.io API. By default it updates existing rows only; pass --insert-new to insert symbols that are not yet in the database.

```
snapper update-polygon-symbols [OPTIONS]
```

Options:

Option	Type	Default	Description
-f, --force	bool	false	Force update
--insert-new	bool	false	Insert new symbols

Note: This operation may take 10-15 minutes (44k+ symbols).

update-underlyings

Syncs underlying asset definitions from YAML to the database. It matches active instruments to underlyings via pattern rules, upserts mappings, and applies YAML-fallback instrument_type / expiry_override values to InstrumentSpec rows when API-sourced values are NULL.

```
snapper update-underlyings [OPTIONS]
```

Option	Type	Default	Description
-f, --force	bool	false	Bypass safety guard for stale cleanup

reconcile-symbol-aliases

Closes active symbol_aliases rows whose owning capability row is no longer tradeable and no longer market-data-enabled. The command is idempotent and can be scoped to one exchange.

```
snapper reconcile-symbol-aliases [OPTIONS]
```

Option	Type	Default	Description
-e, --exchange	string	all	Exchange to reconcile (kraken, kraken_futures, kraken_equities, walutomat, polygon, or all)

Market Snapshots

update-kraken-market-snapshot

Updates Kraken market snapshots with current prices.

```
snapper update-kraken-market-snapshot
```

update-kraken-futures-market-snapshot

Updates Kraken Futures market snapshots with current prices.

```
snapper update-kraken-futures-market-snapshot
```

update-kraken-equities-market-snapshot

Updates Kraken Equities market snapshots with current prices.

```
snapper update-kraken-equities-market-snapshot
```

update-kraken-futures-funding-rates

Backfills historical funding rates for Kraken Futures perpetuals into the funding_rates table. Duplicates are silently skipped via the partial unique index.

```
snapper update-kraken-futures-funding-rates [OPTIONS]
```

Options:

Option	Type	Default	Description
--symbol / -s	list[str]	None	One or more native perpetual symbols (-s BTC-USD-PERP -s ETH-USD-PERP)
--all	bool	false	Backfill every mapped Kraken Futures perpetual

update-walutomat-market-snapshot

Updates Walutomat market snapshots with current prices.

```
snapper update-walutomat-market-snapshot
```

Data Backfill

polygon-backfill-aggregates

Downloads historical aggregated data from the Polygon.io API to the on-disk CSV cache. This command is download-only: it writes CSV files and never touches the database. Populating the candles table is the separate second step, handled by polygon-load-csv.

The full workflow is two steps:

1. polygon-backfill-aggregates downloads candles into the CSV cache.
2. polygon-load-csv reads that cache and upserts the candles into the database (no Polygon API calls).

```
snapper polygon-backfill-aggregates [OPTIONS]
```

Options:

Option	Type	Default	Description
-s, --symbol	string[]	from settings	Symbols to download
--all	bool	false	All mapped symbols
-m, --multiplier	int	1	Timeframe multiplier
-t, --timespan	string	minute	Timespan (minute, hour, day)
-d, --days	int	None (effective default 30)	Days back
--resume/--no-resume	bool	true	Skip days already cached on disk
--csv/--no-csv	bool	true	Save to CSV files

Examples:

```
# Download specific symbols to the CSV cache
snapper polygon-backfill-aggregates -s AAPL -s MSFT --days 365

# Download all mapped symbols
snapper polygon-backfill-aggregates --all --timespan day

# Hourly download, skipping already-cached days
snapper polygon-backfill-aggregates -s SPY -t hour -m 1 --resume

# Then load the downloaded cache into the database
snapper polygon-load-csv --all --timespan day
```

polygon-load-csv

Loads candles from the on-disk Polygon CSV cache into the database. This is the cache-only counterpart to polygon-backfill-aggregates: it reads the CSV files produced by that command and upserts the candles into the candles table. It never contacts the Polygon API.

Run this after polygon-backfill-aggregates has populated the cache. Archive-symbol cache directories with no current Symbol identity are skipped with a warning, so a stale cache directory never aborts the load.

When neither --symbol nor --all is given, the symbols default to the settings-configured Polygon instruments (the instruments setting). The wildcard sentinel ["*"] in that setting is treated like --all and loads every cached archive symbol; an explicit settings list loads only those symbols.

```
snapper polygon-load-csv [OPTIONS]
```

Options:

Option	Type	Default	Description
-s, --symbol	string[]	from settings	Symbols to load (native or Polygon format)
--all	bool	false	Load every archive symbol present in the cache
-t, --timespan	string	day	Timespan subtree to read (minute, hour, day)
--since	string	None	Earliest day to load, inclusive (YYYY-MM-DD)
--until	string	None	Latest day to load, inclusive (YYYY-MM-DD)

Examples:

```
# Load every cached minute symbol into the database
snapper polygon-load-csv --all --timespan minute

# Load a single symbol within a date window
snapper polygon-load-csv -s X:BTCUSD --since 2024-01-01 --until 2024-01-31
```

polygon-load-grouped-candles

Loads the on-disk Polygon **grouped-daily** cache into the database as 1d candles tagged source='native', complete=True. This is the cache-only counterpart to polygon-backfill-grouped (which only writes CSV): it reads the grouped-daily cache — the same corpus the strategy warmup reads — and upserts each leg's days into the candles table so the persisted plane holds the daily history the single-source candle read path and the DB-first warmup depend on. It never contacts the Polygon API.

--exchange is **required**: it is the venue the persisted 1d bars live under, which **must** be the same venue the leg's live read and warmup resolve (e.g. kraken for FET/RENDER) — *not* polygon. Instrument identity is per (symbol, exchange); live synthesized higher-TF bars persist under the live publisher's venue, so a Polygon-keyed backfill would be an orphaned plane that no live read or warmup ever sees. polygon here is only the CSV cache segment that supplies the OHLCV.

--cut-date is **required**: it is the first UTC day that synthesized live persistence may own. The backfill writes only days strictly *before* it, so under one venue native backfilled history and synthesized live bars never share a bar key (the candle unique key excludes source). Before writing, an ownership preflight — reading back under the same venue — fails closed if the persisted plane already holds a synthesized 1d row before the cut date or a native 1d row at/after it, so the operator resolves any overlap rather than silently version-thrashing.

When neither --symbol nor --all is given, the symbols default to the settings-configured Polygon instruments; the wildcard sentinel ["*"] is treated like --all and enumerates every Polygon-mapped native symbol.

```
snapper polygon-load-grouped-candles --exchange VENUE --cut-date YYYY-MM-DD
[OPTIONS]
```

Options:

Option	Type	Default	Description
-e, --exchange	string	*required*	Venue the persisted bars live under (e.g. kraken); the leg's live-read/warmup venue, never polygon
--cut-date	string	*required*	First UTC day synthesis may own (YYYY-MM-DD); writes only days before it
-s, --symbol	string[]	from settings	Native symbols to load
--all	bool	false	Load every Polygon-mapped native symbol
--lookback-days	int	800	Calendar-day cap on the backward cache walk per symbol

Examples:

```
# Load FET/RENDER daily history (under the kraken venue) before the cutover
boundary
snapper polygon-load-grouped-candles --exchange kraken --cut-date 2026-06-16 -s
FET-USD -s RENDER-USD

# Or via Make (EXCHANGE and CUT_DATE are required; unset fails fast)
make run-polygon-grouped-candles EXCHANGE=kraken CUT_DATE=2026-06-16
```

verify-candle-coverage

Read-only gate that verifies the persisted candles plane is ready to be the single source for >1m reads before the read-path cutover. For each (symbol, timeframe) under --exchange it RANGE-reads an explicit canonical window and checks the FULL expected slot

grid (not just the newest rows) is fresh, gap-free, complete and correctly-tagged — 1d is native strictly before --cut-date and synthesized at/after, every other higher timeframe is synthesized. Exits 1 (and prints the failing (symbol, timeframe) reasons) when any pair is incomplete, so it can gate the cutover and the DB-first warmup; exits 0 when every pair passes.

It reads under the live venue — the same --exchange the read path and warmup resolve; passing -e polygon is rejected (that plane is orphaned). For 1d the window is extended back across the --cut-date seam, so the §4e invariant is enforced directly: a [cut_date, publisher_start) gap (synthesis not yet persisting from the cut) surfaces as a missing-bar failure rather than silently staling the warmup. --min-bars is the window DEPTH verified — set it to the consumer's required lookback (e.g. the warmup depth) to verify that far back.

The derive_snaps OHLCV-parity rung (the "no derive regression" check) needs a genuinely live, SUB-socket-warm cache as an independent witness, so the standalone CLI runs **structural checks only**; parity is deferred to an in-process slice-4 cutover check.

```
snapper verify-candle-coverage --exchange VENUE --cut-date YYYY-MM-DD [OPTIONS]
```

Options:

Option	Type	Default	Description
-e, --exchange	string	*required*	Live venue the persisted bars + read path resolve under
--cut-date	string	*required*	Synthesis ownership boundary (YYYY-MM-DD); decides 1d provenance
-s, --symbol	string[]	from settings	Native symbols to verify
-t, --timeframe	string[]	5m 15m 30m 1h 4h 1d	Timeframes to verify
--min-bars	int	30	Window depth (canonical slots) verified per pair
--writer-lag-seconds	int	120	Grace seconds for the writer to flush the just-closed bar

Examples:

```
# Gate the FET/RENDER plane before the single-source cutover
snapper verify-candle-coverage --exchange kraken --cut-date 2026-06-16 -s FET-USD -s RENDER-USD

# Verify only the daily plane
snapper verify-candle-coverage -e kraken --cut-date 2026-06-16 -s FET-USD -t 1d
```

kraken-futures-backfill-candles

Backfills historical OHLCV candles for Kraken Futures perpetuals.

```
snapper kraken-futures-backfill-candles [OPTIONS]
```

Options:

Option	Type	Default	Description
-s, --symbol	str[]	None	Native symbols to backfill (e.g. BTC-USD-PERP)
--all	bool	false	Backfill all mapped Kraken Futures symbols
-t, --timeframe	str	1h	Candle interval (1m, 1h, 4h, 1d)
-d, --days	int	90	Days back to fetch
--resume / --no-resume	bool	--resume	Resume from latest stored candle

kraken-equities-backfill-candles

Backfills historical OHLCV candles for Kraken Equities (TradFi FCM) contracts via the internal iapi.kraken.com ticker-history endpoint. Response is ~10-minute delayed per FCM policy. Per-chunk upstream failures raise RuntimeError so outages are distinguishable from legitimately-empty candle windows.

```
snapper kraken-equities-backfill-candles [OPTIONS]
```

Options:

Option	Type	Default	Description
-s, --symbol	str[]	None	Native symbols to backfill (e.g. MNQM6-CME)
--all	bool	false	Backfill all mapped Kraken Equities symbols
-t, --timeframe	str	1h	Candle interval (1m, 5m, 15m, 30m, 1h, 1d)
-d, --days	int	30	Days back to fetch
--resume / --no-resume	bool	--resume	Resume from latest stored candle

build-continuous

Builds and displays a continuous-contract series for an underlying.

```
snapper build-continuous TICKER EXCHANGE CONTRACT_FAMILY [OPTIONS]
```

Arguments:

Argument	Description
TICKER	Underlying asset ticker (e.g. SPX, GOLD)
EXCHANGE	Exchange to source contracts from
CONTRACT_FAMILY	Product root (e.g. ES, GC)

Options:

Option	Type	Default	Description
-t, --timeframe	str	1d	Candle timeframe
-m, --method	str	panama	Adjustment method (unadjusted, ratio, panama)
--start	str	365 days ago	Series start time (ISO)
--end	str	now	Series end time (ISO)
--rollover-days	int	0	Days before expiry to roll (0..365)

polygon-backfill-grouped

Downloads grouped daily data from the Polygon.io API to the CSV cache. Like polygon-backfill-aggregates, this is download-only and writes CSV files without touching the database.

```
snapper polygon-backfill-grouped [OPTIONS]
```

Options:

Option	Type	Default	Description
--market / -m	str	crypto	Market type (crypto, stocks, fx)
--days / -d	int	3	Number of recent days to fetch
--locale / -l	str	global	Market locale (global, us)
--csv / --no-csv	flag	--csv	Save data to CSV.gz files
--adjusted / --unadjusted	flag	--adjusted	Use adjusted prices

Data Archive

archive

Export data to CSV archive files. Supports candle cache projection, append-only event tables (ticks, trades, signals, executions, telemetry, control), and SCD2 state tables (orders, positions, instruments, instrument_specs, settings, symbols, symbol_aliases, symbol_exchange_capabilities, users, user_login_events, market_snapshots, process_runs, underlying_assets, instrument_underlying_mappings, continuous_contract_configs). Merges with existing files and deduplicates rows.

```
snapper archive [OPTIONS]
```

Options:

Option	Type	Default	Description
--table	TEXT	candles	Table to archive. Accepts candles, candles-audit, any event table (ticks, trades, signals, executions, telemetry, control), or any state table (see the section intro for the full list).
--exchange	TEXT	None	Exchange filter (e.g. polygon, kraken)
--symbol	TEXT	None	Native symbol filter (e.g. BTC-USD), resolved to stable archive_symbol
--timeframe	TEXT	1m	Candle timeframe (1m, 5m, 1h, 1d) — candles and candles-audit only
--day	str	None	Single day to archive (YYYY-MM-DD; parsed in the handler)
--from	str	None	Start of date range (YYYY-MM-DD; parsed in the handler)
--to	str	None	End of date range (YYYY-MM-DD; parsed in the handler)
--dry-run	FLAG	False	Report counts without writing files
--purge	FLAG	False	Delete exported rows from DB after writing (event tables, plus candles-audit/state tables when combined with --closed-only)
--closed-only	FLAG	False	Only closed SCD2 versions (candles-audit and state tables)
--output-dir	str	data	Base output directory

Output structure:

```
Candles:  {output_dir}/{exchange}/cache/{timespan}/{archive_symbol}/{year}/
{date}.csv
Events:   {output_dir}/archive/{table}/{exchange}/{archive_symbol}/{year}/
{date}.csv
Flat:     {output_dir}/archive/{table}/{year}/{date}.csv  (telemetry, control)
```

Examples:

```
snapper archive --day 2024-03-15 --exchange polygon --symbol BTC-USD
snapper archive --from 2024-01-01 --to 2024-03-31 --dry-run
snapper archive --day 2024-03-15 --timeframe 1d
snapper archive --table ticks --day 2024-03-15 --exchange polygon
snapper archive --table trades --from 2024-01-01 --to 2024-03-31 --purge
snapper archive --table control --day 2024-03-15
snapper archive --table candles-audit --day 2024-03-15 --timeframe 1m
snapper archive --table candles-audit --day 2024-03-15 --closed-only --purge
snapper archive --table orders --day 2024-03-15
snapper archive --table symbols --day 2024-03-15 --closed-only --purge
snapper archive --table instruments --day 2024-03-15
```

Notes:

- --symbol resolves the current native_symbol to the stable archive_symbol via the Symbol anchor row. After a symbol rename, the CLI argument and the output directory may differ.
- --day or --from/--to is required.
- Daily timespan (1d) produces monthly CSV files to match Polygon layout.
- --purge is supported for event tables, plus candles-audit and state tables when combined with --closed-only (which protects active rows); it is **not** supported for the candle cache export.
- Event archive CSV files include full temporal metadata (public_id, timestamp, known_to, session_id, sequence_id).
- Merge/dedup for events uses (public_id, timestamp, known_to) as key.
- Executions are partitioned by exchange/archive_symbol via order -> instrument.
- candles-audit exports all SCD2 versions (closed + active) grouped by open_at date. --purge requires --closed-only to protect active rows.

restore

Restore archived CSV data back into the database. Reads CSV files exported by snapper archive and inserts rows, deduplicating against existing data by (public_id, timestamp, known_to).

```
snapper restore [OPTIONS]
```

Options:

Option	Type	Default	Description
--table	TEXT	(required)	Table name to restore into
--file	str	None	Single CSV file to restore

Option	Type	Default	Description
--dir	str	None	Directory to scan recursively for CSV files
--source	TEXT	audit	Restore mode (audit for full history)

Examples:

```
snapper restore --table ticks --file data/archive/ticks/polygon/BTC-USD/
2024/2024-01-01.csv
snapper restore --table settings --dir data/archive/settings/
snapper restore --table candles --dir data/archive/candles/polygon/BTC-USD/
```

Notes:

- At least one of --file or --dir is required.
- Rows already present in DB (matching public_id + timestamp + known_to) are skipped.
- Audit restore inserts all temporal columns exactly as exported.

Encryption Management

settings-rotate-encryption

Rotates encryption keys for settings in the database.

```
snapper settings-rotate-encryption [OPTIONS]
```

Options:

Option	Type	Default	Description
--new-password	string	*(prompted, hidden, confirmed)*	New master password — no default; the command prompts with hidden input and confirmation when the flag is omitted, so the value never lands in shell history. Inline --new-password X is still accepted for non-interactive automation but the same shell-history caveats apply.
--old-password	string	from env	Current master password (optional; reads from env/ bootstrap when omitted). Avoid inline use — passing a secret on the command line leaks it to shell history.
--dry-run	bool	false	Show changes without applying

Example:

```
# Simulate rotation — prompts hidden for the new master password
snapper settings-rotate-encryption --dry-run

# Actual rotation — same prompt flow, then re-encrypts every encrypted setting
snapper settings-rotate-encryption
```

The CLI prompts for --new-password with hidden input and confirmation when the flag is omitted (preferred), so the master password never lands in shell history, scrollback, or CI logs. After a successful rotation the command prints only a reminder to update MASTER_PASSWORD in .env — it does NOT echo the value you entered. Inline --new-password X is still accepted for non-interactive automation but the same shell-history caveats apply.

Example Workflows

New Installation Initialization

```
# 1. Initialize database
snapper db-init

# 2. Create administrator (prompts for password — never put it on the cmdline)
snapper init-admin --username admin

# 3. Synchronize symbols
snapper update-kraken-symbols
snapper update-polygon-symbols

# 4. Start server
snapper server
```

Full Trading System Startup

```
# Terminal 1: ZMQ Broker
snapper broker

# Terminal 2: Data feed
snapper feed --symbols "BTC-USD,ETH-USD"

# Terminal 3: Executor
snapper executor -e kraken

# Terminal 4: Trade runtime
snapper trade-zmq

# Terminal 5: Server with dashboard
SERVER_API_ONLY=true snapper server
```

Historical Data Backfill

Backfilling Polygon candles is a two-step workflow: download to the CSV cache, then load the cache into the database.

```
# Step 1 - download last 30 days for SPY to the CSV cache (no DB write)
snapper polygon-backfill-aggregates -s SPY --days 30 --timespan minute

# Step 1 - download daily data for all symbols to the CSV cache
snapper polygon-backfill-aggregates --all --timespan day --days 365

# Step 2 - load the downloaded CSV cache into the database (no API calls)
snapper polygon-load-csv --all --timespan day
```

Notifications

notify

Long-running iOS Push Foundation sidecar (ZMQ alerts.* → APNs HTTP/2). Subscribes to the alerts. ZMQ prefix, fans out each received AlertEventData to the target user's active devices via the outbox-backed NotifySidecar, and retries server/throttled failures on its own 30-second scheduler. Configuration is read from the apns_* settings (provisioned through the seed profile).

```
snapper notify
```

Run under systemd / K8s with Restart=always — the sidecar exits only on unhandled errors or SIGINT; the outbox drain on the next start recovers any queued deliveries left behind.

AI Integration

dev-mint-pat

Mints a long-lived AI delegate JWT into data/dev-pat.json (mode 0600) for the local Snapper MCP bridge. See ai-integration.md for the wider Claude Code / Cursor / Windsurf wire-up.

```
snapper dev-mint-pat [OPTIONS]
```

Options:

Option	Env var	Default	Description
--base-url URL	SNAPPER_DEV_BASE_URL	http://localhost:8000	Base URL of the running backend
--admin-username USER	SNAPPER_DEV_ADMIN_USERNAME	None (resolved from seed profiles, mcp then dev)	Admin login
--admin-password PASS	SNAPPER_DEV_ADMIN_PASSWORD	None (resolved from seed profiles, mcp then dev)	Admin password
--output PATH	SNAPPER_DEV_PAT_OUTPUT	data/dev-pat.json	Output file path
--label LABEL	—	Local Dev MCP	Delegate label

The script drives the production REST endpoints (POST /api/auth/login + POST /api/ai-delegates) so the seeded delegate exercises the same code path the AI Integration UI uses. Tokens never appear in stderr — HTTP error bodies are passed through a JWT-shape redaction helper before printing.

Backtesting

backtest-run

Run a backtest synchronously. Creates a DB record, executes the engine in-process, and persists results (signals, trades, equity, metrics).

```
snapper backtest-run \
    --strategy MACDCrossover \
    --instrument BTC-USD \
    --exchange kraken \
    --start 2026-01-01 \
    --end 2026-06-01 \
    --timeframe 1h \
    --initial-cash 10000 \
    --params '{"fast": 12, "slow": 26, "signal_period": 9}'
```

Options:

Option	Type	Default	Description
--strategy	str	required	Registered strategy class name
--instrument	str	required	Native instrument symbol

Option	Type	Default	Description
--exchange	str	required	Exchange name
--start	str	required	Start date (ISO format)
--end	str	required	End date (ISO format)
--timeframe	str	1h	Candle timeframe
--initial-cash	float	10000	Starting cash balance
--wallet	str	cli	Wallet public ID
--params	str	{}	Strategy params as JSON
--execution-mode	str	direct_db	Engine type (direct_db or zmq_replay)
--fill-model	str	market	Fill simulation model
--slippage-bps	float	0.0	Per-fill slippage in basis points
--commission-bps	float	0.0	Per-fill commission in basis points

--execution-mode is persisted on the run, but the CLI computes the pairing config hash with execution mode excluded and currently instantiates DirectDbEngine directly. Use POST /api/backtests for a run that actually executes through zmq_replay.

backtest-list

List backtest runs with optional filters.

```
snapper backtest-list --status completed --limit 10
```

Options:

Option	Type	Default	Description
--strategy	str	None	Filter by strategy name
--status	str	None	Filter by status (pending, running, completed, failed, cancel_requested, cancelled)
--wallet	str	None	Filter by wallet public ID
--limit	int	20	Maximum number of rows to return

backtest-cancel

Cancel a pending or running backtest run.

```
snapper backtest-cancel <run-public-id>
```

backtest-rerun

Re-run a CLI-compatible backtest with the fields exposed by backtest-run, including strategy params. The CLI rerun path does not preserve target_execution_exchange; use the API/process runner for cross-asset reruns.

```
snapper backtest-rerun <run-public-id>
```

Configuration

Snapper's configuration system is based on two layers: environment variables (bootstrap) and database settings.

Environment Variables

Settings loaded from .env file or environment variables.

Database

Variable	Default	Description
DB_URL	sqlite+aiosqlite:///./data/snapper.db	SQLAlchemy connection URL

Database URL examples:

```
# SQLite (development default)
DB_URL=sqlite+aiosqlite:///./data/snapper.db

# PostgreSQL (production)
DB_URL=postgresql+asyncpg://user:password@localhost:5432/snapper
```

Encryption

Variable	Default	Description
SNAPPER_ENV	development	Deployment environment: development, dev, test, testing, and ci allow local placeholders; production, prod, and staging refuse placeholder secrets at startup/runtime. Any other value fails startup; matching is case-insensitive
MASTER_PASSWORD	snapper_default_master_password_v1	Password for encrypting settings in database (salt derived automatically)

Important: Set SNAPPER_ENV=production and change default secret values in production. Production-like modes fail fast when MASTER_PASSWORD, auth_secret_key, or csrf_secret_key still use development placeholders.

HTTP Server

Variable	Default	Description
SERVER_HOST	127.0.0.1	Listen address
SERVER_PORT	8000	Server port
SERVER_RELOAD	false	Auto-reload for development
SERVER_PROXY_HEADERS	true	Enable proxy header parsing in uvicorn
SERVER_FORWARDED_ALLOW_IPS	127.0.0.1	Trusted proxy IPs/CIDRs for forwarded headers
SERVER_API_ONLY	false	Skip process autostart; serve API + WS bridge only (separate-engine boots). Multi-worker uvicorn (multiple FastAPI processes sharing a broker) is supported — see docs/architecture.md Deployment Modes for the AI-review fanout dedup contract under N>1. Set SNAPPER_COORDINATOR_INSTANCE_ID + SNAPPER_COORDINATOR_INSTANCE_COUNT per worker to enable the shared partitioning.
PROCESS_AUTOSTART_PROFILE	all	Select which enabled process configs this node starts: all for single-container/dev, api for backend-without-market-publishers, feed for the dedicated feed container.
TELEMETRY_RECORDING_ENABLED	false	Record pings, heartbeats, and GET reads to telemetry table. High volume — enable for debugging only

The Default column reflects the Pydantic defaults on BootstrapSettingsLoader (src/snapper/config/bootstrap.py) for the server / encryption / ZMQ / coordinator / trade-safety rows. The observability rows further down (SYSTEM_METRICS_*, RETENTION_*, DB_METRICS_*, DB_POOL_*, SNAPPER_*_PROBE) are NOT loaded through that class. application/system_metrics/snapshotter.py and application/db_stats/snapshotter.py read their env vars directly via os.environ. The retention env vars are read in two places: application/retention/scheduler.py (RETENTION_INTERVAL_SECONDS, RETENTION_DISABLED, RETENTION_OUTPUT_DIR) and application/retention/service.py (RETENTION_DRY_RUN), with fallback resolvers defined in application/retention/policies.py. The defaults shown for those rows reflect the per-module fallbacks.

The repo-root .env.example intentionally ships with Docker-friendly overrides for two server values: SERVER_HOST=0.0.0.0 (so the dev server binds inside containers) and SERVER_FORWARDED_ALLOW_IPS=127.0.0.1,172.17.0.1 (so requests forwarded by the default Docker bridge are accepted). Caveat: the make run-server / make dev-backend

targets in the Makefile pass --host 0.0.0.0 on the command line, which wins over the .env value via Typer's host or s.server_host resolution — so editing .env only changes SERVER_HOST for direct snapper server invocations (no --host flag). SERVER_FORWARDED_ALLOW_IPS is honored on every server-start path because no CLI flag overrides it; make migrate-dev does not bind either value.

ZeroMQ

Variable	Default	Description
ZMQ_BROKER_XSUB	tcp://127.0.0.1:7500	XSUB endpoint (publishers connect here)
ZMQ_BROKER_XPUB	tcp://127.0.0.1:7501	XPUB endpoint (subscribers connect here)
ZMQ_BROKER_BIND_XSUB	empty	Optional broker bind endpoint for XSUB. Empty means bind ZMQ_BROKER_XSUB; cross-container deployments set this to tcp://0.0.0.0:7500.
ZMQ_BROKER_BIND_XPUB	empty	Optional broker bind endpoint for XPUB. Empty means bind ZMQ_BROKER_XPUB; cross-container deployments set this to tcp://0.0.0.0:7501.

Bind endpoints exist because ZMQ bind() requires a local interface while cross-container clients connect through the Docker service name. In the compose split, the backend broker binds 0.0.0.0 and both the backend and snapper-feed connect to tcp://snapper:7500/7501.

Database Engine Pool

These optional variables are read directly by the repository engine factory and only affect PostgreSQL engines.

Variable	Default	Description
DB_POOL_SIZE	SQLAlchemy default	Per-process pool size.
DB_MAX_OVERFLOW	SQLAlchemy default	Per-process overflow connections.

Set them on the dedicated feed container when publisher subprocesses would otherwise multiply the default pool toward PostgreSQL max_connections.

Coordinator Sharding

The trade runtime supports horizontal sharding via these variables. With SNAPPER_COORDINATOR_INSTANCE_COUNT > 1 each instance owns a partition of the active wallets / outbox rows, enabling multi-worker uvicorn deployments and parallel trade-zmq instances. Multi-instance coordinator deployments require PostgreSQL; startup fails fast on SQLite when SNAPPER_COORDINATOR_INSTANCE_COUNT > 1 because SQLite does not enforce SELECT ... FOR UPDATE row claims. Process-registry DB sync at server startup runs only on instance 0 (or when the count is 1); other instances skip it with a log line so concurrent workers do not race on the registry setting rows.

Variable	Default	Description
SNAPPER_COORDINATOR_INSTANCE_ID	0	Zero-based shard index for this instance
SNAPPER_COORDINATOR_INSTANCE_COUNT	1	Total number of instances in the cluster
SNAPPER_COORDINATOR_OUTBOX_MAX_SCAN_ROWS	1000	Per-tick upper bound on outbox rows scanned by this shard; set to unbounded, none, or an empty value to disable the cap. Positive integers are accepted.

Trade Runtime Safety

These bootstrap variables gate stale trade dispatch and live multi-leg execution safety.

Variable	Default	Description
TRADE_COMMAND_DISPATCH_TTL_S	30.0	Max age, in seconds, for trade commands. Stale CREATED submits expire in the outbox before publishing; executor stale-frame handling verifies venue state before any order placement; the executor's reconciliation sweep auto-rejects an unresolved DISPATCHED command on verified venue absence only after max(120 s, 2 × TTL). Values <= 0 disable the cap and the absence auto-reject. Keep this below the engine's 60 s in-flight valve.

Variable	Default	Description
PAIRED_EXECUTION_GUARD_ENABLED	false	Fail-closed live multi-leg strategy gate. With the default, live exchanges refuse multi-leg groups while paper multi-leg remains allowed. Enable only after the paired-execution guard is deployed.
PAIRED_EXECUTION_ASSEMBLY_TIMEOUT_S	5.0	Seconds a paired-execution group waits for all sibling legs to register durable commands before the guard can break the group.
PAIRED_EXECUTION_FILL_TIMEOUT_S	30.0	Seconds an armed paired-execution group waits for fills before the guard treats a lagging leg as breakage and compensates.

Observability Pipeline

These variables tune the always-on background pipelines that emit system-metrics snapshots, run retention archive+purge sweeps, and sample DB stats. See observability.md for the underlying snapshots and the retention window math.

Variable	Default	Description
SYSTEM_METRICS_INTERVAL_SECONDS	5	Cadence for the in-process system-metrics snapshotter
SYSTEM_METRICS_HISTORY_CAP	17280	Maximum number of snapshots retained in memory (≈ 24 h at 5 s cadence)
RETENTION_INTERVAL_SECONDS	3600	Cadence for the retention archive+purge tick
RETENTION_DISABLED	false	Disable the retention loop entirely (still allow on-demand archive via CLI)
RETENTION_DRY_RUN	false	Compute the window and counts but skip writes/purges
RETENTION_OUTPUT_DIR	data	Base directory for archive CSV writes (matches the CLI --output-dir default)
DB_METRICS_INTERVAL_SECONDS	60	Cadence for the per-table SCD2 row-count sampler
DB_METRICS_DISABLED	false	Disable the DB stats sampler

Variable	Default	Description
SNAPPER_TICK_PROBE	unset	Enable per-stage tick hot-path histograms in publisher logs when truthy (1, true, yes)
SNAPPER_TRADE_PROBE	unset	Enable per-stage trade hot-path histograms in publisher logs when truthy (1, true, yes)

The two probe variables flush one log line roughly every 10 seconds with per-stage count, rate, p50, p95, p99, and max. Leave them unset outside targeted throughput investigations.

Database Settings

Sensitive data is stored encrypted in the settings table. Wallet-scoped exchange credentials (api keys, secrets, PEM material) live in a separate wallet_credentials table — see Wallet Credentials below. Shared market-data provider keys stay in settings.

Market Data API Keys

Key	Description
polygon_api_key	Polygon.io API key (shared market data — not wallet-scoped)

Push Notifications (APNs)

The notification sidecar reads its APNs credentials and the push-beta rollout gate from the settings table:

Key	Description
apns_team_id	Apple Developer team ID
apns_key_id	APNs auth key ID
apns_bundle_id	iOS app bundle identifier (reverse-DNS form)
apns_topic	APNs topic — normally identical to apns_bundle_id for alert pushes
apns_environment	sandbox, production, or sandbox_and_production; the latter runs both APNs clients and routes by each device's registered environment
apns_private_key_p8_base64	APNs auth key (.p8 PKCS#8 PEM) encoded as base64 so it round-trips through the TOML seed

Key	Description
push_beta_config	JSON {"enabled": bool, "user_public_ids": [...]} rollout gate (category notifications). Disabled by default — every user receives pushes; when enabled, only allowlisted users do. An absent or malformed value falls back to the disabled default so a misedit can never silence pushes. Managed via Manage Push-Beta Gate

All six apns_* keys are required when the notification sidecar starts: load_apns_config fails loud at startup with the full list of missing keys rather than producing cryptic APNs errors at first-send time. They are seeded under category apns via the proprietary seed profile (proprietary/data/seed/{dev,prod}.toml).

Wallet Credentials

Per-exchange trading credentials live in the wallet_credentials table and are loaded by CredentialResolver during per-wallet executor startup. Each row carries:

Column	Description
wallet_public_id	UUID of the owning wallet ((label, is_paper) unique in wallets table)
exchange	Exchange identifier (kraken, kraken_futures, walutomat, paper)
credential_type	Envelope shape: api_key_secret, rsa_pem, oauth, or paper
encrypted_payload	Fernet-encrypted JSON envelope (shape depends on credential_type)
label	Human-readable description of the credential row

Envelope shapes by credential_type:

- api_key_secret — {"api_key": "...", "api_secret": "..."} (Kraken, Kraken Futures)
- rsa_pem — {"api_key": "...", "private_key_pem": "..."} (Walutomat)
- oauth — {"client_id": "...", "client_secret": "...", "refresh_token": "..."}
- paper — {"initial_balance": "10000.0"} (paper wallets)

Seed profiles are resolved in three tiers: data/seed/{profile}.toml, then proprietary/data/seed/{profile}.toml, then the package-bundled src/snapper/data/seed/{profile}.toml. The bundled dev.toml seeds an open-source paper wallet with a paper credential; maintainers may have a proprietary override with live wallet credentials. On a fresh database, db-seed creates the default operator, every [[wallets]] entry, nested [[wallets.credentials]] rows, and the first admin user's primary operator membership. If the profile has no wallets, the loader falls back to a single default paper wallet with a 10000.0 initial

balance. Re-running seed on an established database does not merge wallets: users are skipped when any user exists, settings are inserted only when the key is absent, and the whole operator/wallet bootstrap is skipped when either operators or wallets already exist.

The seed loader supports the api_key_secret, rsa_pem, and paper envelope shapes. The oauth shape is accepted at the credential-resolver layer but **not** by the current seed loader, so OAuth credentials must be inserted through the REST wallet_credentials create/rotate routes (POST /api/wallets/{wallet_public_id}/credentials, POST /api/wallets/{wallet_public_id}/credentials/{credential_public_id}/rotate) rather than via seed.

The credential management REST surface AND the matching frontend UI (frontend/src/features/admin/CredentialManagement/) are both shipped. Day-to-day operators use the dashboard's Credential Management view; headless callers can hit the REST routes directly (curl / Postman — the snapper-mcp bridge does NOT expose credential CRUD as MCP tools). Other recovery options are re-running the seed against a clean DB or direct SQL surgery on the encrypted payload as a last resort.

Seed file structure:

```toml
[[wallets]]
label = "default"
is_paper = false

[[wallets.credentials]]
exchange = "kraken"
credential_type = "api_key_secret"
api_key = "your-kraken-api-key"
api_secret = "your-kraken-api-secret"

[[wallets.credentials]]
exchange = "walutomat"
credential_type = "rsa_pem"
api_key = "your-walutomat-api-key"
private_key_pem_base64 = "your-pem-base64-encoded"
```

Two wallets named default — one with is_paper=true and one with is_paper=false — coexist via the (label, is_paper) unique index. The seed loader encrypts every payload with the master-password Fernet key before insert.

Trading Parameters

Key	Default	Description
instruments	{"kraken": ["*"], "kraken_futures": ["*"], "kraken_equities": ["*"], "walutomat": ["*"], "polygon": ["*"]}	Instruments per exchange (* = all instruments)
timeframes	["1m"]	Candle timeframes (list)
risk_max_leverage	1.0	Maximum leverage

Key	Default	Description
risk_r_per_trade	0.005	Risk per trade (0.5%)

Additional runtime settings are also DB-backed and can be managed through the Settings API/UI:

Key	Default	Description
ai_integration_enabled	true	Feature gate for /api/mcp and /api/ai-delegates/*; false returns 503 {"error_code":"feature_disabled"} from /api/mcp and hides AI integration in the frontend
feed_egress_enabled	false	Route feed publishers through their own process-local egress pools at startup
egress_pool	unset	JSON tunnel-pool definition, validated as EgressPoolConfig at startup. An absent, malformed, or validation-failing value disables egress routing with a log line. See snapper-egress.md
paper_instruments	{"kraken": ["BTC-USD", "EUR-USD"], "kraken_futures": [], "walutomat": ["EUR-PLN", "USD-PLN"]}	Source exchanges and symbols replayed by paper feeds
backfill_days	30	Default historical backfill window
risk_max_drawdown	0.15	Maximum portfolio drawdown threshold
log_level	INFO	Application log level
log_json	false	Emit structured JSON logs
ws_reconnect_max_delay	10	Maximum WebSocket reconnect delay in seconds
rest_retry_max_attempts	5	REST retry attempt cap
rest_retry_backoff_base	0.2	REST exponential-backoff base delay in seconds
rest_retry_max_delay	2.0	REST retry maximum delay in seconds
rest_circuit_failure_threshold	5	Failures before the REST circuit opens
rest_circuit_reset_timeout	30	Seconds before the REST circuit attempts reset

Key	Default	Description
zmq_heartbeat_interval_ms	1000	ZMQ heartbeat interval
recon_balance_threshold	1.0	Absolute balance mismatch threshold for reconciliation warnings

The trade runtime always uses the durable (outbox-driven) dispatch path. The engine writes TradeCommand rows to the database; the OutboxDispatcher picks them up and publishes to ZMQ. Stale CREATED create/submit commands older than TRADE_COMMAND_DISPATCH_TTL_S (see Trade Runtime Safety) are CAS-expired to terminal EXPIRED instead of published — cancels and replaces are exempt — and the engine releases its in-flight intent via a synthetic expired event. Executor-side VenueEvent writes are fail-closed for pre-acceptance events — a failed persist raises before the executor acknowledges the venue event; once the venue has accepted an order, a failed order_accepted persist no longer aborts the flow and the executor's recon loop retries the durable write each cycle.

Market Persist Policy

Market-data publishers always publish ZMQ frames, but DB persistence is controlled by these settings so high-volume feeds can be cache-first:

Key	Description
market_persist_ticks	Persistence mode for tick rows
market_persist_trades	Persistence mode for trade rows
market_persist_candles	Persistence mode for candle rows
persist_intermediate_candles	When false (default) only the FINAL bar per native window is persisted (the native finalizer eliminates intra-minute SCD2 churn); true also persists in-progress complete=false bars, superseded by the final. ZMQ publishes every frame regardless.
spot_trade_built_shadow_enabled	When false (default) Kraken Spot trade-built 1m candles are not persisted; true writes them to shadow_candles only for native-vs-trade-built A/B checks.
spot_candle_source	Kraken Spot live 1m source. Default native keeps venue OHLC unchanged; explicit trade_built switches live 1m bars to the trade stream. Read at feed startup: the live subscription source is fixed when the publisher subscribes, so a change takes effect only after a snapper-feed restart (a live settings broadcast refreshes the cached value but does not re-subscribe).

Key	Description
trade_built_finalize_grace_seconds	Seconds to wait after a trade-built minute closes before finalizing it. Default 12 absorbs late Kraken trades.
candle_forward_fill	When false (default) the higher-TF synthesis flush emits only data-backed bars; true forward-fills empty higher-TF windows with a flat carried-close bar. Enable ONLY for continuous-corpus venues (24/7 crypto like Kraken spot); leave off for session-based equities where it would manufacture non-trading-day bars.
candle_single_source	When false (default) the /api/candles smart route derives 5m/15m/30m on-read from the 1m cache; true serves those frames from the persisted candles plane (Phase 3 read-cutover, closing the cache-vs-persisted dual-source hazard). Keep off until the persisted higher-TF plane is populated and verify-candle-coverage passes. 1m stays cache-served and 1h/4h/1d are already DB-served.
market_persist_extra	Explicit extra instrument allowlist
market_persist_exclude	Explicit instrument denylist

MarketPersistPolicy refreshes from settings and admin bus events, then injects immutable allow/deny snapshots into publishers. Invalid edits are rejected for that refresh cycle and the previous policy remains active. Use /api/market/cache/health and /api/market/coverage to verify the effective persisted universe.

Publisher Micro-Batch

Key	Default	Description
write_buffer_flush_ms	50	Flush age threshold in ms (cached at start)
write_buffer_candle_max_rows	100	Candle batch size trigger
write_buffer_tick_max_rows	500	Tick batch size trigger
write_buffer_trade_max_rows	500	Trade batch size trigger

These settings are cached when the publisher starts and require a process restart to take effect.

Authentication

Key	Default	Description
auth_secret_key	change-me-in-production-use-openssl-rand-hex-32	JWT signing secret
csrf_secret_key	change-me-in-production-csrf-key	CSRF token signing secret
auth_algorithm	HS256	JWT signing algorithm
auth_access_token_expire_minutes	15	Access token lifetime
auth_refresh_token_expire_days	7	Refresh token lifetime
auth_refresh_token_expire_days_extended	30	Extended refresh token lifetime
ws_token_ttl_seconds	900	WebSocket token lifetime
csrf_token_expire_minutes	60	CSRF token lifetime
session_secure	false	Require HTTPS for session cookies
session_same_site	lax	Session cookie SameSite mode
session_domain	""	Session cookie domain override
ui_origin	""	Additional allowed UI origins for CORS and WS origin validation

When SNAPPER_ENV is production, prod, or staging, these database-backed placeholders are refused. Configure real values in the settings table before switching a deployment into production mode.

.env.example File

The block below mirrors the variables documented in .env.example at the repo root. The inline comments are abbreviated here for readability; the canonical version of the file (with full per-variable rationale and rotation guidance) lives in .env.example itself.

```
# Database connection
DB_URL=sqlite+aiosqlite:///./data/snapper.db
SNAPPER_ENV=development
```

```
# Encryption settings for sensitive data (API keys, secrets)
# IMPORTANT: Change this default in production!
MASTER_PASSWORD=snapper_default_master_password_v1

# Server settings
SERVER_HOST=0.0.0.0
SERVER_PORT=8000
SERVER_RELOAD=false
SERVER_API_ONLY=false
PROCESS_AUTOSTART_PROFILE=all
SERVER_PROXY_HEADERS=true
SERVER_FORWARDED_ALLOW_IPS=127.0.0.1,172.17.0.1

# Telemetry recording (pings, heartbeats, GET reads — high volume)
TELEMETRY_RECORDING_ENABLED=false

# ZMQ broker settings
ZMQ_BROKER_XSUB=tcp://127.0.0.1:7500
ZMQ_BROKER_XPUB=tcp://127.0.0.1:7501
# ZMQ_BROKER_BIND_XSUB=tcp://0.0.0.0:7500
# ZMQ_BROKER_BIND_XPUB=tcp://0.0.0.0:7501

# PostgreSQL pool clamps for multi-process feed deployments
# DB_POOL_SIZE=2
# DB_MAX_OVERFLOW=3

# Coordinator sharding (multi-instance trade-zmq / multi-worker uvicorn)
SNAPPER_COORDINATOR_INSTANCE_ID=0
SNAPPER_COORDINATOR_INSTANCE_COUNT=1
SNAPPER_COORDINATOR_OUTBOX_MAX_SCAN_ROWS=1000
TRADE_COMMAND_DISPATCH_TTL_S=30.0
# PAIRED_EXECUTION_GUARD_ENABLED=false
# PAIRED_EXECUTION_ASSEMBLY_TIMEOUT_S=5.0
# PAIRED_EXECUTION_FILL_TIMEOUT_S=30.0

# Observability pipeline (system-metrics snapshotter)
SYSTEM_METRICS_INTERVAL_SECONDS=5
SYSTEM_METRICS_HISTORY_CAP=17280

# Retention policy framework
RETENTION_INTERVAL_SECONDS=3600
RETENTION_DISABLED=false
RETENTION_DRY_RUN=false
RETENTION_OUTPUT_DIR=data

# DB stats sampler (per-table row-count snapshots)
DB_METRICS_INTERVAL_SECONDS=60
DB_METRICS_DISABLED=false

# Publisher hot-path probes, disabled by default
# SNAPPER_TICK_PROBE=1
# SNAPPER_TRADE_PROBE=1
```

Accessing Configuration in Code

Bootstrap settings (without database)

```
from snapper.config.settings import get_settings

settings = get_settings()
db_url = settings.db_url
host = settings.server_host
```

With database access

```
from snapper.config.settings import get_settings_with_service

settings = get_settings_with_service(settings_service)
polygon_key = settings.polygon_api_key  # Decrypted from database
```

Wallet-scoped exchange credentials do NOT appear on AppSettings. They are loaded by CredentialResolver during per-wallet executor startup:

```
from snapper.config.credentials import CredentialResolver

resolver = CredentialResolver(repository)
envelope = await resolver.get_credentials(
    exchange="kraken",
    wallet_public_id="01975a8b-3c7d-7000-8000-0000000000a1",
)
# envelope == {"api_key": "...", "api_secret": "..."}
```

Managing Settings via API

List Settings

```
GET /api/settings
GET /api/settings?category=api
GET /api/settings?category=api&as_of=2026-01-18T12:00:00Z
```

Public Feature Flags

```
GET /api/settings/features
```

This route does not require authentication. It returns the public feature-flag projection used by the frontend, currently ai_integration_enabled.

List Categories

```
GET /api/settings/categories
GET /api/settings/categories?as_of=2026-01-18T12:00:00Z
```

Manage Push-Beta Gate

```
GET /api/settings/push-beta/users
POST /api/settings/push-beta/users
X-CSRF-Token: <csrf_token>
Content-Type: application/json

{
    "type": "update_push_beta_users_command",
    "public_id": "<uuid7>",
    "session_id": "<client-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "payload": {
        "enabled": true,
        "user_public_ids": ["<user-public-id>"]
    }
}
```

The POST body replaces the full allowlist; it is not a merge.

Set Setting

```
POST /api/settings/{key}/set
X-CSRF-Token: <csrf_token>
Content-Type: application/json

{
    "type": "setting_update",
    "public_id": "<uuid7>",
    "session_id": "<client-session>",
    "sequence_id": 1,
    "timestamp": "2026-01-18T12:00:00Z",
    "payload": {
        "value": "new_value",
        "category": "system",
        "description": "optional description"
    }
}
```

Remove Setting

```
POST /api/settings/{key}/remove
X-CSRF-Token: <csrf_token>
Content-Type: application/json
```

```
{
    "type": "remove_setting_request",
    "public_id": "<uuid7>",
    "session_id": "<client-session>",
    "sequence_id": 2,
    "timestamp": "2026-01-18T12:00:01Z",
    "payload": {}
}
```

Environment Configuration

Development

```
# .env.development
DB_URL=sqlite+aiosqlite:///./data/snapper.db
SNAPPER_ENV=development
SERVER_HOST=127.0.0.1
SERVER_PORT=8000
SERVER_RELOAD=true
SERVER_PROXY_HEADERS=true
SERVER_FORWARDED_ALLOW_IPS=127.0.0.1
```

Production

```
# .env.production
DB_URL=postgresql+asyncpg://user:pass@db.example.com/snapper
SNAPPER_ENV=production
MASTER_PASSWORD=<strong_random_password>
SERVER_HOST=0.0.0.0
SERVER_PORT=8000
SERVER_RELOAD=false
SERVER_PROXY_HEADERS=true
SERVER_FORWARDED_ALLOW_IPS=127.0.0.1,172.17.0.1
```

Docker

Variables can be passed via docker-compose.yml:

```
services:
  snapper:
    env_file: .env
    environment:
      - DB_URL=postgresql+asyncpg://snapper:password@postgres/snapper
```

Settings Encryption

Sensitive settings are encrypted with Fernet (AES-128-CBC + HMAC-SHA256):

1. Salt is derived from MASTER_PASSWORD via SHA-256
2. Encryption key is derived via PBKDF2-HMAC-SHA256 (100,000 iterations)
3. Values are encrypted with Fernet and stored in database

If MASTER_PASSWORD changes, encrypted settings become inaccessible. Use snapper settings-rotate-encryption to re-encrypt encrypted settings rows before changing it. Wallet credential payloads use the same master-password Fernet key but are not rewritten by that command; rotate or recreate wallet credentials separately before changing MASTER_PASSWORD.

Configuration Validation

The system validates configuration at startup via the shared .env allowlist check. The encrypt/decrypt round-trip is not a startup step; it runs as part of the snapper settings-rotate-encryption command.

The allowlist is the union of BootstrapSettingsLoader aliases and each subsystem's exported ENV_VARS set. Unknown keys fail startup with suggestions, so typos such as MASTER_PASWORD do not silently fall back to defaults. When adding a new env var, register it on the owning module's ENV_VARS or add it as a bootstrap settings field. Production-like SNAPPER_ENV values additionally reject placeholder secret material so a typo cannot silently sign tokens or encrypt settings with development defaults.

Development

Guidelines for developers working on the Snapper project.

Requirements

- Python 3.14+
- Poetry
- Node.js 26+ and pnpm 11+ (frontend engines, Docker UI build, and CI workflows all standardize on Node 26)
- Build tools for Python packages. TA-Lib is optional at runtime: the indicator adapter uses TA-Lib when importable and falls back to pure Python implementations otherwise.
- Pre-commit hooks

Environment Setup

```
# Install system dependencies (Linux/macOS)
make system-deps

# Install Python dependencies
make setup

# Install frontend dependencies
make ui-setup

# Synchronize pre-commit hooks
make pre-refresh

# Refresh dependencies, local tools, and Dockerfile tool pins
make update
```

Quality Gates

Main Command

```
make check-all
```

Executes the complete quality gate:

1. Formatting (ruff, black, isort)
2. Linting (ruff)

3. Type checking (mypy)
4. Docstring compliance
5. No # comments in Python
6. Canonical __main__ guards
7. Empty __init__.py files
8. No forbidden temporal mutations
9. Vendor-neutrality check (check-vendor-neutral)
10. Pydantic-only FastAPI I/O (check-pydantic-routes)
11. Egress compose safety check (check-egress-compose)
12. Frontend checks (ui-typecheck, ESLint, Prettier, dead code, i18n checks)
13. No pragma/noqa/ignore exclusions
14. Backend tests with 100% coverage
15. Frontend tests with coverage

Individual Steps

Command	Description
make fmt	Check formatting
make fmt-fix	Fix formatting
make lint	Ruff linting
make lint-fix	Fix linting issues
make typecheck	Mypy type checking
make test	Tests (parallel)
make test-serial	Tests (sequential, debug)
make test-integration	Paper-mode E2E tests (tests/integration/)
make cov	Tests with coverage
make cov-xml	Export coverage XML for SonarCloud (after make cov)
make check-egress-compose	Reject unsafe snapper-egress compose wiring

make check-egress-compose scans Compose files and override files for the snapper-egress sidecar contract. The sidecar must share the unified Snapper image, run through command: ["egress"], run as root with NET_ADMIN, avoid host ports: / network_mode: host, and share the SQLite data mount when the monolith uses /app/data. The monolith stays unprivileged: no cap_add and no user: override.

Automatic Fixes

```
make fix        # Backend: fmt-fix + lint-fix + move-imports
make fix-all    # Backend + frontend
```

Code Standards

Types (MANDATORY)

All functions must have type annotations:

```python
def calculate_rsi(prices: pd.Series, period: int = 14) -> pd.Series:
    """Calculate RSI indicator."""
    ...

async def fetch_data(symbol: str) -> list[Candle]:
    """Fetch candle data."""
    ...
```

Test functions must have -> None:

```python
async def test_rsi_calculation() -> None:
    """Test RSI calculation returns expected values."""
    ...
```

Docstrings (Google style)

```python
def process_signal(signal: Signal, config: StrategyConfig) -> Order | None:
    """Process trading signal and generate order.

    Takes a trading signal and configuration, validates the signal,
    and generates an appropriate order if conditions are met.

    Args:
        signal: Trading signal to process.
        config: Strategy configuration dictionary.

    Returns:
        Generated order if valid, None otherwise.

    Raises:
        ValidationError: If signal data is invalid.
    """
    ...
```

No # Comments

comments are forbidden. Use docstrings:

```
# WRONG
def foo():
    # This calculates something
    x = 1 + 1

# CORRECT
def foo():
    """Calculate the sum."""
    x = 1 + 1
```

Imports

Imports at the top of file, single lines:

```
from datetime import UTC
from datetime import datetime

import pandas as pd
from sqlalchemy import select

from snapper.config.settings import get_settings
from snapper.data.models import Candle
```

Forbidden:

- from __future__ import annotations
- if TYPE_CHECKING: blocks
- Re-exports in __init__.py

__init__.py

__init__.py files must be empty (or docstring only):

```
"""Snapper strategies module."""
```

Tests

Structure

```
tests/
├── conftest.py          # Shared fixtures
├── api/                 # API tests
├── application/         # Business logic tests
```

```
├── cli/                # CLI tests
├── data/               # Data layer tests
├── indicators/         # Indicator tests
├── integration/        # Paper-mode E2E tests
├── messaging/          # ZMQ tests
├── strategies/         # Strategy tests
└── ...
```

Writing Tests

```python
import pytest
from snapper.indicators.rsi import rsi


class TestRSI:
    """Test suite for RSI indicator."""

    def test_rsi_returns_series_with_correct_length(self) -> None:
        """
        Given a price series of length N
        When RSI is calculated with period P
        Then the result has length N.
        """
        prices = pd.Series([100, 102, 101, 103, 105])
        result = rsi(prices, period=14)
        assert len(result) == len(prices)

    def test_rsi_values_in_valid_range(self) -> None:
        """
        Given any price series
        When RSI is calculated
        Then all values are between 0 and 100.
        """
        prices = pd.Series([100, 102, 101, 103, 105, 104, 106])
        result = rsi(prices, period=3)
        assert (result >= 0).all()
        assert (result <= 100).all()
```

Async Tests

```python
import pytest


@pytest.mark.asyncio
async def test_fetch_candles() -> None:
    """Test fetching candles from repository."""
    repo = get_repository(db_url)
    candles = await repo.get_candles("BTC-USD", limit=10)
    assert len(candles) <= 10
```

Tests must produce zero pytest warnings. A common pitfall: AsyncMock() makes **every** attribute on the mock async, including synchronous methods such as session.add() or session.expunge(). The fix is to override the synchronous attributes explicitly:

```
from unittest.mock import AsyncMock, MagicMock

session = AsyncMock(add=MagicMock(), expunge=MagicMock())
```

Run python -m pytest tests/ -W error::RuntimeWarning to fail the suite on any unawaited-coroutine warnings.

Fixtures

```
@pytest.fixture
def sample_candles() -> list[Candle]:
    """Provide sample candle data for tests."""
    return [
        Candle(open=100, high=105, low=95, close=102, volume=1000),
        Candle(open=102, high=110, low=100, close=108, volume=1200),
    ]
```

Integration Tests

```
make test-integration
```

Runs the paper-mode end-to-end tests in tests/integration/ with a 120-second per-test timeout. The fixtures spin up a real ZmqBrokerThread, a PaperOrderExecutor, and a TraderCoordinator against the SQLite test fixture, so the full order path is exercised without touching a live venue. These tests carry the integration pytest marker and are excluded from make test / make cov (the default addopts select -m 'not integration').

100% Coverage

The project requires 100% code coverage (TDD). Check:

```
make cov
```

Coverage is configured under [tool.coverage.run] / [tool.coverage.report] in pyproject.toml. The fail_under = 100 threshold enforces the TDD requirement.

Frontend

Setup

```
make ui-setup
```

Development

```
make ui-dev
```

Frontend at http://localhost:3000/ with backend proxy.

Quality Checks

Command	Description
make ui-lint	ESLint
make ui-format	Prettier check
make ui-typecheck	TypeScript type check
make ui-dead-code	Knip dead code
make ui-i18n-check	Frontend locale/catalog consistency
make ui-test	Vitest tests
make ui-cov	Tests with coverage

Fixes

```
make ui-fix   # lint-fix + format-fix + dead-code-fix
```

Type Generation

```
make ui-gen-types   # Frontend OpenAPI/WS types, Zod schemas, entity aliases,
permissions
make ios-gen-types  # Swift models from OpenAPI + WebSocket schemas
make ts-bridge      # snapper-mcp bridge wire contract only
make bridge-regen   # Regenerate bridge contract and run bridge checks
make bridge-check   # Verify committed bridge contract without regenerating
make ui-check-types # Non-destructive drift check for generated types and
backend i18n catalogs
```

The shared generator is scripts/generate_types.py. With no explicit target it runs --all, which covers the main-repo frontend/iOS export path but intentionally excludes the snapper-mcp bridge because that target writes across the integration boundary. Use --bridge (or the Makefile targets above) when the bridge wire contract must be updated. make bridge-check verifies the committed wire-contract file against the current backend schemas (scripts/bridge_check/check_drift.py and check_oss_prose.py) and runs the

bridge npm stack (typecheck, lint, tests, stdout gate) without rewriting any files. scripts/check_type_drift.py backs up generated files, runs make ui-gen-types ios-gen-types gen-backend-i18n-catalog, compares the result, then restores the working tree.

Internationalization

The frontend i18n catalogs in frontend/src/locales/<lang>/*.json are the source of truth for translated UI strings across 45 locales. iOS mirrors the subset of strings used on the native client by porting them into ios/Snapper/Resources/Localization/Localizable.xcstrings. Backend user.default_language validation accepts the union of the iOS and frontend catalog code forms: 45 iOS cases plus 45 frontend cases, with five naming divergences (pt-BR/pt, nb/no, zh-Hans/zh, sr-Latn/sr, my/my-MM) for 50 accepted codes.

Backend alert catalog

scripts/gen_backend_i18n_catalog.py reads the iOS xcstrings file and writes committed backend JSON catalogs to src/snapper/i18n/catalogs/<lang>.json. The backend catalog is limited to alert title/body templates: 12 keys across 45 languages, one JSON file per language. It is generated with make gen-backend-i18n-catalog and checked by scripts/check_type_drift.py.

Market catalog

The market.description.*, market.assetClass.*, market.sector.*, market.related.*, market.pairStats.*, and market.cacheBanner.* namespaces are ported from the frontend JSON into the iOS xcstrings by scripts/port_market_catalog.py. Run after editing any frontend/src/locales/<lang>/market.json file under one of those namespaces:

```
python scripts/port_market_catalog.py        # write
python scripts/port_market_catalog.py --check  # CI parity gate
```

The --check mode is wired into make ui-check via ui-i18n-check-market, so a forgotten regeneration fails the backend + frontend quality gate. Placeholder tokens ({{name}} / {{assetClass}}) are rewritten to xcstrings positional codes (%1$@, %2$@) based on appearance order in the EN template.

iOS-side coverage of the resulting catalog is enforced by CatalogParityTests (45 locales × every key declared in SnapperTests/I18n/ExpectedKeys.swift); adding a new key to the catalog requires adding it to ExpectedKeys.swift in the same iOS PR.

make ios-i18n-check (run from the main repo) delegates into the ios submodule Makefile and lints Swift call sites: bare String(localized:) usage under ios/Snapper is rejected in favor of the LocaleStrings helper, so catalog lookups follow the selected app locale rather than Locale.current. An allowlist file (ios/scripts/check_i18n_allowlist.txt) exempts specific paths.

Alerts catalog (historical direction: xcstrings → JSON)

Historically the iOS alerts catalog was the source of truth and frontend caught up via scripts/port_ios_alert_catalog.py. That direction is preserved for back-compatibility; new namespaces should flow frontend → iOS via the market-catalog pattern above. The alert port currently processes 32 alerts.* keys across 45 locales, writes one frontend/src/locales/<lang>/alerts.json file per locale, and updates common.nav.alerts from the alerts.navTitle xcstrings key. The five divergent locale directory names are remapped between iOS and frontend in the same way as SUPPORTED_LANGUAGES.

Scanner Behavior

The backend quality scanners are intentionally source-tree scanners, not import-time checks. check_docstrings.py scans src, tests, proprietary when present, and scripts; with the Makefile flags it enforces module/public docstrings, BDD-style test docstrings, and Google-style Args: / Returns: sections. check_no_comments.py tokenizes Python under src, tests, scripts, proprietary/src, and proprietary/tests; hashes inside strings and docstrings are allowed, but real comment tokens fail strict mode. check_pydantic_routes.py walks src/snapper/server, src/snapper/auth, src/snapper/config, and src/snapper/api, then filters for FastAPI route functions whose I/O must use concrete Pydantic schemas rather than dict, Any, or raw response types.

The remaining scanners are similarly path-scoped: coverage/lint exclusions scan Python in src, tests, scripts, optional proprietary code, and frontend TypeScript; temporal-mutation and vendor-neutrality scans walk src/snapper; check_init_files.py checks configured __init__.py roots for docstring-only files; and check_main_guard.py requires every discovered entry point to end with raise SystemExit(main()).

Database Migrations

Dev DB Location

The local server SQLite database lives at ./data/snapper.db. Test and coverage Make targets use an isolated SQLite fixture at ./data/dev.db by setting DB_URL=sqlite+aiosqlite:///./data/dev.db inside the Makefile. This keeps make test, make cov, and make check-all from mutating the database used by a running local server, even when .env points at ./data/snapper.db or PostgreSQL.

The fixture is built automatically on the first test run and reused afterwards. make migrate-dev-sqlite builds it explicitly (Alembic migrations via snapper db-init, then snapper db-seed --profile dev); delete ./data/dev.db first to rebuild it from scratch.

The fixture URL is the TEST_DB_URL Make variable (default sqlite+aiosqlite:///./data/dev.db). Override it to run the suite against a PostgreSQL staging database:

```
make test TEST_DB_URL=postgresql+asyncpg://USER:PASS@HOST/DB
```

Never point TEST_DB_URL at a live server's database: tests mutate state, and the isolated_sqlite_db conftest fixture only protects SQLite paths.

Migration Strategy (Development)

The schema is built from a chain of sequential Alembic revisions (0001_init.py, 0002_instrument_feed_health.py, ...), each of which revises the previous one. To change the schema, add a new numbered revision rather than editing an existing one, then apply and reseed:

```
make migrate-dev
```

make migrate-dev applies Alembic migrations (via snapper db-init) then seeds development data (via snapper db-seed --profile dev). There is no standalone make migrate target -- always use make migrate-dev or make migrate-prod.

Applying (Production)

```
make migrate-prod
```

Rollback

```
snapper db-downgrade
```

Docker

Build

```
make docker-build-dev    # Development image
make docker-build-prod   # Production image
make docker-migrate-dev  # Initialize and seed the Docker SQLite database
```

Run

```
make docker-migrate-dev
make docker-run
```

Docker Compose

```
make docker-migrate-dev
docker compose up -d
```

CI/CD

The pipeline (.github/workflows/ci.yml) wraps the consolidated gate used locally in additional build and smoke stages:

1. Setup — make system-deps, make setup, make ui-setup
2. make ui-build — Production frontend bundle
3. make migrate-dev — Initialize and seed the database
4. make ui-check-types — Drift check for generated types and backend i18n catalogs
5. make check-all — Backend checks, frontend checks, exclusion scan, and tests with coverage
6. make cov-xml — Export coverage XML, consumed by the SonarCloud scan step
7. Docker smoke stage — make docker-build-dev, make docker-migrate-dev, docker compose up for the snapper and snapper-web services, then a make server-check health check

For debugging a failing make check-all locally, the equivalent steps are:

1. make check — Backend quality checks
2. make ui-check — Frontend lint, format, dead-code, type, and i18n catalog checks (ui-lint ui-format ui-dead-code ui-typecheck ui-i18n-check ui-i18n-check-alerts ui-i18n-check-market)
3. make check-exclusions — No pragma/noqa/ignore bypasses

4. make cov — Backend tests with coverage
5. make ui-cov — Frontend tests with coverage

Workflow

Before Commit

```
make check-all
```

Before PR

1. Ensure make check-all passes
2. Add/update tests
3. Update documentation if needed
4. Verify 100% coverage

Code Review Checklist

- [ ] Types for all functions
- [ ] Docstrings (Google style)
- [ ] No # comments
- [ ] Tests for new functionality
- [ ] 100% coverage
- [ ] make check-all passes

Project Structure

```
snapper/
├── src/snapper/           # Source code
│   ├── api/               # API schemas
│   ├── application/       # Business logic
│   ├── auth/              # Authentication
│   ├── cli/               # CLI interface
│   ├── config/            # Configuration
│   ├── core/              # Core types
│   ├── data/              # Data layer
│   ├── egress/            # snapper-egress sidecar entry point
│   ├── i18n/              # Backend i18n catalogs
│   ├── indicators/        # Technical indicators
│   ├── infrastructure/    # External integrations
│   ├── interface/        # WebSocket
│   ├── mcp/              # MCP server (delegate-facing tool surface)
│   ├── messaging/        # ZeroMQ
│   ├── server/            # FastAPI
```

```
│   ├── strategies/        # Trading strategies
│   └── utils/             # Utilities
├── tests/                 # Tests
├── frontend/              # React dashboard submodule
├── ios/                   # SwiftUI iOS client submodule
├── docs/                  # Documentation
├── scripts/               # Helper scripts
└── data/                  # Local data
```

Tools

Ruff

Linter and formatter. Configuration in pyproject.toml:

```toml
[tool.ruff.lint]
select = ["E", "F", "W", "I", "B", "UP", "C4", "SIM", "N", "PERF", "ISC",
"PLW", "PLC", "A", "TID", "D"]
```

Mypy

Type checker. Strict mode enabled:

```toml
[tool.mypy]
python_version = "3.14"
strict = true
```

Black

Formatter. Line length 100:

```toml
[tool.black]
line-length = 100
```

Pre-commit

Hooks run before commit:

```
pre-commit install
pre-commit run --all-files
```

Debugging

Logging

```
from loguru import logger

logger.info(f"Processing signal: {signal}")
logger.debug(f"Calculated RSI: {rsi_value}")
logger.error(f"Failed to execute order: {e}")
```

ZMQ Logger

```
snapper zmq-logger --payload
```

Serial Tests

```
make test-serial
```

pdb/ipdb

```
import ipdb; ipdb.set_trace()
```

Documentation

Docstrings

Every module, class, and public function must have a docstring.

docs/

Markdown documentation in docs/ directory.

PDF

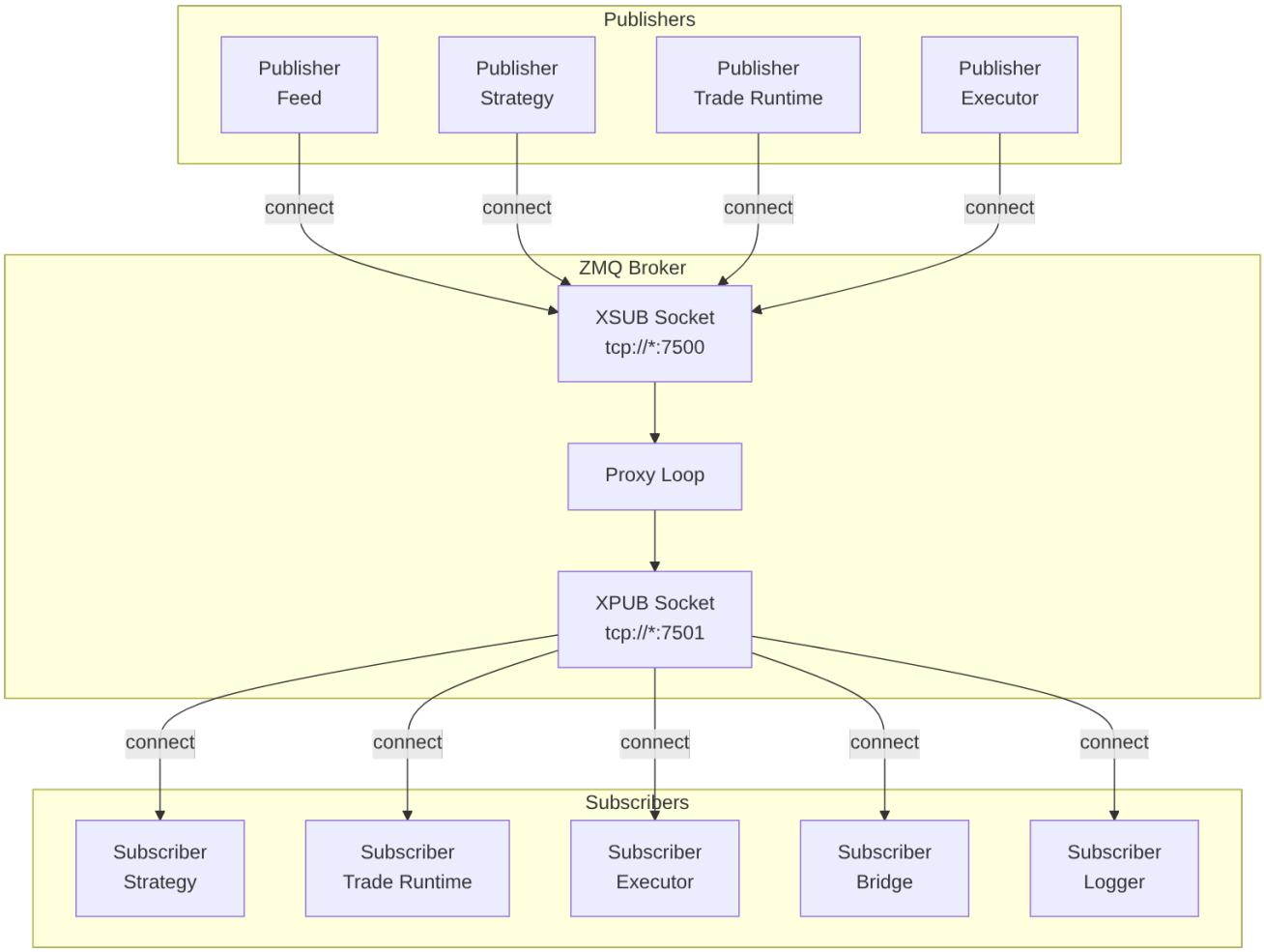
```
make docs-pdf
```

Generates frontend/public/snapper.pdf from README + docs/*.md so the docs ship with the frontend bundle and are linked from the login page. Because frontend/ is the snapper-frontend submodule, commit and push frontend/public/snapper.pdf from inside that submodule, then commit the updated frontend submodule pointer in the parent Snapper repository.

Messaging (ZeroMQ)

Snapper uses ZeroMQ for inter-process communication in a pub/sub architecture. The central component is an XPUB/XSUB broker that routes messages.

Architecture



XPUB/XSUB Broker

The broker is the central message hub:

- **XSUB** (tcp://*:7500) — Publishers connect here
- **XPUB** (tcp://*:7501) — Subscribers connect here

The broker forwards messages in both directions:

- Data: XSUB → XPUB
- Subscriptions: XPUB → XSUB

Starting the Broker

```
snapper broker
```

Or programmatically:

```
from snapper.messaging.infrastructure.broker import ZmqBrokerThread

broker = ZmqBrokerThread()
broker.start()
# ...
broker.stop()
```

Topics

Topic format varies by category (see per-category tables below).

Market Data

Topic	Description
market.kraken.BTC-USD.candles.1h	Hourly BTC/USD candles from Kraken
market.kraken.BTC-USD.ticks	BTC/USD ticks from Kraken
market.kraken_equities.MNQU6-CME.candles.1d	Daily Nasdaq-100 Micro futures candles from Kraken Equities (note: live market.* topics exclude polygon — Polygon is replay-only via market.paper.polygon.…)

Signals

Topic	Description
signals.paper.BTC-USD.rsi_btc_1h	Paper trading signals
signals.kraken.BTC-USD.live	Live signals from Kraken

Orders & Executions

Topic	Description
orders.commands.kraken.BTC-USD.submit	Order requests for Kraken

Topic	Description
orders.commands.kraken.BTC-USD.cancel	Cancel requests for an existing Kraken order
orders.commands.kraken.BTC-USD.replace	Replace/modify requests; executors currently reject with a lightweight rejected replace event because atomic replace is not implemented
orders.events.kraken.BTC-USD.executed	Order executions from Kraken
orders.events.kraken.BTC-USD.submitted	Order submitted events
orders.events.kraken.BTC-USD.accepted	Order accepted events
orders.events.kraken.BTC-USD.rejected	Order rejected events
orders.events.kraken.BTC-USD.cancelled	Order cancelled events
orders.events.kraken.BTC-USD.expired	Order expired events
orders.events.kraken.BTC-USD.replaced	Order replaced events
orders.events.kraken.BTC-USD.unknown	Ambiguous submit outcome; non-terminal until venue verification resolves it

TradeCommand, VenueEvent, and TradeProjectionCheckpoint are durable database artifacts used by the trade runtime. They are not additional ZMQ topics.

System

Topic	Description
system.heartbeats.executor.{exchange}	Executor heartbeat for the single-wallet template path (4 segments)
system.heartbeats.executor.{exchange}.{wallet_short}	Per-wallet executor heartbeat (5 segments). wallet_short is the **last** 12 lowercase hex characters of the wallet UUID (see snapper.core.wallet_short)
system.heartbeats.strategy.{name}	Strategy heartbeat (e.g. strategy.rsi_btc_1h)

Topic	Description
system.heartbeats.feed.{exchange}	Feed heartbeat (e.g. feed.kraken, feed.paper.kraken)
system.egress.snapshot	Read-only process-local egress pool snapshot for API aggregation
system.egress.transfer	Read-only snapper-egress WireGuard transfer samples for API route load
system.settings	Configuration change notifications
system.symbol_aliases	Symbol cache invalidation

Heartbeat component values use dot notation matching the topic path after system.heartbeats.: executor.kraken, executor.kraken.019d6ca45f2e, strategy.rsi_btc_1h, feed.kraken, feed.paper.kraken. The per-wallet executor heartbeat envelope additionally carries meta.wallet_public_id so subscribers that prefix-match the 4-segment parent topic can still disambiguate by reading the payload.

Pool-bearing processes publish system.egress.snapshot on the heartbeat cadence with an EgressPoolSnapshotEventData payload. The payload carries container (<role>@<hostname>) and snapshot, where snapshot is the process-local EgressPoolStatusSnapshot, including per-route connections rows for open WebSocket host counts and capped REST last-seen hostnames. Only hostnames are emitted; URL paths, query strings, headers, bodies, and authentication material are stripped before the snapshot leaves the process. The API process subscribes to the exact topic, caches the latest frame per container by local receive time, and merges those snapshots into GET /api/health/egress without changing routing decisions.

The snapper-egress sidecar publishes system.egress.transfer on the same cadence with an EgressTransferEventData payload. Each interface row contains interface, socks5_listen_port, cumulative rx_bytes and tx_bytes, optional byte-per-second rates, latest_handshake_at, counter_reset, and sampled_at. The API caches the latest sample per interface by receive time and joins transfer data onto GET /api/health/egress routes by SOCKS5 listener port. Ambiguous port joins are left null.

Executor heartbeats derive status from supervised-loop health instead of reporting a fixed healthy value. ERROR fires on an active loop death streak of at least 600 s, no successful reconciliation pass for 900 s, or an order command in flight for 600 s; WARNING fires on 2+ deaths within an active streak, reconciliation progress age of at least 300 s, a command in flight for 120 s, an unhealed accept-event backlog, or parked ambiguous submits awaiting venue verification. A death streak counts as active only while its last death is younger than 300 s, so a loop that died once and then self-healed never ages into a false alarm. lag_ms carries the reconciliation progress age and meta.status_reasons lists the human-readable reasons; if the status computation itself

raises, the frame still publishes as WARNING with the failure recorded in status_reasons — a broken computation can cause neither heartbeat absence nor a false healthy report. Executor heartbeats also carry venue REST reconciliation health in meta.venue_recon_failure_count, meta.venue_rest_reachable, meta.venue_health_halt_recommended, and meta.last_venue_recon_error. After three consecutive venue reconciliation failures, the coordinator uses the heartbeat to halt known real shard keys for the matching exchange/wallet scope and to drop new cold-path signals for that scope.

The launcher also publishes on the per-wallet executor topics: when a per-wallet executor exhausts its restart budget and is parked, the launcher emits bursts of three synthetic ERROR heartbeats on the instance's own topic, spaced 2 s apart so every frame clears the bridge's per-subscription forwarding throttle, re-bursting hourly while the name stays parked. These frames are recognizable by meta.synthetic: true, meta.origin: "launcher", and meta.reason: "restart_budget_exhausted"; they drive the existing critical-system-error alert pipeline without any new topic family. A successful restart or stop unparks the name and cancels the burst mid-flight.

Admin

Cross-cutting administrative notifications fanned out to every backend subscriber that cares about user/operator state changes.

Topic	Description
admin.user_deactivated	A user (or AI delegate) was deactivated; immediate revocation hook for in-process token caches, WebSocket close fanout, and APNs push fan-out via the notify sidecar. Auth listeners also poll the DB-backed users.is_active=False registry as broker-outage fallback
admin.scope_granted	create_grant published a new active scope row (instrument-exclusive grant just inserted)
admin.scope_handed_over	handover finalized a scope handover (old row closed, new row open, both committed)
admin.scope_revoked	revoke_grant closed an active scope row; revoke hook for any subscriber holding cached principal state

Shape: admin.{resource} — exactly 2 segments. The validator only enforces the 2-segment shape; the resource token itself is not character-class-checked, so consumers must use the payload's type discriminator as the authoritative routing hint.

Accruals

Funding / rollover / borrow ledger entries that affect mark-to-market.

Topic	Description
accruals.kraken_futures.BTC-USD-PERP.funding	Perpetual funding
accruals.kraken.EUR-USD.rollover	FX rollover
accruals.kraken.BTC-USD.borrow	Borrow interest

Shape: accruals.{exchange}.{instrument}.{accrual_type} (4 segments). accrual_type must be one of funding, rollover, borrow.

Alerts

Per-user push-notification fanout consumed by the WS bridge and the APNs sidecar.

Topic	Description
alerts.<user_public_id>.order_fill_full	Order filled in full
alerts.<user_public_id>.order_rejected	Venue rejected the order
alerts.<user_public_id>.order_unknown	Order submit outcome ambiguous — parked as non-terminal UNKNOWN pending venue verification (safety-critical; user-scoped with admin fan-out for strategy orders)
alerts.<user_public_id>.position_stop_loss_fired	Stop-loss fired on an open position
alerts.<user_public_id>.margin_warning	Margin warning
alerts.<user_public_id>.critical_system_error	Critical system error

Shape: alerts.{user_public_id}.{alert_type} (3 segments). user_public_id must be a UUID7; alert_type is the discriminated union in the AlertType Literal — the validator pulls the set via typing.get_args(AlertType) so adding a new alert type to the schema flows through automatically.

Plan Decisions

Bracket / trailing-stop / execution-plan decisions, emitted by the PlanExecutorService through a durable outbox written in the same transaction as each ExecutionPlanDecision SCD2 insert. The service attempts the ZMQ publish immediately, marks the outbox row sent after success, and retries pending rows with capped exponential backoff from a background drain loop after broker outages; the same drain also runs once at startup, so

decisions that never published before a restart replay from their durable rows. A row whose third publish attempt fails is marked failed terminally. The notify sidecar subscribes to turn stop-loss / take-profit fires into iOS pushes.

Topic	Description
plans.decisions.<plan_public_id>	One topic per execution plan; payload carries the decision frame

Shape: plans.decisions.{plan_public_id} (3 segments) with plan_public_id in UUID7 form.

AI Reviews

Outbound delegate-consultation frames for the human-in-the-loop review queue. The bridge applies a per-frame scope filter so only the selected AI delegate receives frames for wallets and instruments covered by its live scope grant. Non-delegate WebSocket principals fail closed for this topic family even if they somehow request a subscription.

Topic	Description
ai_reviews.<user_public_id>.<strategy_public_id>.request	Strategy is asking for review
ai_reviews.<user_public_id>.<strategy_public_id>.decision_ack	Operator's decision acknowledged
ai_reviews.<user_public_id>.<strategy_public_id>.caps_violation	Caps violation after approval

Shape: ai_reviews.{user_public_id}.{strategy_public_id}.{suffix} (4 segments). Both ID segments are UUID7; suffix is one of the three frame discriminators above. The payload must include string wallet_public_id and instrument_public_id values so the bridge can check the delegate's grant before forwarding.

Backtest

Per-run lifecycle and progress events.

Topic	Description
backtest.<wallet_public_id>.<run_public_id>.started	Run kicked off
backtest.<wallet_public_id>.<run_public_id>.progress	Periodic progress tick
backtest.<wallet_public_id>.<run_public_id>.milestone	Engine progress milestone — payload's milestone field is one of 25pct/50pct/75pct (BacktestProgressMilestone Literal)

Topic	Description
backtest.<wallet_public_id>.<run_public_id>.completed	Run finished
backtest.<wallet_public_id>.<run_public_id>.failed	Run failed
backtest.<wallet_public_id>.<run_public_id>.cancelled	Run cancelled

Shape: backtest.{wallet_public_id}.{run_public_id}.{event} (4 segments). wallet_public_id and run_public_id are UUID7; event is drawn from BacktestProgressEvent in data.py — the validator's BACKTEST_EVENTS set is derived via typing.get_args(BacktestProgressEvent) so the wire schema and the validator share one source of truth. Subscription-only prefixes (backtest., backtest.{wallet}., backtest.{wallet}.{run}.) are accepted on the subscribe.py / bridge.py paths.

Bus

Internal cross-service event bus over the shared broker.

Topic	Description
bus.delegate_offline	Delegate session torn down
bus.ai_review_decision	AI review decision recorded — fast-path fanout to the in-process AI review bus listener that resolves the strategy's await future (WS fanout of the operator's ack is the separate external ai_reviews.<user>.<strategy>.decision_ack topic). The matching bus.ai_review_request topic is intentionally NOT published today
bus.caps_violation_after_ai_approve	Caps violation post-approval

Shape: bus.{name} (2 segments) where name matches [a-z][a-z0-9_]* (snake_case). The payload's StrictDataSchema.type discriminator is the authoritative routing hint; the validator only enforces structural shape so the schema layer can reject unknown payload types.

Process Manager Events

Snapshot + per-run events emitted by ProcessLauncherService whenever the configured set of processes changes or a run transitions. Frontend subscribes via wsDispatcher and uses these to invalidate React Query caches (no REST polling).

Topic	Description
processes.events.summary.<instance_id>	Full snapshot of every configured process's current status (start/stop/crash/completion transitions)
processes.events.configured.<instance_id>	Snapshot of currently-known process names. Emits in two places: create_process_config (persisted-config-row create), and spawn_per_wallet_executors (per-wallet executor instances appear). It does NOT fire on persisted-row delete or on per-wallet teardown today. The snapshot's process_names is the union of persisted config names and instance_configs.keys().
processes.events.runs.<process_name>	Per-run lifecycle transitions: running / succeeded / failed / cancelled
strategies.events.list.<instance_id>	Snapshot of canonical class paths for every STRATEGY-role process config (drives the Strategies view)

Shape: 4 segments. The instance-id / process-name tail must match [A-Za-z][A-Za-z0-9_-]* (regex _PROCESSES_TOPIC_NAME_PATTERN). Numeric SNAPPER_COORDINATOR_INSTANCE_ID values are converted to topic-safe slugs by ProcessLauncherService.coordinator_topic_slug(); the default instance therefore emits on coord-0, and instance 7 emits on coord-7. The launcher emits these best-effort: if no MessagePublisher is wired the helper no-ops, and send failures are logged without corrupting process state.

Message Data Classes

All messages use typed Data classes (Pydantic) from snapper.messaging.schemas.data. Every Data class inherits from StrictDataSchema and carries:

Field	Type	Description
public_id	string	UUID7 external identifier, stable across REST and WS
type	string	Literal type discriminator
timestamp	datetime	Message creation time
session_id	string	UUID7 of the producer process session (required at construction)
sequence_id	int	Monotonic counter per topic within the session (required at construction)

session_id and sequence_id are allocated by producers from a shared SequenceTracker before the payload is constructed. MessagePublisher.send() serializes and forwards the already-complete payload without modifying provenance. Consumers can use these fields to detect message loss and producer restarts.

messaging.schemas.messages remains the parser/registry module: it exposes parse_message(), MessageParseError, GapEnvelope, MarketDataMessage, and MESSAGE_TYPE_MAP.

TickData

```
from datetime import UTC, datetime

from snapper.messaging.schemas.data import TickData

tick = TickData(
    public_id="019e1a2b-0000-7000-8000-000000000101",
    session_id="019e1a2b-0000-7000-8000-000000000001",
    sequence_id=1,
    timestamp=datetime.now(UTC),
    instrument="BTC-USD",
    exchange="kraken",
    bid=41990.0,
    ask=42010.0,
    last=42000.0,
    volume=0.5,
)
```

Fields:

Field	Type	Description
type	string	"tick"
public_id	string	UUID7 external identifier
instrument	string	Instrument symbol
exchange	string	Exchange name
bid	float | None	Best bid price
ask	float | None	Best ask price
last	float | None	Last traded price
volume	float	Volume
is_delayed	bool	

Field	Type	Description
		Whether the feed delivers exchange-delayed ticks (e.g. ~10 min for TradFi index futures); strategies must gate on this before treating the price as current
is_extended_hours	bool | None	Whether the tick occurred during an extended-hours TradFi session; None means the feed does not distinguish extended-hours ticks
timestamp	datetime	Timestamp
session_id	string	Producer session (inherited)
sequence_id	int	Monotonic counter per topic or logical stream (inherited)

CandleData

```
from datetime import UTC, datetime

from snapper.messaging.schemas.data import CandleData

candle = CandleData(
    public_id="019e1a2b-0000-7000-8000-000000000102",
    session_id="019e1a2b-0000-7000-8000-000000000001",
    sequence_id=2,
    timestamp=datetime.now(UTC),
    instrument="BTC-USD",
    exchange="kraken",
    timeframe="1h",
    open_at=datetime(2026, 1, 18, 12, 0, tzinfo=UTC),
    open=42000.0,
    high=42500.0,
    low=41800.0,
    close=42300.0,
    volume=1234.56,
)
```

Fields:

Field	Type	Description
type	string	"candle"
public_id	string	UUID7 external identifier
instrument	string	Symbol
exchange	string	Exchange
timeframe	string	Timeframe (1m, 5m, 1h, etc.)
open_at	datetime	Exchange-provided candle interval start time

Field	Type	Description
open	float	Open price
high	float	High
low	float	Low
close	float	Close price
volume	float	Volume
vwap	float | null	Volume-weighted average price (optional)
trades	int | null	Number of trades in the candle (optional)
complete	bool	Whether the bar's window has closed. false marks a provisional intra-minute update of a still-forming bar (the "living" candle a UI redraws in place); true marks the final bar. Defaults true. Native 1m bars derive it best-effort from the timeframe window; synthesized bars carry the aggregator's trustworthy-boundary flag.
timestamp	datetime	Timestamp

SignalData

```python
from datetime import UTC, datetime

from snapper.messaging.schemas.data import SignalData

signal = SignalData(
    public_id="019e1a2b-0000-7000-8000-000000000201",
    session_id="019e1a2b-0000-7000-8000-000000000010",
    sequence_id=1,
    timestamp=datetime.now(UTC),
    instrument="BTC-USD",
    exchange="paper",
    side="buy",
    strength=0.85,
    reason="RSI <= 30",
    strategy_name="rsi_btc_1h",
    price=42000.0,
    fired_at=datetime.now(UTC),
)
```

Fields:

Field	Type	Description
type	string	"signal"
public_id	string	UUID7 external identifier

Field	Type	Description
instrument	string	Symbol
exchange	string	Target exchange
side	string	"buy" or "sell"
strength	float	Signal strength 0.0-1.0
reason	string	Reason
strategy_name	string	Strategy name (required for paper exchange signals)
price	float	Price
fired_at	datetime	Domain time when signal was generated
timestamp	datetime	System timestamp
paired_group_id	string | None	Paired-execution group identifier; unset for standalone signals
paired_group_size	int | None	Number of legs in the group (must be >= 2)
paired_group_index	int | None	This leg's position in the group (0 <= index < size)
paired_group_policy	string | None	Coordination policy: simultaneous or sequential_handoff
paired_group_key	string | None	Canonical sorted {exchange}:{instrument}:{mode} leg-set key

Paper signals (exchange == "paper") require strategy_name to be set. The schema enforces this invariant at construction time so invalid paper signals cannot be created.

The paired-group descriptor is fail-closed and all-or-nothing: a model validator requires either every paired_group_* field unset (a standalone signal) or all five set together, with paired_group_size >= 2, 0 <= paired_group_index < paired_group_size, and a non-empty paired_group_key. The descriptor (without paired_group_key) is also carried transport-only on the downstream order schemas — OrderRequestData, OrderData, ExecutionData, and OrderEventData — so subscribers can attribute order flow to its paired group.

OrderRequestData

```
from datetime import UTC, datetime

from snapper.messaging.schemas.data import OrderRequestData

order = OrderRequestData(
    public_id="019e1a2b-0000-7000-8000-000000000301",
    session_id="019e1a2b-0000-7000-8000-000000000020",
```

```
    sequence_id=1,
    timestamp=datetime.now(UTC),
    instrument="BTC-USD",
    exchange="kraken",
    client_order_id="ord_123",
    side="buy",
    order_type="limit",
    price=42000.0,
    quantity=0.1,
    strategy_id="rsi_btc_1h",
    mode="live",
)
```

order_type carries the core order-type vocabulary — market, limit, stop, or stop_limit (the OrderType Literal). Venue wire spellings such as stop-loss / stop-loss-limit never appear in orders.commands.* or orders.events.* payloads: the executor translates to each venue's wire contract at the venue boundary, immediately before the exchange call. stop_price is the trigger price, required for stop / stop_limit orders — before any venue send, the executor refuses a stop-typed request without a trigger (and any order type outside the core vocabulary), publishing a definitive rejected and recording a durable order_rejected venue event.

ExecutionData

```
from datetime import UTC, datetime

from snapper.messaging.schemas.data import ExecutionData

execution = ExecutionData(
    public_id="019e1a2b-0000-7000-8000-000000000401",
    session_id="019e1a2b-0000-7000-8000-000000000030",
    sequence_id=1,
    timestamp=datetime.now(UTC),
    instrument="BTC-USD",
    exchange="kraken",
    client_order_id="ord_123",
    exchange_order_id="KRAKEN-456",
    side="buy",
    price=42000.0,
    size=0.1,
    last_size=0.1,
    last_price=42000.0,
    fee=0.001,
    fee_asset="USD",
    status="filled",
    executed_at=datetime.now(UTC),
)
```

OrderData

OrderData is the canonical event for order state. It carries the request-time margin metadata (leverage, reduce_only) all the way to subscribers and the REST /orders endpoint, so the frontend can render shorts and reduce-only intent.

```
from datetime import UTC, datetime

from snapper.messaging.schemas.data import OrderData

order_status = OrderData(
    public_id="019e1a2b-0000-7000-8000-000000000501",
    session_id="019e1a2b-0000-7000-8000-000000000020",
    sequence_id=2,
    timestamp=datetime.now(UTC),
    instrument="BTC-USD",
    exchange="kraken",
    client_order_id="ord_123",
    exchange_order_id="KRAKEN-456",
    side="sell",
    order_type="limit",
    size=0.1,
    filled_size=0.0,
    status="accepted",
    price=42000.0,
    created_at=datetime.now(UTC),
    leverage=3,
    reduce_only=False,
)
```

HeartbeatData

```
from datetime import UTC, datetime

from snapper.messaging.schemas.data import HeartbeatData

heartbeat = HeartbeatData(
    public_id="019e1a2b-0000-7000-8000-000000000601",
    session_id="019e1a2b-0000-7000-8000-000000000001",
    sequence_id=1,
    timestamp=datetime.now(UTC),
    component="zmq_broker",
    status="healthy",
    sequence=1,
    lag_ms=5,
)
```

ProcessSummaryEventData

Full snapshot published on processes.events.summary.<instance_id> whenever the launcher detects a status transition. Frontend replaces its React Query cache entry wholesale — no diff reconciliation.

Fields:

Field	Type	Description
type	string	"process_summary_event"
coordinator	string	Topic-safe slug of the emitting node (default coord-0); mirrors the topic suffix so consumers can attribute rows to the coordinator that sampled them
processes	list[ProcessSummaryItem]	Unordered snapshot of per-process rows (configured first in repository row order, then runtime per-wallet instances — consumers must not rely on this order)
snapshot_at	datetime	Bus time the snapshot was assembled

ProcessSummaryItem fields: name, running, enabled, role, lifecycle, active_public_id, rss_bytes (int | None, subprocess resident set size in bytes; None for thread-mode/ unsampled processes), and cpu_percent (float | None, whole-process CPU% which can exceed 100 on multi-threaded children; None under the same conditions and 0.0 on the first sample after a restart). PID / command / exit_code are intentionally absent — they only exist for native subprocesses, so detailed status is fetched via REST when needed.

ProcessConfiguredEventData

Published on processes.events.configured.<instance_id> from two launcher sites only: create_process_config (persisted-config-row create) and spawn_per_wallet_executors (per-wallet executor instances appear). The launcher does NOT emit on persisted-row delete or on per-wallet teardown today. process_names is the sorted union of persisted config names AND instance_configs.keys() at snapshot time.

Fields:

Field	Type	Description
type	string	"process_configured_event"
process_names	list[string]	All configured process names at snapshot time
snapshot_at	datetime	Bus time the snapshot was assembled

ProcessRunEventData

A single process-run lifecycle transition. Per-run (not snapshot) so consumers can append to their run-history view without re-fetching.

Fields:

Field	Type	Description
type	string	"process_run_event"
process_name	string	Owning process
run_id	string	Stable identifier for this run
status	string	running / succeeded / failed / cancelled (mirrors ProcessRunStatusEnum.value)
started_at	datetime	Bus time the run kicked off
completed_at	datetime | None	Bus time the run finished; None while in flight
exit_code	int | None	Native subprocess exit code; None for async-task processes (the launcher folds non-zero exits into the run record's error field) and None while in flight

StrategyListEventData

Published on strategies.events.list.<instance_id> whenever a STRATEGY-role process config is created or its start/stop state transitions. The launcher derives the payload from **persisted** process configs filtered by role == STRATEGY — NOT from the in-memory StrategyFactory.STRATEGY_CLASSES registry (which is static after import time). Frontend uses this to refresh the Strategies view without REST polling.

Fields:

Field	Type	Description
type	string	"strategy_list_event"
strategy_classes	list[string]	Sorted canonical class paths of every active STRATEGY-role process config
snapshot_at	datetime	Bus time the snapshot was assembled

Publisher

MessagePublisher (recommended)

MessagePublisher sends fully constructed payloads to an explicit stream_key. Producers allocate session_id and sequence_id from a shared SequenceTracker before constructing the payload, then pass both the topic and payload to MessagePublisher.send().

```
import zmq.asyncio
from snapper.messaging.infrastructure.validated_socket import
ValidatedPublisher, apply_hwm, HWM_MARKET_DATA
from snapper.messaging.infrastructure.publisher import MessagePublisher,
SequenceTracker
from snapper.messaging.topics.builders import topic_for_message

ctx = zmq.asyncio.Context()
raw_socket = ctx.socket(zmq.PUB)
apply_hwm(raw_socket, sndhwm=HWM_MARKET_DATA)
raw_socket.connect("tcp://127.0.0.1:7500")

tracker = SequenceTracker()
publisher = MessagePublisher(ValidatedPublisher(raw_socket), tracker)

topic = topic_for_message(candle_data)

await publisher.send(topic, candle_data)
publisher.close()
```

For brevity, the example assumes candle_data was already constructed with session_id and sequence_id allocated from tracker.

One SequenceTracker per component process; all MessagePublisher instances within that component share it. Counters survive socket reconnects — only a full component restart creates a new session.

Per-Topic Counters

SequenceTracker maintains monotonic counters keyed by **ZMQ topic** (e.g. market.kraken.BTC-USD.candles.1m, orders.events.kraken.BTC-USD.executed). Each topic has its own independent counter. Producers allocate provenance using the final ZMQ topic string and then pass that same topic to MessagePublisher.send().

For DB-only writes that do not flow over ZMQ (instruments, users, settings seed data), the counter key is the destination table name as a logical identifier.

For WS control and REST middleware paths, logical channel names are used: server.control, server.telemetry, rest.control, rest.telemetry.

For paper market data or other cases where the topic cannot be derived from the payload alone, pass an explicit topic override:

```
topic = "market.paper.kraken.BTC-USD.ticks"
await publisher.send(topic, tick_data)
```

ValidatedPublisher (low-level)

Direct access to the socket wrapper without StrictDataSchema serialization helpers:

```
import zmq.asyncio
from snapper.messaging.infrastructure.validated_socket import
ValidatedPublisher, apply_hwm, HWM_MARKET_DATA

ctx = zmq.asyncio.Context()
raw_socket = ctx.socket(zmq.PUB)
apply_hwm(raw_socket, sndhwm=HWM_MARKET_DATA)
raw_socket.connect("tcp://127.0.0.1:7500")
publisher = ValidatedPublisher(raw_socket)

topic = "market.kraken.BTC-USD.candles.1h"
payload = candle_data.publish_to(topic)
await publisher.send_multipart(topic, payload)
publisher.close()
```

Subscriber

Subscribing to messages via validated socket wrapper around a raw ZMQ SUB socket:

```
import zmq.asyncio
from snapper.messaging.infrastructure.validated_socket import
ValidatedSubscriber, apply_hwm, HWM_MARKET_DATA
from snapper.messaging.schemas.messages import parse_message

ctx = zmq.asyncio.Context()
raw_socket = ctx.socket(zmq.SUB)
apply_hwm(raw_socket, rcvhwm=HWM_MARKET_DATA)
raw_socket.connect("tcp://127.0.0.1:7501")  # broker XPUB
subscriber = ValidatedSubscriber(raw_socket)
subscriber.subscribe("market.kraken.BTC-USD.")

while True:
    topic, payload = await subscriber.recv_multipart()
    envelope = parse_message(payload.decode())
    print(f"Received {envelope.type} on {topic}")
```

Market Data Publisher

Built-in publisher for exchange data:

```
snapper feed --symbols "BTC-USD,ETH-USD"
```

Programmatically:

```
from snapper.messaging.publishers.kraken import KrakenMarketDataPublisher

async def run_feed():
    publisher = KrakenMarketDataPublisher(symbols=["BTC-USD", "ETH-USD"])
    await publisher.start()
```

Candle Identity Cache

On startup the publisher loads the latest candle public_id per (instrument, timeframe) pair from the database into an in-memory cache. The startup load is bounded to a recent open_at window (_CANDLE_ID_CACHE_LOOKBACK, 2 days) so the warm-up rides the (instrument, open_at) index instead of scanning the full (multi-hundred- million-row) candles table — an unbounded scan otherwise blocked the publisher's startup for minutes before its WebSocket connected. Bounding it is safe: only the *current* open interval needs identity continuity, and intervals older than the window are closed (no further updates). When a new tick arrives for an existing candle interval (open_at matches), the cached public_id is reused so the same logical candle keeps a stable identity across ZMQ, WebSocket, and the database. When open_at advances to a new interval (or falls outside the warm-up window), a fresh UUID7 is minted and the cache entry is replaced. This avoids any DB reads on the hot path.

Higher-Timeframe Candle Synthesis

When more than one timeframe is configured (timeframes setting, e.g. ["1m", "1h", "1d"]), the publisher subscribes to the venue's 1m stream **only** and synthesizes every higher timeframe client-side from the finalized 1m candles. Each synthesized bar is published on the same topic family as a native candle (market.{exchange}.{instrument}.candles.{timeframe}), so subscribers cannot tell a synthesized 1h/1d bar from a venue-native one.

Key properties:

- **UTC boundaries** — every timeframe aligns to UTC (1d closes at 00:00 UTC, fixed intervals to floor(unix / interval) * interval). This matches Kraken's native OHLC boundaries and the Polygon historical corpus, so live bars line up with backtest/warm-up data.
- **Finalize once** — the venue emits many in-progress frames for the open minute; a minute is folded into the higher timeframes exactly once, with its final value, when a later minute is observed (per symbol).
- **Emit on close** — a higher-TF bar is published only when its window has closed (a finalized 1m at or after the window end). open_at is the window START; consumers must treat closed_at = open_at + timeframe as the true close — do not act on a bar before then.
- **Persisted (provenance-tagged)** — synthesized bars are published to ZMQ and, when a higher timeframe is configured for persistence (off by default — timeframes defaults to ["1m"]), also written to the candles table tagged source='synthesized' and carrying the aggregator's complete trustworthy-boundary flag, under the SAME persist policy (_should_persist_row) as the 1m stream. Provenance is source ∈ {native, calculated, synthesized} + complete (a trustworthy-boundary bool, NOT a full-minute-coverage assertion). native = venue-precomputed upstream OHLC

(Kraken spot ohlc:1m, Polygon history, Kraken futures/equities 1m REST-aggregate backfills); calculated = Snapper-built live 1m from the trade/quote stream (Kraken futures/equities, Walutomat — tagged from migration 0012 forward); synthesized = higher-TF rollups. The candle READ path single-source cutover is gated by the candle_single_source setting (default OFF): while OFF, the /api/candles smart route still derives 5m/15m/30m on-read from the 1m cache; when ON, those frames serve single-source from the persisted plane (get_candles), retiring the on-read derive_snaps rollup and closing the dual-source hazard. Enable it only after the persisted plane is populated and verify-candle-coverage passes (else the read would be empty/short). 1m stays cache-served and 1h/4h/1d are already DB-served regardless.

- **Native 1m persistence is final-only by default** — every native 1m frame is published to ZMQ (the living candle), but the NativeCandleFinalizer holds the in-progress minute and persists exactly ONE complete=True bar per window — on the next minute's first frame (boundary), or via a 30s wall-clock flush for an illiquid/stalled symbol, or on shutdown drain. This eliminates the intra-minute SCD2 "temporary candle" churn. Operator caveat: a 1m bar's DB row therefore lags the live minute by up to one flush interval (~35s); the cache//api/candles live reads are ZMQ-fed and unaffected, and on a hard crash the current held minute self-heals on restart (the venue re-sends it). Set persist_intermediate_candles=true to also persist the in-progress complete=false frames (superseded by the final).
- **Restart rebuild** — on startup the open window of each higher timeframe is rebuilt from the persisted 1m history (excluding the current open minute, which the live stream finalizes), so a mid-window restart does not truncate that window's bar. The seed reads each row's durable complete flag (not a write-time heuristic) to decide finality, so a finalized illiquid bar sealed with a pre-close write timestamp is correctly seeded.
- **Forward-fill (opt-in, default off)** — with candle_forward_fill enabled, a time-driven flush (every 30s) seals a wall-clock-ended window that no later 1m arrived to close, and forward-fills empty windows with a flat bar (open = high = low = close = vwap = prior close, volume = 0, trades = 0) so a thin instrument still yields a contiguous series. It operates only on the live region (windows that opened at or after the publisher's live epoch), so seeded/restart windows stay owned by the data path, and a late 1m for an already-sealed window is dropped (counted in late_rolls_after_close), never re-emitted. Enable it ONLY where the warm-up corpus is contiguous at the timeframe — 24/7 crypto qualifies; session-based equities (and 24/5 FX without verified weekend rows) must leave it off, or the live series would carry bars the strategy was never validated on. This is ENFORCED per venue: the global candle_forward_fill setting is gated by each publisher's _supports_forward_fill() (only the kraken-spot publisher returns true), so on any other venue — or in paper replay —

forward-fill is forced OFF with a warning even if the flag is set. The flush wall-clock also applies a small grace so a just-ended minute is not sealed before the venue can deliver its final frame.

- **Native-feed replacement (kraken spot)** — kraken spot is the one venue with a native higher-TF OHLC feed; synthesizing those timeframes from 1m REPLACES it (a deliberate one-mechanism choice — the rolled-up VWAP/trades approximate the native bar). The publisher logs a one-time warning naming the replaced timeframes so the fidelity tradeoff is not silent.
- **Paper replay** — the paper publisher synthesizes higher timeframes from the replayed 1m history exactly as a live publisher does (so backtest bars match forward-test bars and no persisted higher-TF rows are required), but anchors the aggregator's live epoch at the REPLAY START rather than wall-clock now — so historical windows are trustworthy and emitted instead of being suppressed as pre-epoch. Paper does not run the restart seed (the full replay rebuilds every window) and never forward-fills (it is not a continuous-corpus live feed).

Micro-Batch DB Persistence

DB writes are decoupled from the ZMQ publish path via dedicated per-stream writer tasks. The producer loops (candles, ticks, trades) are ingest-only: each drains its exchange subscription iterator, publishes to ZMQ, and enqueues the resulting row onto a bounded per-stream write queue. A dedicated writer task per stream consumes its queue with a long-lived database session, accumulating rows and flushing on a size or age threshold — DB commit latency never blocks the producer loop:

- **Size trigger**: candles flush at 100 rows, ticks/trades at 500 rows
- **Age trigger**: all streams flush after 50 ms since the first batch item
- **Publish-first**: ZMQ delivery happens before DB persistence; subscribers receive data without waiting for the database

Thresholds are configurable via write_buffer_flush_ms, write_buffer_candle_max_rows, write_buffer_tick_max_rows, and write_buffer_trade_max_rows settings (cached at publisher start, require process restart to change).

The write queues are bounded (20 000 rows for ticks and candles, 5 000 for trades) with drop-oldest overflow: when a queue is full, the enqueue helper evicts the oldest queued row to make room for the new one and emits a rate-limited WARNING with the drop count. This is a persistence backlog, not data loss on the wire — subscribers already received the evicted rows over ZMQ; only their DB persistence is skipped.

If a writer task's database session is lost mid-flush, the task holds its unflushed rows, reopens the session with capped exponential backoff (1 s doubling up to 30 s), and resumes draining — queued and in-flight rows survive session loss.

On shutdown (launcher cancellation or direct stop), remaining rows are flushed by the writer drain loops, which keep draining the queue and the in-flight batch after the running flag goes false until both are empty; stop() joins each queue before awaiting its writer task. The producer loop's asyncio.wait poll still uses a persistent future that is never cancelled on timeout, preserving the underlying exchange subscription iterator — the age-based flush itself is driven by the writer task's batch timer.

Candle flushes that hit an IntegrityError (e.g. out-of-order timestamps) fall back to row-by-row retry, isolating the bad row without losing the rest of the batch. The repository upsert_candles() contract is unchanged.

Per-stream flush error counters (_flush_errors) drive the heartbeat status field: "warning" if any stream has errors, "healthy" otherwise. Counters reset to zero on the next successful flush of that same stream.

Order Executor

Service executing orders:

```
snapper executor -e kraken
```

Executor:

1. Subscribes to orders.commands.{exchange}.{instrument}.submit, .cancel, and .replace topics, plus system.symbol_aliases and system.settings
2. Receives OrderRequestData and OrderCancelData from the durable trade outbox, and accepts direct OrderReplaceData command frames from compatible publishers
3. Executes order via exchange API
4. Persists VenueEvent rows for accepted (order_accepted), fill (fill_observed), terminal (order_terminal), rejected (order_rejected), ambiguous-submit (order_submit_unknown), and breaker-open (order_breaker_open) observations
5. Publishes ExecutionData, OrderData, or lightweight OrderEventData

Before any venue call, each submit passes two gates. The duplicate-submit guard drops outbox replays of an already-evidenced client_order_id, checking the live pending entry, the unhealed-accept queue, and the durable has_order_submit_evidence probe (which excludes order_rejected rows so a legitimate retry after a definitive reject still flows). A failed durable probe is fail-closed — the command is dropped as if duplicate — and dropped duplicates publish nothing, with one exception: a replay whose durable evidence includes order_breaker_open reruns the breaker-open disposition described below instead of dropping silently, so an executor crash mid-disposition cannot leave the engine's intent held forever. The staleness gate then handles commands older than the dispatch TTL (TRADE_COMMAND_DISPATCH_TTL_S, default 30 s, measured from the published frame's signaled_at, which the outbox derives from the durable trade-command row's created_at)

with venue-truth-backed outcomes: a single client-id lookup runs first; an order found on the venue is adopted as accepted, an unverifiable venue (lookup unsupported or unreachable) makes the frame drop silently, and only a verified absence publishes rejected and records an order_rejected venue event — the durable row is written only after a confirmed publish, so a failed publish leaves the command for the reconciliation loop to release durably.

When a submit fails ambiguously (the order may exist on the venue), the executor never fabricates a rejection. It parks the pending entry and verifies venue truth by client id via find_order_by_client_id: a found order is adopted as accepted, while a definitive rejection requires two consecutive authoritative not-found answers — a lookup that could not be checked never counts as the venue saying no. If verification cannot resolve, the executor publishes OrderData on orders.events.{exchange}.{instrument}.unknown (non-terminal; the engine holds its in-flight guard) and the reconciliation loop re-verifies parked entries each cycle until the order resolves to accepted or rejected.

A submit refused because the venue circuit breaker is open gets a distinct disposition instead of a fabricated venue rejection: the executor records a durable order_breaker_open venue event (counted as duplicate-submit evidence, so a late redispatch of the same command never resubmits), marks the durable command FAILED, and only then publishes rejected with reason="circuit_breaker_open" in the OrderData payload — subscribers can distinguish the local infrastructure refusal from a venue rejection. If any step fails, the pending entry is parked and the reconciliation loop reruns the sequence; the engine releases its in-flight intent only on the confirmed publish.

Fills arrive on the venue's private execution WebSocket stream. A supervisor task owns the stream's lifecycle: any termination — SDK reconnect-budget exhaustion during an outage, a raised error, even a clean generator return — counts as death and triggers a respawn with capped jittered backoff (1 s doubling to 60 s; a stream that survived 5 minutes resets the backoff), so a dead stream can no longer leave fills permanently dark while the process keeps reporting healthy. Before each respawn the supervisor runs a best-effort reconciliation pass that heals the dark-window fill gap first, and the resubscribe is idempotent: both Kraken clients suppress subscribe-time snapshots (spot requests snap_orders/snap_trades as explicit false, futures drops fills_snapshot frames), so neither a respawn nor an in-budget SDK reconnect that replays the cached subscription can re-deliver already-booked fills. Venues without WebSocket executions skip the stream entirely; the periodic reconciliation loop is their only fill source.

Fill booking is atomic per order: a lock on the pending entry serializes the dedupe gate, delta computation, durable fill_observed write, execution publish, watermark advances, and lifecycle removal across the live stream task, reconciliation correctives, and cancel handling. Duplicates are dropped via an exec-id LRU plus a cumulative-monotonic guard; the committed cumulative advances only after a successful publish, so a fill whose publish failed is absorbed by the venue's redelivery or by the next fill's cumulative-anchored delta and published deltas always sum to venue truth.

Failed VenueEvent persistence is fail-closed on the fill path: the fill_observed write precedes the execution publish, and a write failure aborts the publish. The accepted path differs because the order is already live on the venue: the accepted status is published anyway, the failed order_accepted write is parked in _unhealed_accept_events, and the reconciliation loop retries the durable write each cycle until it heals.

Replace commands are parsed and wallet-scoped like submit/cancel commands, but no exchange client implements atomic replace today; the executor logs the request and publishes a lightweight OrderEventData(event="rejected") on orders.events.{exchange}.{instrument}.rejected so callers can use an explicit cancel + new-order workflow.

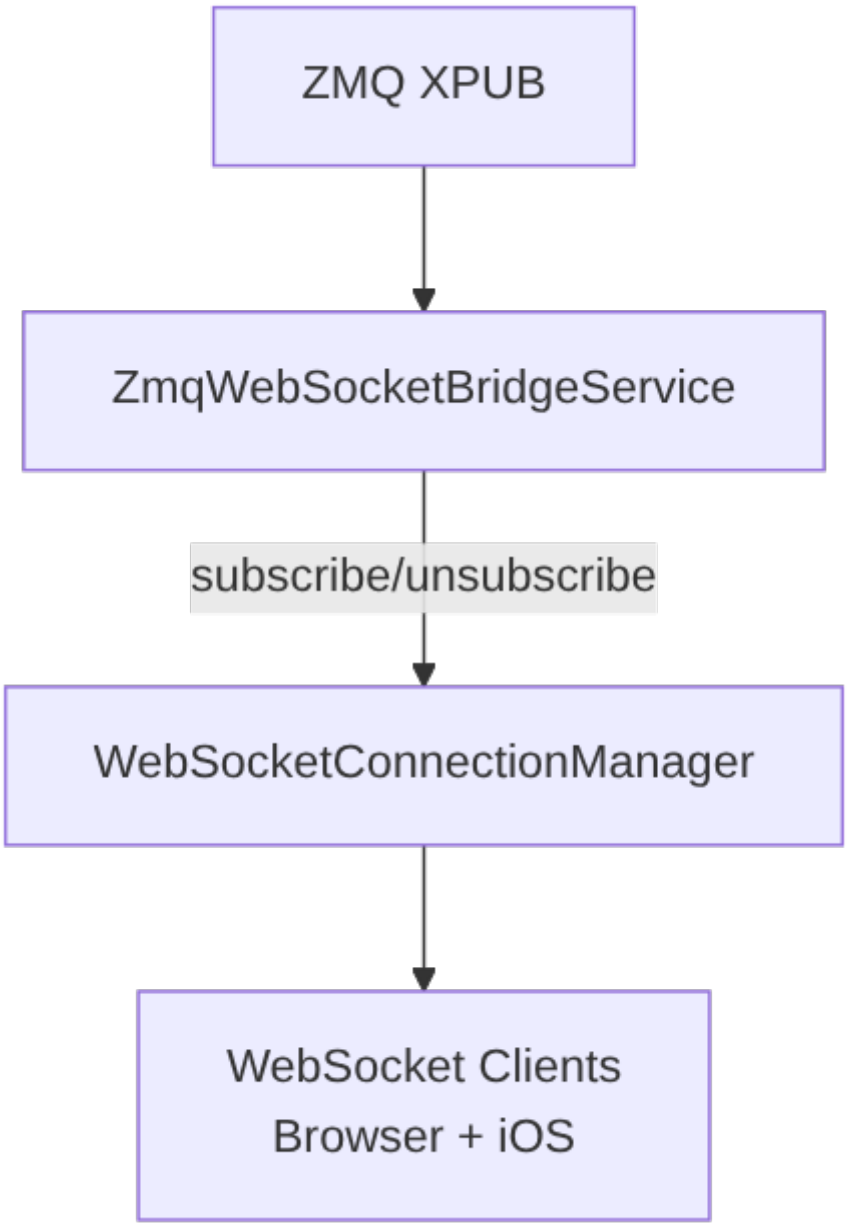
The trade runtime subscribes to orders.events.* to keep TradeService and BalanceService in sync during normal operation. VenueEvent rows are used as the durable recovery and reconciliation backbone. At startup the executor recovers open orders from those rows and the executions log (rows exist only for successfully published fills): recorded-but-unpublished fills are republished under their original exec ids (every consumer dedupes by exec id), and the remaining venue-ahead gap is emitted as corrective fills with deterministic synthetic ids — fills that landed while the executor was down are projected instead of being silently re-baselined away (see docs/operations.md → Recovery-time corrective fills). Corrective fills carry real venue fees: the venue's cumulative commission is tracked per fee currency against what was already published, and each corrective's fee/fee_asset carries exactly the not-yet-attributed remainder, so per-fill live fees are never re-charged. A corrective emits fee-less only when the venue exposes no usable fee source, or after the bounded deferral for a transiently unavailable source expires with a CRITICAL log.

Because parked entries are in-memory, the executor's reconciliation loop also closes restart gaps against venue truth: open venue orders carrying our client order ids but missing from pending state are adopted under the original request rebuilt from the durable command row, and dispatched commands with no resolving venue evidence are verified by client id — a found order is adopted, while repeated authoritative absence (only with the dispatch TTL enabled, and bounded by an age cutoff) publishes rejected. Adoption-shaped accepted events carry reason="adopted" in the OrderData payload — the engine uses it to re-arm its in-flight guard for the adopted order — and a failed adoption publish is parked and retried by the reconciliation loop until it lands. Subscribers may therefore see accepted or rejected events for orders the current executor process never submitted.

PlanExecutorService (see docs/architecture.md → Execution Plans) also subscribes to orders.events. and market. on the broker XPUB, routes ExecutionData → fill propagation, OrderData → plan status transitions, and TickData → evaluator dispatch. Plan cancels flow through the same TradeCommand outbox as submits, with command_type="cancel" branching _outbox_publish to emit OrderCancelData on orders.commands.{exchange}.{instrument}.cancel.

ZMQ-WebSocket Bridge

Bridge between ZMQ and WebSocket clients (browser and iOS):



Bridge automatically:

- Subscribes to ZMQ topics when WebSocket client subscribes
- Validates each inbound ZMQ message as JSON and runs GapDetector.check() before forwarding
- Drops messages with malformed JSON (logged as WARNING; counted in invalid_messages per topic)
- Applies per-frame scope filters for ai_reviews.*, orders.events.*, and alerts.*; malformed scoped frames fail closed before fan-out
- Forwards messages via _forward_to_clients with per-subscription backpressure

- Throttles forwarding per subscriber: the live WebSocket subscribe path registers every subscription with a flat 100 ms interval, regardless of topic (TOPIC_REGISTRY carries per-prefix throttle_ms metadata, but only the bridge's schema-aware registration path reads it, and the WebSocket subscribe handler does not use that path)
- Drops market data when a client exceeds MAX_PENDING_MESSAGES_MARKET (100)
- Disconnects slow clients on trade topics when exceeding MAX_PENDING_MESSAGES_TRADE (1000)
- Unsubscribes when last client disconnects
- Records control events (subscribe errors, client disconnects) to the control table

Gap Detection

GapDetector tracks session_id and sequence_id partitioned by the tuple (received_topic, wallet_public_id) — wallet scoping avoids false gaps when interleaved per-wallet streams share a topic — and emits log messages when it finds sequence gaps or producer session resets.

```
from snapper.messaging.infrastructure.gap_detector import GapDetector

detector = GapDetector()
detector.check(received_topic, session_id, sequence_id,
wallet_public_id=wallet_public_id)
```

Rules:

- Messages without provenance (session_id == "" or sequence_id == 0) are **rejected** with a WARNING log and a rejected_unstamped counter increment. This makes unstamped traffic observable rather than silently passing through.
- First message on a (topic, wallet_public_id) stream: sets baseline. If sequence_id > 1 the subscriber joined mid-stream (logged INFO).
- Same (topic, wallet_public_id) stream, new session_id: producer restarted (logged INFO, counter resets).
- In-order message (sequence_id == expected): accepted silently.
- Gap (sequence_id > expected): logs WARNING with the missing range, then advances.
- Duplicate / reorder (sequence_id < expected): logs DEBUG, state unchanged.

GapDetector.check() returns True when the message carries valid provenance and was processed, False when rejected as unstamped.

GapDetector is wired into the ZMQ-WebSocket bridge and into subscriber loops inside the executor, trade runtime coordinator, and strategy _listen_loop().

Message Logger

Debug tool for monitoring ZMQ traffic:

```
snapper zmq-logger --payload --audit-file logs/zmq.jsonl
```

Logs all messages passing through the broker.

Configuration

Endpoints in .env:

```
ZMQ_BROKER_XSUB=tcp://127.0.0.1:7500
ZMQ_BROKER_XPUB=tcp://127.0.0.1:7501
```

Throttling

TopicSchema defines throttling per topic:

```
TopicSchema(
    pattern="market.",
    category="market",
    throttle_ms=100,  # Max 10 msg/s
)
```

Message Validation

ValidatedPublisher and ValidatedSubscriber validate **topic strings**, not message payloads. ValidatedPublisher.send_multipart() calls validate_topic() before sending; ValidatedSubscriber.subscribe() calls validate_subscription_pattern() before subscribing. Both raise TopicValidationError on invalid topics.

High Water Mark (HWM) Policy

ZMQ sockets use explicit high water marks via apply_hwm() from validated_socket.py to bound queue depth and prevent silent message loss:

Tier	Constant	Value	Used by
Order flow	HWM_ORDER_FLOW	0 (unlimited)	Executors, trade runtime coordinator
Broker	HWM_BROKER	50 000	XSUB (rcvhwm), XPUB (sndhwm)

Tier	Constant	Value	Used by
Market data	HWM_MARKET_DATA	20 000	Publishers, strategies, bridge, settings, symbol updaters
Audit	HWM_AUDIT	10 000	Message logger

apply_hwm() must be called **before** connect() or bind().

Retry and Resilience

Sockets automatically:

- Reconnect on connection loss
- Buffer messages when broker unavailable
- LINGER=0 on close (discard pending)
- HWM applied before connect/bind (see above)

Topic Derivation

topic_for_message() in snapper.messaging.topics.builders derives the canonical ZMQ topic from a typed Data class. Producers typically call it before MessagePublisher.send().

```
from snapper.messaging.topics.builders import topic_for_message

topic = topic_for_message(candle_data)
await publisher.send(topic, candle_data)
```

Paper market data topics include a source_exchange segment that is not carried in the payload, so they must always be published with an explicit topic override.

WebSocket Message Provenance

All WebSocket protocol messages (authentication, subscription management, ping/pong, errors) inherit directly from StrictDataSchema, so they carry the same provenance fields as ZMQ data payloads: public_id, session_id, sequence_id, and timestamp. Every payload item in the system — whether it flows over ZMQ, REST, or WebSocket — has a uniform provenance envelope.

Audit Tables: Control and Telemetry

Two destination tables provide always-available observability for non-domain traffic:

- **control** — Always-on audit for commands, authentication events, subscribe/unsubscribe messages, and REST mutation requests. Recording is wired in three places:
- **WS handlers** — _record_ws_control() records auth, subscribe, unsubscribe, and error events with transport="ws". Client causation linkage is extracted from inbound messages via _extract_client_provenance() and stored as client_session_id and client_public_id on the control row.
- **REST middleware** — ClientProvenanceMiddleware._record_control() records every mutation (POST / PUT / DELETE / PATCH per _MUTATION_METHODS) with transport="rest", redacted payload, outcome (ok/error/exception), and server-side provenance.
- **ZMQ bridge** — _record_bridge_control() records subscribe errors and client disconnects with transport="zmq".

All control writes use a finally block so the record is persisted regardless of whether the request succeeded or failed. The write is non-blocking: any DB failure is logged and swallowed so the response already sent to the client is never invalidated.

- **telemetry** — Toggleable high-volume table for pings, heartbeats, pongs, and GET read requests. Recording is gated by the TELEMETRY_RECORDING_ENABLED environment variable (default false). When disabled, SequenceTracker counters still increment — only the DB write is skipped. Recording is wired in:
- **WS handlers** — _record_ws_telemetry() records ping/pong/heartbeat events
- **REST middleware** — _record_telemetry() records GET reads (health, status, entity endpoints)

The non-blocking audit invariant applies to both tables: audit writes never reject, delay, or invalidate the primary request/message flow.

Best Practices

1. **One broker per system** — All components connect to the same broker
2. **Topic hierarchy** — Use hierarchy for filtering (market.kraken.*)
3. **Data types** — Always use typed Data classes from messaging.schemas.data
4. **Provenance** — Use MessagePublisher (not ValidatedPublisher directly) so every message carries session_id and sequence_id for gap detection
5. **One SequenceTracker per component** — Create it once at start() and share across all publisher instances; restart creates a new session
6. **Heartbeats** — Keep component-specific heartbeat cadences small and regular

7. **Graceful shutdown** — Close sockets with LINGER=0

Observability

Snapper exposes operator-facing health metrics so a dashboard can see the gradient toward exhaustion before a crash. The current observability surface is **Cluster A** of the three-cluster monitoring initiative: process-level counters sampled into an in-memory ring buffer with a REST query API.

Cluster B (per-table SCD2 DB stats — /api/metrics/db/tables) and Cluster C (retention policy framework — /api/metrics/retention) are both live alongside Cluster A. Cluster C defines what "archivable" means, which Cluster B's archivable counter then mirrors.

Endpoints

All metrics routes are gated by Permission.READ_SYSTEM_STATUS (held by AI_DELEGATE, VIEWER, OPERATOR, ADMIN). The two POST routes also require CSRF (cookie-auth path); Authorization: Bearer requests bypass CSRF per the project-wide auth contract.

Method	Path	Purpose
GET	/api/metrics/system	Latest snapshot
GET	/api/metrics/system/history?since&until&limit	Windowed slice of the ring buffer
POST	/api/metrics/system/tracemalloc/start?duration_s	Arm Python tracemalloc with auto-stop deadline
POST	/api/metrics/system/tracemalloc/stop	Disarm tracemalloc + cancel pending deadline
GET	/api/metrics/notifications	Notify-sidecar outbox delivery counters
GET	/api/metrics/rest-rate	Rolling REST call rates + rate-limit utilization per exchange
GET	/api/metrics/retention	Retention scheduler status and policy counters
GET	/api/metrics/db/tables	Per-table row-count and SCD2 lifecycle counters

Sibling operator health endpoint

GET /api/health/egress is a sibling operator health endpoint outside the /api/metrics/* surface. It shares the same Permission.READ_SYSTEM_STATUS gate as the metrics routes and returns a per-container egress pool status snapshot, aggregated cross-process over the system.egress.snapshot ZMQ topic, with sidecar system.egress.transfer samples joined to matching SOCKS5 route rows. See api.md (GET /api/health/egress) for the full wire schema.

Data-liveness watchdog

Beyond the REST surface, each exchange feed runs a per-subscription data-liveness watchdog (SubscriptionHealthTracker, infrastructure/exchanges/_subscription_health.py): a confirmed subscription that receives no data for data_stale_threshold_s (default 300.0s; ack_timeout_s default 15.0s guards the initial subscribe ACK) is marked stale and emits a one-shot log line. It is log-only — stream-gap signals surface in the feed logs, not as a REST metric.

Failure contract

If the snapshotter singleton failed to start at lifespan time, the app.state.system_metrics_snapshotter attribute stays None and every /api/metrics/system* route returns HTTP 503 with body {"detail": "system metrics snapshotter not available"}. The rest of the application continues to serve other endpoints.

Cold-start contract

SystemMetricsSnapshotter.start() takes one eager synchronous sample BEFORE returning, so the first request after lifespan startup completes hits a populated buffer.

Notification delivery metrics

GET /api/metrics/notifications does not touch the snapshotter or the ring buffer (the failure contract above does not apply to it). Each request aggregates active alert_deliveries rows (SCD2 predecessor versions excluded) into per-status delivery totals — sent, failed, APNs-410 unregistered, cancelled_scope — plus the queued backlog depth of the notify sidecar's retry loop. Statuses absent from the database report 0. See api.md for the NotificationMetricsResponse wire schema.

Snapshot fields

Each sample captures eight nested groups, a top-level bus_time timestamp, and two top-level flags. All field names are stable; downstream consumers (frontend, iOS) regenerate from the backend OpenAPI / JSON-Schema export and pin the wire shape.

process

Field	Type	Description
pid	int	Process ID.
uptime_seconds	float	Monotonic-clock seconds since the snapshotter was constructed.
status	str	psutil process status (running, sleeping, ...).
num_threads	int	Live thread count.
num_fds	int	Open file descriptor count.
num_connections	int	Open TCP/UDP socket count.

cpu

Field	Type	Description
process_percent	float	psutil.Process.cpu_percent reading at sample time.
user_time_seconds	float	Cumulative user-mode CPU.
system_time_seconds	float	Cumulative system-mode CPU.
cgroup_quota_microseconds	int | null	cgroup CPU quota microseconds per period; null when absent or unlimited.
cgroup_throttled_count	int | null	Cumulative throttle event count from cpu.stat nr_throttled; null when cgroup absent.

memory

Field	Type	Description
rss_bytes	int	Resident set size (RSS).
rss_peak_bytes	int	Peak RSS from /proc/self/status VmHWM; falls back to rss_bytes on non-Linux.

Field	Type	Description
vms_bytes	int	Virtual memory size.
python_traced_bytes	int | null	Bytes tracked by tracemalloc; null when tracemalloc is OFF.
native_bytes	int | null	max(0, rss - python_traced) when tracemalloc is active; null otherwise — exposes "native dark matter" (numpy / pandas / zmq / aiosqlite native cache / pydantic-core Rust).
cgroup_limit_bytes	int | null	cgroup memory cap; null when absent or unlimited.
cgroup_current_bytes	int | null	Current cgroup memory consumption.
saturation_pct	float | null	cgroup_current / cgroup_limit when both available; null otherwise.

asyncio

Field	Type	Description
active_tasks	int	len(asyncio.all_tasks()).
pending_tasks	int	Subset of all_tasks whose done() is False.

gc

The fields below describe the API/wire shape returned by /api/metrics/system. Internally, the snapshotter stores the generation counters as a tuple (collections_per_gen: tuple[int, int, int] on the GcMetrics group reached via SystemMetricsSnapshot.gc); the route mapper flattens that tuple into the three collections_gen{N} fields below before serializing.

Field	Type	Description
collections_gen0	int	Cumulative GC collection count for generation 0.
collections_gen1	int	Generation 1.
collections_gen2	int	Generation 2.
uncollectable	int	Cumulative count of objects GC could not free.
current_objects	int	Sum of gc.get_count() across all generations at sample time.

limits

Field	Type	Description
rlimit_nproc	int	Soft RLIMIT_NPROC.
rlimit_nofile	int	Soft RLIMIT_NOFILE.
rlimit_as_bytes	int	Soft RLIMIT_AS (address space).

saturation

Percentage toward exhaustion (0.0 - 1.0). The gradient operators actually need to see (vs raw counts) — the rate of increase here is the leading indicator of a thread leak or fd leak.

Field	Type	Description
threads_pct	float | null	num_threads / rlimit_nproc; null when rlimit_nproc is RLIM_INFINITY or zero.
fds_pct	float | null	num_fds / rlimit_nofile; null under the same conditions.

db_internal

Field	Type	Description
aiosqlite_live_connections	int	len(_live_aiosqlite_connections) — atomic read; each live aiosqlite connection runs its own worker OS thread.
pool_size	int | null	Reserved for future pool instrumentation. Currently always null; the sampler does not yet reflect the SQLAlchemy queue pool.
pool_checked_out	int | null	Reserved for future pool instrumentation. Currently always null.

Top-level flags

Field	Type	Description
bus_time	datetime (UTC)	Sample timestamp; window queries match against this field.
tracemalloc_active	bool	True iff tracemalloc.is_tracing() was True at sample time.

Field	Type	Description
cgroup_version	"v1" | "v2" | null	cgroup detection result; null on hosts without cgroup.

Configuration

The snapshotter reads two environment variables directly (no AppSettings extension this iteration). Defaults match the in-code constants in snapper.application.system_metrics.snapshotter.

Variable	Default	Effect
SYSTEM_METRICS_INTERVAL_SECONDS	5	Seconds between sampler ticks. Empty / unparseable / non-positive falls back to the default.
SYSTEM_METRICS_HISTORY_CAP	17280	Ring buffer cap (snapshot count). 17280 ≈ 24h at the default 5s interval. Empty / unparseable / non-positive falls back to the default.

Operators can drop the ring buffer cap to reduce resident memory (720 ≈ 1h ≈ 360 KB).

Publisher hot-path probes are separate, opt-in diagnostics:

Variable	Default	Effect
SNAPPER_TICK_PROBE	unset	Logs per-stage tick publisher timing histograms every ~10 seconds when truthy.
SNAPPER_TRADE_PROBE	unset	Logs per-stage trade publisher timing histograms every ~10 seconds when truthy.

Use these only during targeted throughput investigations; when unset the hot path pays only a branch and return.

Sampling cadence + ring buffer

- Sample interval: configurable, default **5 seconds**.
- Per-snapshot byte budget: ~500 bytes (≈30 metrics, ~15 bytes each).
- Default cap 17280 snapshots ≈ **8.6 MB resident**.
- Eviction: oldest-first via collections.deque(maxlen=N) semantics.
- All buffer access goes through an asyncio.Lock so concurrent route reads see a consistent view (single-writer / multi-reader).

cgroup detection

Runs at every sample and degrades silently when paths are absent:

- v2 default in modern Docker (kernel 5.0+, Docker Engine 20.10+): /sys/fs/cgroup/memory.max, cpu.max, cpu.stat, memory.current.
- v1 fallback: memory/memory.limit_in_bytes, memory/memory.usage_in_bytes, cpu,cpuacct/cpu.cfs_quota_us (or the older cpu/cpu.cfs_quota_us mount), and the matching cpu.stat file for nr_throttled.
- cgroup_version reports the detected layout; null on dev hosts without cgroup (macOS, non-containerised Linux without unified hierarchy).

Tracemalloc

Off by default — enabling tracemalloc costs **5-10% CPU** while active and holds extra metadata in process memory. Operators arm it briefly to capture the python_traced_bytes byte counter for the native_bytes diagnostic, then auto-stop fires after a bounded window.

Knob	Default	Cap
duration_s (POST query)	600s	3600s (clamped)

Calling start while already armed REPLACES the deadline (cancels the previous timer, starts a fresh one with the new duration). The route response carries the clamped requested_duration_seconds, so the operator sees what was actually applied.

Example invocations

Latest snapshot:

```
curl -fsS \
  -H "Authorization: Bearer ${SNAPPER_ACCESS_TOKEN}" \
  https://snapper.example.com/api/metrics/system \
  | jq '.payload | {threads_pct: .saturation.threads_pct,
rss: .memory.rss_bytes, aiosqlite: .db_internal.aiosqlite_live_connections}'
```

Last hour, capped at 60 samples:

```
SINCE=$(date -u -d '-1 hour' +%FT%TZ)
curl -fsS \
  -H "Authorization: Bearer ${SNAPPER_ACCESS_TOKEN}" \
```

```
  "https://snapper.example.com/api/metrics/system/history?since=${SINCE}
&limit=60" \
  | jq '.count, [.payload[].memory.rss_bytes]'
```

Arm tracemalloc for 5 minutes to inspect the native gap:

```
curl -fsS -X POST \
  -H "Authorization: Bearer ${SNAPPER_ACCESS_TOKEN}" \
  "https://snapper.example.com/api/metrics/system/tracemalloc/start?
duration_s=300" \
  | jq
sleep 30
curl -fsS \
  -H "Authorization: Bearer ${SNAPPER_ACCESS_TOKEN}" \
  https://snapper.example.com/api/metrics/system \
  | jq '.payload.memory | {rss: .rss_bytes, traced: .python_traced_bytes,
native: .native_bytes}'
```

Multi-instance note

The ring buffer is per-process. With N>1 instances each hosts its own buffer; aggregation across instances is out of scope for Cluster A. Operators who need cross-instance views should poll each instance directly; no cross-instance collector exists.

Retention

Cluster C ships a **policy-driven retention loop** that periodically archives + purges old rows from event tables, complementing the existing manual snapper archive CLI. Policies are declarative (RetentionPolicy(table, retain_days, backlog_lookback_days)) and evaluated by RetentionService on every scheduler tick.

Endpoint

Method	Path	Purpose
GET	/api/metrics/retention	Most recent scheduler tick's per-policy summary

The route is gated by Permission.READ_SYSTEM_STATUS (same gate as the system metrics surface).

Failure / cold-start contract

The route returns HTTP 503 with one of three details:

- "retention scheduler not available" — singleton failed to start at lifespan time (the lifespan hook leaves app.state.retention_scheduler as None so the route falls through to 503).
- "retention scheduler disabled" — the operator set RETENTION_DISABLED=true; the scheduler is parked.
- "retention scheduler not yet run" — eager run did not populate the summary before the first request (defensive; should not happen in production because start() awaits the eager run_once).

Policy semantics

For each scheduler tick, with today_utc = datetime.now(UTC).date():

```
oldest_kept_day      = today_utc - retain_days       # rows on this day stay in
DB
last_eligible_day    = oldest_kept_day - 1d          # day_end (inclusive)
earliest_scanned_day = last_eligible_day - backlog_lookback_days  # day_start
(inclusive)
```

The archiver call covers the inclusive day-range [day_start, day_end], matching EventArchiver.export(...) semantics. Steady state after the backlog drains processes one new day per tick.

Concrete example with today_utc = 2026-05-01, retain_days = 1, backlog_lookback_days = 30:

- oldest_kept_day = 2026-04-30 (rows with timestamp on 2026-04-30 stay in DB).
- last_eligible_day = 2026-04-29 (last day archived + purged this tick).
- earliest_scanned_day = 2026-03-30 (oldest day touched this tick).
- The archiver query covers [2026-03-30 00:00 UTC, 2026-04-30 00:00 UTC) per the repository semantics.

The shipped default policy:

Table	retain_days	backlog_lookback_days
telemetry	1	30

Configuration

Variable	Default	Effect
RETENTION_INTERVAL_SECONDS	3600	Scheduler loop period. Parsed as a positive float; empty / unparseable / non-positive values silently fall back to the default.
RETENTION_DISABLED	false	Truthy ("1"/"true"/"yes", case-insensitive) parks the scheduler entirely.
RETENTION_DRY_RUN	false	Truthy forces purge=False on every archiver call regardless of policy. Operators flip to true for one cycle when adding a NEW high-volume policy to verify the window before any DB row is deleted.
RETENTION_OUTPUT_DIR	data	Filesystem root passed to EventArchiver; matches the snapper archive CLI default.

Roll-out checklist for a new high-volume policy

1. Add the new RetentionPolicy entry to RETENTION_POLICIES in src/snapper/application/retention/policies.py.
2. Set RETENTION_DRY_RUN=true for one full interval and inspect GET /api/metrics/retention — check that the day_start / day_end window matches expectations and archived_rows is non-zero.
3. If the new policy targets a deeply backlogged table, run a one-time snapper archive --table=<...> --from=<old> --to=<recent> --purge to drain backlog before flipping RETENTION_DRY_RUN=false.
4. Flip RETENTION_DRY_RUN=false. Subsequent ticks will both archive AND purge.

Multi-instance note

The retention scheduler is single-instance only for v1. With N>1 instances, RETENTION_DISABLED=true on N-1 instances avoids double-archive / double-purge races. A coordinator-based extension is a future iteration.

Cluster B — per-table SCD2 stats (DB metrics)

GET /api/metrics/db/tables returns the latest sampled per-table row counters for every registered event + state SCD2 table. The endpoint is operator telemetry: at a glance it answers "is telemetry growing as expected", "are there orders accumulating closed versions", "how many rows are eligible for the next retention cycle".

Cluster B endpoint

GET /api/metrics/db/tables — READ_SYSTEM_STATUS permission gate. Returns a DbStatsResponse envelope wrapping a DbStatsData payload with tables: list[TableStatsItem] in the sampler's deterministic order (STATE-table block first alphabetical, EVENT-table block second alphabetical).

Cluster B failure / cold-start contract

State	Status	Detail	Retry-After
Snapshotter helper failed before assigning	503	DB metrics snapshotter not initialized	(none)
DB_METRICS_DISABLED=true (operator opt-out)	503	DB metrics snapshotter disabled via DB_METRICS_DISABLED	(none)
Started but no sample yet (cold-start)	503	DB metrics snapshotter has not completed a sample yet	<interval_seconds>
Healthy	200	—	—

The cold-start window lasts up to DB_METRICS_INTERVAL_SECONDS (default 60s) — by design, the sampler does NOT block lifespan startup on the first sample. Frontend dashboards must poll-with-backoff using the Retry-After header.

Snapshot fields

TableStatsItem

Field	Type	Notes
table	str	Table name. Usually a key in EVENT_TABLES or STATE_TABLES; candles is also sampled (as a state-kind table) despite living outside both archiver registries.
table_kind	"event" | "state"	Discriminates wire semantics.
total	int | null	

Field	Type	Notes
		Total row count. On PostgreSQL this is a pg_class.reltuples planner estimate (GREATEST(reltuples, 0) — fast, accurate within ANALYZE/autovacuum drift, intended for growth-trend monitoring); on the SQLite dev fixture it is an exact COUNT(*). null only on per-table query failure with no prior sample to clone.
current	int | null	Active SCD2 versions (rows whose known_to equals the SCD2 sentinel) for state tables; null for event tables (no SCD2 lifecycle — reporting 0 would imply the dimension exists).
closed	int | null	Superseded SCD2 versions for state tables; null for event tables.
archivable	int | null	Row count in the policy retention window when a RETENTION_POLICIES entry applies; null when no policy applies (semantically distinct from 0).
is_stale	bool	True when the row was reused from a prior sample after a per-table query timeout or exception.
last_sampled_at	datetime	UTC timestamp of the row's source sample. On a stale clone this is the original timestamp, NOT the current cycle's clock.

DbStatsData

Field	Type	Notes
snapshot_started_at	datetime	When the sampler began the cycle.
snapshot_completed_at	datetime	When the sampler finished the cycle.
interval_seconds	int	Echo of the configured cadence.
tables	list[TableStatsItem]	One entry per registered table.

Per-kind semantics

- **EVENT tables** (append-only — ticks, trades, signals, executions, telemetry, control): total comes from the dialect-aware row-count primitive — a pg_class.reltuples planner estimate on PostgreSQL, an exact COUNT(*) on the SQLite fixture. The current / closed axes are null because event tables have no SCD2 lifecycle. archivable is non-null only for tables with a registered policy (currently only telemetry).
- **STATE tables** (SCD2-versioned — orders, positions, instruments, etc.): current = COUNT(known_to == sentinel) (exact) counts active SCD2 versions; total comes from the dialect-aware row-count primitive (PG planner estimate / SQLite exact); closed =

max(0, total - current) is derived and clamped at 0 to absorb stale PostgreSQL planner estimates where the exact current temporarily exceeds the estimated total. archivable is null until a policy is registered for the table.

Cluster B/C alignment

The archivable counter computes the same window as Cluster C's compute_retention_window(today_utc, policy): the half-open timestamp >= day_start midnight UTC AND timestamp < day_end + 1d midnight UTC predicate, with the policy's (retain_days, backlog_lookback_days) knobs. This means dashboard archivable and the next retention cycle's archived_rows MUST agree exactly — if they drift, one of the two has a boundary bug. The tests/application/db_stats/test_cluster_alignment.py integration test pins the equality contract.

Cluster B configuration

Variable	Default	Effect
DB_METRICS_INTERVAL_SECONDS	60	Sampler loop period. Empty / unset → default; a non-integer or out-of-range value raises ValueError at startup.
DB_METRICS_DISABLED	false	Truthy ("1"/"true"/"yes", case-insensitive) parks the snapshotter entirely. PG operators with deployments lacking psycopg2 in deps boot cleanly with this flag (the underlying repo factory is never called).

PER_TABLE_TIMEOUT_SECONDS is a module constant (30s, not env- configurable for v1). On per-table timeout, the snapshotter clones the prior TableStats with is_stale=True; if no prior sample exists the row carries all-null counters. Per-table failures NEVER abort the sampler tick.

Sampling order + cold-start

STATE tables sample first (alphabetical by name), EVENT tables second. Atomic swap of _latest_snapshot happens at the END of the cycle — readers wait one full interval_seconds before the first 200, regardless of order. STATE-first ordering is for resilience to EVENT timeouts: if a heavy EVENT table (e.g. ticks at 50M rows) times out, STATE counters were already computed and the snapshot still publishes with EVENT entries marked is_stale=True from a prior run.

telemetry.timestamp index

Cluster B's archivable query on telemetry filters on Telemetry.timestamp (the bus-time column inherited from TemporalMixin). Without an index on that column, the query is a full table scan over a high-volume audit table. The ix_telemetry_timestamp index over Telemetry.timestamp is part of the consolidated 0001_init migration (src/snapper/data/migrations/versions/0001_init.py), so both Cluster B's counter and Cluster C's existing retention scan run in O(log n) from the first deployed schema.

Operations runbook — multi-instance trade coordinator

The trade runtime supports static-hash shard partitioning. At SNAPPER_COORDINATOR_INSTANCE_COUNT=1 (the default) a single TraderCoordinator owns every shard. Scaling to N >= 2 deploys multiple coordinators against the same DB and ZMQ broker, each owning ~1/N of the shards deterministically via SHA-256 of the shard key.

This document covers operating the N-instance trade runtime: scale-up, scale-down, and crash recovery. It is intentionally opinionated about systemd because systemd template units are the stable recipe. A contract for alternative orchestrators (Docker Compose, Kubernetes, Nomad) is also specified so operators can adopt them with empirical verification.

Prerequisites: systemd template unit

All subsequent recipes assume a template unit snapper-trade-zmq@.service that consumes %i as the instance id:

```
# /etc/systemd/system/snapper-trade-zmq@.service
[Unit]
Description=Snapper Trade Coordinator (instance %i)
After=network.target snapper-broker.service

[Service]
Type=simple
User=snapper
EnvironmentFile=/etc/snapper/coordinator.env
Environment=SNAPPER_COORDINATOR_INSTANCE_ID=%i
ExecStart=/opt/snapper/bin/snapper trade-zmq
Restart=always
RestartSec=2

[Install]
WantedBy=multi-user.target
```

With this template, snapper-trade-zmq@0.service and snapper-trade-zmq@1.service are distinct systemd units with separate cgroups and separate journal streams on the same host. Per-instance PID is queried via systemctl show -p MainPID snapper-trade-zmq@0.service.

/etc/snapper/coordinator.env holds the shared bootstrap env, including SNAPPER_COORDINATOR_INSTANCE_COUNT=2 during N=2 operation:

```
DB_URL=postgresql+asyncpg://snapper:...@db.internal/snapper
SNAPPER_ENV=production
ZMQ_BROKER_XSUB=tcp://broker.internal:7500
ZMQ_BROKER_XPUB=tcp://broker.internal:7501
MASTER_PASSWORD=...
SNAPPER_COORDINATOR_INSTANCE_COUNT=2
```

SNAPPER_ENV=production (also prod / staging) arms the placeholder-secret gate: every process loading this env file — coordinators included — refuses to start while MASTER_PASSWORD is unset or still the development placeholder (snapper_default_master_password_v1); BootstrapSettingsLoader raises ValueError naming the variable before anything binds. The API server additionally refuses the placeholder auth_secret_key / csrf_secret_key DB settings with a RuntimeError naming the key. Remediation: set a real MASTER_PASSWORD in the env file (and real secret rows in the settings table) before flipping SNAPPER_ENV to a production-like value — see docs/configuration.md.

SNAPPER_COORDINATOR_INSTANCE_COUNT >= 2 requires a PostgreSQL DB_URL. On a SQLite backend the coordinator refuses to start with a count above 1, raising ValueError before any signal is dispatched: SQLite compiles SELECT ... FOR UPDATE to a plain SELECT, so cross-instance row claims cannot serialize — use PostgreSQL for multi-instance deployments. A single-instance SQLite coordinator starts but logs a writer-concurrency warning.

Scale up N=1 → N=2 (systemd)

```
# 1. Edit /etc/snapper/coordinator.env — set
SNAPPER_COORDINATOR_INSTANCE_COUNT=2.

# 2. Stop the currently-running single-instance coordinator.
sudo systemctl stop snapper-trade-zmq@0.service

# 3. Verify graceful shutdown.
sudo systemctl status snapper-trade-zmq@0.service | grep -E "Active:|Main PID:"

# 4. Start both instances.
sudo systemctl start snapper-trade-zmq@0.service
sudo systemctl start snapper-trade-zmq@1.service

# 5. Verify both are live.
sudo systemctl is-active snapper-trade-zmq@0.service  # expect: active
sudo systemctl is-active snapper-trade-zmq@1.service  # expect: active
sudo journalctl -u snapper-trade-zmq@0.service -n 20 | grep "instance 0/2"
sudo journalctl -u snapper-trade-zmq@1.service -n 20 | grep "instance 1/2"
```

Scale down N=2 → N=1 (systemd)

```
# 1. Stop BOTH coordinator instances first — full cutover is required.
sudo systemctl stop snapper-trade-zmq@0.service
sudo systemctl stop snapper-trade-zmq@1.service

# 2. Verify nothing remains.
sudo systemctl list-units 'snapper-trade-zmq@*' --state=active --no-legend
# (expect empty output)

# 3. Edit /etc/snapper/coordinator.env — set
SNAPPER_COORDINATOR_INSTANCE_COUNT=1.

# 4. Start instance 0 only.
sudo systemctl start snapper-trade-zmq@0.service
```

Why full cutover is required

Scale up and scale down both require stopping ALL old-N coordinators BEFORE starting any new-N coordinator. Overlapping old-N and new-N instances is the failure mode to prevent.

Scenario: improper restart with overlap. If an operator skipped the cutover step and left instance 0 running as count=1 while starting instance 1 as count=2:

- Instance 0 believes it owns ALL shards (count=1 → every owns() returns True).
- Instance 1 believes it owns shards where hash(shard_key) % 2 == 1.
- Both coordinators dispatch the same TradeCommand for shards where hash() % 2 == 1 → duplicate venue orders OR a race on status='created' → 'dispatched'.

The ~10s dispatch pause for owned shards during the cutover window is the explicit no-HA trade-off. HA was excluded from scope by design.

Crash recovery (systemd)

If instance K crashes:

```
# 1. Diagnose.
sudo journalctl -u snapper-trade-zmq@1.service -n 100 --no-pager

# 2. Check whether Restart=always brought it back.
sudo systemctl is-active snapper-trade-zmq@1.service

# 3a. If active: confirm the coordinator resumed with its original ownership.
sudo journalctl -u snapper-trade-zmq@1.service -n 200 | grep "instance 1/2"
sudo journalctl -u snapper-trade-zmq@1.service -n 200 | grep "recovery complete"

# 3b. If inactive (Restart=always gave up): manually restart.
sudo systemctl restart snapper-trade-zmq@1.service
```

During the outage window, shards owned by the crashed instance pause until the coordinator comes back. Typical outage is under 10s if Restart=always is configured.

Recovery rebuilds engines from:

1. Checkpoints (carry shard_key directly, filtered by ownership).
2. Live executions (filtered by recovered shard_key; paper rows are EXCLUDED under N>1 because ExecutionRow has no strategy_tag).
3. Active orders (same treatment as executions).

Under N>1 paper mode, state that never produced a checkpoint is not recovered by the restarted coordinator. Paper strategies that need to survive restart MUST persist checkpoints.

REST orders with wallet_public_id under N>1

REST manual orders are supported under N>1. POST /api/orders uses the same compute_shard_key(...) helper as the signal engine, executor, and MCP manual-order path. When wallet_public_id is non-empty, the shard key includes the .w{wallet_short} segment; REST manual orders pass strategy_tag=None, so they do not add the paper strategy segment.

The route writes that shard key to both execution_plans and trade_commands, inserts the command with ownership=None so any API worker may accept the request, and lets the trade coordinator outbox ownership filter decide which instance dispatches it. During outbox publish, the coordinator registers client_order_id -> shard_key, so the N>1 CID guard accepts the venue ACK/fill/cancel events only on the owning coordinator.

Use wallet_public_id="" only for backward-compatible or explicitly single-wallet workflows where the wallet segment should be omitted. Signal-driven paper orders may still add a strategy_tag; REST manual orders intentionally do not.

Non-systemd deployments (contract)

The operational recipes are anchored on the systemd template above. For non-systemd platforms, the deployment author writes a recipe that satisfies the same contract:

1. Two distinct supervised processes on the same host (or across hosts) with SNAPPER_COORDINATOR_INSTANCE_ID=0 and =1 respectively, and SNAPPER_COORDINATOR_INSTANCE_COUNT=2 in both.
2. Per-instance lifecycle (stop / start / restart / logs) via the orchestrator's native mechanism — never by process pattern match (pkill -f is too broad for multi-instance).
3. Scale-up and scale-down follow the same stop-all → verify-empty → start-new cutover flow as the systemd recipes above.

4. At least one concrete recipe per chosen orchestrator, verified empirically, with a provenance line:

```
Verified on YYYY-MM-DD against <platform-version>
```

The regex enforced by tests/meta/test_operations_runbook.py is ^Verified on \d{4}-\d{2}-\d{2} against \S+$ — exact casing, anchored to the start and end of the line. The meta-test searches the entire file body for at least one matching line; there is no required section placement.

Acceptable orchestrators for the non-systemd recipe are docker compose, kubectl (Kubernetes), or nomad — the meta-test rejects bare docker because docker run alone does not provide the per-instance lifecycle the contract requires.

Verified environments

Verified on 2026-04-18 against docker-compose-v2.29.7

Kraken Equities (TradFi) market data

TradFi index futures (Kraken FCM: MNQ/ES/YM/RTY/NKD/M2K) are shipped in market-data-only mode. The symbol updater remains disabled by default at code level, while the publisher is registered as an enabled market-data process. Persisted DB Setting rows still control runtime enable/autostart state, so rollout + rollback happen without code deploys.

Contracts used by this flow:

- REST: iapi.kraken.com/api/internal/markets/all/futures-contracts for instrument metadata; .../{ws_symbol}/ticker/history for historical candles. Requires Origin: https://pro.kraken.com + Referer: https://pro.kraken.com/ headers.
- WS: wss://ws-equities.kraken.com for live delayed (~10 min) ticks + trades. The outer envelope's delayed flag propagates into every TickData message as is_delayed.
- No order API — every submit route (POST /api/orders, POST /api/execution-plans, POST /api/trailing-stops) calls snapper.server._capability_guard.require_tradable and rejects TradFi instruments with HTTP 422 error_code=instrument_market_data_only.

Market diagnostics

Use these REST endpoints during feed rollout and incident response:

- GET /api/market/feed-health — current per-symbol publisher health. Filter with exchange=kraken_equities and optionally fresh_within_seconds=<n> to hide stale rows from stopped publishers.
- GET /api/market/coverage — per-exchange counts for active instruments, fresh ticks, fresh candles, gated-off instruments, and dark instruments. Tune tick_window_seconds and candle_window_seconds to match the venue's expected cadence.
- GET /api/market/cache/health — cache and persist-policy snapshot: number of cached instruments, cached pair-stat entries, and persisted instrument universe size.
- GET /api/market/cache/stats/configured and /api/market/cache/stats/{exchange_a}/{symbol_a}/{exchange_b}/{symbol_b} — cached Pearson/cointegration diagnostics for configured pairs.

The feed-health table is current-state rather than SCD2 history. Its natural key is (coordinator, exchange, channel, symbol), where channel encodes the stream kind and timeframe together (e.g. ohlc:1m) and symbol is the wire-format product id, so split-feed deployments can show which coordinator is publishing a stale or missing stream.

Feed resilience and outage recovery

Market-data publishers self-recover from multi-minute network outages on both the public (egress) and private (direct) paths without operator action:

- **Persistent recovery.** When a feed goes silent past its liveness threshold, recovery runs as a single-flight loop with capped exponential backoff (1→60 s + jitter) that retries until messages resume or the publisher stops — it never gives up on a timeout.
- **Per-venue liveness thresholds.** Kraken Spot and Futures recover after 60 s of message silence (their wildcard/continuous universes always tick, so 60 s reliably means a dark feed); Kraken Equities uses 120 s and is fully suppressed during scheduled CME closure windows.
- **Native-candle liveness.** The message watchdog above shares one watermark across ticks, trades, and candles, so a venue whose 1m bars arrive on a dedicated native channel (Kraken Spot's ohlc:1m) can have that channel stall silently while ticks/trades keep the watermark fresh (the 2026-06-18 incident: trades alive ~21 h, candles dark). Spot therefore also runs a venue-wide candle watchdog: if no native 1m candle arrives for 300 s (well above the ~60 s venue-wide cadence) while a candle subscription is active, it spawns the same WS-restart recovery (which re-subscribes the dead channel). This recovery is candle-aware — it is only declared

successful once a fresh native candle resumes, not on trades alone — and it never escalates to the dark-feed process exit, so a candle-only stall cannot kill the still-live trade feed.

- **Transport keepalive.** Every Kraken WebSocket handshake is opened with an explicit ping_timeout/close_timeout so a silently dead socket (a yanked link with no close frame) surfaces in tens of seconds rather than waiting out the app-level silence threshold.
- **Bounded connect.** The WebSocket connect itself (SpotWSClient/ FuturesWSClient start()) is wrapped in a 20 s timeout (_WS_CONNECT_TIMEOUT_S). The vendored SDK's start() polls for the socket with a connect timeout that never fires, so on a prolonged blackout — where the connector exhausts its reconnect ceiling and never establishes a socket — an unbounded start() would hang forever and wedge in-process recovery (only a process restart would clear it). Bounding it makes a stalled handshake fail fast so the recovery loop tears the partial client down and retries with a fresh one, reconnecting as soon as the network returns.
- **Bounded teardown.** Tearing a client down is bounded too: every venue WS close runs under a 10 s timeout, and a close that times out or raises escalates to a last-resort force close that cancels the SDK's connector run tasks, drains them (5 s bound), and closes the SDK's internal aiohttp session directly. Previously a close landing during the SDK's reconnect backoff sleep (up to ~3 min; the SDK's stop flag did not interrupt it) abandoned the SDK's own session cleanup, so each rebuild cycle during a prolonged blackout leaked one HTTP session and the abandoned reconnect later spawned orphaned background tasks. The reconnect backoff is now hardened to re-check the stop flag every 0.5 s and to reap its child tasks on every exit path, so blackout-driven rebuild cycles no longer accumulate leaked sessions. This covers every process that tears down a Kraken WS client — the executors' private clients included, not just the publishers. A close timed out - forcing cleanup warning followed by kraken WS force-close: leaked aiohttp session closed is the cleanup working, not a fault to act on.
- **Bounded subscribe sends + serialized reconnects.** Every Kraken Futures SDK subscribe/unsubscribe send is wrapped in a 5 s timeout (_SDK_SEND_TIMEOUT_S): the Futures SDK's send_message spins forever on a client whose socket never came up, and one such send once wedged the shared subscribe throttle lock — starving the post-reconnect subscription replay and leaving the feed connected-but-dark until a process restart. Spot and Equities market-data subscribe sends are not wrapped — the spinning send_message is a Futures-SDK behavior, so those venues rely on the bounded connect, the ping/close timeouts, and the 180 s per-attempt recovery bound instead; the Spot *private* executions subscribe send IS bounded by the same 5 s timeout (see "Private fill-stream supervision"). Client (re)connects are serialized per venue (_ws_connect_lock), slot writes are compare-and-clear, and a connect raced by a disconnect closes its own client instead of leaking it. Each recovery attempt is additionally bounded to 180 s (_RECOVERY_ATTEMPT_TIMEOUT_S) so even an

unforeseen hang becomes a logged, retried failure rather than a silent permanent wedge. Futures dark auto-recovery covers the trade channel too (its candles are synthesized from trades).

- **Dark-feed watchdog.** If a feed stays dark for ~25 min despite recovery (a wedged SDK or recovery bug), the publisher exits non-zero and ProcessLauncherService respawns it. The exit ceiling is kept above the launcher's lifetime-restart reset window, so a prolonged outage produces an unbounded slow restart-and-retry cadence that self-heals the instant connectivity returns — it never permanently abandons the feed.

Watch recovery via GET /api/market/feed-health / GET /api/market/coverage and the pub:<exchange> log lines (liveness recovery triggered, feed recovered after N attempt(s), feed dark for …s … exiting for launcher restart).

Order-path retry semantics (Kraken Spot REST)

Order **creation** is deliberately excluded from the REST network-retry wrapper: a network-class failure such as a request timeout is ambiguous (the order may have reached Kraken and executed even though the response was lost), Kraken deduplicates cl_ord_id only among *open* orders, and strategy orders are market orders that fill instantly — so a blind re-send could double a real position. During a network blip an order submit therefore fails fast (one venue call, logged as Network error (no retry, non-idempotent call)) instead of silently retrying up to three times; the failure still feeds the REST circuit breaker so a sustained outage trips fail-fast mode. Idempotent calls (cancel, fetch, balance) keep the 3-attempt network retry. Expect more transient submit failures in logs during connectivity incidents — that is the guard working as intended, not a regression.

Ambiguous submits — the UNKNOWN order state

On every venue (Kraken Spot ccxt + native, Kraken Futures, Walutomat) an order submit that fails *after the request may have left the process* (request timeout, connection reset, gateway 5xx, unparseable success body) no longer fabricates a REJECTED event. The executor first verifies against venue truth — up to three lookups by client id (cl_ord_id on Kraken Spot via open+closed orders, cliOrdId on Futures via order status; 2s/5s/10s backoff, 15s bound each). A found order finalizes as accepted (an already-terminal one additionally projects its fills through the disappeared-order reconciler); two consecutive authoritative not-found answers make the rejection venue-truth-based and safe. Only when verification cannot resolve — venue unreachable or lookup unsupported

(Walutomat active-order miss) — does the executor park the order, record a non-terminal order_submit_unknown venue event, and publish a single orders.events.*.unknown message:

- the engine **holds its in-flight guard indefinitely** (the 60 s timeout valve is disabled while UNKNOWN) so no replacement order can double exposure while the original may be live;
- a safety-critical order_unknown operator alert fires ("do not assume flat", 5-minute dedup window);
- failures that provably happened *before* any send (circuit breaker open, credentials, symbol/order-type validation) and authoritative venue answers (4xx, success=false, exhausted 429) still reject immediately on every venue — only genuine ambiguity parks. A breaker-open refusal additionally lands a durable terminal disposition before its REJECTED — a venue event recording the refusal plus the command row moved to FAILED — and publishes the rejection with reason circuit_breaker_open, so consumers can tell an infra refusal from a venue rejection and a redispatched frame can never resubmit the command after the breaker closes; if any step fails, the entry parks and the 60s reconciliation loop reruns the disposition until engine intent is released. Connection refused rejects immediately only on Walutomat, where httpx surfaces connection-setup errors distinctly; on Kraken Spot and Futures connection-setup failures park as UNKNOWN by design, because the ccxt and requests transports cannot reliably distinguish sent from not-sent (a connection error can fire mid-body), and a false park merely alerts while a false rejection could double a position.

Resolution: a late accepted event clears the UNKNOWN flag (the order resumes its normal lifecycle with the guard still held until fills); a COMPLETE fill or a rejection releases the guard entirely. A partial fill alone neither clears the flag nor the guard — the order stays guarded until its terminal event. A PARKED UNKNOWN order is retried by the 60s reconciliation loop (fresh venue-verification round each cycle); if it stays unresolved across cycles, check the venue's open and closed orders for the client id from the alert before taking any manual action. The same 60s loop also heals fill gaps: when the venue reports more filled quantity than the executor has recorded, it emits a corrective fill, pricing market orders that lack a snapshot price from the VWAP in the venue's own per-order fills summary (get_order_fill_summary). Only when no venue price resolves at all does it log Recon: fill gap … no price on market order, skipping corrective fill — a fill mismatch needs manual reconciliation against the venue's fill history only if that ERROR persists across cycles. Correctives also carry the venue's real cumulative fee — the order snapshot's running commission, or the fills-summary total — instead of fabricating a zero fee. When the snapshot carries no fee and the fills summary cannot answer (failed lookup, partial page, an implemented source with no rows), the corrective is deferred with a WARNING for at most 5 cycles, then emitted fee-less under a single CRITICAL log so the order can still reach its terminal state. After that CRITICAL the quantity is healed but the fee is not — reconcile the fee manually against the venue's fill history.

On Walutomat, where fills come from polling, an order that disappears from the active list is never guessed terminal: a failed final-state query keeps the order tracked and retries every poll cycle (a wrong FILLED would create phantom position; a wrong CANCELED would free engine intent while the order may have filled). Each failed attempt logs a WARNING; the 10th consecutive failure escalates to a single CRITICAL and the retries continue. On that CRITICAL, check the order on the venue — no manual terminal injection is needed, and the 60s reconciliation loop converges the pending entry independently once the venue answers.

Stop orders — venue support and pre-send rejects

Order commands speak the core order-type vocabulary end-to-end — market, limit, stop, stop_limit — across REST, MCP, the durable trade_commands rows (a stop's trigger is the trade_commands.stop_price column), and the outbox frames. Venue wire names (stop-loss, stop-loss-limit) appear only at the executor's venue boundary, where the request is translated immediately before the exchange call — never on the bus or in trade_commands. Older execution plans may still carry the legacy venue_order_type wire value in their params; readers normalize it back to the core vocabulary. Venue support:

- **Kraken Spot** and **Kraken Futures** accept stop and stop_limit: the trigger comes from stop_price; once triggered, stop executes at market and stop_limit places a limit order at the command's price.
- **Walutomat** (its FX market API has no stop orders) and **paper trading** (no trigger simulation — a stop would fill immediately, misrepresenting the protective semantics) reject every stop-typed submit.

A stop-typed submit that cannot be sent whole is rejected PRE-SEND: the failure is provably-not-placed, so it takes the definitive-reject disposition (REJECTED published, durable order_rejected venue event, engine in-flight intent released) — never an UNKNOWN park, never a venue round-trip. The pre-send gates are:

- an order type with no venue wire mapping (vocabulary error);
- stop / stop_limit with no stop_price;
- stop_limit with no price (the limit leg) on the Kraken venues;
- any stop on Walutomat or paper.

POST /api/orders and the MCP submit_manual_order tool refuse the same malformed shapes up front — unknown order_type, limit/stop_limit without price, stop/stop_limit without stop_price — before any command row is persisted (HTTP 422 on REST; the same evaluator rule surfaces as a tool error on MCP). An executor-side pre-send stop rejection in the logs (Error processing order ... followed by the REJECTED publish) is therefore safe to act on: nothing reached the venue, engine intent is released, and the command can be corrected and resubmitted.

Duplicate-dispatch guard and dispatch TTL

Two executor/outbox gates close the remaining order-flow loss windows:

- **Duplicate-submit guard.** A coordinator crash between the ZMQ publish and the CREATED→DISPATCHED commit (or a failed bulk write) re-publishes the same client_order_id. The executor now drops such replays silently: an entry already pending, an accepted order awaiting its durable-event heal, or durable venue-event evidence (accepted/fill/terminal/unknown/breaker-open — rejections excluded so legitimate retries still flow) all block the re-submit before any venue call. A replay carrying breaker-open evidence is the one exception to the silent drop: it reruns the breaker disposition (above) so a crash mid-disposition cannot leave engine intent held. A failed evidence check drops FAIL-CLOSED.
- **Dispatch max-age TTL** (TRADE_COMMAND_DISPATCH_TTL_S, default 30s, <= 0 disables). Stale CREATED create/submit commands expire at the outbox to the terminal EXPIRED status (never published; engine intent released through the regular expired pipeline), and stale frames buffered through an outage (the order-flow socket has no high-water mark) are resolved at the executor against venue truth: a frame whose order EXISTS under the client id is adopted as accepted (a crash-window replay that actually placed), an unverifiable frame drops silently (the row stays DISPATCHED for the verification sweep below and the engine valve), and only a venue-verified-absent frame rejects. A stale row that DOES carry submit evidence publishes anyway and is absorbed by the duplicate guard — expiring it would fabricate a terminal state for a possibly-live order. Cancels and replaces are exempt: expiring a stale cancel would strand a live order, and replace outcomes are reported on the lightweight cancel/replace event path. Keep the TTL below the engine's 60s in-flight valve.

Behind the two gates, the durable command plane reconciles itself. The coordinator's reconciliation loop folds the append-only venue_events rows into trade_commands status advances (ack, partial/full fill, terminal), which rescopes its stale-command logging: a stale command ... no venue evidence WARN fires only for an over-age create/submit command with ZERO venue evidence (a real anomaly); unknown-only evidence logs at INFO (executor verification is already working it); a command with real evidence is silent — an old open limit order is healthy, not stale. Cancel commands keep the legacy WARN. The executor's 60s cycle works the zero-evidence queue from the venue side: each unresolved DISPATCHED command gets a client-id lookup — a found order is adopted under its original request (this also recovers parked-UNKNOWN entries lost to an executor restart; the DISPATCHED row is the durable record of the send intent), and only two consecutive authoritative absences on a command younger than an hour auto-REJECT to release engine intent. Absence never auto-rejects when the dispatch TTL is disabled (nothing expires frames, so one may legitimately still be in flight at any age) or past the hour bound (venue closed-order lookback makes absence non-authoritative) — those cases WARN for operator attention instead. Consecutive executor venue reconciliation

failures are exposed on executor heartbeats; after three failures, the coordinator halts known real shard keys for that exchange/wallet scope and drops new cold-path signals for the same scope until a healthy heartbeat clears the scope gate. Existing shard halts remain explicit operator/release decisions.

The same cycle adopts ghost orders — open at the venue under our client id with no in-memory entry — by rebuilding the request from the command row; an open order with NO command row is foreign/manual, warned about once and never touched. A durably REJECTED command later found open (or with live venue evidence postdating the rejection) is healed loudly: the order is adopted and the command resurrected to ACCEPTED, so a false rejection cannot silently strand a live order.

Adoption also re-arms the engine. Every adoption of a venue-LIVE order — ghost adoption, ambiguous-submit verification, the false-reject heal, startup recovery of a venue-verified-open row — publishes its ACCEPTED with reason="adopted"; terminal-snapshot adoptions stay reason-less (re-arming for a dead order would block honest emission). On that marker the coordinator re-arms a running engine's RELEASED in-flight guard (it consumed the false REJECTED, or the 60s timeout valve had already cleared it), so the next signal cannot emit a second order while the adopted one still works the book. A RE-ARMED in-flight intent for adopted order … WARN in the coordinator log is that guard working, not a fault. An engine already in flight for a DIFFERENT order is never clobbered (WARN — both orders stay live and fills of the adopted one still book position), and a client id retired by an honest terminal (FILLED fill or cancelled/expired confirm; rejections never retire) refuses late duplicate adopted frames. A failed adopted ACCEPTED publish parks on the executor and the 60s reconciliation loop retries it until it lands.

Private fill-stream supervision

The private execution (fills) WebSocket in each executor is supervised, not spawned once. Previously any termination of that stream — typically SDK reconnect-budget exhaustion after ~2 min (Futures) / ~5 min (Spot) of network outage — left fills permanently dark while the executor kept reporting RUNNING until a manual restart: Futures surfaced a single error line, Spot exited silently and kept treating the dead client as connected, blocking any rebuild. Now:

- **Death is detectable.** A Spot stream whose connection is terminally lost raises instead of ending cleanly and closes the poisoned client, and on both venues ensure-connected rebuilds a slot holding a terminally-failed client instead of returning it as connected. The Spot private subscribe send is bounded by a 5 s timeout (_SDK_SEND_TIMEOUT_S) so a wedged socket fails the attempt fast instead of hanging the resubscribe.

- **A supervisor respawns dead streams.** Any termination — a clean generator return included — triggers a respawn with capped jittered exponential backoff (1→60 s); it never abandons the stream. A stream that ran healthy for 5+ minutes resets the backoff so an old incident's ceiling is not inherited by the next blip.
- **The dark window is healed before re-entry.** Every post-death respawn first runs a best-effort reconciliation pass (serialized with the periodic 60 s cycle so two concurrent cycles cannot double-emit the same corrective fill), then resubscribes.
- **Resubscribe is idempotent.** Spot subscribes without order/trade snapshots and Futures drops fills_snapshot frames at the source, so neither a supervised respawn nor an SDK in-budget reconnect replays already-booked fills.
- **Per-order fill booking is atomic.** A per-order lock plus exec-id dedupe serializes booking across the live stream, recon corrective fills, and cancel handling, so concurrent paths cannot double-book a fill, and a publish failure no longer silently corrupts fill tracking (the gap is re-absorbed by later frames or the recon corrective).

During a private WS outage expect recurring executor-log warnings — Execution stream died (...) - respawn #N after Ns backoff or Execution stream returned cleanly - respawn #N ... — that is the supervisor retrying; no manual restart is needed, and fills plus the dark-window gap reconcile automatically once connectivity returns. The supervisor exits permanently only on shutdown and on venues without WebSocket execution streaming, where the 60 s reconciliation loop is the only fill source.

Recovery-time corrective fills

Executor startup recovery no longer re-baselines fill tracking to the venue's current cumulative (which silently swallowed every fill that landed while the stack was down). Instead, recovery reads both truth planes per order — the durable venue_events fill rows and the executions log (rows exist only for successfully published fills) — seeds the committed and durable watermarks from what each plane proves, republishes recorded-but-unpublished fills through the normal pipeline under their original exec ids (every consumer dedupes by exec id, so this is idempotent), and emits the remaining venue-ahead gap as a recon corrective with a deterministic id (recon-{order}-c{cum}), so repeated restarts and publish retries converge instead of double-applying. Orders that went terminal during the downtime get their fill gap healed BEFORE the terminal event is projected. Orders the venue cannot verify at startup are parked in tracking with DB-derived seeds — the 60s recon loop retries them every cycle instead of dropping them. If the durable plane is unreadable for an order, recovery degrades to the legacy venue-truth seeding for that order only and logs an ERROR (the downtime gap for it stays unhealed — fix the DB and restart).

Executor task supervision and service restarts

Every executor core loop (order handler, reconciliation, heartbeat — plus the fill stream covered above) runs under an in-process supervisor: a loop that dies (exception or unexpected clean return) respawns with capped jittered backoff (1s doubling to 60s; a 5-minute healthy run resets it), preserving all in-memory order state. The order handler's supervisor rebuilds its SUB socket before re-entry; poison frames (already consumed by ZMQ) are logged and skipped, never respawn-looped. Reconciliation cycles are bounded (300s) so a hung venue call cannot hold the recon lock forever, and parked-UNKNOWN verification is fairness-capped (3 per cycle, index-rotated) so a large parked set cannot starve its tail. On the core loops (order handler, reconciliation, heartbeat — the fill stream's supervisor only backs off, it never escalates), a death streak older than 25 minutes escalates: the whole service crashes out of start() (siblings cancelled, sockets closed) and the process launcher — whose task-completion handler now drives the same restart watchdog as native subprocesses — rebuilds a FRESH instance, made safe by recovery-time corrective fills. The escalation ceiling deliberately exceeds the launcher's healthy-uptime reset, so escalations retry forever at a slow cadence instead of exhausting the restart budget; only fast startup crash-loops (bad credentials, DB down at boot) park the executor after the budget.

Executor heartbeats report derived health, not a hardcoded HEALTHY: each beat is computed from the supervised-loop seams, so degradation pages operators through the existing system-degradation alert (critical_system_error: 3 consecutive non-HEALTHY beats, deduped to about one page per hour per instance) instead of staying green while a loop is dead. An active death streak reports WARNING from its second death and ERROR once the streak is 10 minutes old (well before the 25-minute escalation); a reconciliation loop with no completed pass for 5 / 15 minutes reports WARNING / ERROR (recon swallows per-cycle faults by design, so only its progress clock can expose a recon that fails forever without dying); an order command stuck in flight for 2 / 10 minutes reports WARNING / ERROR (a wedged handler neither dies nor progresses); an unhealed accept-event backlog or parked-UNKNOWN submits report WARNING. The executor heartbeat's lag_ms is the age of the last successful reconciliation pass. Per-wallet executor instances publish on per-wallet heartbeat topics (system.heartbeats.executor.{exchange}.{wallet_short}), and the alert rule parses those — previously per-wallet executor heartbeats were silently discarded, so a degraded executor could never page.

A parked executor is paged, not just logged: on giving up, the launcher speaks for the dead instance on its own per-wallet heartbeat topic with a synthetic 3-frame ERROR burst, re-bursting hourly while the instance stays parked — the page is level-triggered (a notify restart that loses one burst is healed by the next) while the alert dedup still caps it at about one page per hour. A successful (re)start or a successful deliberate stop unparks the instance and cancels the burst. GET /health additionally reports ERROR while any process is parked — checked before every other branch, including API-only mode — as the backstop for a lost burst.

Fault-injection testing

scripts/resilience_fault_injection.py simulates a real exchange outage and asserts recovery. It drops outbound HTTPS (:443) **inside the feed container's network namespace** with a throwaway privileged helper (docker run --net=container:<feed> --cap-add=NET_ADMIN), holds the outage, restores it, and measures candle-freshness recovery against the SLA. Only :443 is dropped, so the ZMQ bus and DB writes keep working — the outage is market-data only (order execution, a separate container, is unaffected).

It targets the feed's own netns because the runtime image has no iptables (docker exec <feed> iptables fails). This matches the default direct-feed path (feed_egress_enabled=false). If feed egress is enabled, publishers connect to snapper-egress SOCKS ports instead, so drop the SOCKS/egress path rather than feed-container :443. Restore is safety-critical and proven, not best-effort: every step is exit-code-checked, the helper runs a prebuilt snapper-fault-helper image (no package install while the link is down), removal is proven with iptables -C, and if removal cannot be proven the feed container is recreated (fresh netns) and re-verified — the harness raises loudly rather than reporting an uncertain restore. It builds the helper image on first run.

```
# Default: drop the feed's :443 for 180 s, expect spot+futures back within 180
s.
DB_URL=... python scripts/resilience_fault_injection.py --hold-s 180

# Longer outage + a wider recovery SLA (e.g. validating the dark-feed watchdog).
DB_URL=... python scripts/resilience_fault_injection.py --hold-s 900 --sla-s 600

# Freshness bar vs SLA: --fresh-s is HOW FRESH the newest candle must be to
# count as recovered; --sla-s is the wall-clock budget for getting there.
DB_URL=... python scripts/resilience_fault_injection.py --hold-s 240 --sla-s
360 --fresh-s 120

# Override the target container, port, or verified exchanges.
DB_URL=... python scripts/resilience_fault_injection.py \
    --container snapper-feed --port 443 --exchanges kraken,kraken_futures
```

Run it in a low-impact window — it darkens live market data for the hold duration.

Egress routing for feeds (feed_egress_enabled)

By default the feed publishers connect **directly** to the exchanges and do not initialize an egress pool in their own processes. Set the feed_egress_enabled setting to enable per-publisher egress routing: each feed subprocess then initializes its own process-local egress pool at startup, so the Kraken connect shim and Walutomat's pooled HTTP transport route through the configured tunnels (per-exchange pins apply; direct stays the fallback).

This is gated off by default because routing live market data through shared VPN IPs changes latency and can make Kraken/Cloudflare throttle the feed differently than the host IP. Roll it out cautiously:

1. Set feed_egress_enabled=true and restart the feed (the setting is read at publisher startup, not live).
2. In snapper-feed, confirm ss -tnp shows connections to snapper-egress:1081-1085 (SOCKS) rather than direct exchange :443.
3. Compare candle freshness, tick/trade lag, and dark-instrument counts to the direct baseline; watch the reconnect logs for throttling/close codes.
4. Revert instantly by setting feed_egress_enabled=false and restarting the feed.

With routing enabled, a TCP/SOCKS connect-level failure through a proxy route — timeout, connection refused, or a tunnel that went dark — quarantines that route for 30 s (_WS_CONNECT_ERROR_QUARANTINE_S), so the next reconnect fails over to another tunnel or the direct fallback instead of re-dialing the dead route. Direct routes are never quarantined: a connect error with no proxy means the exchange or the local uplink is down, not the route.

Spot candle source (spot_candle_source)

By default the Kraken Spot feed reads live 1m candles from the venue-native ohlc:1m WebSocket channel (spot_candle_source=native). Set the spot_candle_source setting to trade_built to switch the live Spot 1m source to candles synthesized locally from the already-consumed spot trade stream: the publisher then consumes KrakenExchangeClient.subscribe_trade_built_candles and **drops** the ohlc:1m subscription entirely (the native-candle subscription set goes empty — a bandwidth saving), tagging the persisted rows source='calculated' instead of source='native'.

The setting value is read at feed startup, where the Spot 1m subscription (native ohlc:1m vs the trade-built stream) is established once — so **a flip only takes effect after a snapper-feed restart**. A live settings broadcast over ZMQ (_handle_settings_update) refreshes the publisher's cached value but does **not** re-establish the subscription, so it cannot switch the live source on its own; always restart the feed. Roll it out cautiously:

1. Set spot_candle_source=trade_built.
2. Restart snapper-feed (required — the subscription source is fixed at startup; the live ZMQ broadcast updates the cached value only).
3. Verify live Spot 1m rows are landing with source='calculated', that no new native ohlc:1m subscription is opened (the native-candle subscription set is empty), and that the feed stays healthy.
4. Note the illiquid-pair tradeoff: a minute with zero trades emits **no** base 1m bar, and candle_forward_fill fills only higher timeframes — not the base 1m — so thinly-traded pairs will show 1m gaps that the native ohlc:1m channel would have carried.

5. Revert instantly by setting spot_candle_source=native and restarting the feed. The default is native.

Per-wallet executors

Executor process configs are templates named executor_<exchange>. At runtime, ProcessLauncherService spawns one instance per active wallet_credentials row, named executor_<exchange>_w<wallet_short>. Those instances load credentials through CredentialResolver and ignore commands for other wallets. Operators should start/stop the generated wallet instances, not the bare templates.

Enable the feed

```
# 1. Populate Symbol + SymbolExchangeCapability rows from iapi.
#    (kraken_equities entry must be present for is_tradeable
#    cache hits; default deny on missing row.)
snapper update-kraken-equities-symbols --force

# 2. Verify the rows landed. SCD2 active rows have known_to set to the
#    canonical sentinel (9999-12-31 23:59:59.000000 on SQLite — see
#    _KNOWN_TO_ACTIVE_SQLITE in src/snapper/data/models.py).
sqlite3 data/snapper.db \
  "SELECT COUNT(*) FROM symbol_exchange_capabilities
   WHERE exchange='kraken_equities'
         AND known_to='9999-12-31 23:59:59.000000';"
# expect >= 38 (indices only; full catalog up to ~180 contracts)

# 3. Trigger an immediate runtime launch of the symbol-updater +
#    feed-publisher processes. `POST /api/processes/<name>/start`
#    invokes `ProcessLauncherService.start_process_by_name`, which
#    reads the existing persisted Setting row and applies the
#    request payload as RUNTIME-ONLY overrides — it does NOT
#    persist a new config row, and it does NOT go through
#    `ProcessRegistrySyncer`. (Persistent enable/autostart state
#    lives on the create / configure path: `POST /api/processes` or
#    the Settings page; the launcher reads that persisted state at
#    startup, while the registry syncer only seeds and updates the
#    DB rows from the code registry.) With
#    SNAPPER_COORDINATOR_INSTANCE_COUNT >= 2 only coordinator
#    instance 0 runs the startup registry sync; other instances log
#    "Skipping process registry sync on coordinator instance
#    <id>/<count>; instance 0 owns it" so replicas never race the
#    same temporal Setting rows.
#    The body is the full request envelope: `json_body` calls
#    `ProcessStartRequest.model_validate_json` on the raw body, and the
#    envelope (StrictDataSchema, extra="forbid") REQUIRES session_id,
#    sequence_id, public_id, and timestamp. A bare `{"payload":{}}` is
#    rejected with HTTP 422 before the handler runs. `payload` may carry
#    optional `mode` / `parameters` runtime overrides.
curl -X POST http://localhost:8000/api/processes/kraken_equities_symbol_updater/
start \
     -H "Authorization: Bearer <token>" \
     -H "X-CSRF-Token: <csrf>" \
```

```
     -H "Content-Type: application/json" \
     -d
'{"type":"process_start_request","session_id":"<sid>","sequence_id":1,"public_id":"<uuid7>
{}}'

curl -X POST http://localhost:8000/api/processes/kraken_equities_feed_publisher/
start \
     -H "Authorization: Bearer <token>" \
     -H "X-CSRF-Token: <csrf>" \
     -H "Content-Type: application/json" \
     -d
'{"type":"process_start_request","session_id":"<sid>","sequence_id":1,"public_id":"<uuid7>
{}}'

# To stop a running process later:
# curl -X POST http://localhost:8000/api/processes/<name>/stop ...

# 4. `/start` returns once the process is launched in the runtime;
#    there is no "wait one syncer interval" step on this path.
#    If you need the new state persisted across restarts, do that
#    via `POST /api/processes` (create / configure) rather than
#    via `/start`.
```

Backfill historical candles

```
# 30-day 1-hour backfill of the configured KRAKEN_EQUITIES instruments.
# The default setting is the wildcard ["*"], which expands at runtime to
# the full mapped venue universe (all ~180 contracts).
snapper kraken-equities-backfill-candles -t 1h -d 30

# Or the Makefile wrapper (equivalent)
make backfill-kraken-equities-candles

# Single symbol, daily candles, 90 days
snapper kraken-equities-backfill-candles -s MNQU6-CME -t 1d -d 90 --no-resume
```

The endpoint is undocumented/internal — upstream application-layer failures (HTTP 200 with result=null or non-empty errors) raise RuntimeError so they are distinguishable from legitimately-empty windows. See src/snapper/infrastructure/exchanges/implementations/kraken_equities.py:get_ohlcv for the error contract.

To seed daily history for the single-source read cutover and the DB-first warmup, load cached Polygon grouped-daily rows as 1d candles. This loader is cache-only — it reads the on-disk grouped-daily cache and never contacts the Polygon API:

```
# Load every cached day strictly before the cut date as
# source='native', complete=True 1d candles under the live-read venue.
snapper polygon-load-grouped-candles --exchange kraken --cut-date 2026-01-01

# Or the Makefile wrapper (equivalent)
make run-polygon-grouped-candles EXCHANGE=kraken CUT_DATE=2026-01-01
```

```
# Single symbol; or --all for every Polygon-mapped native symbol.
snapper polygon-load-grouped-candles -e kraken --cut-date 2026-01-01 -s BTC-USD
```

--exchange is the **live-read venue** the persisted bars live under (e.g. kraken) — never polygon; the loader writes the 1d history under the same venue the read cutover and warmup resolve. --cut-date must sit at or before the first UTC day live synthesis owns: the loader writes only days **strictly before** it, keeping native backfill and synthesized live bars on disjoint day ranges. There is **no** Docker variant of this loader (make docker-polygon-grouped wraps the separate grouped-daily CSV download, not the candle load). See docs/cli.md for the full option reference (--symbol/-s, --all, --lookback-days).

Quarterly rotation

The default KRAKEN_EQUITIES instruments setting is the wildcard ["*"], which expands at runtime to the full mapped venue universe, so the default scope tracks every available contract automatically. Rotation discipline applies when an operator narrows the setting to an explicit allowlist: the chosen TradFi symbols are quarterly expiry contracts (MNQU6-CME = Sep 26, etc.) and must be rotated before the InstrumentSpec.expiry_at timestamp on any listed symbol drops below 14 days. Rotation cadence:

1. Pull the current expiry list. native_symbol lives on the symbols table (joined to instruments via symbol_public_id); expiry data lives on instrument_specs keyed by instrument_public_id:

sqlite3 data/snapper.db \ "SELECT sym.native_symbol, datetime(spec.expiry_at) FROM instruments AS i JOIN symbols AS sym ON sym.public_id = i.symbol_public_id AND sym.known_to = '9999-12-31 23:59:59.000000' JOIN instrument_specs AS spec ON spec.instrument_public_id = i.public_id AND spec.known_to = '9999-12-31 23:59:59.000000' WHERE i.exchange = 'kraken_equities' AND i.known_to = '9999-12-31 23:59:59.000000';" 2. Identify the next quarterly (e.g. MNQZ6-CME Dec 26 when MNQU6-CME Sep 26 drops below 14 days). 3. Add the new nearest contract by either changing the default KRAKEN_EQUITIES entry inside the AppSettings.instruments property default literal in src/snapper/config/app.py (code path — requires a deploy; roll back via git revert), OR overriding the instruments DB Setting row via the Settings page or POST /api/settings/instruments/set (immediate, no deploy; requires CONFIGURE_SYSTEM + CSRF). instruments is a read-only @property, so the DB Setting row is the only runtime override.

Troubleshooting

- **Zero instruments after update-kraken-equities-symbols --force**: iapi reachability / Origin header mismatch. Verify curl -H 'Origin: https://pro.kraken.com' -H 'Referer: https://pro.kraken.com/' \ 'https://iapi.kraken.com/api/internal/markets/all/futures-contracts?delayed=true' | head -c 200 returns JSON with result.data[...], not result:null,errors:[...].
- **WS disconnect loops on ws-equities.kraken.com**: inspect Kraken status page; no operator action required — the WS client backs off on reconnect. If the symbol_updater keeps returning zero rows across two consecutive runs, fire a monitoring alert.
- **Submit returns 422 instrument_market_data_only**: expected. TradFi is observation-only. Point the strategy at a can_trade=True instrument (crypto/xStocks) for execution.
- **Fragmented Kraken Equities 1m candles** (a settled minute holding more than one historical candle version, written from partial trade sets): the live-synthesis builder fix (065463de, live since 2026-06-10) stops new fragmentation, and the historical damage window (2026-05-24 → 2026-06-10) was repaired in full on 2026-06-11 (158 444 candles rebuilt from raw trades, rerun-idempotent). The one-off repair tooling was removed after execution; if fragmentation ever reappears, recover the repair-equity-candle-fragmentation CLI and its service/repository code from commit 50c98684 (ix_trades_executed_at from migration 0007 is still in place).

Operations runbook — paired-execution guard

The paired-execution guard provides atomicity-by-bounded-compensation for multi-leg strategies (e.g. CointegrationPairs): either every leg of a group fills, or the guard cancels the live remainder, flattens the filled exposure with reduce-only orders, and halts the pair scope until the books are square. The whole guard is gated by PAIRED_EXECUTION_GUARD_ENABLED (default false, fail-closed: live multi-leg signals are refused while the flag is off). This runbook covers the operator's share of the lifecycle: reading incidents, resolving manual_intervention groups at the venue, and attesting the resolution so the halt clears.

State model in sixty seconds

A multi-leg signal creates one **group** (assembling → armed → broken → compensating → completed, with manual_intervention as the operator hand-off state) and one **leg** per instrument. Each leg tracks signed accounting:

- filled_signed_qty — cumulative original-order fill (buy +, sell -),
- compensated_signed_qty — cumulative reduce-only flatten fills, negated so it moves toward filled_signed_qty,
- open_qty = filled_signed_qty - compensated_signed_qty — the residual exposure the guard still owes a flatten. open_qty > 0 means a net-long residual (flatten by SELLING |open_qty|); open_qty < 0 means net-short (flatten by BUYING).

A failing group's scope — (wallet, strategy, group_key) where group_key is the canonical sorted exchange:instrument:mode token set — carries one durable halt row in paired_execution_halts. The halt blocks NEW grouped signals on the same pair and is mirrored into each trade coordinator's in-memory shard halts. The guard scanner interval is max(1.0, PAIRED_EXECUTION_ASSEMBLY_TIMEOUT_S / 2), which is 2.5 seconds with the default 5-second assembly timeout.

The automatic lifecycle (no operator action)

Breaks (assembly timeout, fill timeout, a leg rejected/cancelled/expired before filling) are handled end-to-end by the scanner: held commands are cancelled, live originals are cancelled at the venue, filled exposure is flattened with reduce-only market orders (re-flattening on late fills with a fresh compensation_seq each round), and once every leg is settled at zero open exposure the group COMPLETES, the scope's halt clears, and the pair may trade again — typically within one scanner cycle of the last fill settling. If you see a broken or compensating incident with automation still in flight, the correct action is usually to wait one cycle.

A submit the executor refuses locally because its venue circuit breaker is open counts as a leg rejection too: the order is rejected with reason circuit_breaker_open and its command is durably FAILED, so it can never fire late once the breaker closes. A flatten refused the same way counts as that flatten's terminal — the leg reopens to filled and the sweep re-flattens it next cycle, retrying until the breaker closes. Breakage does not depend on live messages alone: the coordinator's reconciliation loop folds durable venue events into command statuses, and the scanner projects a durably-terminal original command onto its stuck leg, so a lost reject/cancel message still breaks the group within a cycle.

When the guard hands off to you: manual_intervention

The guard escalates a leg (and its group) to manual_intervention instead of flattening when an automatic flatten would be unsafe:

- the venue/instrument has no reduce-only capability,
- the instrument spec is missing, or its lot size is missing/non-positive,
- the residual rounds below one lot or the venue minimum order size (dust),
- the instrument cannot be resolved (e.g. delisted mid-compensation).

A manual_intervention group stays HALTED every cycle by design until you resolve and attest it. Clearing the halt row by hand does NOT help — the scanner re-creates it on the next cycle while the group remains exposed.

Triage: read the incident

```
curl -s -H "Authorization: Bearer $SNAPPER_PAT" \
    "$SNAPPER_BASE_URL/api/paired-execution/incidents" | jq
```

One incident per halted/exposed scope, carrying the durable halt (reason, owning group), every exposed group with its status and failure_reason, and per-leg exposure: status, filled_signed_qty, compensated_signed_qty, open_qty, compensation_seq. Notes:

- halt_missing: true flags an exposed scope whose halt row is transiently absent (a crash/clear window). The scanner re-halts it within a cycle; it is surfaced so you never lose sight of exposure.
- A broken group with zero-fill legs (e.g. an assembly timeout) is NOT an incident — the guard deliberately leaves it free to re-assemble.
- Requires READ_POSITIONS; non-admin callers see only their wallets.

Resolve at the venue

For each leg with status = manual_intervention and open_qty != 0:

1. Confirm the actual venue position for that instrument/wallet.
2. Close the residual at the venue (UI or API): SELL |open_qty| when open_qty > 0, BUY |open_qty| when open_qty < 0. Use reduce-only if the venue supports it.
3. Verify the venue position is flat (or back at the strategy's intended level).

Orders you place outside Snapper's order flow do NOT update compensated_signed_qty — the leg's books will still show the residual. That is expected: the leg keeps the true record of what THE GUARD did, and your attestation (below) is the durable record that the remainder was resolved manually.

Attest: terminalize the group

```
curl -s -X POST -H "Authorization: Bearer $SNAPPER_PAT" \
    "$SNAPPER_BASE_URL/api/paired-execution/groups/$GROUP_ID/terminalize" | jq
```

Requires MANAGE_PAIRED_EXECUTION (OPERATOR or ADMIN). On success the group becomes completed with terminalized_by=<you> stamped into its failure_reason, legs keep their true accounting, and the scanner clears the scope's durable halt and every coordinator's in-memory mirror within one cycle — the pair can trade again. Responses:

- 404 — no such group in your accessible wallets.
- 409 — not attestable right now. The detail's status is freshly read. Causes: the group is not manual_intervention (nor a compensating group re-opened onto a manual leg); a sibling leg still has automation in flight (a working original or an unfinished flatten — wait a cycle and retry); the group has no legs; or a held original command still exists.

If a LATE original fill lands after your attestation, the guard re-opens the group to compensating; automation re-flattens what it can, and if the manual leg is the residual holder the group will refuse auto-completion again — re-resolve at the venue if needed and POST terminalize again (the re-attestation path accepts a compensating group with a manual leg).

Known windows and boundaries

- **Reopen right after a clear**: a late fill landing immediately after a halt cleared leaves the scope exposed and un-halted for at most one scanner cycle; the next cycle re-halts. Self-healing, no action.

- **Missed not-tradeable reject**: a flatten rejected locally as not-tradeable (a delisted instrument) writes no durable venue event; if its live message is also lost, the leg stays compensating + halted, and the group is NOT attestable while it does (terminalize requires every non-manual leg settled, so POST terminalize returns 409). The executor's dispatched-command verification sweep normally heals this: the flatten's command row is still dispatched with no venue evidence, so on a venue that can look an order up by its client order id the sweep verifies it absent twice (no earlier than max(120 s, 2 × dispatch TTL) after creation), publishes the REJECTED status, and records the durable order_rejected event — the compensating sweep then settles the leg, and the residual re-flattens or escalates to manual_intervention under the normal sweep rules. The sweep does NOT auto-reject when the dispatch TTL is disabled (a frame may then legally be in flight at any age), when the command has aged past one hour (venue closed-order lookback makes absence non-authoritative — WARN-only escalation), or when the venue has no client-order-id lookup; in those cases the scope stays halted (fail-safe) until a durable venue event for the flatten appears or the rows are reconciled by hand.
- **Dispatch max-age TTL applies to paired commands too**: a paired CREATED command older than TRADE_COMMAND_DISPATCH_TTL_S (default 30 s) is CAS-expired by the outbox instead of dispatched — a group wedged pre-dispatch longer than the TTL breaks via the assembly/fill deadlines rather than firing stale legs.
- **Late fills older than 7 days**: startup recovery replays fills into groups completed within the last 7 days. Older late fills are reconciliation territory.
- **Reconciliation halts are separate**: a shard halted by the recon circuit breaker is NOT released by paired-execution machinery (and vice versa — the paired release is reason-scoped). A wedged recon halt currently requires a coordinator restart.
- **Do not hand-edit guard tables.** Clearing halts re-halts within a cycle while exposure remains; editing leg statuses falsifies the books the accounting relies on. The only supported operator mutation is the terminalize endpoint.

Enablement

The guard ships dark. Enabling real-money multi-leg execution (PAIRED_EXECUTION_GUARD_ENABLED=true) should follow the staged enablement plan (staging trial, outbox-gate query plan check on the production database, rollback criteria) maintained alongside the deployment docs.

snapper-egress sidecar — operator runbook

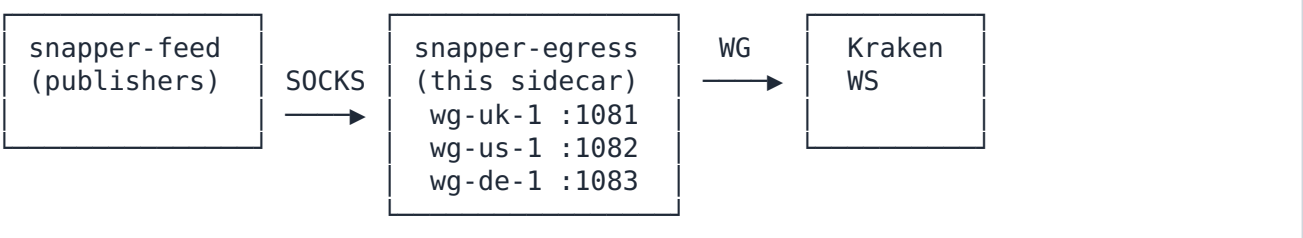
The snapper-egress sidecar runs N WireGuard tunnels plus N matching SOCKS5 listeners inside one container, so Kraken Spot/Futures/Equities WebSocket publishers and Walutomat HTTP polling can route public market-data traffic out through alternate egress IPs without changing source code.

This sidecar is part of the egress-multiplexer subsystem.

Prerequisites

- **Docker Compose v2.** depends_on.condition: service_healthy is Compose v2 syntax. Compose v1 silently ignores it.
- **Kernel WireGuard on the host when tunnels are configured.** Verify with lsmod | grep wireguard or modprobe wireguard. The sidecar bind-mounts /lib/modules and uses the host kernel module via netlink (no userspace WireGuard). Empty/default deployments with no declared tunnels still expose /ready without probing WireGuard. Use /readyz when you need strict "every declared tunnel is up" semantics.
- **MASTER_PASSWORD** in the same .env the snapper (API) container uses. The sidecar reads encrypted egress_tunnel_*_private_key settings via the same Fernet path; mismatched master passwords silently mean "no tunnels load".
- **DB_URL and ZMQ_BROKER_XSUB** are both hard-required by the sidecar entrypoint (snapper.egress.__main__). The process exits with **code 2** at startup if either is missing: DB_URL lets the SettingsService read tunnel descriptors + encrypted keys, and ZMQ_BROKER_XSUB wires SettingsService change-broadcast support. DB_URL is supplied via env_file: .env. In the Compose split, ZMQ_BROKER_XSUB MUST be the routable broker host tcp://snapper:7500, NOT the .env default tcp://127.0.0.1:7500 — the docker-compose.yml snapper-egress service overrides it for exactly this reason. snapper-egress runs in a separate container, so localhost would not reach the backend broker living in the snapper container.

Architecture (short)



Per tunnel: 1. WireGuard interface wg-<id> lives in the sidecar container. 2. Source-based routing rule sends packets from the tunnel's IP through the matching WG interface. 3. SOCKS5 listener binds outbound sockets to the tunnel's IP → kernel routes them via the tunnel.

Kraken WS handshakes go through the EgressPool (socks5h://snapper-egress:<port>) inside the feed-publisher subprocesses of the **snapper-feed** container (the API profile excludes market-data publishers; the FEED profile runs only them). Each publisher process builds its own process-local pool at start, gated on the feed_egress_enabled DB setting (default **off** — publishers dial direct until the gate is flipped). The pool reserves a route per Kraken WS handshake. A route is quarantined — and the next handshake fails over to an alternate (or the direct fallback) — on any of the QuarantineReason values: http-429 (handshake 429), close-1015 (Cloudflare close frame), http-connect-error (REST connect failure via the pooled transport), and ws-connect-error (a WS connect-level failure — TCP/SOCKS timeout, connection refused, or a silent blackhole — through a proxy route, quarantined ~30 s via _WS_CONNECT_ERROR_QUARANTINE_S). Connect-level failover matters because a dead tunnel often blackholes rather than returning 429: without it the reconnect loop would re-dial the same dead route indefinitely.

Adding a tunnel

Preferred path: run the provisioning helper from a checkout that has scripts/ available. The production Docker image does not copy scripts/ into /app, so run it on the host or bind-mount the checkout into a maintenance container with the same DB_URL.

```
poetry run python scripts/provision_egress_tunnel.py \
  --tunnel-id wg-uk-1 \
  --interface wg-uk-1 \
  --address 10.64.12.34 \
  --prefix-length 32 \
  --private-key-file ./secrets/wg-uk-1.key \
  --peer-pubkey "<peer-public-key>" \
  --peer-endpoint vpn.example.com:51820 \
  --socks5-listen-port 1081 \
  --priority 10 \
  --dry-run
```

Remove --dry-run after validation. The helper writes the descriptor, encrypted private/ optional preshared key settings, and merges the SOCKS5 route into egress_pool. It can also restart the sidecar with --restart-sidecar. After egress_pool changes, restart **snapper-feed** (each feed-publisher process builds its own process-local pool at start) in addition to snapper (the API tier) — restarting only the API rebuilds the API process's pool but not the pools the Kraken/Walutomat feeds actually dial through.

Manual path:

Three DB settings per tunnel:

Key	Encrypted?	Type	Example
egress_tunnel_<id>	no (JSON descriptor)	text	{"interface":"wg-uk-1", "address":"10.64.12.34", "prefix_length":32, "peer_pubkey":"...", "peer_endpoint":"vpn.example.com:51820", "socks5_listen_port":1081, "priority":10}
egress_tunnel_<id>_private_key	yes (auto-encrypted because the key contains the private_key substring)	text	base64 WG private key
egress_tunnel_<id>_preshared_key	yes, optional	text	base64 WG PSK (or absent / empty)

Constraints:

- id MUST NOT contain underscores (it would collide with the _private_key / _preshared_key naming convention). Use hyphens.
- interface MUST start with wg- and be ≤ 15 chars total (Linux IFNAMSIZ).
- socks5_listen_port MUST be 1024-65535, unique across tunnels. Convention: 1081 for the first tunnel, then 1082, 1083, ...
- address must parse as IPv4 or IPv6; IPv4 prefix ≤ 32, IPv6 ≤ 128.

Once the three keys exist in the DB, restart the sidecar:

```
docker compose restart snapper-egress
```

The sidecar orchestrator (run_sidecar) calls load_declared_tunnels(service) → wg_control.bring_up(...) → Socks5Server.start() for each tunnel. Failures are best-effort: one bad tunnel does NOT block the others.

Wiring the route into the EgressPool

Edit the egress_pool setting to add the new route:

```
{
  "enabled": true,
  "on_all_quarantined": "wait",
  "private_fallback_route_id": "wg-uk-1",
  "routes": [
    {"id": "default", "kind": "direct", "priority": 100, "enabled": true},
    {"id": "wg-uk-1", "kind": "socks5",
     "proxy_url": "socks5h://snapper-egress:1081",
```

```
      "region": "uk-lon",
      "exit_ip": "203.0.113.20",
      "provider": "wireguard-uk",
      "priority": 10, "enabled": true}
  ]
}
```

region, exit_ip, and provider are optional operator metadata fields. They do not affect routing; they are surfaced through GET /api/health/egress so an operator can identify where a route exits.

private_fallback_route_id is an optional top-level key (default null). It names a declared route used as the **private-traffic fallback only when no healthy direct route exists** — so executor (order) traffic CAN ride a SOCKS5 tunnel when configured, even though private traffic normally goes direct (see *Public vs private traffic* below). It is validated at config-load: when set, it MUST reference an existing route id, otherwise EgressPoolConfig rejects the pool. Its allowed_exchanges is **ignored** on the private fallback path (the route may be publicly pinned to one exchange while still serving as the private fallback for another). Kraken authenticated REST wires this route for private executor REST reads.

Priority semantics: EgressPool.reserve() sorts by (priority, in_use_count) and picks the **lowest** number first. For SOCKS5 to actually win selection, its priority MUST be less than the default route's priority. Bootstrap configurations that ship default: priority=0 and a new tunnel at priority=10 would silently keep using direct egress because 0 < 10. Set the direct route to a high number (e.g. 100) when you want it to act as fallback only. Keep at least one enabled direct route whenever egress_pool.enabled=true; current schema does not support disabling direct fallback for an enabled pool.

Per-exchange routing — pinning a tunnel to one exchange

To restrict a SOCKS5 route to specific exchanges, add an allowed_exchanges field listing the exchange names (matching ExchangeEnum *values* — "walutomat", "kraken", "kraken_futures", "kraken_equities", "polygon", "paper"):

```
{
  "id": "wg-eu-1", "kind": "socks5",
  "proxy_url": "socks5h://snapper-egress:1084",
  "priority": 10,
  "allowed_exchanges": ["walutomat"],
  "enabled": true
}
```

A non-empty allowed_exchanges restricts the pool: when pool.reserve(exchange=...) is called with an exchange name not in the list, this route is filtered out before the (priority, in_use_count) sort. An empty allowed_exchanges (the default) means the route serves any exchange — the back-compatible path for existing pool entries.

The direct fallback path IGNORES allowed_exchanges by design. When every preferred route is quarantined, EgressPool returns the direct route as a last resort regardless of any exchange pin. If you want SOCKS routes preferred for a specific exchange, pin those routes with allowed_exchanges and give the direct route a worse priority; current schema does not support per-exchange denial of direct fallback. Unknown exchange names (typos) are rejected by EgressPoolConfig at config-load time, so a "krakeen" typo never silently makes a route unreachable.

Public vs private traffic

Routing depends on the reservation's **traffic class** (TrafficClass = Literal["public", "private"], defaulting to "public"):

- **Public** market-data traffic (Kraken WS publishers and other feeds) routes over the VPN/SOCKS5 tunnels by the public selection path — the (priority, in_use_count) sort honouring each route's allowed_exchanges allow-list, with the direct route as fallback.
- **Private** executor (order) traffic is routed **direct-first**. Executors wrap their work in egress_identity(traffic_class="private", owner="executor"), and the pool routes traffic_class="private" through a separate selection path (_pick_private_locked) that prefers a healthy direct route regardless of route priority and ignores allowed_exchanges, bypassing the public sort entirely. When direct is unavailable and private_fallback_route_id is configured, private idempotent REST reads and the next private WebSocket reconnect can ride that fallback route. Private mutations remain direct-only. Kraken authenticated REST ops are tagged the same way.

In short: **public = VPN/SOCKS5 preferred** (priority + allow-list, with direct fallback), **private = direct-first**. A healthy direct route always wins for private traffic; private_fallback_route_id is used only when direct is unavailable, and private mutations remain direct-only.

Then enable the feed gate (DB setting feed_egress_enabled=true) if not already on, and restart both the API and the feed tier — the feed publishers hold their own process-local pools:

```
docker compose restart snapper snapper-feed
```

The process-local egress preflight validates each SOCKS5 route before installing the pool: it checks that python-socks is importable, resolves the proxy host, opens the proxy port, and sends a SOCKS5 NO_AUTH greeting. A route that fails preflight is auto-disabled in that process's effective config, so restart every process that should use the updated route set.

The orchestrator logs:

```
egress_pool: configured with 2 route(s), on_all_quarantined=wait
```

Verification

1. **Sidecar healthy?** docker compose exec snapper-egress \ python -c "import urllib.request; print(urllib.request.urlopen('http://127.0.0.1:8081/ready', timeout=2).read())" Expect a 200 with the running + failed tunnel id arrays. This is the Compose health endpoint and intentionally stays 200 after bootstrap even if an individual tunnel failed.
2. **All tunnels up?** docker compose exec snapper-egress \ python -c "import urllib.request; print(urllib.request.urlopen('http://127.0.0.1:8081/readyz', timeout=2).read())" Expect 200 only when every declared tunnel is up; otherwise it returns 503 with the failed tunnel ids.
3. **Tunnel status detail?** docker compose exec snapper-egress \ python -c "import urllib.request; print(urllib.request.urlopen('http://127.0.0.1:8081/tunnels', timeout=2).read())" Each tunnel id maps to {"status": "up", "reason": null}.
4. **Backend egress snapshot visible?** From an authenticated operator session with read:system_status, call GET /api/health/egress. Expect payload.enabled=true, one row per configured route, route metadata (region, exit_ip, provider) when configured, quarantine state, merged in_use_count, per-reservation container, connections rows for open WebSocket host counts and capped REST last-seen hostnames, and a transfer object on SOCKS5 routes when snapper-egress has reported matching WireGuard counters for that listener port. Transfer rows include cumulative rx/tx bytes, current byte rates when a reliable delta exists, the latest WireGuard handshake timestamp, sample age, and stale/reset flags. Direct routes, missing samples, and ambiguous shared ports show transfer=null. Connection rows expose hostnames only, never URL paths, query strings, headers, request bodies, or credentials. A missing snapper-feed row means the API process has not received a system.egress.snapshot frame from that container yet; a stale row remains visible with stale=true.
5. **WireGuard handshake established?** (operator debugging — uses iproute2 shipped in the image) docker compose exec snapper-egress ip -d link show wg-uk-1 docker compose exec snapper-egress ip rule show table 5000 The interface should be UP and the rule should pin from <tunnel_addr> → table N.
6. **Kraken WS uses the tunnel?** The pool is a process-local singleton inside each feed-publisher process — get_egress_pool() in a fresh docker compose exec interpreter returns None, and quarantining inside the API process would not touch the publisher pools, so a REPL-based quarantine cannot exercise the failover. Instead, steer by configuration: give the direct route the worst priority in the egress_pool setting, make sure feed_egress_enabled=true, then restart snapper-feed and observe the next handshake. The publisher log line should show the tunnel route id; Cloudflare's view of the source IP should match the VPN exit. Restore the normal direct-route priority when done.

7. **Stable tick flow?** sql SELECT i.exchange, COUNT(*) AS n FROM ticks t JOIN instruments i ON i.public_id = t.instrument_public_id WHERE t.timestamp > NOW() - INTERVAL '60 seconds' GROUP BY i.exchange ORDER BY n DESC; Kraken should show consistent counts (~1500-2000 ticks/s in steady state). Drops indicate either the tunnel is flapping or the SOCKS5 listener is mis-bound.

Threat-model + security notes

- The SOCKS5 listener runs WITH NO AUTHENTICATION. Isolation comes entirely from the Docker snapper-internal network. The CI lint hook scripts/ check_egress_compose.py rejects any ports: entry, network_mode: host, or env-interpolated bypass on the snapper-egress service so an operator cannot accidentally publish the listener to the host.
- The same lint hook also enforces the unified-image runtime contract: snapper-egress must use the same image as snapper, run command: ["egress"] as user: "0:0", include NET_ADMIN, and mount /app/data when the monolith does. The snapper service must remain unprivileged, with no cap_add and no user override.
- All traffic on snapper-internal is trusted equally. Do NOT attach untrusted containers (other tenants, debugging shells from unknown sources) to that network. If you must, add SOCKS5 username/password auth in socks5_server.py (≈10-line change).
- For extra defense-in-depth, run docker compose config | grep -A5 snapper-egress after any compose edit to verify the effective config still has no host port mapping. The lint hook catches the static cases but not every override pattern.
- WG private keys are stored Fernet-encrypted in the Postgres settings table. The encryption key is derived from MASTER_PASSWORD (env var). Loss of master password = loss of every encrypted setting (not just WG keys).

Decommissioning a tunnel

1. Remove the route from egress_pool (set enabled: false on that route OR delete the entry); restart the API and feed tiers (docker compose restart snapper snapper-feed).
2. Delete the three settings (egress_tunnel_<id>, egress_tunnel_<id>_private_key, egress_tunnel_<id>_preshared_key).
3. Restart snapper-egress. The orchestrator's startup bring_down pass (idempotent cleanup before bring_up) removes any stale interface / ip-rule / routing-table entry left over.

Rotating private keys

1. Generate a new private key + matching peer config on the VPN side.

2. Update the egress_tunnel_<id>_private_key setting (SettingsService re-encrypts on write).
3. Restart snapper-egress to pick up the new key. The orchestrator's bring_up is idempotent and recreates the WG interface from scratch.

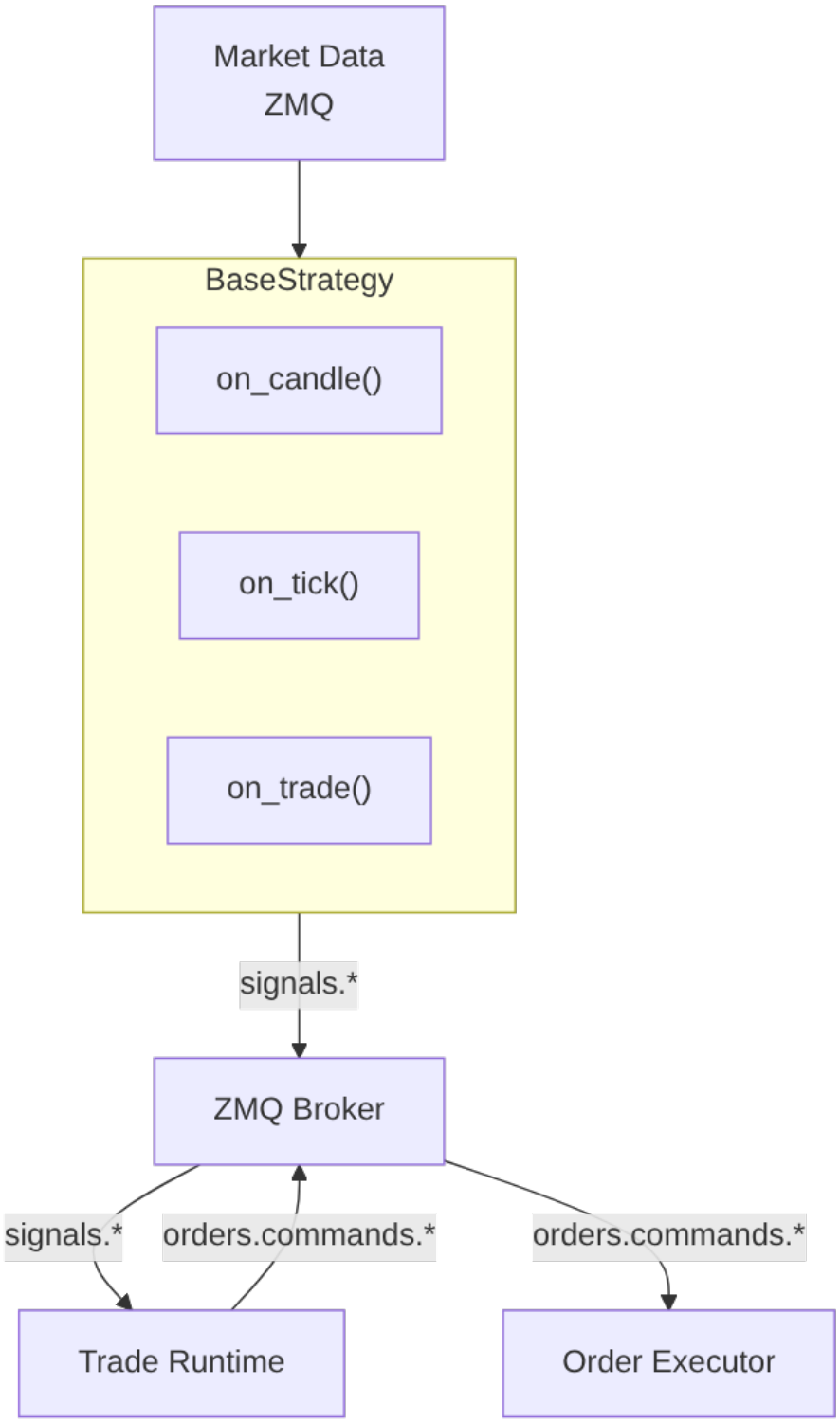
Operational expectations

- Sidecar startup: ~5-15 s (DNS resolution of peer endpoints is the dominant cost).
- HTTP 429 failover: ~1-2 s (route quarantined → next handshake picks alternate).
- WS connect-error failover (ws-connect-error): the dead proxy route is quarantined ~30 s on a connect timeout/refused/blackhole, so the next reconnect rides an alternate route (or the direct fallback) instead of re-dialing the dead tunnel.
- Tunnel handshake age: WireGuard re-keys every 2 minutes; if wg show <iface> latest-handshakes returns > 180 s ago, the peer is unreachable. The backend also surfaces the latest handshake from sidecar system.egress.transfer samples on the matching SOCKS5 route.
- SOCKS5 listener latency: ≤ 1 ms additional vs direct route (single process, single asyncio loop, no auth handshake).

Trading Strategies

Snapper provides a framework for creating trading strategies based on ZeroMQ messaging. Strategies subscribe to market data and publish signals. Strategies produce intent only; the trade runtime owns order lifecycle, positions, and balance projections.

Strategy Architecture



Creating Strategies

Basic Structure

```python
from snapper.strategies.base import BaseStrategy, StrategySignal, StrategyConfig
from snapper.strategies.decorators import register_strategy,
create_strategy_process
from snapper.messaging.schemas.data import CandleData


@register_strategy("MyStrategy")
@create_strategy_process(
    process_name="my_strategy_btc",
    default_config={
        "name": "my_strategy_btc",
        "inputs": ["market.kraken.BTC-USD.candles.1h"],
        "outputs": ["BTC-USD"],
        "exchange": "paper",
        "params": {
            "threshold": 0.5,
            "period": 14,
        },
    },
)
class MyStrategy(BaseStrategy):
    """Strategy description in docstring."""

    def __init__(self, config: StrategyConfig) -> None:
        """Initialize strategy."""
        super().__init__(config)
        self.threshold = self.params.get("threshold", 0.5)
        self.period = self.params.get("period", 14)

    async def on_candle(self, instrument: str, candle: CandleData) ->
StrategySignal | None:
        """Process candle and generate signal."""
        candles = self.candle_buffer.get(instrument, [])
        if len(candles) < self.period:
            return None

        closes = [c.close for c in candles]
        current_price = closes[-1]

        if self._should_buy(closes):
            return StrategySignal(
                instrument=instrument,
                side="buy",
                strength=1.0,
                price=current_price,
                reason="Buy condition met",
            )

        if self._should_sell(closes):
            return StrategySignal(
                instrument=instrument,
                side="sell",
                strength=1.0,
                price=current_price,
                reason="Sell condition met",
            )
```

```
        return None

    def _should_buy(self, closes: list[float]) -> bool:
        """Buy condition logic."""
        return False

    def _should_sell(self, closes: list[float]) -> bool:
        """Sell condition logic."""
        return False

    async def reset(self) -> None:
        """Reset strategy state (for replay)."""
        pass
```

Decorators

@register_strategy(name)

Registers strategy class in factory under given name.

```
@register_strategy("RSIReversion")
class RSIReversion(BaseStrategy):
    ...
```

@create_strategy_process(process_name, default_config)

Creates process wrapper for strategy and registers in process manager.

```
@create_strategy_process(
    process_name="strategy_rsi_eth_1h",
    default_config={
        "name": "rsi_eth_1h",
        "inputs": ["market.kraken.ETH-USD.candles.1h"],
        "outputs": ["ETH-USD"],
        "exchange": "paper",
        "params": {"period": 14, "upper": 70.0, "lower": 30.0, "cooldown": 2},
    },
)
class RSIReversion(BaseStrategy):
    ...
```

Strategy Configuration

StrategyConfig

Field	Type	Description
name	string	Unique strategy instance name
strategy_class	string	Strategy class name
inputs	list[str]	List of ZMQ topics to subscribe
outputs	list[str]	List of instruments for signals
exchange	string	Target exchange (paper, kraken, kraken_futures, walutomat)
params	dict	Strategy-specific parameters
wallet_public_id	string	Wallet that will execute orders for this strategy. Still defaults to empty and is NOT validated by StrategyConfig itself (the dataclass keeps an empty default pending the NOT NULL tightening migration). Process routes validate active operator/wallet grant coverage when both this field and operator_public_id are populated; a wallet without an operator is rejected. The non-empty requirement applies at runtime via the caps guard for any strategy that uses create_ai_review_and_await() — see "AI delegate consultation" below.
operator_public_id	string	Trading-identity operator that owns this strategy instance. Empty default; validated against the launching principal's operator_public_ids when populated, and used with wallet_public_id for active grant and live-output coverage checks.

Candle history warm-up (A3-smoke)

A strategy that needs historical bars before it can compute indicators overrides required_candle_history() -> int (e.g. CointegrationPairs returns its lookback_window). When that value is > 0, BaseStrategy.start() prefills the candle buffer from history BEFORE subscribing to the live feed, so the strategy is indicator-ready from a cold start instead of waiting that many live periods (a restart re-warms the same way).

Warm-up is **DB-first** (Phase 3 slice 5) and **opt-in + crypto-scoped**: it reads the persisted 1d plane (get_candles under the leg's live venue) so the warmed series is continuous with the live synthesized 1d bars and the read path is single-source. Only

when the persisted plane is short for some leg (e.g. before the operator has applied the daily backfill) does it fall back to the local Polygon **crypto daily** cache as a non-canonical bootstrap (logged as such).

- Set params["warmup_market_type"] = "crypto" to enable it (without it, warm-up is skipped and the buffer fills live-only). This keeps a non-crypto strategy from ever loading a crypto ticker.
- Set params["buffer_size"] (default 100) >= the lookback, or warm-up is skipped with a warning (no silent under-warm).
- Optional params["polygon_cache_root"] (default data/polygon/cache) — the bootstrap fallback root, used only when the DB is short.
- Only 1d candle inputs are warmed (others rely on live fill).
- **Aligned all-or-nothing for multi-leg strategies**: a pair (e.g. cointegration) is warmed only if every leg shares at least the required number of common UTC days; on any shortfall NO leg is warmed (symmetric live-only fill), so the spread can never start date-misaligned. A warm-up error never crashes startup.

Note: warm-up does NOT arm a strategy. For the FET/RENDER daily cointegration forward-test, the spread's per-leg positional alignment is a separate hard pre-arming gate (it must align legs by open_at, not list position) — see the A3 plan.

AI delegate consultation

Strategies that consult an AI delegate via create_ai_review_and_await() and then forward the approved outcome to emit_signal(outcome=...) MUST set StrategyConfig.wallet_public_id to a non-empty value. The trader-coordinator's caps gate (TradingCapsEnforcer.guard_with_ai_review_attribution) fail-closes early when the engine submission's wallet_public_id is empty:

```
CapsViolationError: missing_wallet_for_ai_review_attribution
    submission.wallet_public_id required for the strategy AI gate;
    engine state must populate wallet on the submission before
    invoking this guard
```

The error is loud and operator-actionable: it identifies the exact missing config field. Without wallet_public_id, the engine cannot map the AI delegate's approved trade to a wallet for cap evaluation (per-user notional / open-orders / quantity caps key on wallet+user). Pure non-AI strategies that never invoke create_ai_review_and_await() continue to run without a wallet; they bypass caps via guard_service_principal() per the existing audit-bypass contract.

The ai_review_public_id carried on the published SignalData is transport-only end-to-end through the ZMQ wire. The companion ai_review_dispatch_version (dedup key) is also transport-only — the strategy citation validator does not compare it; the bus publisher reads dispatch_version from the cited row at publish time so downstream caps-violation fanout events are always tagged with the row-of-record version.

Input Topics

Common formats:

- market.{exchange}.{instrument}.candles.{timeframe} — Candle streams
- market.{exchange}.{instrument}.ticks — Tick streams
- market.{exchange}.{instrument}.trades — Trade streams
- market.paper.{source_exchange}.{instrument}.{type} — Paper replay streams from a historical source

Examples:

- market.kraken.BTC-USD.candles.1h — Hourly BTC/USD candles from Kraken
- market.kraken.ETH-USD.ticks — ETH/USD ticks from Kraken
- market.paper.polygon.AAPL.trades — Paper replay of AAPL trade tape from Polygon

Output Topics (Signals)

Generated automatically:

- Paper: signals.paper.{instrument}.{strategy_name}
- Live: signals.{exchange}.{instrument}.live

StrategySignal Structure

```
@dataclass
class StrategySignal:
    """Trading signal."""

    instrument: str
    side: TradeSide
    strength: float
    reason: str
    price: float
    timestamp: datetime | None = None
```

Built-in Strategies

RSIReversion

Mean-reversion strategy based on RSI.

Parameters:

Parameter	Default	Description
period	14	RSI period
upper	70.0	Overbought threshold (sell)
lower	30.0	Oversold threshold (buy)
cooldown	0 (class fallback) / 2 (registered process preset)	Candles between signals

Logic:

- Buy when RSI <= lower **or** RSI was at or below lower on the previous candle (r_prev <= lower) and price has since reversed upward.
- Sell when RSI >= upper **or** RSI was at or above upper on the previous candle (r_prev >= upper) and price has since reversed downward.
- The previous-threshold + reversal branch lets the strategy capture exits that miss the strict edge-touch but confirm via price action.

```
from snapper.strategies.rsi import RSIReversion
```

MACDCrossover

Trend-following strategy based on MACD.

Parameters:

Parameter	Default	Description
fast	12	Fast EMA
slow	26	Slow EMA
signal_period	9	StrategySignal line period

Logic:

- Buy when MACD histogram crosses from negative to positive
- Sell when histogram crosses from positive to negative

```
from snapper.strategies.macd import MACDCrossover
```

Cointegration

Pairs trading strategy based on cointegration. Trades the spread between two cointegrated instruments, entering when the z-score crosses entry_threshold and exiting on mean reversion past exit_threshold.

The spread aligns the two legs **by open_at** (the days both legs have a bar), never by list position — a live feed delivering one leg's bar before the other cannot desync the regression. A signal fires only when the triggering candle is **both legs' current bar** (open_at == latest common day == latest seen), so the z-score and both legs' order prices are always the same executable day; the completing (second-arriving) leg of each period carries the signal. At most one signal is emitted per day, and no signal fires on a day at/under the warm-up high-water mark (prefilled history is context, not tradeable). This matches the date-aligned daily backtest.

```
from snapper.strategies.cointegration import CointegrationPairs
```

CointegrationPairs is the canonical 2-leg consumer of MultiLegSpreadMixin — its 2-leg invariants are enforced by self._init_legs(expected_count=2) and instrument1 / instrument2 are backwards-compatible aliases for self.legs[0] / self.legs[1]. It declares PAIRED_EXECUTION_POLICY = SIMULTANEOUS, so its entry/exit pairs emit as one synchronized paired-execution group. See the next section for the generalized N-leg API.

Parameters:

Parameter	Default	Description
beta	0.05	Hedge ratio between legs (long leg − beta × short leg)
entry_threshold	2.0	Z-score threshold for entry (in standard deviations)
exit_threshold	0.5	Z-score threshold for mean-reversion exit
lookback_window	50	Window length for rolling spread statistics
min_data_points	30	Minimum buffered candles before any signal is emitted

Two processes register this class out of the box: strategy_cointegration_btc_eth trades hourly BTC-USD/ETH-USD paper candles with the defaults above, and strategy_cointegration_fet_render is a forward-test preset that trades daily FET-USD/RENDER-USD paper candles with a screened hedge ratio (beta=0.257463, lookback_window=60). It opts into the A3-smoke Polygon crypto daily warm-up and sets buffer_size=100, so a fresh default process can prefill the 60-bar spread window when the local cache is present.

Proprietary strategies

When the proprietary submodule is present, additional strategy classes live under proprietary/src/strategies. They are registered by their modules and use the same BaseStrategy and process-wrapper contracts as open-source strategies. Ignore proprietary/plans and proprietary/memory historical notes unless a current README explicitly points to them as live docs.

SPYBTCMomentum

SPYBTCMomentum is a cross-asset momentum strategy: completed/current SPY hourly movement drives same-direction BTC signals. It does **not** trade a correlation spread and does **not** emit SPY orders.

The default paper process subscribes to 1-minute SPY and BTC candles, aggregates both into hourly bars, and emits at most one BTC signal per UTC hour during the configured US session window. A positive SPY return above spy_threshold emits BUY on BTC; a negative SPY return below -spy_threshold emits SELL on BTC. Signal price comes from the current BTC hourly aggregate.

Parameters:

Parameter	Default	Description
spy_threshold	0.002	Absolute SPY hourly return threshold before a BTC signal can fire.
us_session_start_utc	14	Inclusive UTC hour at which signals may start.
us_session_end_utc	21	Exclusive UTC hour at which signals stop.
spy_instrument	SPY	Instrument identifier used to route incoming SPY candles.
btc_instrument	BTC-USD	Target BTC instrument emitted in StrategySignal.instrument.

ParlayCascade

ParlayCascade is a sequential N-leg compounding strategy. It uses MultiLegSpreadMixin for the ordered leg roster, but it does not open all legs at once. It starts on leg 0 only after the trailing-window drift gate clears, then advances through the configured legs one at a time.

Each active leg exits by one of three paths:

1. Favourable move reaches tp_pct: exit the current leg. If another leg remains, the callback returns [exit_current, enter_next] so the handoff uses the multi-leg list return contract.
2. Adverse move reaches sl_pct: exit the current leg, mark the cascade busted, and start the cooldown.
3. max_bars_per_leg candles elapse: exit the current leg as a timeout bust and start the cooldown.

The min_edge_bps gate is measured over lookback_window candles on leg 0. For long entries, positive drift is favourable; for short entries, the sign is flipped so positive still means "edge in our favor". After a bust, cooldown_bars candles must drain before the next attempt can start. A successful full cascade moves to complete state without cooldown and then returns to idle on the next eligible cycle.

Parameters:

Parameter	Default	Description
tp_pct	0.01	Per-leg take-profit threshold as a fraction. Must be positive.
sl_pct	0.01	Per-leg stop-loss threshold as a fraction. Must be positive.
max_bars_per_leg	24	Per-leg timeout in candles. Must be positive.
entry_side	buy	Direction used for every leg entry; sell inverts favourable/ adverse tests.
lookback_window	20	Number of trailing candles used by the drift edge gate.
min_edge_bps	5.0	Minimum favourable drift, in basis points, required before leg 0 can enter.
cooldown_bars	4	Number of candles to wait after a bust before another attempt.

Multi-leg / N-leg basket strategies

Snapper ships a reusable mixin for strategies that emit synchronized signals across N legs (pair-trades, baskets, calendar spreads, cross-exchange arb). The 2-leg cointegration strategy is the canonical consumer; the same mixin generalizes to N ≥ 2.

```
from snapper.strategies.multi_leg import MultiLegSpreadMixin, resolve_legs
```

Two invocation shapes

resolve_legs(config, expected_count=None) accepts either:

1. **Live ZMQ process** — inputs is a length-N list of market-data candle topics, one per leg. Leg instruments are parsed from each topic via parse_market_topic.

```
config = StrategyConfig(
    name="basket_btc_eth_sol",
    inputs=[
        "market.paper.kraken.BTC-USD.candles.1h",
        "market.paper.kraken.ETH-USD.candles.1h",
        "market.paper.kraken.SOL-USD.candles.1h",
    ],
    outputs=["BTC-USD", "ETH-USD", "SOL-USD"],
    ...
)
legs = resolve_legs(config, expected_count=3)
```

2. **Direct-DB backtest** — DirectDbEngine synthesises a single candles.{exchange}.synthetic.{timeframe} input and passes the leg instruments through outputs.

```
config = StrategyConfig(
    name="basket_btc_eth_sol",
    inputs=["candles.kraken.synthetic.1h"],
    outputs=["BTC-USD", "ETH-USD", "SOL-USD"],
    ...
)
legs = resolve_legs(config, expected_count=3)
```

resolve_legs raises ValueError if neither shape matches or if expected_count is set and the resolved tuple has the wrong length.

Mixin API

Subclassing both BaseStrategy and MultiLegSpreadMixin gives you:

Attribute / method	Purpose
self.legs: tuple[str, ...]	Resolved leg instruments in declaration order. Populated by self._init_legs(expected_count=N) from __init__.
self._partner_legs(current)	Tuple of leg names other than current, in declaration order.
self._partner_prices(current)	{leg: last_close} for partners with at least one buffered candle. Partners with empty buffers are omitted (same warmup gate as 2-leg cointegration).

Attribute / method	Purpose
self._build_partner_signals(current, builder)	Pure builder: calls builder(leg, last_close) for each buffered partner and returns the resulting list[StrategySignal]. The host strategy concatenates these after its primary signal and returns [primary, *partners] from on_candle; BaseStrategy emits every leg atomically (see "Signal pairing semantics"). No side-channel queue.

Worked example — 3-leg equal-weight basket

The strategy builds its partner legs with _build_partner_signals and returns the whole group as [primary, *partners] from on_candle. BaseStrategy group-validates and emits every leg atomically on BOTH the live/paper path and the backtest path (process_time_batch), so the synchronized N-leg rebalance behaves identically across paths.

```
from snapper.messaging.schemas.data import CandleData
from snapper.core.types import PairedExecutionPolicyEnum, TradeSide
from snapper.strategies.base import BaseStrategy, StrategyConfig, StrategySignal
from snapper.strategies.base import StrategySignalResult
from snapper.strategies.decorators import register_strategy,
create_strategy_process
from snapper.strategies.multi_leg import MultiLegSpreadMixin


@register_strategy("EqualWeightBasket")
@create_strategy_process(
    process_name="basket_btc_eth_sol",
    default_config={
        "name": "basket_btc_eth_sol",
        "inputs": [
            "market.paper.kraken.BTC-USD.candles.1h",
            "market.paper.kraken.ETH-USD.candles.1h",
            "market.paper.kraken.SOL-USD.candles.1h",
        ],
        "outputs": ["BTC-USD", "ETH-USD", "SOL-USD"],
        "exchange": "paper",
        "params": {"lookback_window": 50, "z_entry": 2.0, "z_exit": 0.5},
    },
)
class EqualWeightBasket(BaseStrategy, MultiLegSpreadMixin):
    """Z-score each leg vs the basket mean and rebalance when it diverges."""

    PAIRED_EXECUTION_POLICY = PairedExecutionPolicyEnum.SIMULTANEOUS

    def __init__(self, config: StrategyConfig) -> None:
        super().__init__(config)
        self.lookback_window = int(self.params.get("lookback_window", 50))
        self.z_entry = float(self.params.get("z_entry", 2.0))
        self.z_exit = float(self.params.get("z_exit", 0.5))
        self._init_legs(expected_count=3)
```

```
    async def on_candle(
        self, instrument: str, candle: CandleData
    ) -> StrategySignalResult:
        partner_prices = self._partner_prices(instrument)
        if len(partner_prices) < len(self.legs) - 1:
            return None

        basket_mean = (candle.close + sum(partner_prices.values())) /
len(self.legs)
        z = (candle.close - basket_mean) / basket_mean

        if abs(z) < self.z_entry:
            return None

        side: TradeSide = "sell" if z > 0 else "buy"
        primary = StrategySignal(
            instrument=instrument,
            side=side,
            strength=min(1.0, abs(z) / self.z_entry),
            price=candle.close,
            reason=f"basket_z={z:+.2f}",
        )
        opposite: TradeSide = "buy" if side == "sell" else "sell"
        partners = self._build_partner_signals(
            instrument,
            lambda leg, price: StrategySignal(
                instrument=leg,
                side=opposite,
                strength=primary.strength,
                price=price,
                reason=f"basket_partner_of_{instrument}",
            ),
        )
        return [primary, *partners]
```

Signal pairing semantics

A strategy callback (on_candle / on_tick / on_trade) returns its legs directly: None, a single StrategySignal, or a list[StrategySignal] in the order the strategy chooses. There is no side-channel queue. BaseStrategy normalizes the return to a list and runs a **fail-closed group preflight** at the callback boundary before anything is published or recorded:

- every entry must be a StrategySignal;
- no two legs may share an instrument;
- every leg's instrument must map to a configured output topic;
- a multi-leg group on a **live** (non-paper) exchange is rejected outright unless PAIRED_EXECUTION_GUARD_ENABLED is true — the fail-closed gate that keeps real-money spreads dark until the paired-execution guard is deliberately switched on.

If any check fails the whole group is rejected (the call raises) and **nothing is emitted** — a malformed multi-leg return can never leave one leg of a spread naked. On success, _listen_loop emits each leg in order and process_time_batch records each leg, so the live/paper path and the backtest path behave identically: the full N-leg basket is one synchronized group.

Every multi-leg strategy **must declare a coordination policy** via the PAIRED_EXECUTION_POLICY class attribute (PairedExecutionPolicyEnum.SIMULTANEOUS for same-instant spreads, as CointegrationPairs declares, or SEQUENTIAL_HANDOFF for exit-then-enter chains); emitting a list[StrategySignal] without one raises ValueError. On emission BaseStrategy stamps the group with a shared paired_group_id (uuid7), paired_group_size, per-leg paired_group_index, the declared paired_group_policy, and a canonical paired_group_key — the descriptors the paired-execution guard correlates on.

> **Scope — emission, not venue atomicity.** This wires up paired *emission*: both legs are published together or not at all. It does NOT provide venue-level execution atomicity. If one leg rejects, partially fills, or fills late, exposure is still possible because the two legs are independent sends to independent executors/coordinators (and at N≥2 instances the legs can be owned by *different* coordinators, since the shard key includes the instrument). Venue-side atomicity is the **paired-execution guard**'s job: group-id correlation through an arming barrier, halt-both-shards on one-leg failure/timeout, and compensating reduce-only flattens. The guard is implemented but ships dark — with PAIRED_EXECUTION_GUARD_ENABLED false (the default) BaseStrategy refuses to emit live multi-leg groups entirely, while paper multi-leg is always allowed. Operator surface: GET /api/paired-execution/incidents and POST /api/paired-execution/groups/{id}/terminalize — see docs/paired-execution.md.

When NOT to use the mixin

- Single-leg strategies (RSI, MACD, VWAP) — there are no partners to pair with; subclassing the mixin adds no value.
- Strategies whose legs use heterogeneous timeframes — legs assumes one timeframe per strategy process. Run one process per timeframe and coordinate via signals.
- Calendar spreads where one leg trades on a derived feed (e.g. funding-rate vs spot) — the input topic shape doesn't fit resolve_legs; subclass BaseStrategy directly.

Data Access

Candle Buffer

```
async def on_candle(self, instrument: str, candle: CandleData) ->
StrategySignal | None:
    candles = self.candle_buffer.get(instrument, [])

    recent = candles[-20:]

    import pandas as pd
    closes = pd.Series([c.close for c in candles])
```

CandleData

```
from snapper.messaging.schemas.data import CandleData
```

Fields:

Field	Type	Description
type	string	"candle"
instrument	string	Trading pair symbol (e.g. BTC-USD)
exchange	string	Source exchange
timeframe	string	Candle duration (1m, 1h, 1d, ...)
open_at	datetime	Exchange-provided candle interval start time
open	float	Opening price
high	float	High price
low	float	Low price
close	float	Closing price
volume	float	Total traded volume
vwap	float | None	Volume-weighted average price (optional)
trades	int | None	Number of trades in the candle (optional)
complete	bool	True marks the final bar for its window; False marks a provisional intra-minute update of a still-forming "living" bar (default True)

> Note: strategies receiving non-final bars (complete == False) may want to gate signal logic on candle.complete so they act only on closed candles and ignore provisional intra-minute updates.

Technical Indicators

RSI

```
from snapper.indicators.rsi import rsi
import pandas as pd

closes = pd.Series([c.close for c in candles])
rsi_values = rsi(closes, period=14)
current_rsi = rsi_values.iloc[-1]
```

MACD

```
from snapper.indicators.ta_lib_adapter import macd

macd_line, signal_line, histogram = macd(closes, fast=12, slow=26, signal=9)
```

TA-Lib (via adapter)

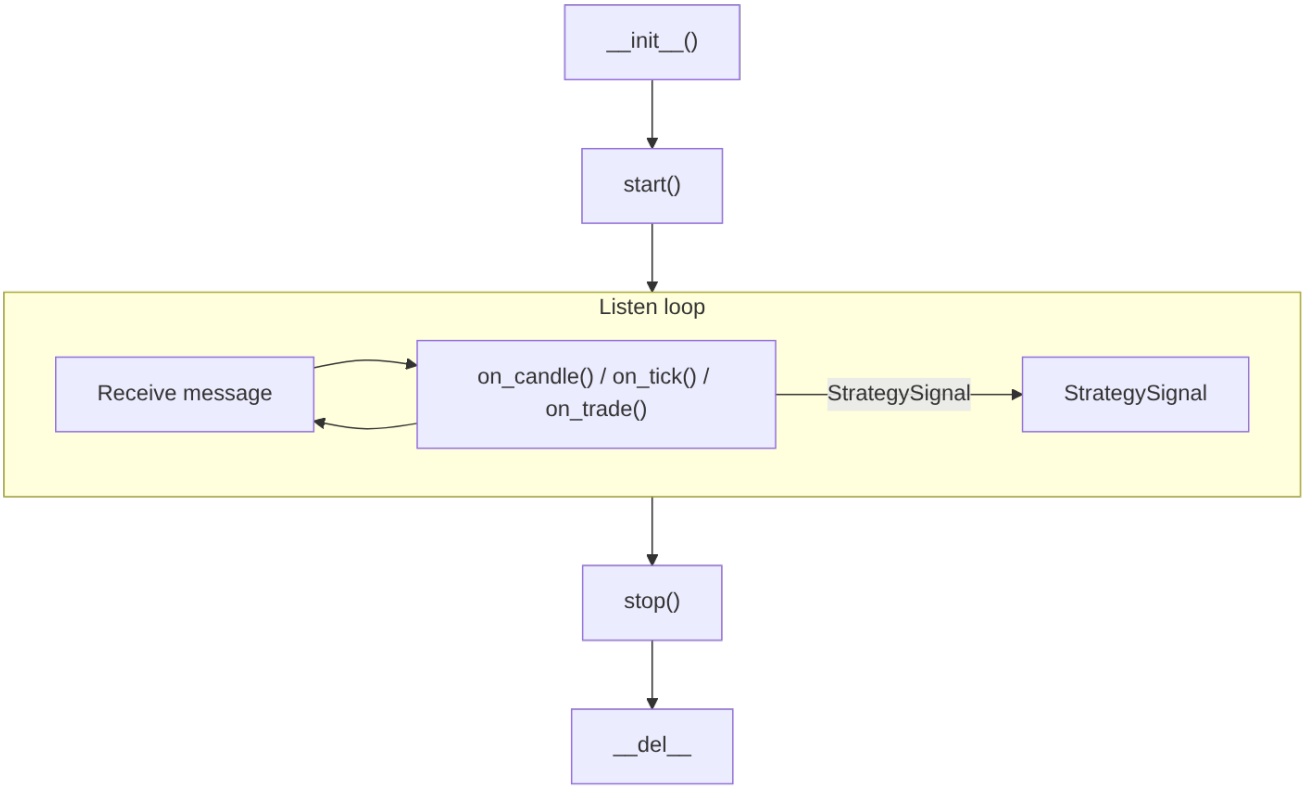
```
from snapper.indicators.ta_lib_adapter import rsi, macd
```

Configuration Validation

Framework automatically validates:

1. **Strategy name** — cannot be empty
2. **Inputs** — at least one required
3. **Outputs** — at least one instrument must be defined
4. **Exchange** — must be one of: paper, kraken, kraken_futures, walutomat
5. **Instruments** — must be available on selected exchange
6. **Paper/live mixing** — mixing paper inputs with live exchange not allowed

Strategy Lifecycle



BaseStrategy does not expose a separate cleanup() hook — teardown happens in stop() (cancel heartbeat + listen tasks, unsubscribe inputs, close the ZMQ publisher + context) and the implicit __del__ (cancel any surviving tasks, close subscriber/publisher sockets, terminate the ZMQ context).

Running Strategies

Via Process Manager (recommended)

Strategies are automatically managed by the server:

```
snapper server
```

Strategies can be enabled/disabled in the dashboard.

Standalone (for testing)

```
import asyncio
from snapper.strategies.factory import StrategyFactory
from snapper.strategies.base import StrategyConfig

# Importing the strategy module fires the @register_strategy
```

```python
# side effect that populates the factory's registry. Snapper's
# strategies/__init__.py is intentionally empty (no re-exports
# per project convention), so each concrete strategy must be
# imported by hand before the factory can resolve it by name.
import snapper.strategies.rsi  # noqa: F401  # register RSIReversion

config = StrategyConfig(
    name="test_rsi",
    strategy_class="RSIReversion",
    inputs=["market.kraken.BTC-USD.candles.1h"],
    outputs=["BTC-USD"],
    exchange="paper",
    params={"period": 14, "upper": 70, "lower": 30},
)

factory = StrategyFactory()
strategy = factory.create_strategy(config)

async def main() -> None:
    # `BaseStrategy.start()` is non-blocking — it wires sockets and
    # spawns the listen + heartbeat tasks, then returns. Keep the
    # event loop alive while the background tasks process data.
    await strategy.start()
    try:
        await asyncio.Event().wait()  # block until Ctrl-C / cancel
    finally:
        await strategy.stop()

asyncio.run(main())
```

Testing Strategies

Unit Tests

```python
import pytest
from snapper.strategies.rsi import RSIReversion
from snapper.strategies.base import StrategyConfig
from snapper.messaging.schemas.data import CandleData


@pytest.fixture
def rsi_strategy() -> RSIReversion:
    config = StrategyConfig(
        name="test_rsi",
        strategy_class="RSIReversion",
        inputs=["market.kraken.BTC-USD.candles.1h"],
        outputs=["BTC-USD"],
        exchange="paper",
        params={"period": 14, "upper": 70, "lower": 30},
    )
    return RSIReversion(config)


async def test_buy_signal_on_low_rsi(rsi_strategy: RSIReversion) -> None:
    # Prepare data with low RSI
```

```
    ...
    signal = await rsi_strategy.on_candle("BTC-USD", candle)
    assert signal is not None
    assert signal.side == "buy"
```

Backtesting

For BacktestConfig runs, direct_db instantiates strategies with a synthetic input prefix (candles.{exchange}.synthetic.{timeframe}), while zmq_replay publishes market.{exchange}.{instrument}.candles.{timeframe} on a private broker. The separate paper_feed_publisher replay process is the path that uses the dedicated market.paper.{source_exchange}... topic shape:

```
default_config={
    "inputs": ["market.paper.kraken.BTC-USD.candles.1h"],
    "exchange": "paper",
    ...
}
```

replay.* is not a valid ZMQ topic — BaseStrategy._listen_loop filters incoming frames to market.* and routes them through BaseStrategy._dispatch_market_data. The canonical builder is market_topic(...) in src/snapper/messaging/topics/builders.py; paper topics are built by passing exchange="paper" together with the venue under source_exchange=... (there is no separate paper_market_topic symbol).

Trailing Stop Plans

Trailing stops are server-side evaluated execution plans that ratchet a stop price as the market moves favorably and fire a market close when the stop is breached. Attach a trailing stop to an open position_cycle via POST /api/trailing-stops (see docs/api.md).

Parameters

Field	Type	Bounds	Meaning
trailing_pct	float	0 < x < 100	Distance from peak price, as percentage. 5.0 means "trigger 5 % below the highest favorable price seen".
min_lock_pct	float	0 <= x < 100 (default 0)	Profit threshold the market must cross before the trailing stop arms. 0 means "arm immediately on attach"; 5.0 means "wait until price has moved 5 % in your favor before tracking a stop".

Behavior

- **Peak ratchet:** every tick where the last price moves favorably updates the peak; the stop level recomputes as peak * (1 - trailing_pct/100) for longs or peak * (1 + trailing_pct/100) for shorts.
- **Stop monotonicity:** for longs, the stop only moves UP (never down); for shorts the stop only moves DOWN. An adverse tick never loosens the stop.
- **min_lock dead zone:** while min_lock_pct > 0 and the market has not yet crossed that threshold, the trailing stop is attached and armed but NOT tracking — a close is NOT emitted no matter how far the price moves against you. A decision warning is logged on the trailing_stop_created row at create time.
- **Single trailing stop per cycle:** enforced by the partial unique index uq_ep_active_trailing_stop_per_cycle. A second create on the same open cycle returns HTTP 409.
- **Cycle close sweeps the plan:** if the cycle closes externally (reconciliation flat, manual close, bracket fill), _sweep_cycle_closures() cancels the trailing stop on the next clock tick.
- **Checkpoint persistence:** peak_price and current_stop are persisted every 10 s via the plan-executor checkpoint loop. On restart, state is restored and peak_price is floored at entry_price (long: max(peak, entry); short: min(peak, entry)).
- **Downtime semantics:** if the service is down when the true market price crosses the stop, the stop fires on the first tick received after restart. There is no placed order on the exchange — the evaluator is purely server-side.

Interaction with brackets

A position cycle may carry both a bracket and a trailing_stop plan at the same time (the two partial unique indexes are disjoint per plan_type). Whichever fires first closes the cycle; the other is swept by the cycle-close handler.

Limitations

- entry_price and total_quantity are frozen at attach time. Dollar-cost averaging (scaling into a position) requires POST /cancel + re-attach after the new average is known.
- No exchange-side native trailing stops — every decision is made by PlanExecutorService against live ticks.
- Futures only (same capability gate as brackets: supports_reduce_only). Not supported on Kraken spot.
- No breakeven-move feature (ratchet stop to entry after N % profit).
- No time-based activation (arm after N minutes regardless of price).

- Live rollout gated on the same follow-ups (wallet-safe routing + orphan-cycle admin) that brackets depend on.

Cross-asset market-data pattern

Snapper enables strategies to subscribe to a market-data-only feed (SymbolExchangeCapability.can_trade=False) and emit signals whose target is an execution-capable instrument on a different venue. Kraken FCM index futures (kraken_equities) are the primary driver: they publish ~10-minute-delayed candles + ticks but have no order API.

Rules

- Strategies MAY subscribe to any combination of can_market_data=True topics regardless of the can_trade state of the source instrument.
- Every emitted StrategySignal.instrument MUST point at an can_trade=True instrument on the strategy's configured exchange. Snapper's order-entry capability guard (snapper.server._capability_guard.require_tradable) rejects submits against can_trade=False pairs with HTTP 422 error_code='instrument_market_data_only' — the signal would be dropped before reaching the venue.
- Tick-driven strategies that consume delayed feeds MUST gate on TickData.is_delayed=True before treating the price as current. Candle-driven cross-asset strategies inherit the source-feed latency implicitly; document the lag in the live runbook.
- Output-coverage enforcement at startup is conditional: when both operator_public_id and wallet_public_id are populated on the process config, _enforce_strategy_outputs_covered checks that every instrument in StrategyConfig.outputs falls within the operator's active scope grants. The empty-default path (neither field set) is permitted for backwards compatibility, and operator-only launches skip the output-coverage check.

Reference implementation

See src/snapper/strategies/examples/tradfi_observe_crypto_execute.py for a minimal EMA-crossover strategy that observes MNQM6-CME on kraken_equities and targets BTC-USD on kraken. The file is NOT registered with the process registry — it ships as a copy-paste starting point. Activation steps live in the module docstring; tests/meta/test_reference_strategy_not_registered.py enforces the non-registration invariant so future maintainers cannot accidentally promote the illustration into a live process.